DEPI GRADUATION PROJECT — FINAL MASTER NETWORK REPORT (TASKS T1 – T68)

Secure AWS Network Architecture, Traffic Flow & Hybrid Connectivity

AWS Account: 102380858878 | Primary Region: us-east-1 (N. Virginia) | DR Region: us-west-2 (Oregon)

Core Principle: Secure -> Segment -> Connect -> Monitor -> Detect -> Remediate -> Recover

Executive Summary & Master Architecture Overview

This report documents the architecture, implementation, security testing, and operational runbooks for the Digital Egypt Pioneers Initiative (DEPI) graduation project. The system provides a multi-tier, defense-in-depth cloud network built on AWS to host production web and database workloads while maintaining strict isolation, hybrid connectivity, and automated incident response.

The infrastructure spans two AWS regions and three primary Virtual Private Clouds: a Production VPC (prod-vpc, 10.0.0.0/16) and a Shared Services VPC (shared-vpc, 10.1.0.0/16) in us-east-1 (N. Virginia), paired with a Pilot Light Disaster Recovery VPC (dr-vpc, 10.2.0.0/16) in us-west-2 (Oregon) and a simulated on-premises environment (192.168.0.0/16). Traffic is secured from the global edge using Amazon Route 53, CloudFront Origin Access Control (OAC), and AWS WAF, segmented internally through least-privilege Security Groups and stateless Network ACLs, and monitored continuously via VPC Flow Logs, Amazon Athena, GuardDuty, and an EventBridge-to-Lambda auto-quarantine pipeline.

Cost Governance & Lifecycle Management: The environment was engineered to operate within a $55–$75 target budget (out of a $200 allocation) by deploying a single zonal NAT Gateway in prod-public-a, routing S3 traffic through a free Gateway Endpoint, keeping the us-west-2 DR environment at zero running compute, and executing same-session deploy, test, and teardown cycles for hourly-billed hybrid components (Transit Gateway, Site-to-Site VPN, and Client VPN). Upon completion of all verification tests, a dependency-ordered teardown (Task T68) was executed across both regions to return account billing to $0.00.

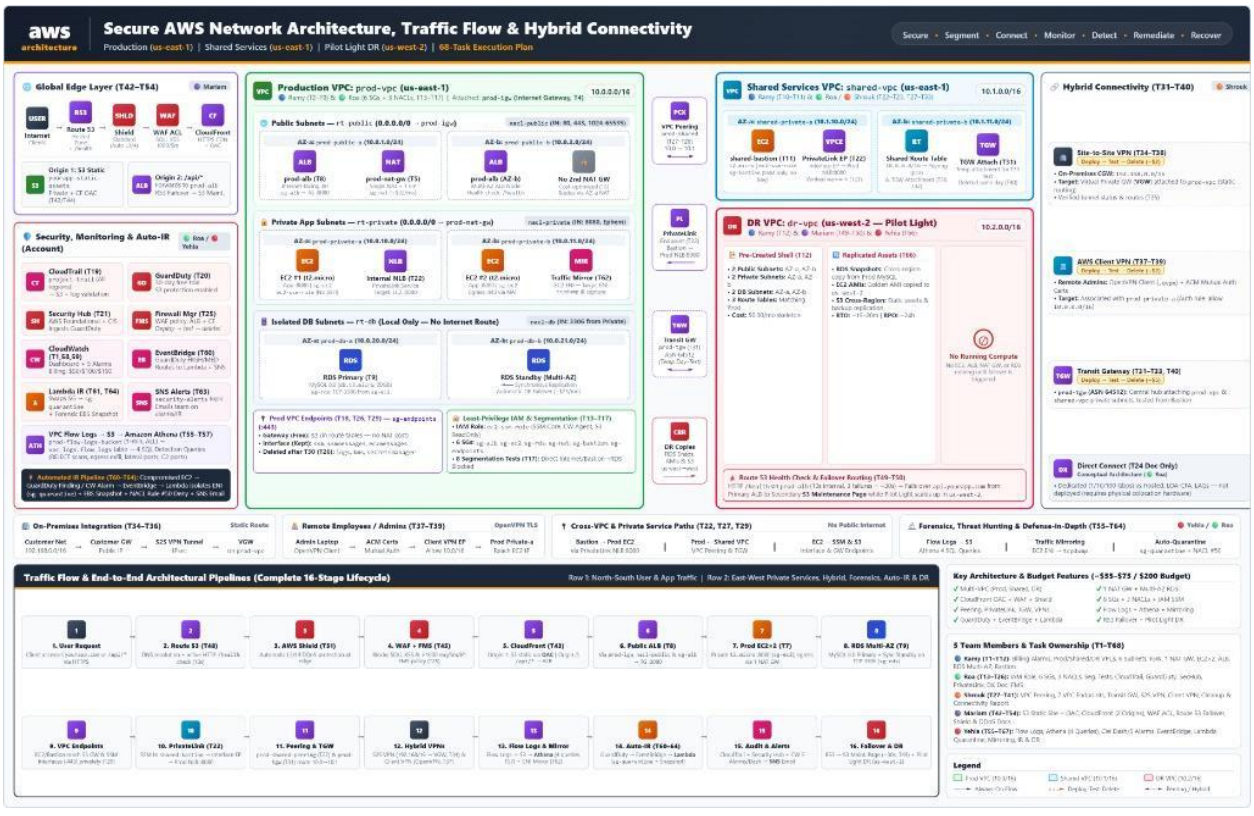


Figure 0.1: Complete 68-Task Secure AWS Network Architecture, Traffic Flow & 16-Stage Lifecycle Diagram.

Team Members, Role Ownership & Deliverable Status Matrix (Tasks T1 – T68)

Team Member	Role & Workstream Ownership	Assigned Tasks	Deliverable Status
1. Ramy (Lead)	VPC Foundation: Billing Alarms ($50/$100/$150), Prod/Shared/DR VPCs, 8 Subnets, IGW, NAT GW, Route Tables, EC2s, ALB, Multi-AZ RDS & Backend /api/* + /health/ Fix	T1 – T12 + API Fix	Completed (100% Verified)
2. Roa	Security & Segmentation: IAM Role (ec2-ssm-role), 7 Security Groups, 3 Stateless NACLs, 8 Segmentation Tests, CloudTrail, GuardDuty, Security Hub, PrivateLink & Direct Connect Doc	T13 – T26	Completed (T13–T24 & T26 Done; T25 Deferred by Design)
3. Shrouk (with Yehia)	Connectivity & Hybrid: VPC Peering, S3 Gateway + 9 Interface Endpoints, Transit Gateway Hub, Site-to-Site VPN, Dual-Method Client VPN Verification (Simulation + Local Hotspot) & Cost Cleanup	T27 – T41	Completed (100% Verified & Cost-Cleaned)
4. Mariam	Edge Security & Delivery: Private S3 Static Website, CloudFront OAC, Dual-Origin CDN (/api/* & /health/*), AWS WAF Rules & XSS Testing, Route 53 Failover, Shield, CF vs. GA & DDoS Plan	T42 – T54	Completed (100% Verified)
5. Yehia	Monitoring, Auto-IR & DR: SNS Alerts, VPC Flow Logs, Athena Partitioned Table + 4 Threat Queries, CW Dashboard + 8 Alarms, EventBridge + Lambda Quarantine, Traffic Mirroring, Live IR Sim, Runbook, DR Plan & Final Report	T55 – T67	Completed (100% Verified)
All 5 Members	Final Account Cleanup: Dependency-ordered teardown of all AWS resources across us-east-1 and us-west-2 (plus automated CLI teardown script) to return billing to $0.00	T68	Completed (100% Cleaned Up — $0.00 Active Billing)

SECTION 1 — WORKSTREAM 1 (TASKS T1 – T12 + BACKEND API FIX)

Ramy — VPC Foundation, Multi-AZ Network Core, Compute, Load

Balancing, RDS & DR Shell

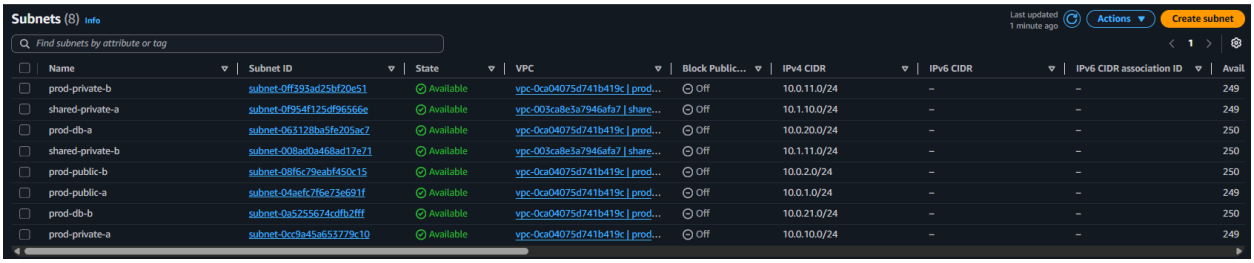
Lead Engineer: Ramy | Primary Region: us-east-1 (N. Virginia) & DR Region: us-west-2 (Oregon)

1.1 VPC Foundation, Subnets, Gateways & Route Tables (Tasks T1 – T6, T10)

Ramy established the foundational multi-VPC topology in us-east-1 after configuring three CloudWatch billing alarms ($50, $100, $150) tied to SNS notifications (T1). He provisioned prod-vpc (vpc-0ca04075d741b419c, CIDR 10.0.0.0/16) with DNS hostnames and DNS resolution enabled (T2) and shared-vpc (vpc-003ca8e3a7946afa7, CIDR 10.1.0.0/16) (T10). Across two Availability Zones (us-east-1a, us-east-1b), he created 6 production subnets (2 public, 2 private app, 2 isolated DB) and 2 shared private subnets (T3), attached prod-igw (T4), deployed a single cost-optimized NAT Gateway prod-nat-gw in prod-public-a (T5), and associated 4 custom route tables (T6).

prod-vpc	vpc-0ca04075d741b419c	Available	–	–	Off	10.0.0.0/16	–	dopt-02ae25b019ad8a...
shared-vpc	vpc-003ca8e3a7946afa7	Available	–	–	Off	10.1.0.0/16	–	dopt-02ae25b019ad8a...

Figure 1.1 (T2 & T10): Production VPC (vpc-0ca04075d741b419c, 10.0.0.0/16) and Shared Services VPC (vpc-003ca8e3a7946afa7, 10.1.0.0/16) Available in us-east-1.



Subnets (8) Info — Last updated 1 minute ago — Actions — Create subnet

Find subnets by attribute or tag

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6 CIDR association ID	Avail
prod-private-b	subnet-0ff393ad25bf20e51	Available	vpc-0ca04075d741b419c | prod...	Off	10.0.11.0/24	–	–	249
shared-private-a	subnet-0f954f125df96566e	Available	vpc-003ca8e3a7946afa7 | share...	Off	10.1.10.0/24	–	–	249
prod-db-a	subnet-063128ba5fe205ac7	Available	vpc-0ca04075d741b419c | prod...	Off	10.0.20.0/24	–	–	250
shared-private-b	subnet-008ad0a468ad17e71	Available	vpc-003ca8e3a7946afa7 | share...	Off	10.1.11.0/24	–	–	250
prod-public-b	subnet-08f6c79eabf450c15	Available	vpc-0ca04075d741b419c | prod...	Off	10.0.2.0/24	–	–	250
prod-public-a	subnet-04aefc7f6e73e691f	Available	vpc-0ca04075d741b419c | prod...	Off	10.0.1.0/24	–	–	249
prod-db-b	subnet-0a5255674cdfb2fff	Available	vpc-0ca04075d741b419c | prod...	Off	10.0.21.0/24	–	–	250
prod-private-a	subnet-0cc9a45a653779c10	Available	vpc-0ca04075d741b419c | prod...	Off	10.0.10.0/24	–	–	249

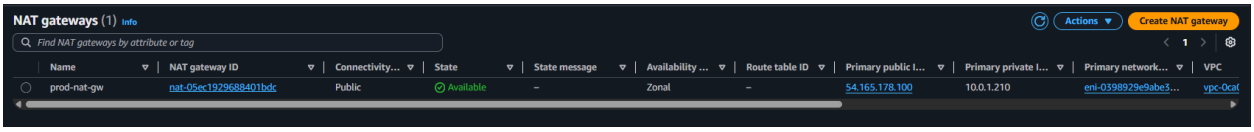
Figure 1.2 (T3 & T10): All 8 Subnets — prod-public-a/b (10.0.1.0/24, 10.0.2.0/24), prod-private-a/b (10.0.10.0/24, 10.0.11.0/24), prod-db-a/b (10.0.20.0/24, 10.0.21.0/24), and shared-private-a/b (10.1.10.0/24, 10.1.11.0/24).

Internet gateways (2) Info — Actions — Create internet gateway

Find internet gateways by attribute or tag

Name	Internet gateway ID	State	VPC ID	Owner
–	igw-0969814a15c4736fe	Attached	vpc-0c3c799b107b10674	102380858878
prod-igw	igw-0ffe7fea11225f8ec	Attached	vpc-0ca04075d741b419c | prod-vpc	102380858878

Figure 1.3 (T4): Internet Gateway prod-igw (igw-0ffe7fea11225f8ec) Attached to prod-vpc.



NAT gateways (1) Info — Actions — Create NAT gateway

Find NAT gateways by attribute or tag

Name	NAT gateway ID	Connectivity...	State	State message	Availability ...	Route table ID	Primary public i...	Primary private i...	Primary network...	VPC
prod-nat-gw	nat-05ec1929688401bdc	Public	Available	–	Zonal	–	54.165.178.100	10.0.1.210	eni-0398929e9abe3...	vpc-0ca0

Figure 1.4 (T5): NAT Gateway prod-nat-gw (nat-05ec1929688401bdc) Available in prod-public-a with Elastic IP 54.165.178.100 (Private IP 10.0.1.210).

Route tables (7) Info — Last updated 2 minutes ago — Actions — Create route table

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
rt-private	rtb-0bc021a16cb11a61d	2 subnets	–	No	vpc-0ca04075d741b419c | prod-vpc	102380858878
–	rtb-0bdb46ce208ad163d	–	–	Yes	vpc-0c3c799b107b10674	102380858878
rt-shared	rtb-001fc230175b09ae5	2 subnets	–	No	vpc-003ca8e3a7946afa7 | shared-vpc	102380858878
–	rtb-0abbb753b55141fca	–	–	Yes	vpc-003ca8e3a7946afa7 | shared-vpc	102380858878
–	rtb-0efda5cf4e5f02f1e	–	–	Yes	vpc-0ca04075d741b419c | prod-vpc	102380858878
rt-db	rtb-0b3419cb66da46a74	2 subnets	–	No	vpc-0ca04075d741b419c | prod-vpc	102380858878
rt-public	rtb-056b0aad7de0cbb3a	2 subnets	–	No	vpc-0ca04075d741b419c | prod-vpc	102380858878

Figure 1.5 (T6 & T10): Custom Route Tables — rt-public (rtb-056b0aad7de0cbb3a), rt-private (rtb-0bc021a16cb11a61d), rt-db (rtb-0b3419cb66da46a74, local only), and rt-shared (rtb-001fc230175b09ae5), each explicitly associated with 2 subnets.

1.2 Compute, Application Load Balancer, RDS Multi-AZ & DR Shell (Tasks T7 – T9, T11 – T12)

In T7 & T11, Ramy launched two private application servers (prod-ec2-a i-00653e2f9fdc1a01b at 10.0.10.214 in us-east-1a, and prod-ec2-b i-0967ccda13818d5f5 at 10.0.11.64 in us-east-1b) alongside shared-bastion (i-007c4c2f38f78cbe6 at 10.1.10.130) with zero public IPs and no SSH key pairs, relying strictly on AWS Systems Manager (SSM). In T8, he deployed the internet-facing Application Load Balancer prod-alb across the public subnets forwarding HTTP:80 to Target Group prod-tg (HTTP:8080). In T9, he deployed prod-rds (MySQL 8.0, db.t3.micro, Multi-AZ enabled, 20GB gp2, database name appdb, master user admin) inside isolated DB subnets (endpoint: prod-rds.cav40wgss3cy.us-east-1.rds.amazonaws.com). Finally, in T12, he switched to us-west-2 and built the Pilot Light DR VPC skeleton dr-vpc (vpc-056006a43fdc16bd8, 10.2.0.0/16) with 6 matching subnets and route tables, maintaining zero running compute for $0 idle cost.

Figure 1.6 (T7 & T11): All 3 EC2 Instances Running with 0 Public IPs — prod-ec2-a (i-00653e2f9fdc1a01b, 3/3 checks passed), prod-ec2-b (i-0967ccda13818d5f5, 3/3 checks passed), and shared-bastion (i-007c4c2f38f78cbe6, 2/2 checks passed).

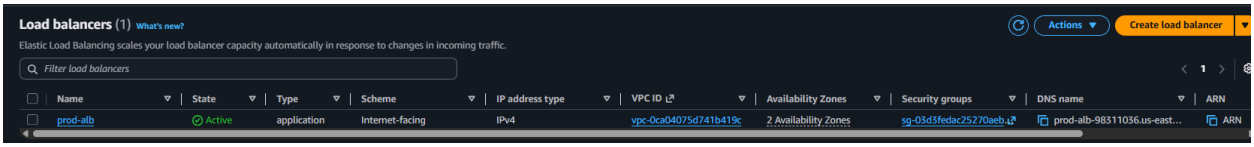


Figure 1.7 (T8): Internet-facing Application Load Balancer prod-alb Active across 2 AZs (DNS: prod-alb-98311036.us-east-1.elb.amazonaws.com) protected by sg-03d3fedac25270aeb (Sg-alb).

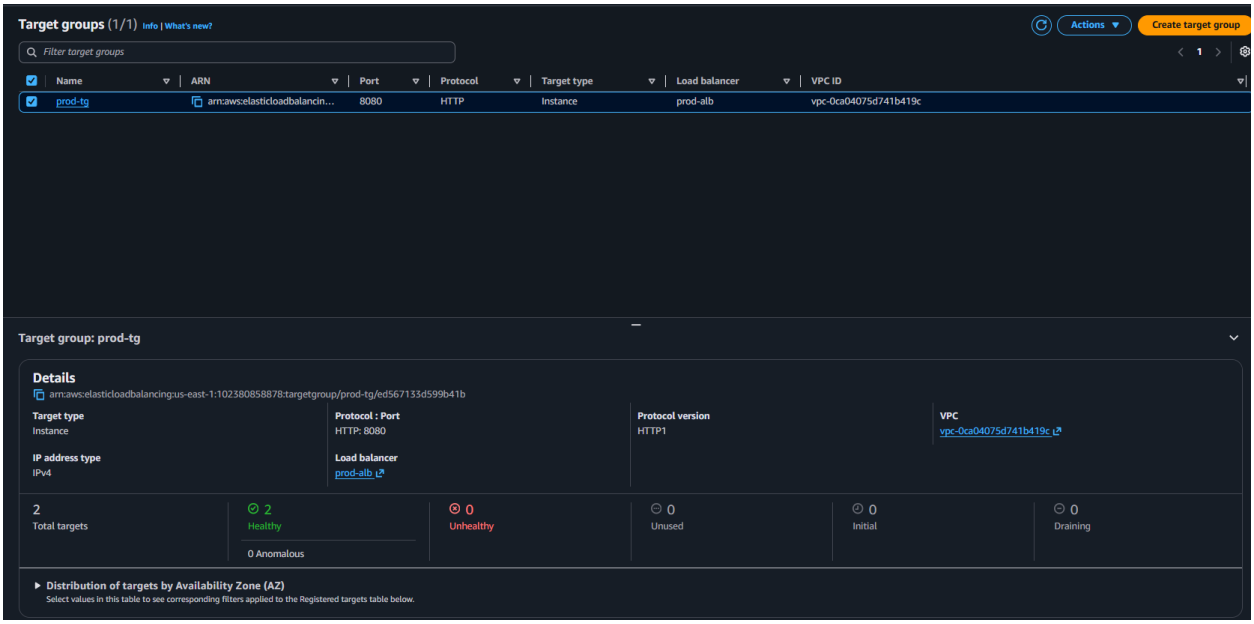


Figure 1.8 (T8): Target Group prod-tg (HTTP:8080) showing 2 Total Targets — 2 Healthy, 0 Unhealthy across us-east-1a and us-east-1b.

Figure 1.9 (T12): Pilot Light Disaster Recovery VPC Skeleton dr-vpc (vpc-056006a43fdc16bd8, CIDR 10.2.0.0/16) Available in us-west-2 (Oregon).

1.3 Backend API & Health Endpoint Resolution (/api/* & /health/)

During Phase 4 handover, Mariam reported that CloudFront's /api/* behavior was forwarding requests to prod-alb, but backend EC2 instances returned HTTP 404 because only the root /var/www/html/index.html page existed. Ramy connected to both prod-ec2-a and prod-ec2-b via SSM Session Manager, provisioned the /var/www/html/api/index.html JSON payload and /var/www/html/health/index.html endpoint (for Route 53 T49 health checks), reloaded Apache httpd, and verified live HTTP/1.1 200 OK responses both locally and end-to-end through prod-alb.

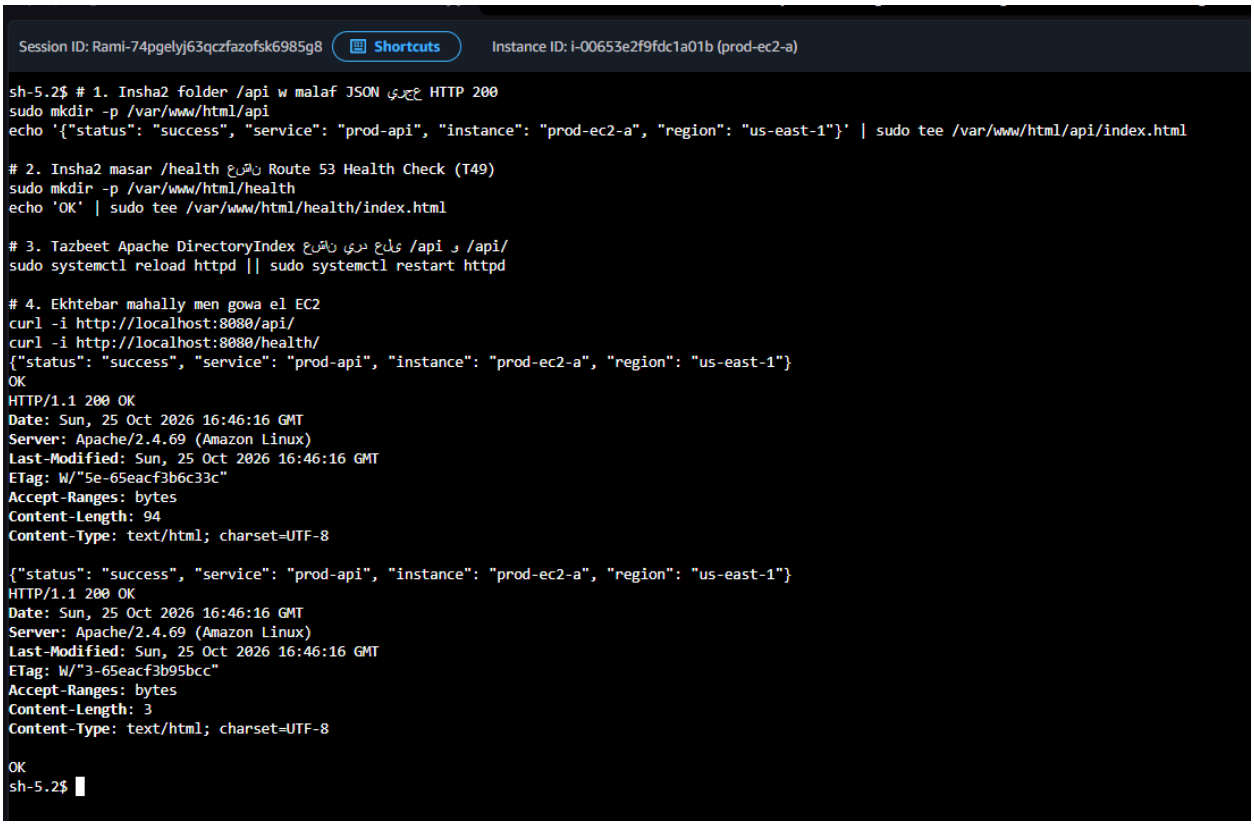


Figure 1.10a: SSM Session on prod-ec2-a (i-00653e2f9fdc1a01b) configuring & verifying /api/ and /health/ (HTTP/1.1 200 OK).

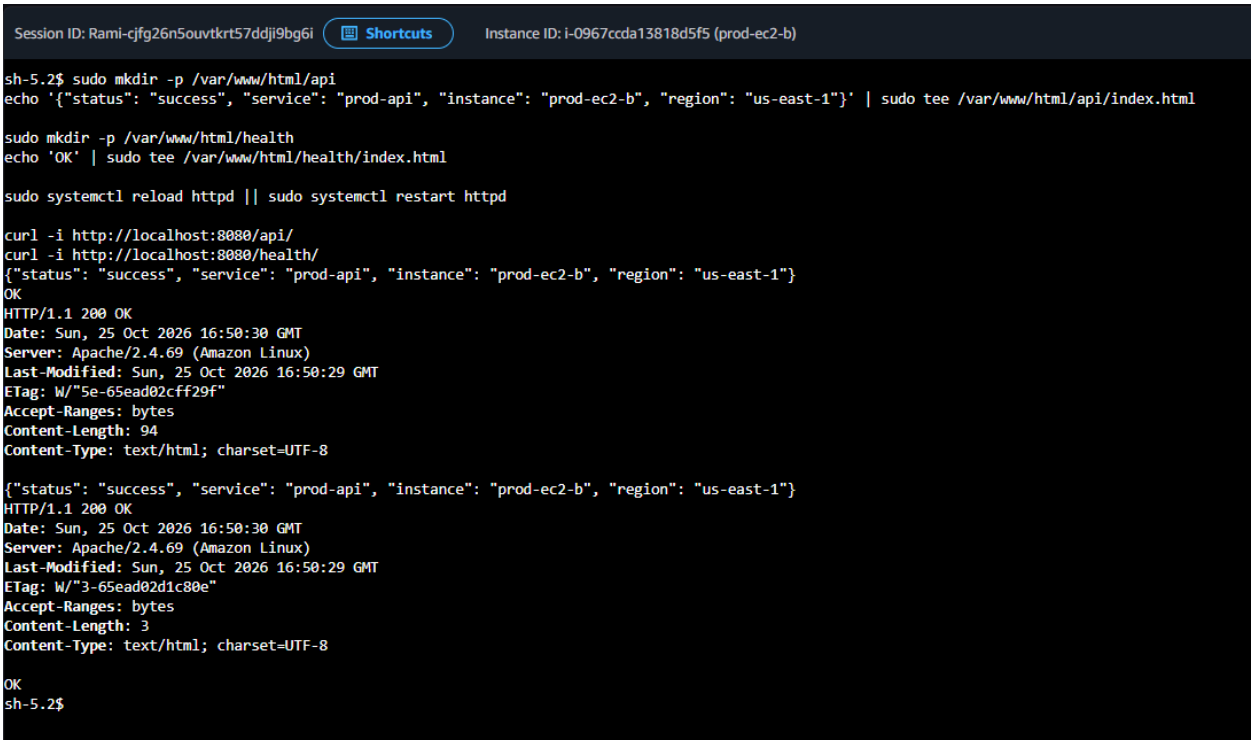


Figure 1.10b: SSM Session on prod-ec2-b (i-0967ccda13818d5f5) configuring & verifying /api/ and /health/ (HTTP/1.1 200 OK).

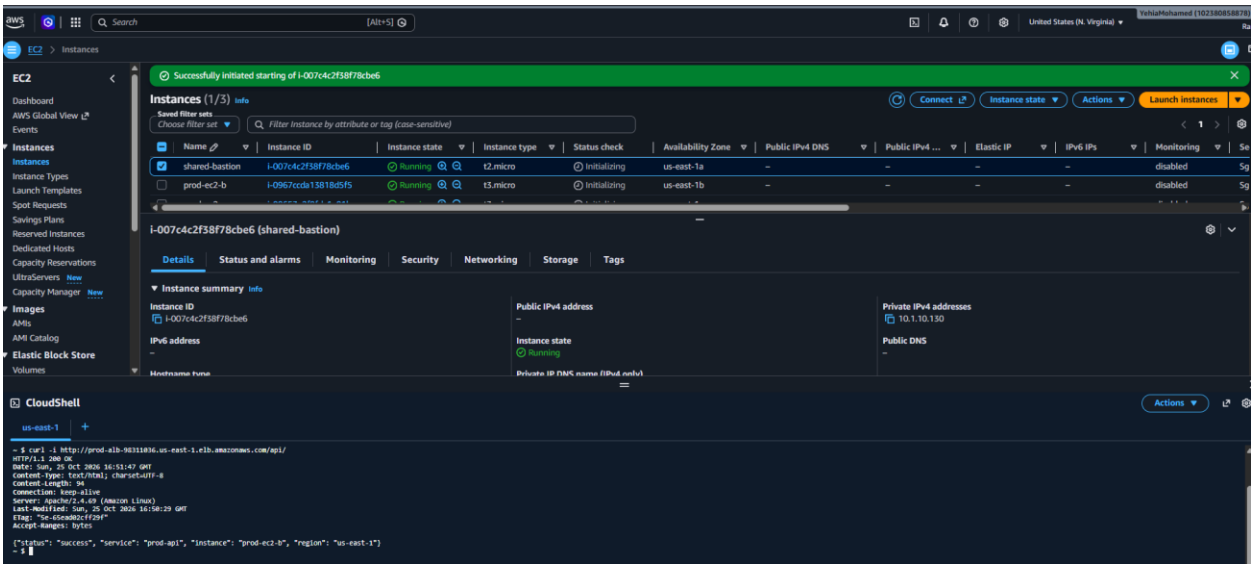


Figure 1.11: End-to-End Verification from AWS CloudShell — curl -i http://prod-alb-98311036.us-east-1.elb.amazonaws.com/api/ returning HTTP/1.1 200 OK and JSON {"status": "success", "service": "prod-api", "instance": "prod-ec2-b", "region": "us-east-1"}.

Resource Tier	Resource Name	AWS Resource ID / Endpoint	Configuration Summary (Ramy T1 – T12)
VPCs	prod-vpc / shared-vpc / dr-vpc	vpc-0ca04075d741b419c / vpc-003ca8e3a7946afa7 / vpc-056006a43fdc16bd8	10.0.0.0/16 (us-east-1), 10.1.0.0/16 (us-east-1), 10.2.0.0/16 (us-west-2)
Prod Subnets (6)	prod-public-a/b, prod-private-a/b, prod-db-a/b	subnet-04aefc7f6e73e691f, subnet-08f6c79eabf450c15, subnet-0cc9a45a653779c10, subnet-0ff393ad25bf20e51, subnet-063128ba5fe205ac7,	Public: 10.0.1.0/24, 10.0.2.0/24 | Private App: 10.0.10.0/24, 10.0.11.0/24 | Isolated DB: 10.0.20.0/24, 10.0.21.0/24

		subnet-0a5255674cdfb2fff	
Shared Subnets (2)	shared-private-a / shared-private-b	subnet-0f954f125df96566e / subnet-008ad0a468ad17e71	10.1.10.0/24 (us-east-1a) and 10.1.11.0/24 (us-east-1b)
Gateways	prod-igw & prod-nat-gw	igw-0ffe7fea11225f8ec & nat-05ec1929688401bdc	IGW attached to prod-vpc; Zonal NAT GW in prod-public-a (EIP 54.165.178.100, Private IP 10.0.1.210)
Route Tables	rt-public, rt-private, rt-db, rt-shared	rtb-056b0aad7de0cbb3a, rtb-0bc021a16cb11a61d, rtb-0b3419cb66da46a74, rtb-001fc230175b09ae5	Explicitly associated with 2 subnets each; rt-db has local route only
EC2 Instances	prod-ec2-a, prod-ec2-b, shared-bastion	i-00653e2f9fdc1a01b (10.0.10.214), i-0967ccda13818d5f5 (10.0.11.64), i-007c4c2f38f78cbe6 (10.1.10.130)	Zero public IPs, no SSH keys, managed via SSM with ec2-ssm-role; Apache serving /, /api/, /health/ on port 8080
Load Balancing	prod-alb & prod-tg	prod-alb-98311036.us-east-1.elb.amazonaws.com / prod-tg (HTTP:8080)	Internet-facing ALB across us-east-1a/1b forwarding port 80 to prod-tg (2/2 Healthy)
Database (RDS)	prod-rds (appdb)	prod-rds.cav40wgss3cy.us-east-1.rds.amazonaws.com	MySQL 8.0, db.t3.micro, Multi-AZ enabled, 20GB gp2 in prod-db-a/b

SECTION 2 — WORKSTREAM 2 (TASKS T13 – T26) — FULL UNABRIDGED DOCUMENTATION & ALL 91 FIGURES

Roa — Security, Segmentation, NACLs, CloudTrail, GuardDuty, Security Hub, PrivateLink & Direct Connect

Security Engineer: Roa | AWS Account: 102380858878 (us-east-1)

Security & Segmentation Documentation

Tasks T13 to T24 (T25 deferred) | Roa | AWS account 102380858878, us-east-1 | Evidence captured 19 October 2026 (UTC+03:00)

Contents

1. Overview 2

2. T13: IAM role for the EC2 instances 2

3. T14: Security groups 5

4. T15: Security group rules and resource assignment 5

5. T16: Network ACLs 9

6. T17: Segmentation tests 12

7. T18: Endpoint security groups and peering rules 19

8. T19 to T21: Account-level services 24

9. T22 and T23: PrivateLink 29

10. T24: AWS Direct Connect concepts 39

11. Resource inventory for cleanup (T26 and T68) 42

1. Overview

This document records the Security & Segmentation work for the secure AWS network project (principle: Secure, Segment, Connect, Monitor, Detect, Remediate, Recover). It covers the critical path (T13 to T17), the endpoint and peering rules (T18), the account-level services (T19 to T21), PrivateLink (T22 and T23) and the Direct Connect concepts document (T24). T25 is deferred and T26 and the final cleanup are being done by Yehia; section 11 lists the resources to remove. Every claim points to a numbered figure. Figure numbers match the order of the screenshots in the source Word file (Figure 38 is an exact visual repeat of Figure 36 and is left out). Figures 59 to 69 were taken on 21 October 2026 and Figures 70 to 91 on 22 October 2026. Figure 92 is a diagram drawn for T24.

Task	What	Status	Evidence
T13	IAM role ec2-ssm-role attached to 3 EC2s	Done	Figures 1-5
T14	6 security groups in prod-vpc, plus one in shared-vpc	Done	Figures 6-7
T15	Rules applied and resources moved to their SGs	Done (bastion swap not evidenced)	Figures 8-25
T16	3 NACLs with rules and subnet associations	Done	Figures 27-35
T17	8 segmentation tests	Done with evidence gaps (section 12)	Figures 26, 36-37, 39-49
T18	Interface endpoints use Sg-endpoints; peering rules added	Done with notes	Figures 59-69
T19	CloudTrail project-trail	Done	Figures 50-52
T20	GuardDuty with sample findings	Done (protection plans page not shown)	Figures 53-54
T21	Security Hub with 2 standards	Enabled, scores still generating	Figures 55-58
T22	PrivateLink NLB, endpoint service and endpoint	Done	Figures 70-89, 91
T23	Test through the endpoint from shared-bastion	Done	Figure 90
T24	Direct Connect concepts document	Done (concepts only)	Section 10, Figure 92
T25	Firewall Manager	Deferred, not covered here	None
T26	Endpoint cleanup	Assigned to Yehia	Section 11 inventory

2. T13: IAM role for the EC2 instances

Purpose. The EC2 instances were launched without an IAM profile (no key pairs, no public IPs). The role lets the SSM agent register so Session Manager can be used instead of SSH.

What was built.

- IAM role ec2-ssm-role, created 19 October 2026 at 12:27, with an instance profile of the same name (Figure 1).

- Three AWS managed policies attached: AmazonSSMManagedInstanceCore, CloudWatchAgentServerPolicy and AmazonS3ReadOnlyAccess (the policy names are truncated in Figure 1).
- Role attached to prod-ec2-a, prod-ec2-b and shared-bastion (Figures 2-4).

Result. Passed. A Session Manager shell opened on prod-ec2-a. It shows whoami = ssm-user, hostname ip-10-0-10-214.ec2.internal, and the web app answering on port 8080 (Figure 5).

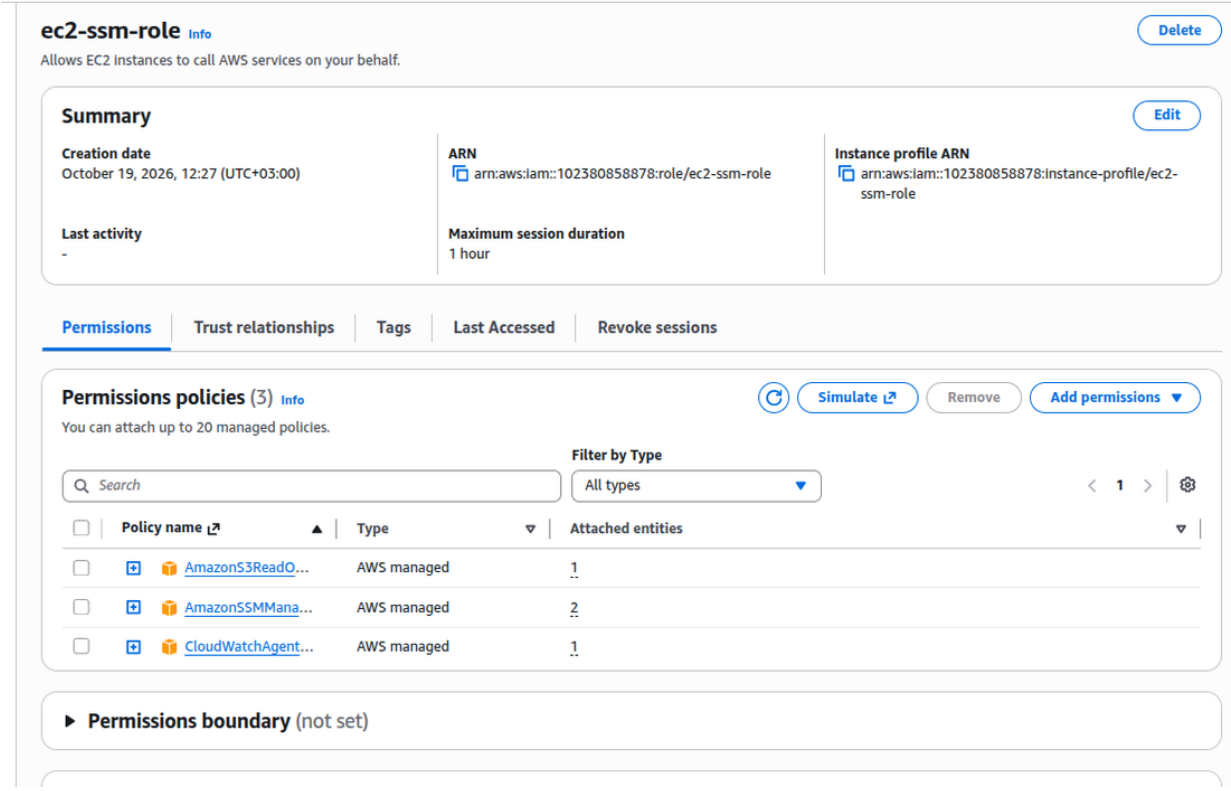


Figure 1. ec2-ssm-role summary: three managed policies attached.

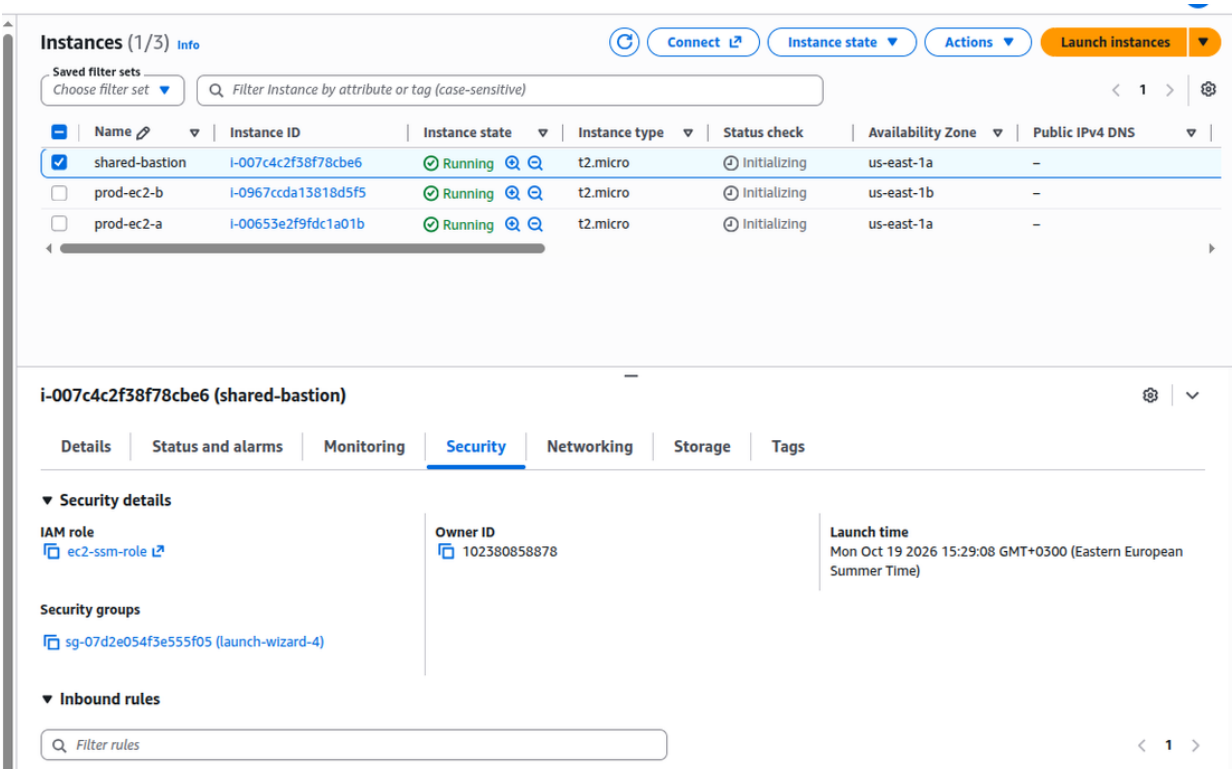


Figure 2. shared-bastion, Security tab: IAM role ec2-ssm-role (still on its original launch-wizard-4 SG at this point).

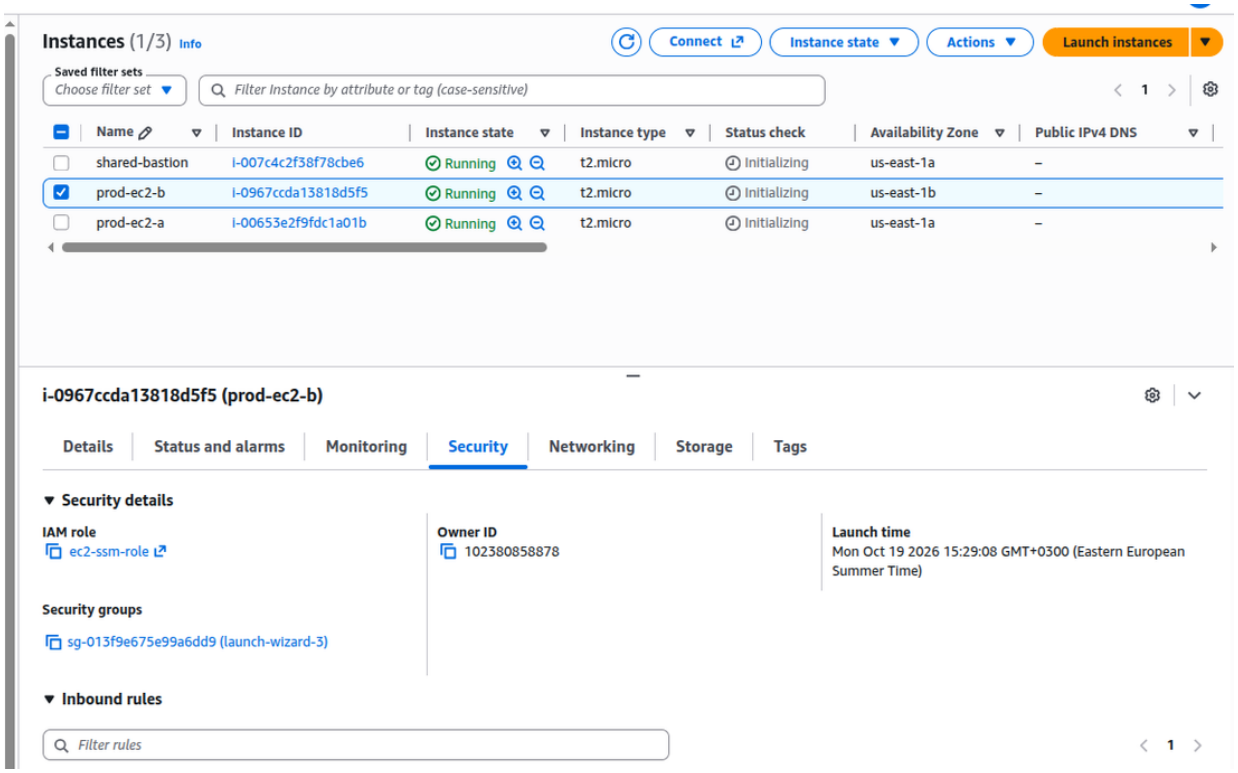


Figure 3. prod-ec2-b, Security tab: IAM role ec2-ssm-role.

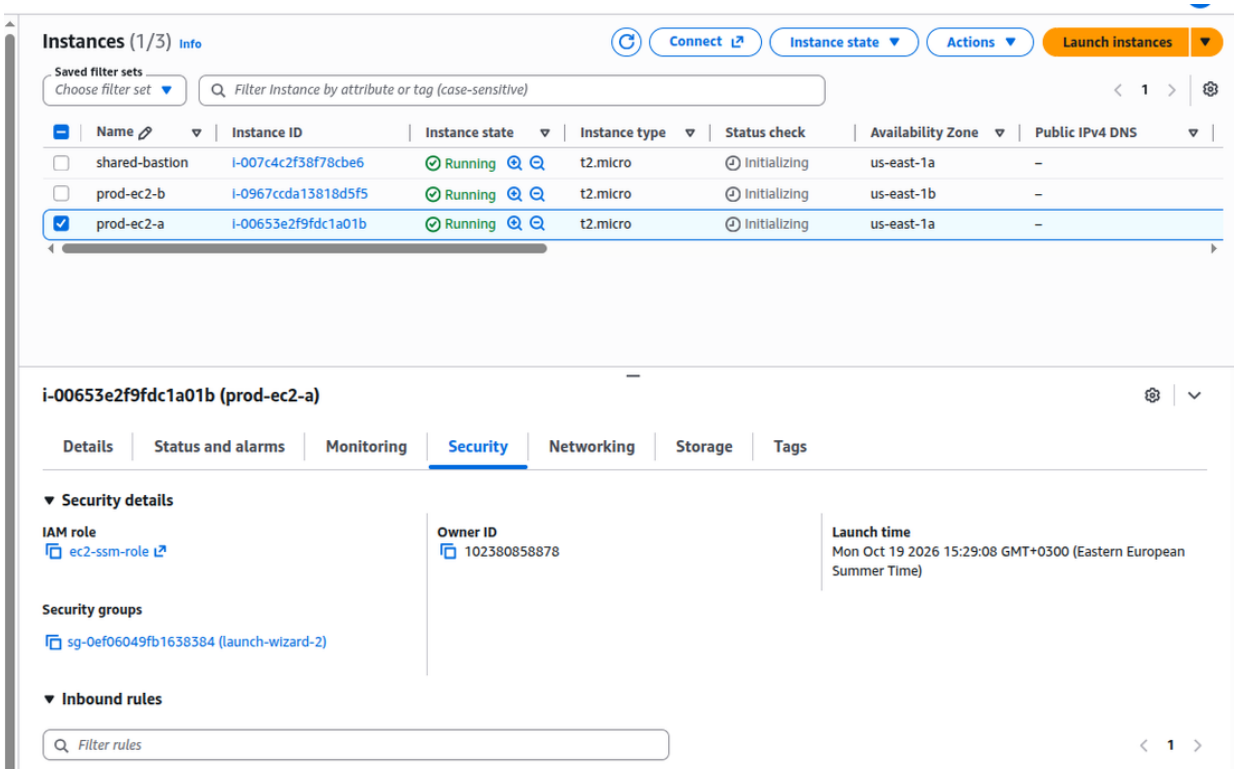


Figure 4. prod-ec2-a, Security tab: IAM role ec2-ssm-role.

Figure 5. Session Manager shell on prod-ec2-a: whoami, hostname and curl localhost:8080.

3. T14: Security groups

Purpose. One security group per tier, so each resource gets only the traffic it needs.

What was built.

- prod-vpc (vpc-0ca04075d741b419c): Sg-alb (ALB: internet-facing web tier), Sg-ec2 (App servers behind ALB), Sg-rds (MySQL database tier), Sg-nat (NAT, documentation only), Sg-bastion (SSM bastion), Sg-endpoints (Interface endpoints). The list also shows three launch-wizard groups and the default group (Figure 6).
- shared-vpc (vpc-003ca8e3a7946afa7): Sg-shared-bastion (SSM bastion in shared-vpc), because a security group cannot be attached across VPCs (Figure 7).

Result. Passed. Note: the console shows the names with a capital S (Sg-...), while the plan writes sg-.... See section 12.

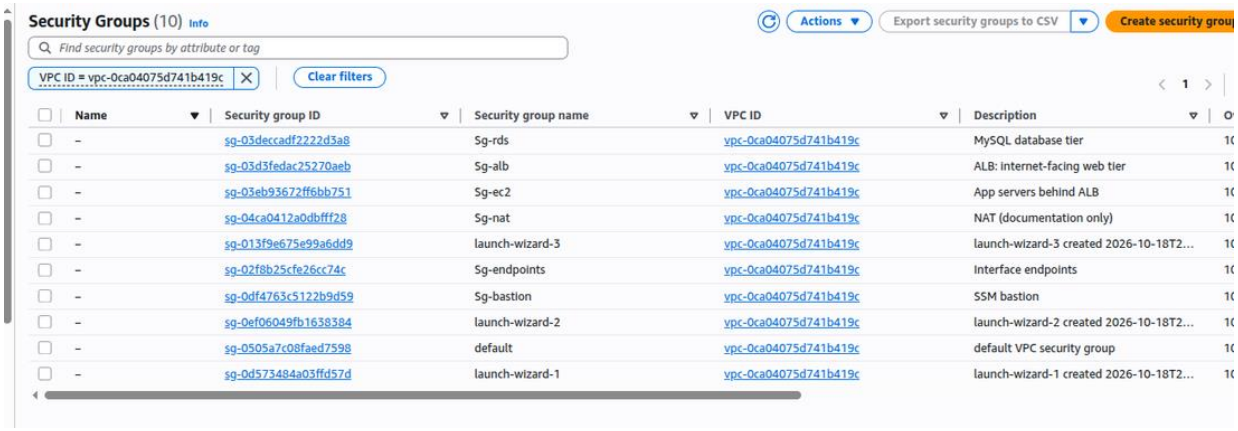


Figure 6. Security groups in prod-vpc: six new groups with descriptions.

Figure 7. Security groups in shared-vpc: Sg-shared-bastion.

4. T15: Security group rules and resource assignment

Purpose. Apply least-privilege rules, then move every resource off its original launch-wizard group.

SG	Inbound (as shown)	Outbound (as shown)	Matches plan
Sg-alb	TCP 80 and 443 from 0.0.0.0/0 (Fig 12)	TCP 8080 to Sg-ec2 (Fig 13)	Yes

Sg-ec2	TCP 8080 from Sg-alb (Fig 8)	TCP 443 to Sg-endpoints, 3306 to Sg-rds, 443 to 0.0.0.0/0 (Fig 9)	Yes
Sg-rds	TCP 3306 from Sg-ec2 (Fig 14)	None (Fig 15)	Yes
Sg-nat	All traffic from 10.0.0.0/16 (Fig 18)	All traffic to 0.0.0.0/0 (Fig 19)	Yes
Sg-bastion	None (Fig 17)	TCP 443 to Sg-endpoints (Fig 16)	Yes
Sg-endpoints	TCP 443 from Sg-bastion and Sg-ec2 (Fig 10)	All traffic to 0.0.0.0/0, default kept (Fig 11)	Yes (roadmap lists no outbound)
Sg-shared-bastion	None (Fig 20)	TCP 443 to 0.0.0.0/0 (Fig 21)	Added for shared-vpc

Resource assignment. prod-alb uses Sg-alb (Figure 22). prod-ec2-a and prod-ec2-b use Sg-ec2 (Figures 23-24). prod-rds shows Sg-rds with its inbound rule sourced from Sg-ec2 (Figure 25). A managed NAT Gateway cannot take a security group, so Sg-nat is documentation only.

Result. Passed. Not evidenced: the move of shared-bastion to Sg-shared-bastion.

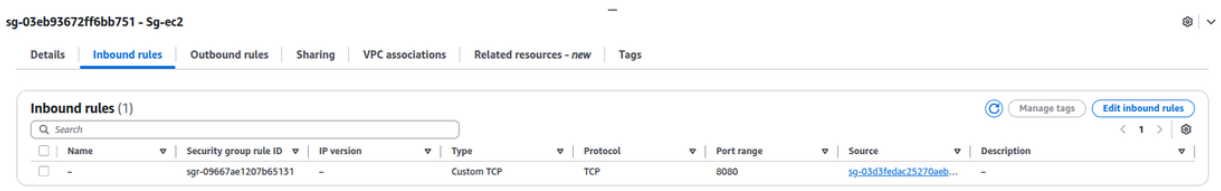


Figure 8. Sg-ec2 inbound: TCP 8080 from Sg-alb.

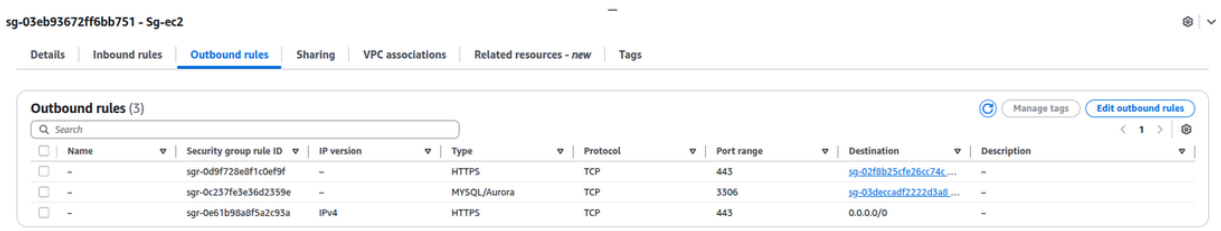


Figure 9. Sg-ec2 outbound: 443 to Sg-endpoints, 3306 to Sg-rds, 443 to 0.0.0.0/0.

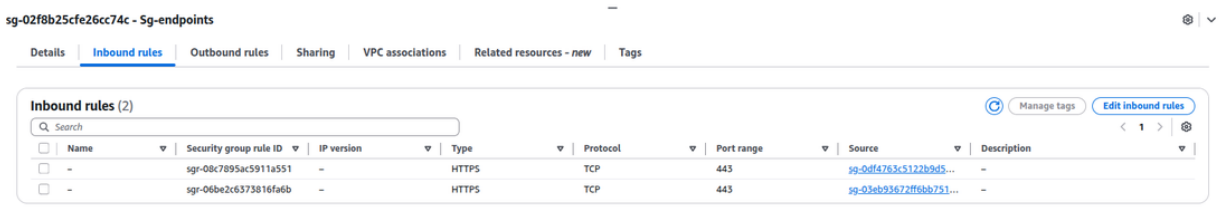


Figure 10. Sg-endpoints inbound: TCP 443 from Sg-bastion and Sg-ec2.

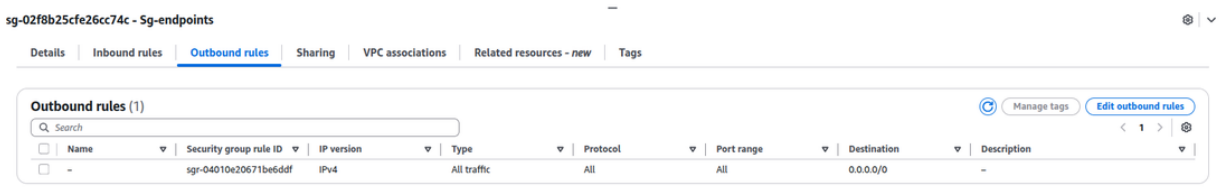


Figure 11. Sg-endpoints outbound: default all traffic kept.

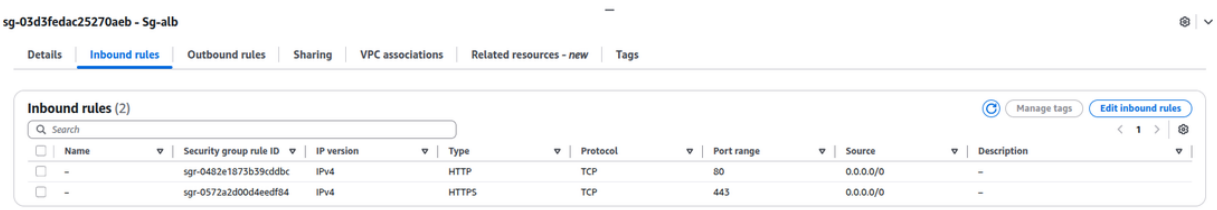


Figure 12. Sg-alb inbound: TCP 80 and 443 from 0.0.0.0/0.

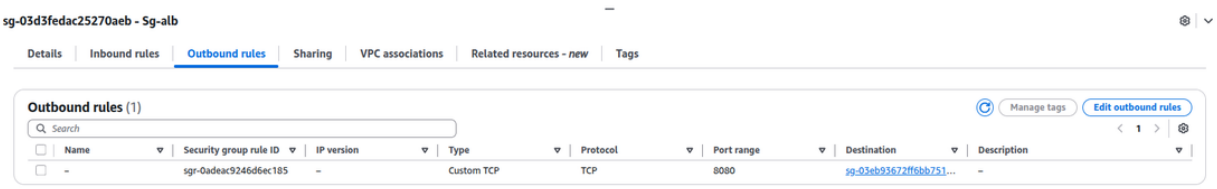


Figure 13. Sg-alb outbound: TCP 8080 to Sg-ec2.

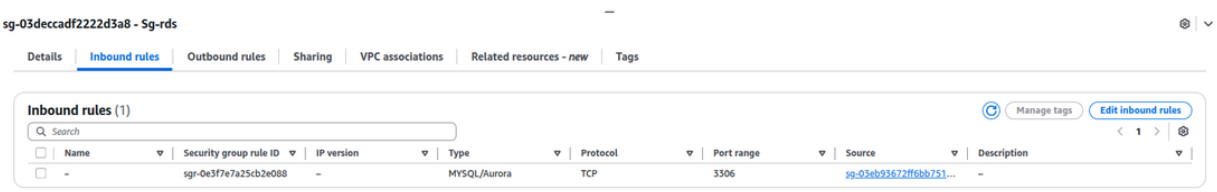


Figure 14. Sg-rds inbound: TCP 3306 from Sg-ec2.

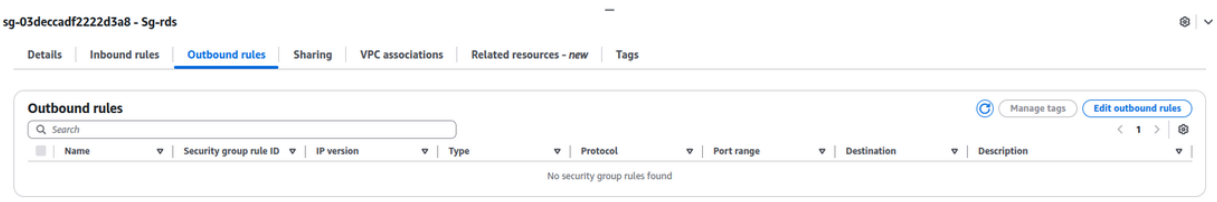


Figure 15. Sg-rds outbound: no rules.

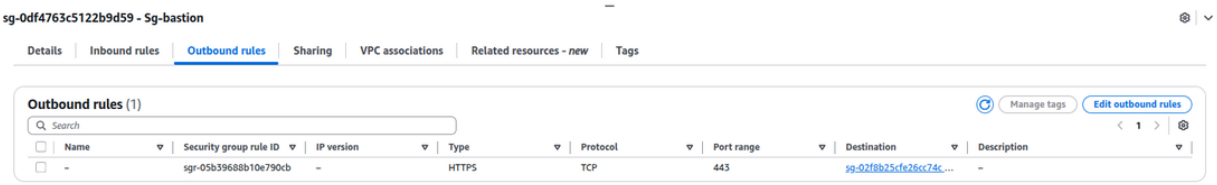


Figure 16. Sg-bastion outbound: TCP 443 to Sg-endpoints.

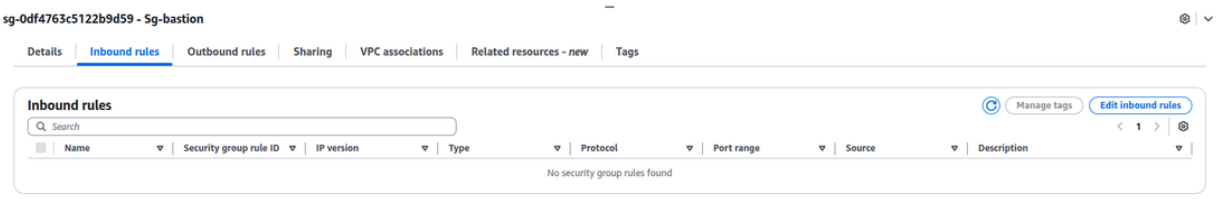


Figure 17. Sg-bastion inbound: no rules.

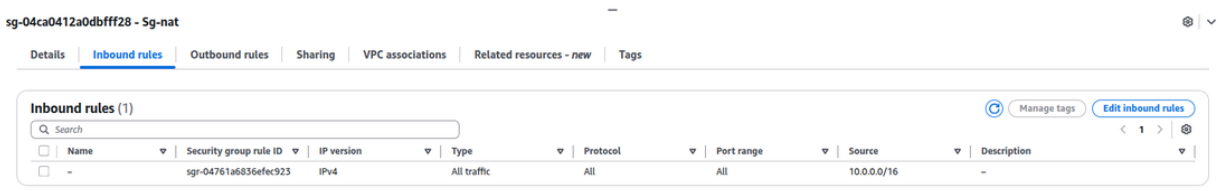


Figure 18. Sg-nat inbound: all traffic from 10.0.0.0/16.

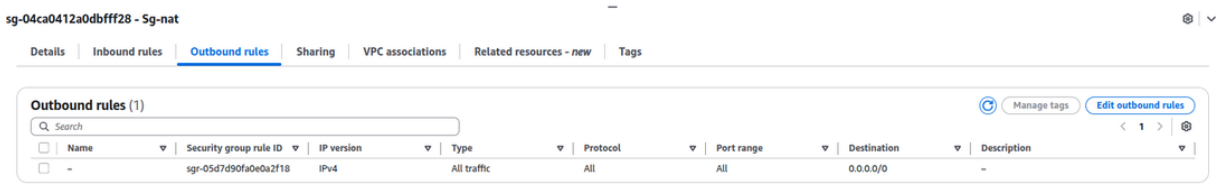


Figure 19. Sg-nat outbound: all traffic to 0.0.0.0/0.

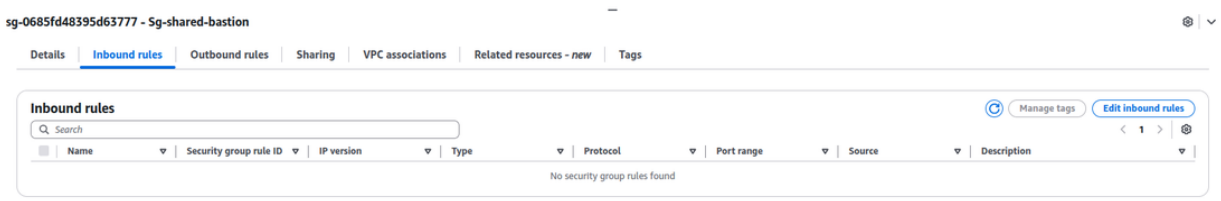


Figure 20. Sg-shared-bastion inbound: no rules.

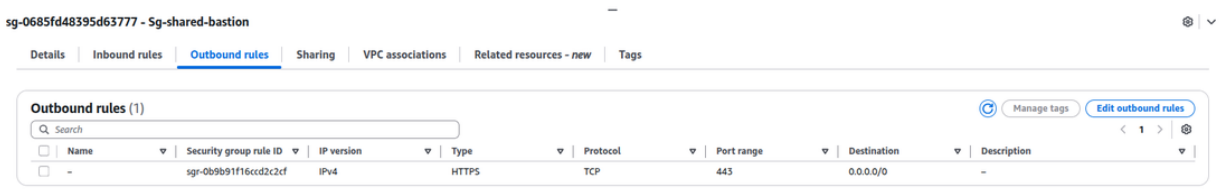


Figure 21. Sg-shared-bastion outbound: TCP 443 to 0.0.0.0/0.

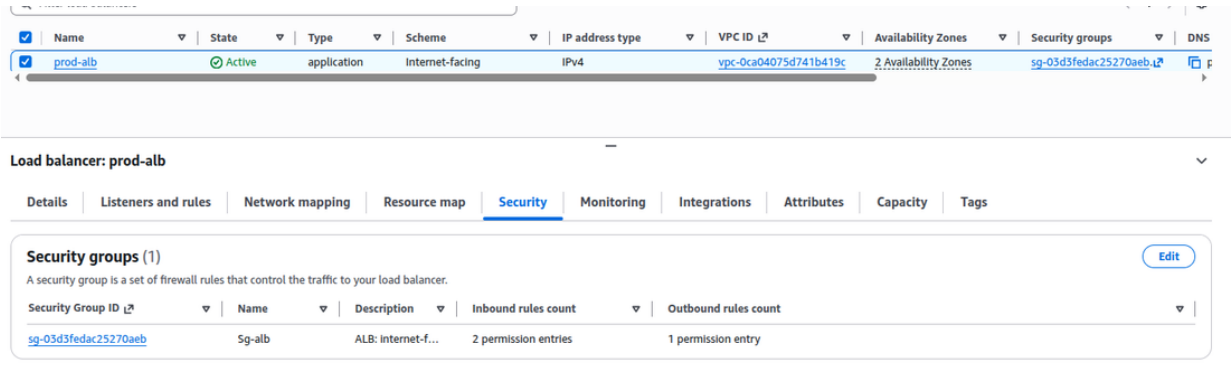


Figure 22. prod-alb (Active, internet-facing, 2 AZs): Security tab shows Sg-alb.

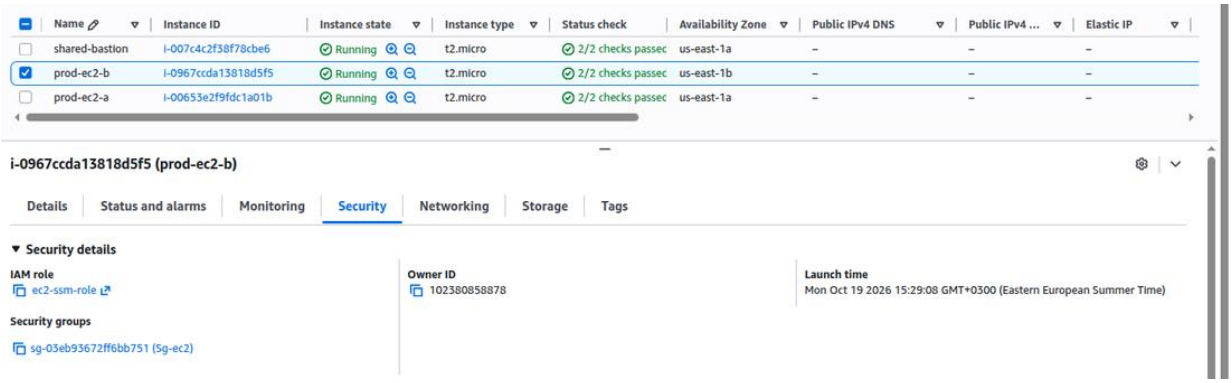


Figure 23. prod-ec2-b, Security tab: Sg-ec2, status checks 2/2.

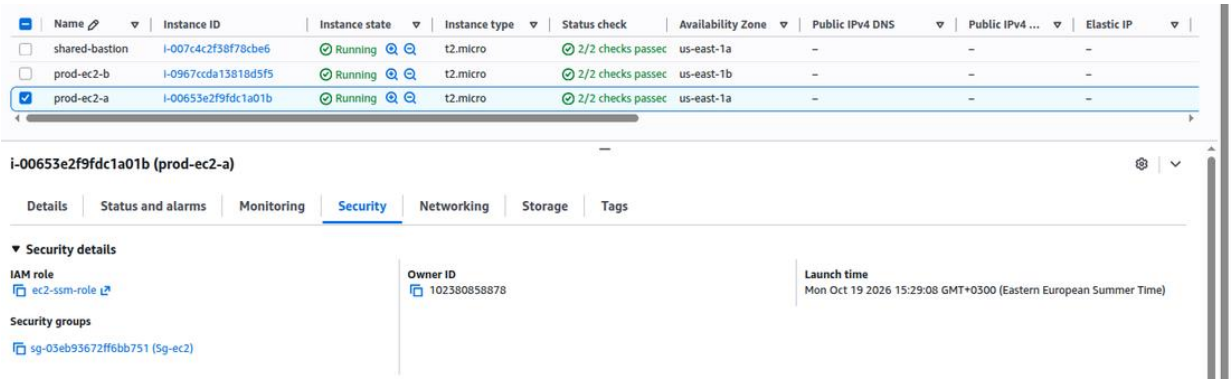


Figure 24. prod-ec2-a, Security tab: Sg-ec2, status checks 2/2.

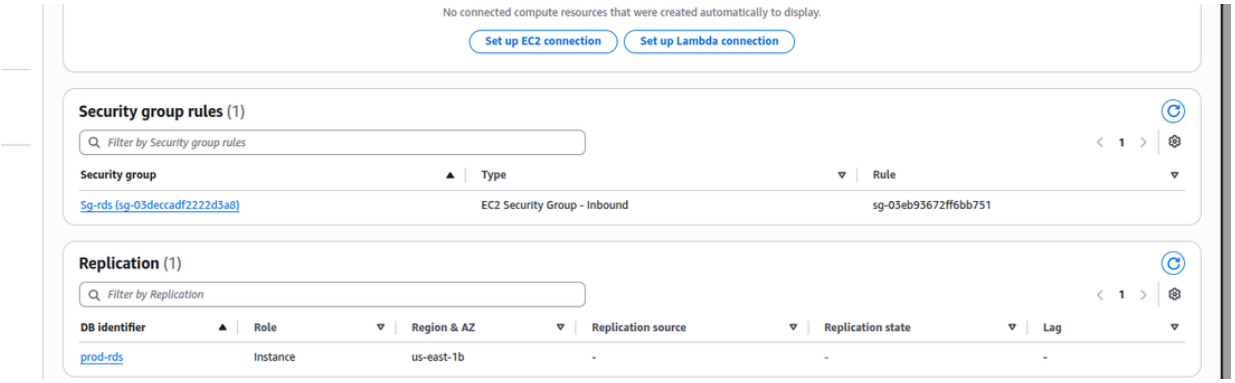


Figure 25. prod-rds: Sg-rds referenced by an EC2 security group inbound rule; instance in us-east-1b.

5. T16: Network ACLs

Purpose. Subnet-level filtering that backs up the security groups. NACLs are stateless, so each one has explicit return-traffic rules. Every rule number is 100 or higher, which keeps rule #50 free for the auto-quarantine deny used later in the IR pipeline.

NACL	Inbound	Outbound
nacl-public	100 TCP 80, 110 TCP 443, 120 TCP 1024-65535 from 0.0.0.0/0	100 TCP 8080 to 10.0.10.0/23, 110 TCP 80, 120 TCP 443, 130 TCP 1024-65535 to 0.0.0.0/0
nacl-private	100 TCP 8080 and 110 TCP 443 from 10.0.0.0/16, 120 TCP 1024-65535 from 0.0.0.0/0	100 TCP 3306 to 10.0.20.0/23, 110 TCP 443 to 0.0.0.0/0, 120 TCP 1024-65535 to 10.0.0.0/16
nacl-db	100 TCP 3306 from 10.0.10.0/23	100 TCP 1024-65535 to 10.0.10.0/23

Associations. nacl-public: prod-public-a (10.0.1.0/24) and prod-public-b (10.0.2.0/24). nacl-private: prod-private-a (10.0.10.0/24) and prod-private-b (10.0.11.0/24). nacl-db: prod-db-a (10.0.20.0/24) and prod-db-b (10.0.21.0/24). Each NACL ends with the default deny-all rule.

Result. Passed. The ALB targets were healthy afterwards (Figure 26).

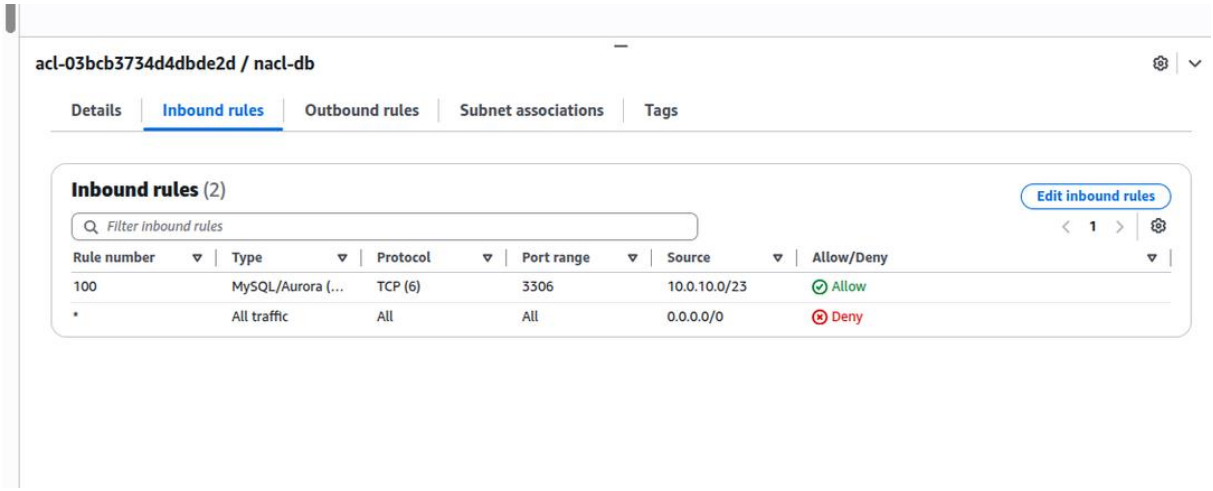


Figure 27. nacl-db inbound rules.

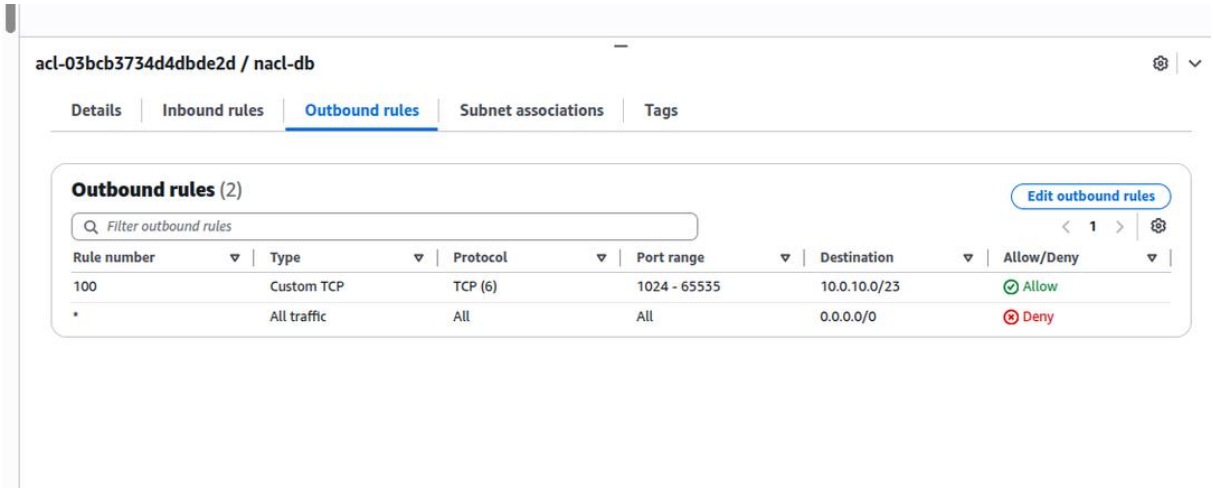


Figure 28. nacl-db outbound rules.

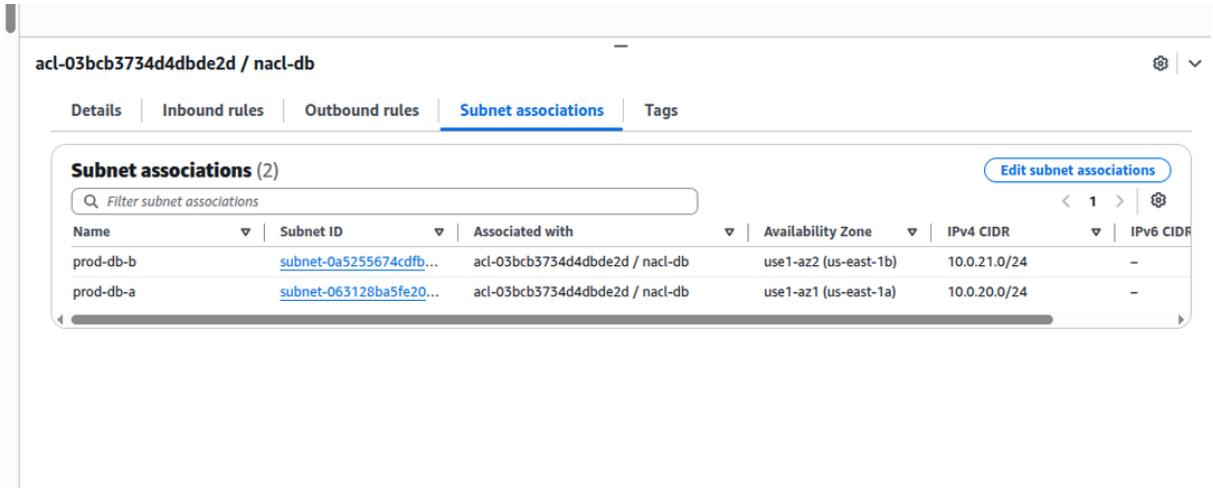


Figure 29. nacl-db subnet associations: prod-db-a and prod-db-b.

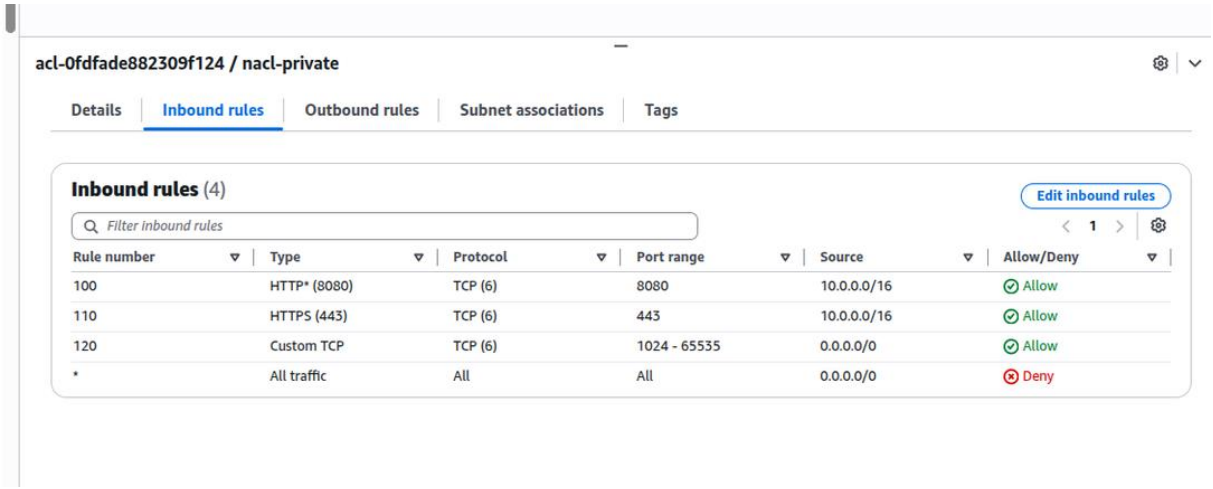


Figure 30. nacl-private inbound rules.

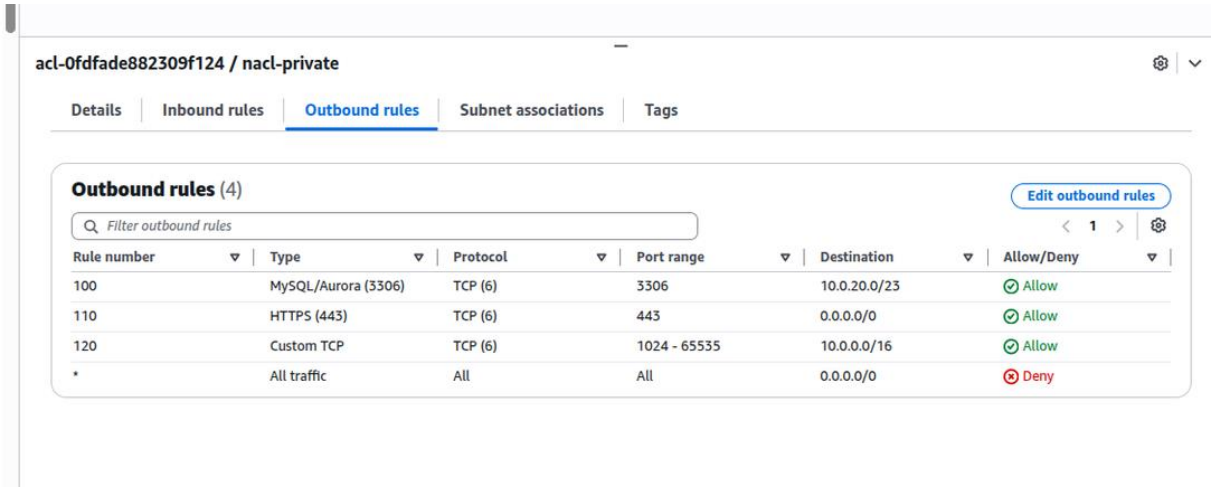


Figure 31. nacl-private outbound rules.

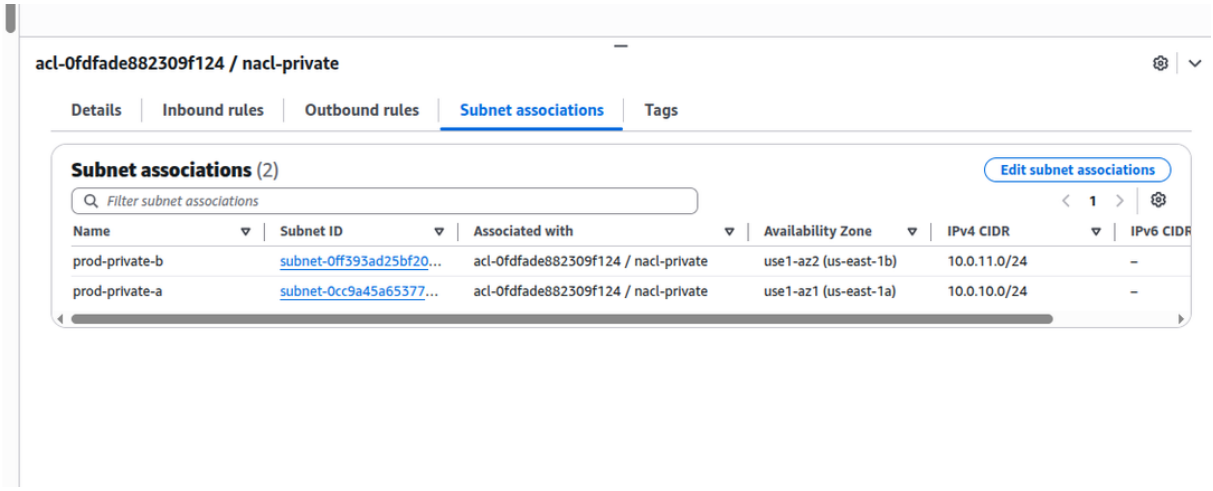


Figure 32. nacl-private subnet associations: prod-private-a and prod-private-b.

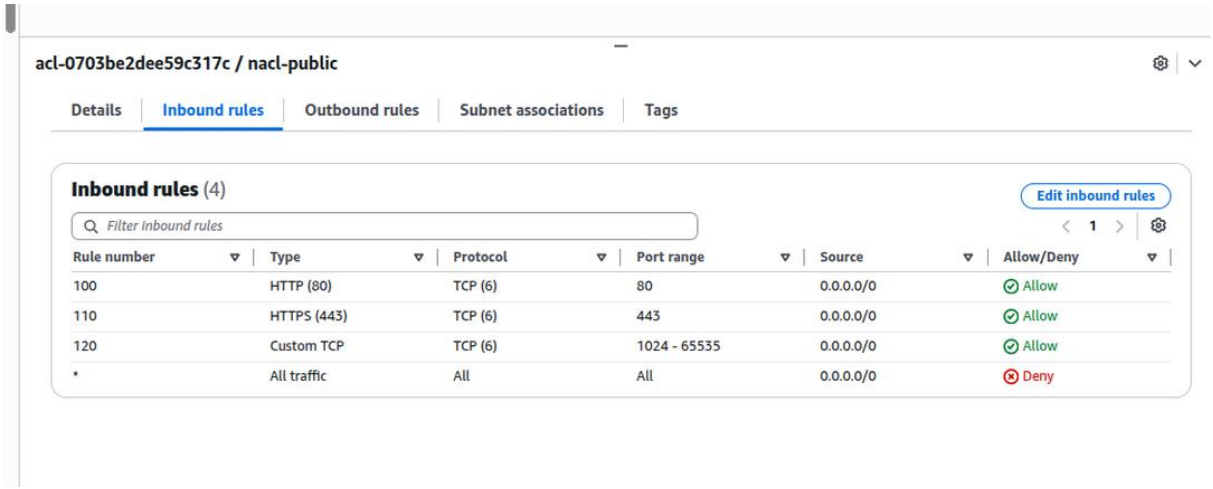


Figure 33. nacl-public inbound rules.

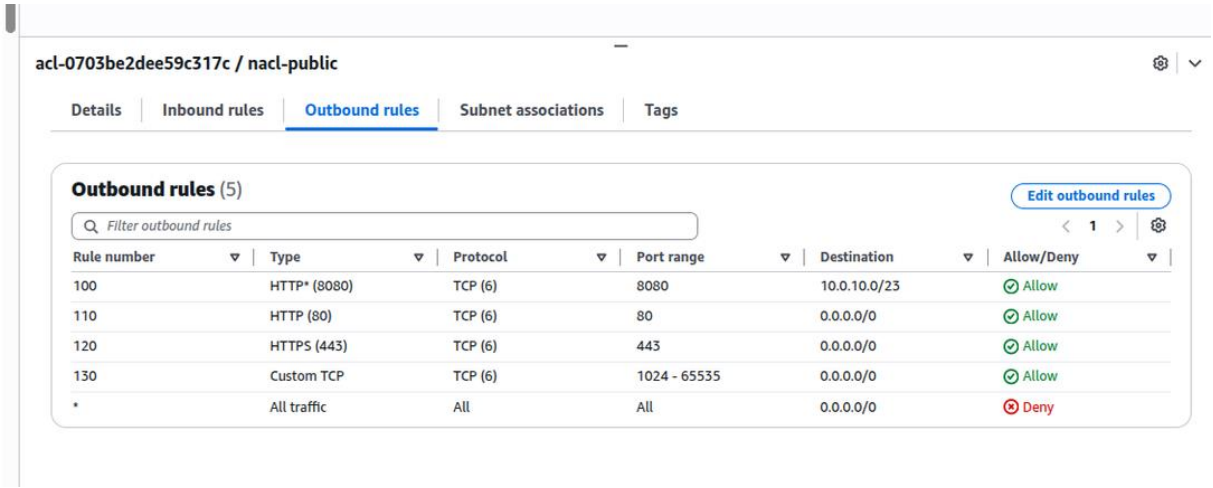


Figure 34. nacl-public outbound rules.

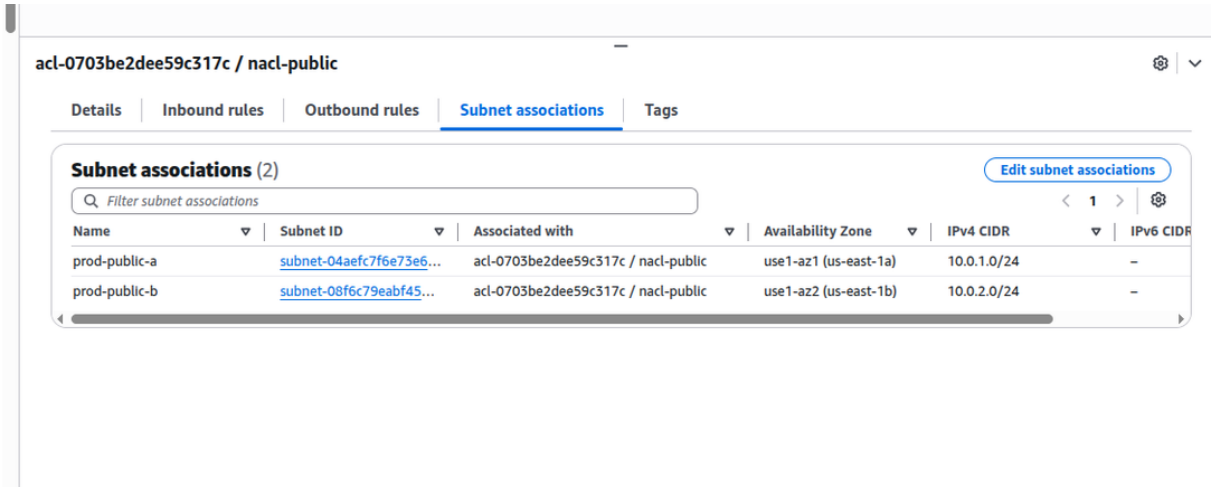


Figure 35. nacl-public subnet associations: prod-public-a and prod-public-b.

6. T17: Segmentation tests

Tests used VPC Reachability Analyzer (five saved paths, Figure 49) and live commands over Session Manager. All eight returned the expected result.

#	Test	Expected	Actual	Figure	Notes

1	Internet to EC2 private IP	Blocked	Not reachable	36, 37	IGW cannot reach an address that is not an ENI public IP. prod-ec2-a has no public IPv4 (private 10.0.10.214). Port field is blank.
2	ALB to EC2 :8080	Allowed	Reachable	39, 26	ALB network interface to prod-ec2-a. Port field is blank. Target group shows 2 healthy, 0 unhealthy on 8080.
3	ALB to EC2 :22	Blocked	Not reachable	40	Explanation: no egress rule of the ALB security group applies. The port is not shown in the summary.
4	EC2 to RDS :3306	Allowed	OPEN	41	/dev/tcp check from prod-ec2-a to the RDS endpoint returned OPEN.
5	Bastion to RDS :3306	Blocked	Not reachable	42, 14	Different VPCs with no connecting resource. Sg-rds only allows Sg-ec2.
6	EC2 to Internet via NAT :443	Allowed	HTTP/2 200	43	curl to https://aws.amazon.com from prod-ec2-a.
7	RDS to Internet	Blocked	Not reachable	46, 15, 44	No egress rule of Sg-rds applies. rt-db has only the local route.
8	SSM console to EC2	Allowed	Session opened	47, 48, 5	Session Manager shell on prod-ec2-a.

Result. Passed (8 of 8). Evidence gaps are listed in section 12.

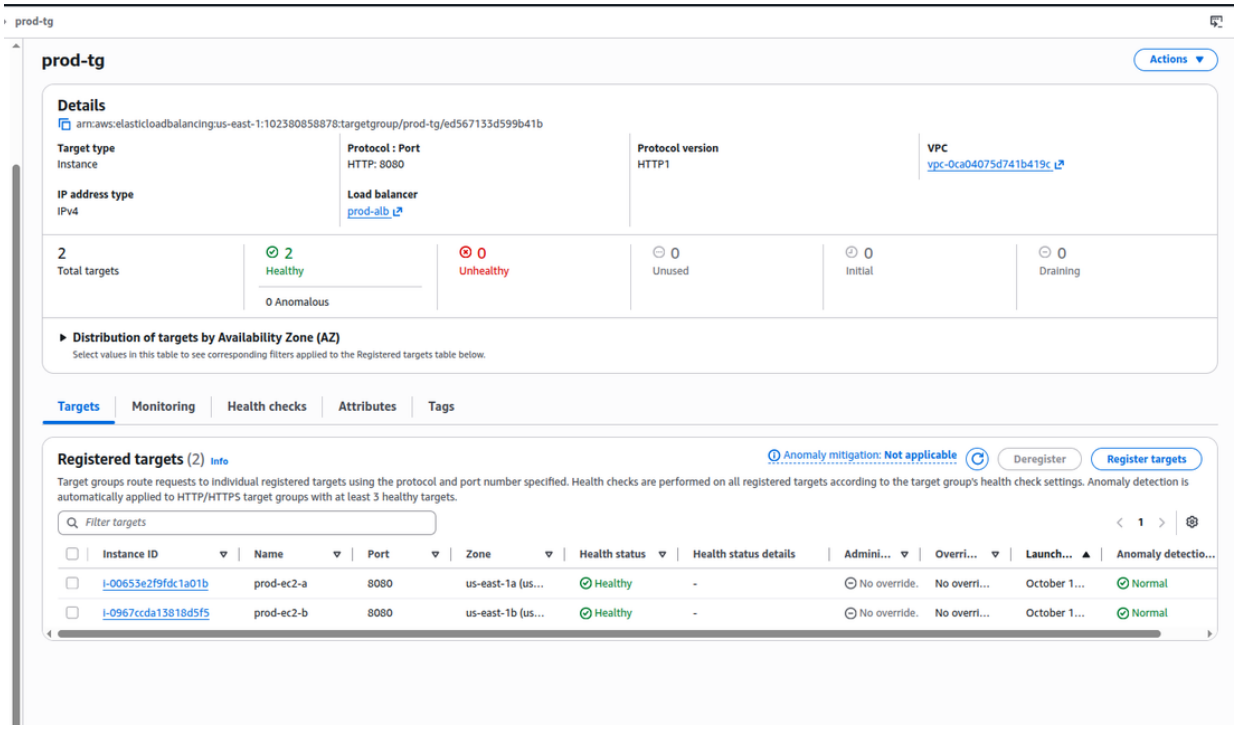


Figure 26. Target group prod-tg (HTTP 8080): 2 targets, 2 healthy, 0 unhealthy.

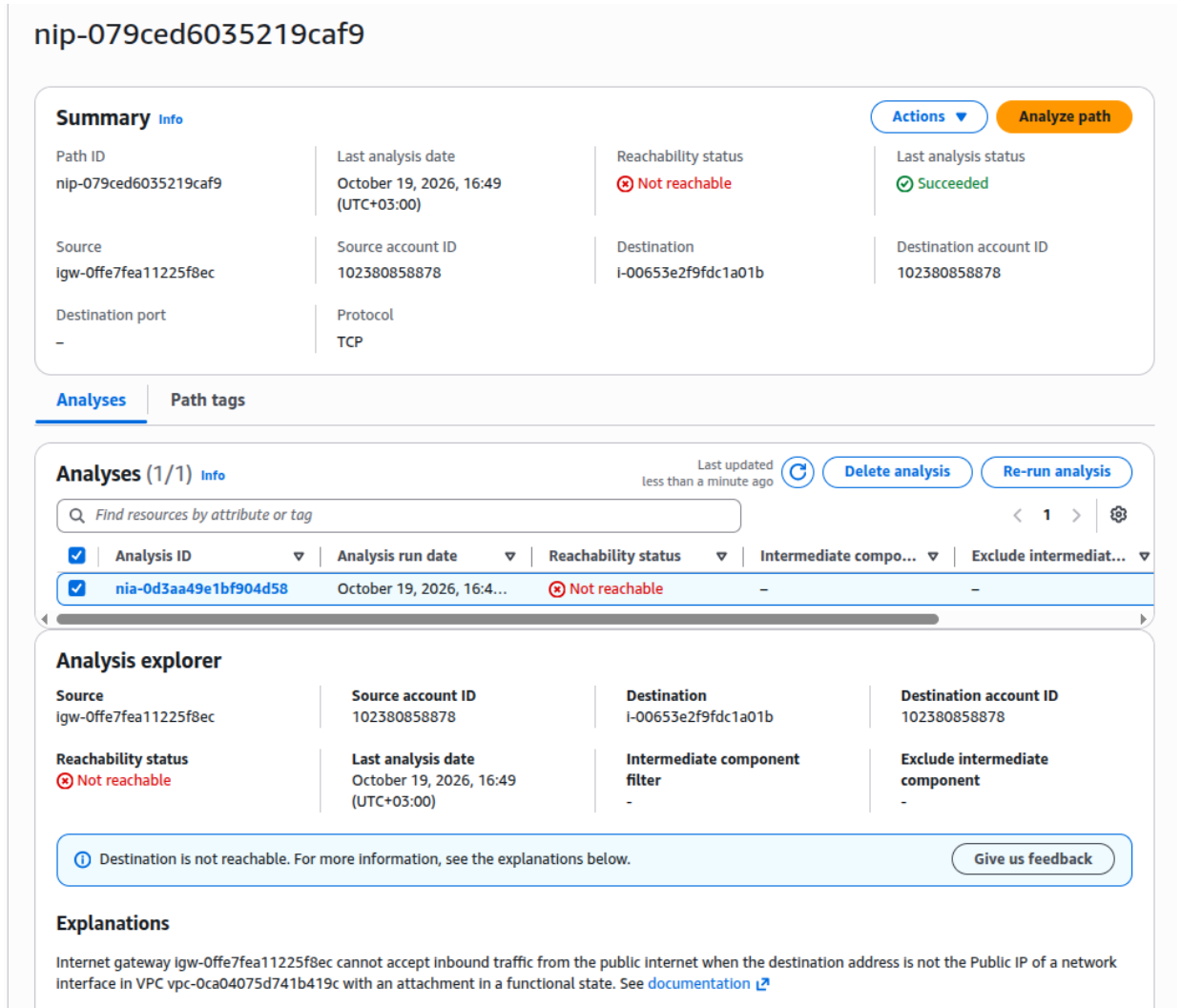


Figure 36. Test 1: Reachability Analyzer, Internet gateway to prod-ec2-a: Not reachable.

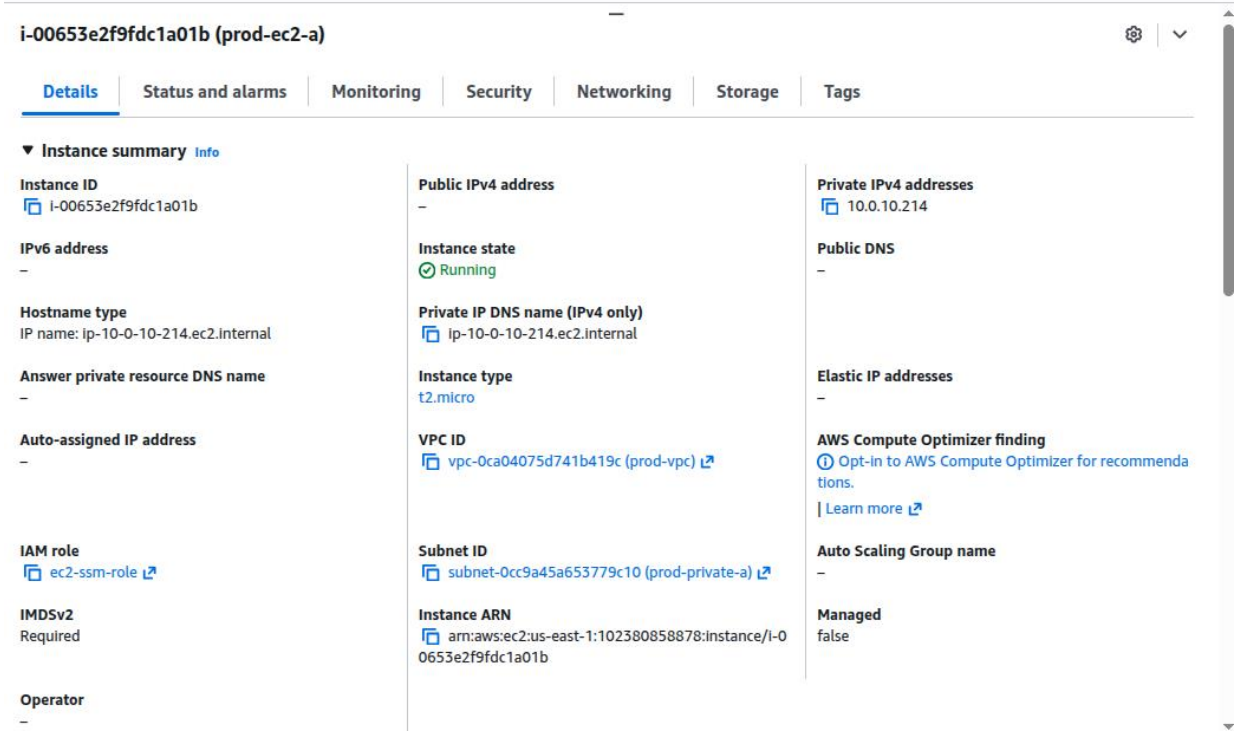


Figure 37. prod-ec2-a details: no public IPv4 address, private IP 10.0.10.214, subnet prod-private-a.

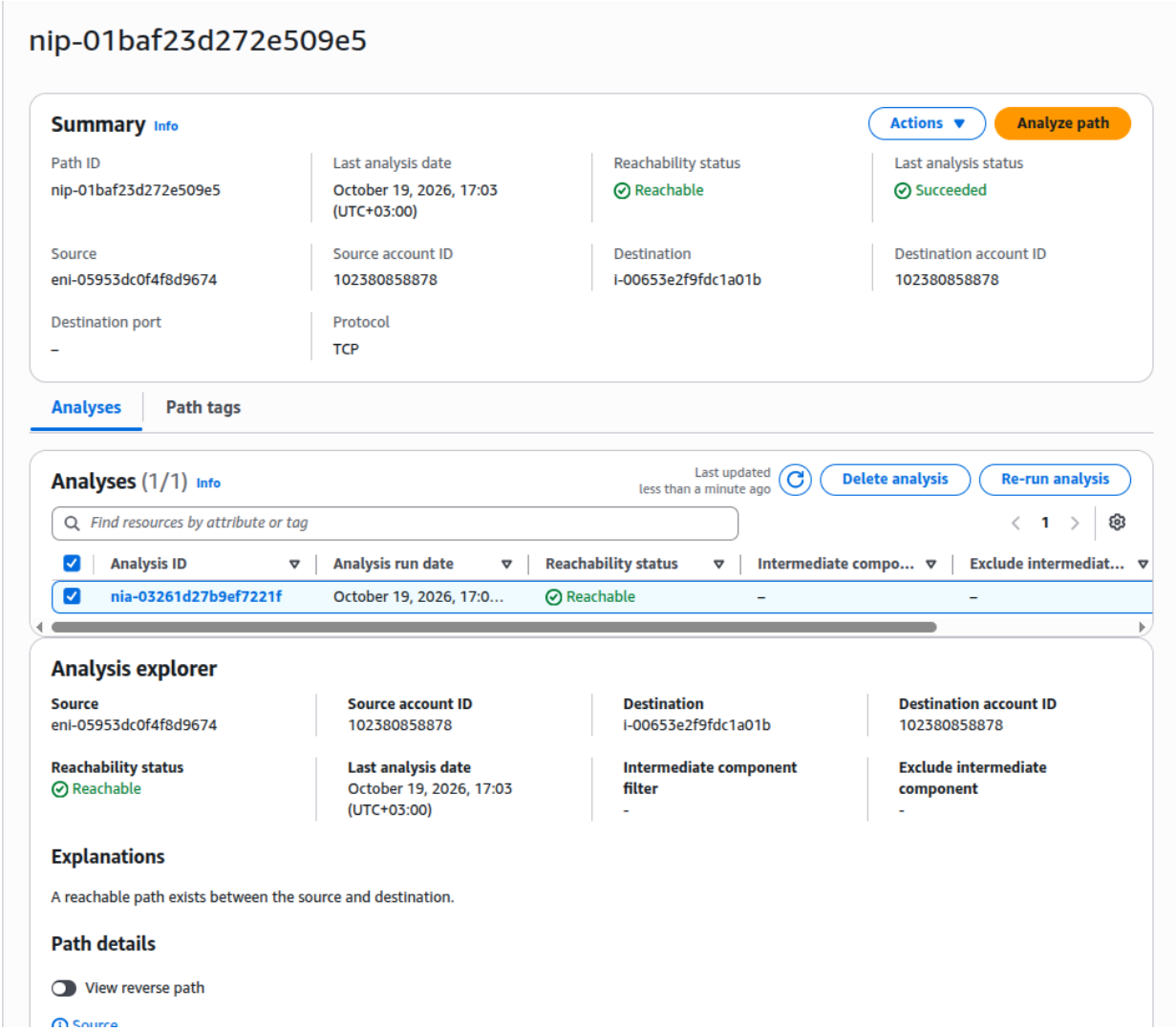


Figure 39. Test 2: ALB network interface to prod-ec2-a: Reachable.

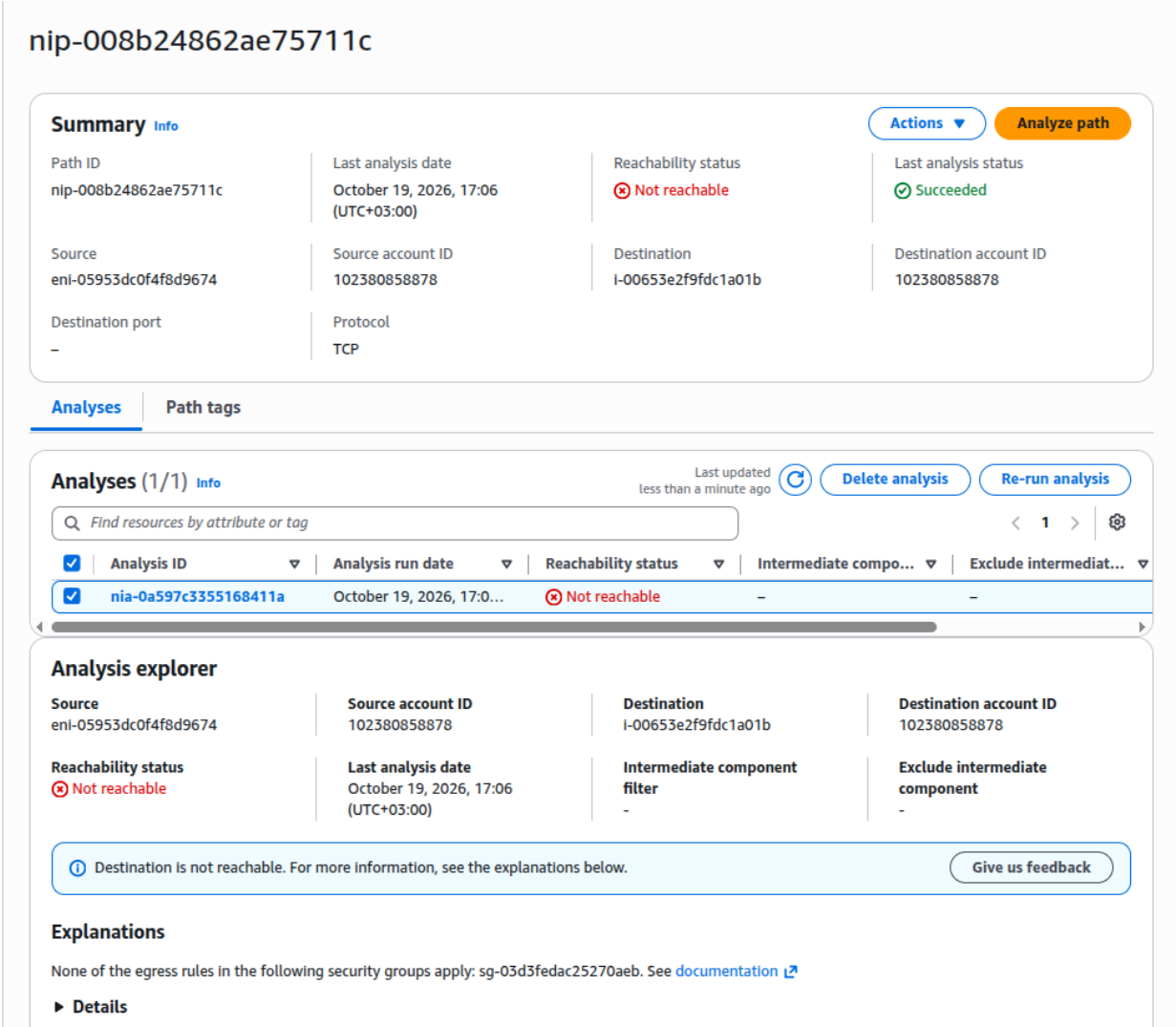


Figure 40. Test 3: ALB network interface to prod-ec2-a: Not reachable (no ALB egress rule applies).

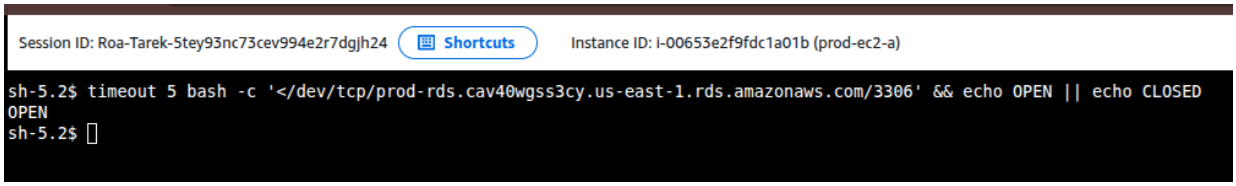


Figure 41. Test 4: prod-ec2-a to the RDS endpoint on 3306: OPEN.

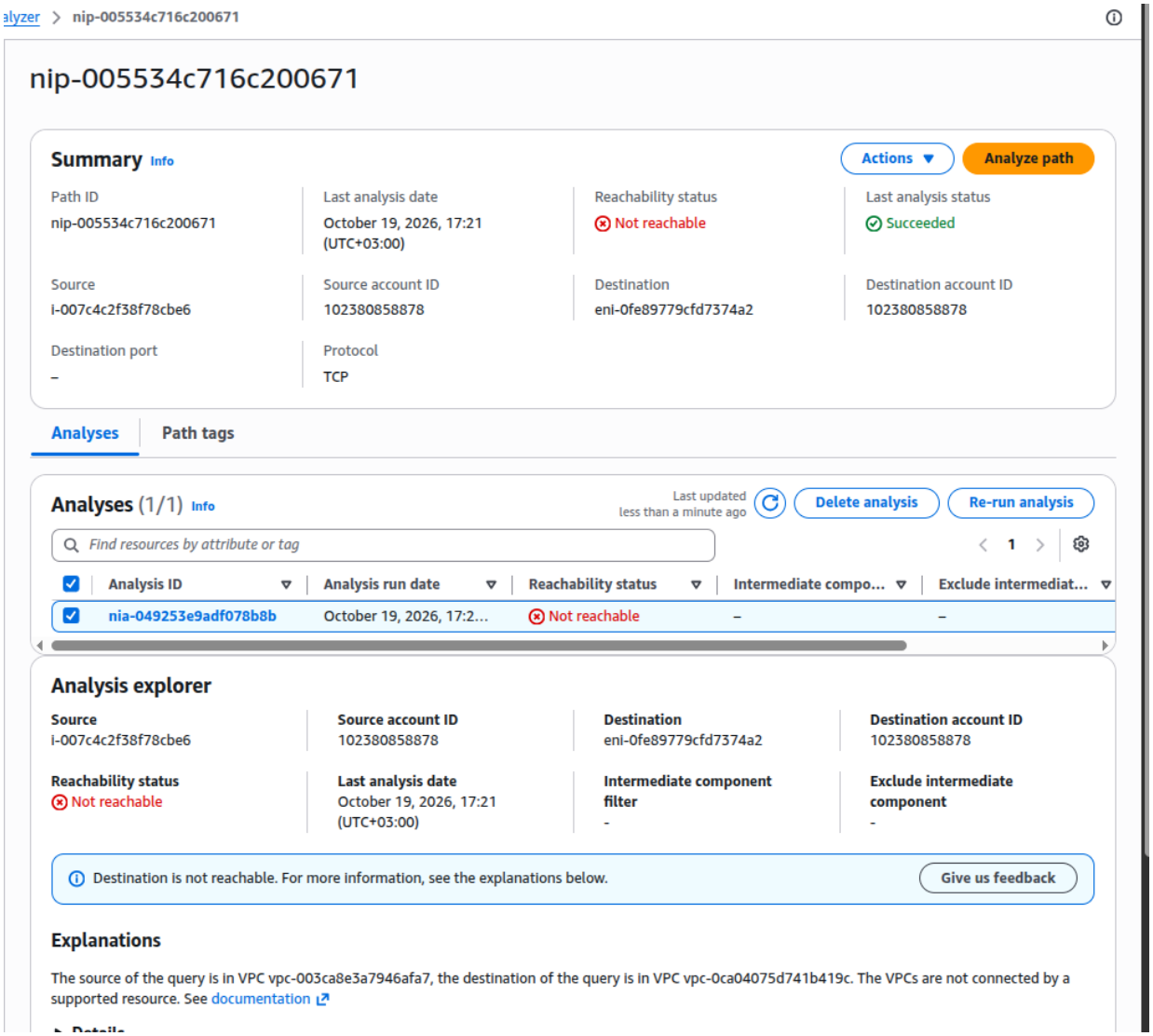


Figure 42. Test 5: shared-bastion to the RDS network interface: Not reachable (VPCs not connected).

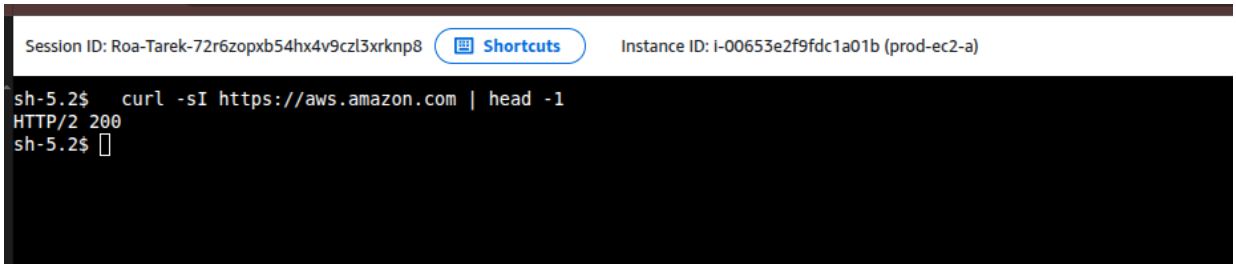


Figure 43. Test 6: curl https://aws.amazon.com from prod-ec2-a returns HTTP/2 200.

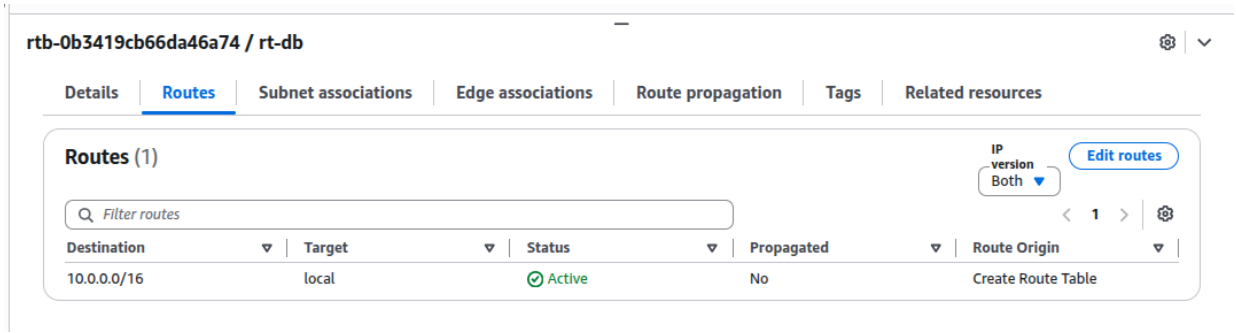


Figure 44. rt-db route table: only 10.0.0.0/16 local.

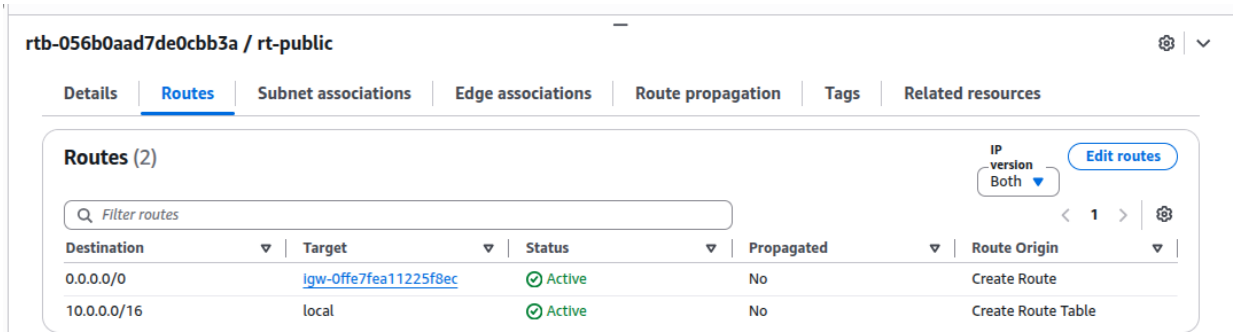


Figure 45. rt-public route table: 0.0.0.0/0 to the internet gateway and the local route.

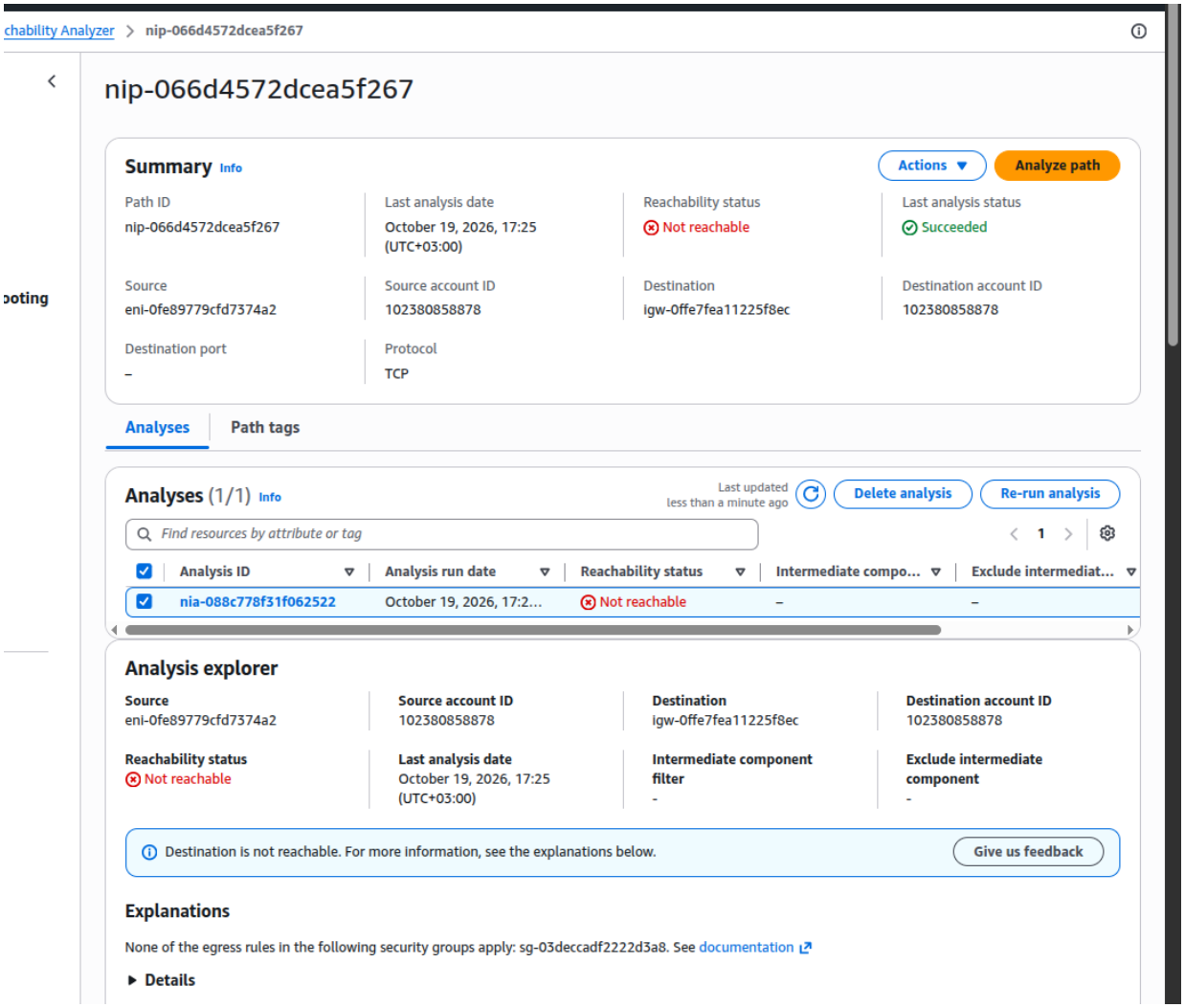


Figure 46. Test 7: RDS network interface to the internet gateway: Not reachable.

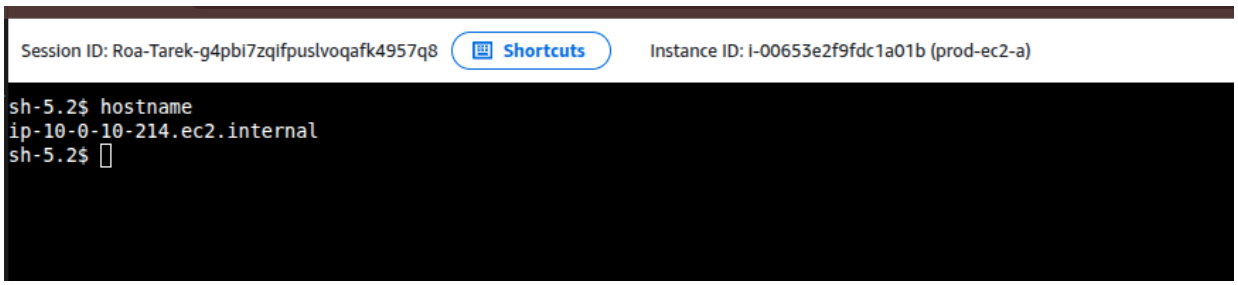


Figure 47. Test 8: Session Manager shell on prod-ec2-a (hostname).

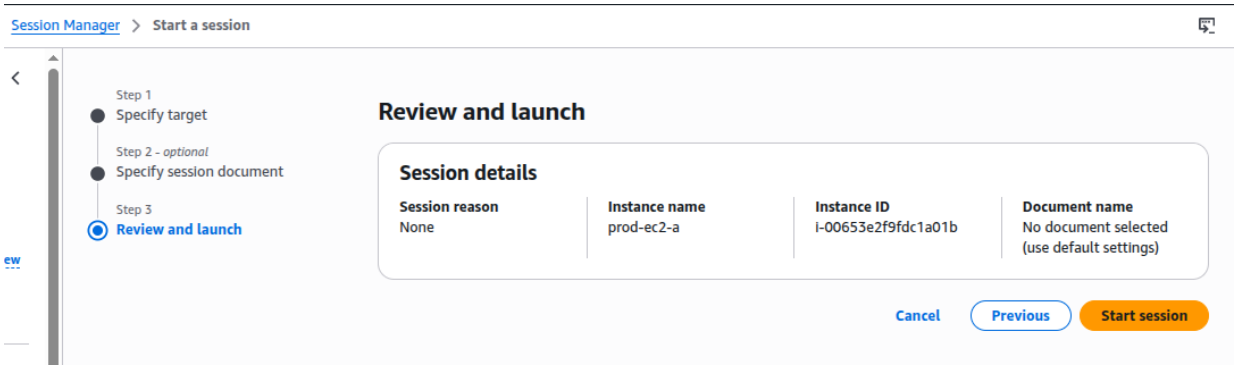


Figure 48. Session Manager launch page for prod-ec2-a.

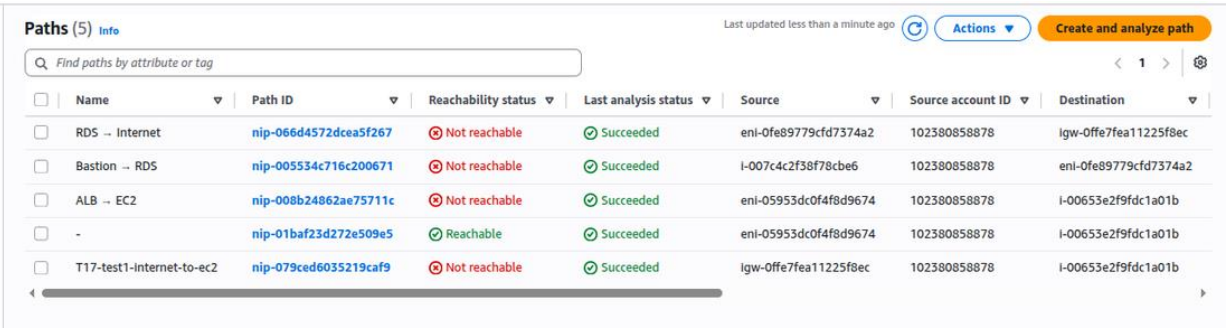


Figure 49. Reachability Analyzer paths list (5 paths).

7. T18: Endpoint security groups and peering rules

Purpose. The roadmap does not describe T18. It is interpreted from the dependency map (after T29, before T30) and the architecture diagram as: confirm the interface endpoints use the endpoint security group, and open the paths Shrouk's peering test (T30) needs. Confirm this scope with the team.

What was checked and changed.

- Seven endpoints exist in prod-vpc and all are Available: prod-s3-gateway (Gateway) and six interface endpoints for ssm, ssmmessages, ec2messages, logs, kms and secretsmanager (Figures 59-64).
- Each interface endpoint's Security Groups tab shows Sg-endpoints (sg-02f8b25cfe26cc74c) in prod-vpc (Figures 59-64).
- Sg-endpoints allows TCP 443 inbound from Sg-bastion and Sg-ec2 (Figure 65).
- Sg-ec2 inbound now has 3 rules: the original TCP 8080 rule from Sg-alb, a second TCP 8080 IPv4 rule and an All ICMP IPv4 rule. The Source column is cut off in Figure 66, so the CIDRs are not visible (the plan is 10.1.0.0/16).
- nacl-private gained rule 130 inbound, TCP 8080 from 10.1.0.0/16 (Figure 67), and rule 130 outbound, TCP 1024-65535 to 10.1.0.0/16 (Figure 68).
- A Session Manager shell on prod-ec2-a still opens after the changes (Figure 69).

Result. Passed with notes. The secretsmanager endpoint uses the service name com.amazonaws.us-east-1.secretsmanager-fips (the FIPS variant), not plain secretsmanager as in the plan. Sg-ec2 now allows ICMP, but nacl-private has no ICMP rule, so a ping across the peering will be blocked while curl on 8080 works.

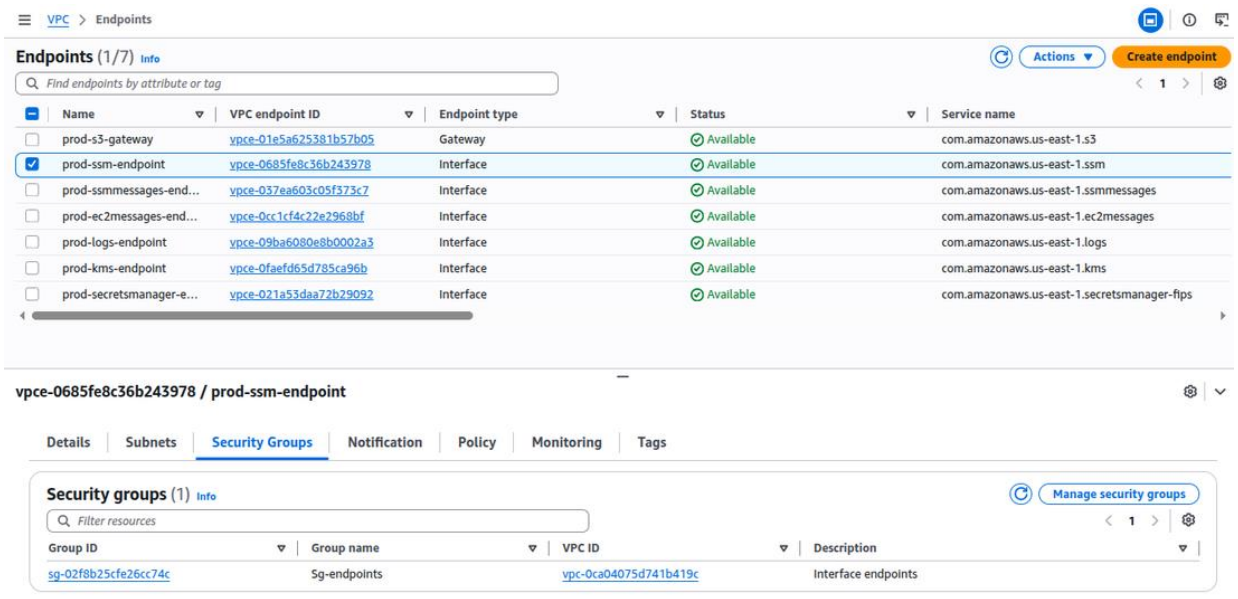


Figure 59. prod-ssm-endpoint (7 endpoints, all Available): Security Groups tab shows Sg-endpoints.

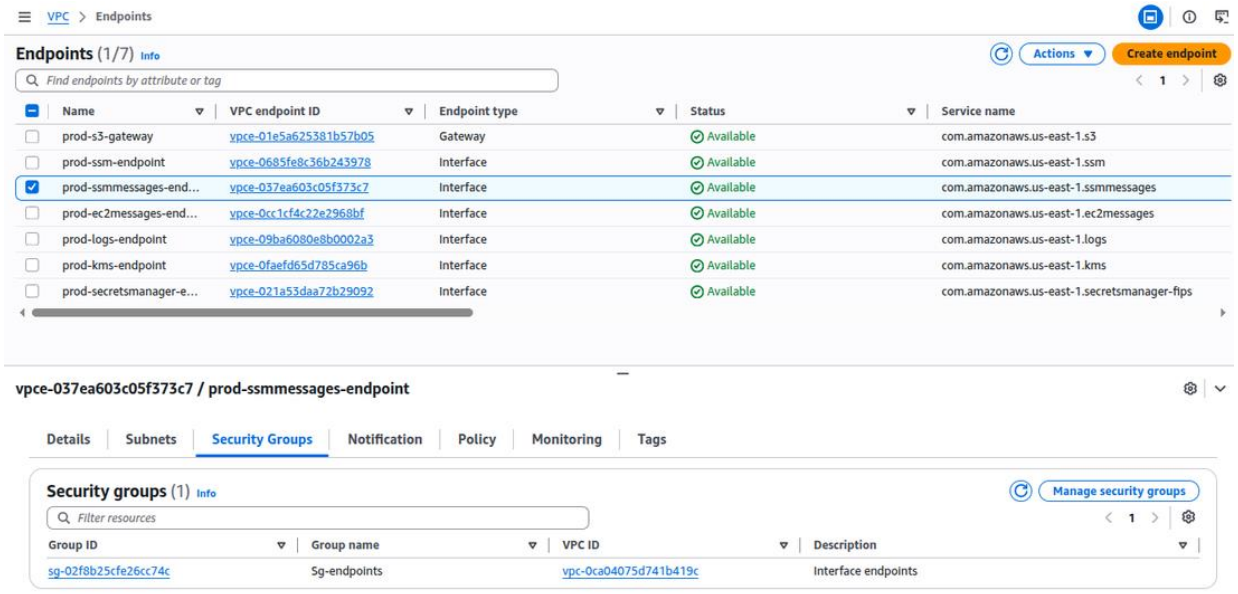


Figure 60. prod-ssmmessages-endpoint: Security Groups tab shows Sg-endpoints.

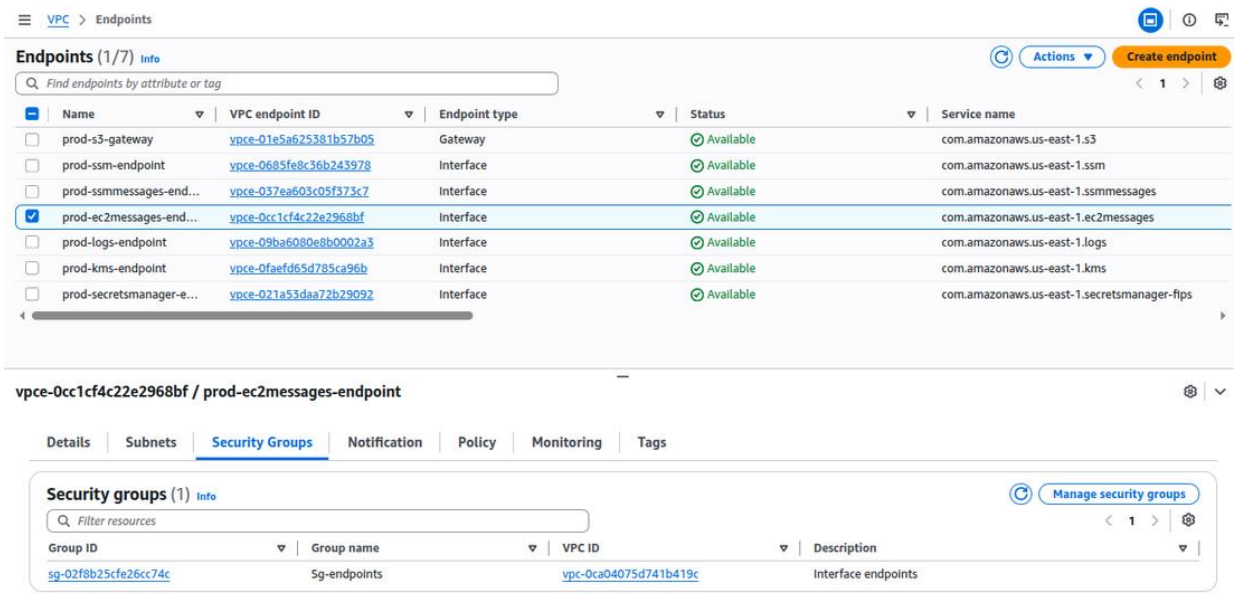


Figure 61. prod-ec2messages-endpoint: Security Groups tab shows Sg-endpoints.

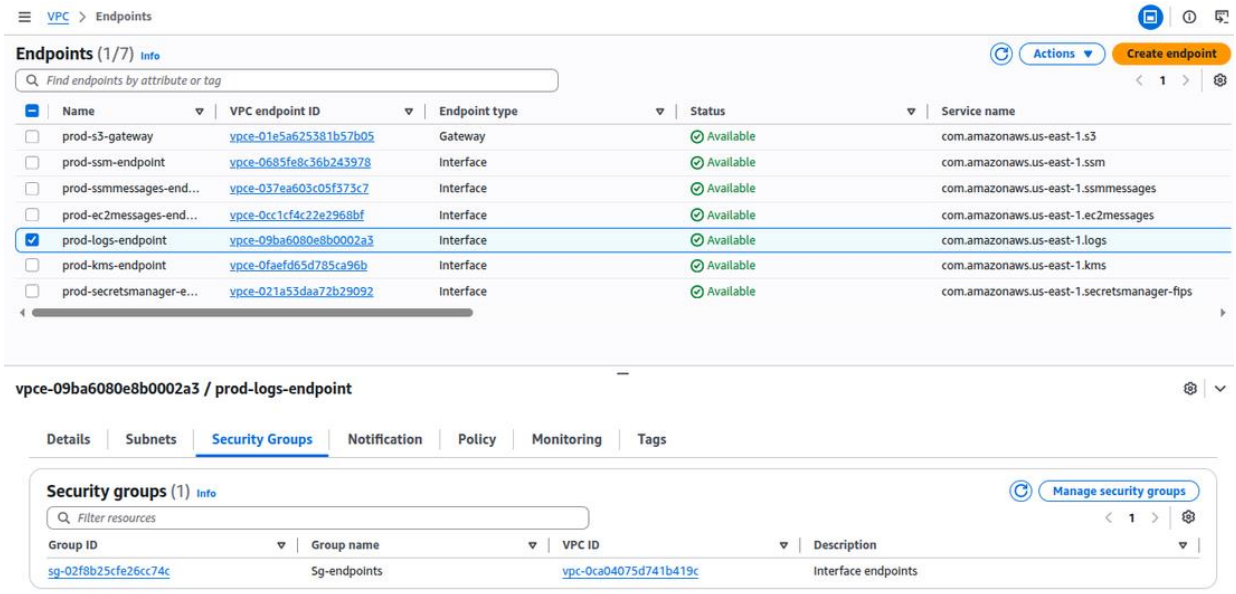


Figure 62. prod-logs-endpoint: Security Groups tab shows Sg-endpoints.

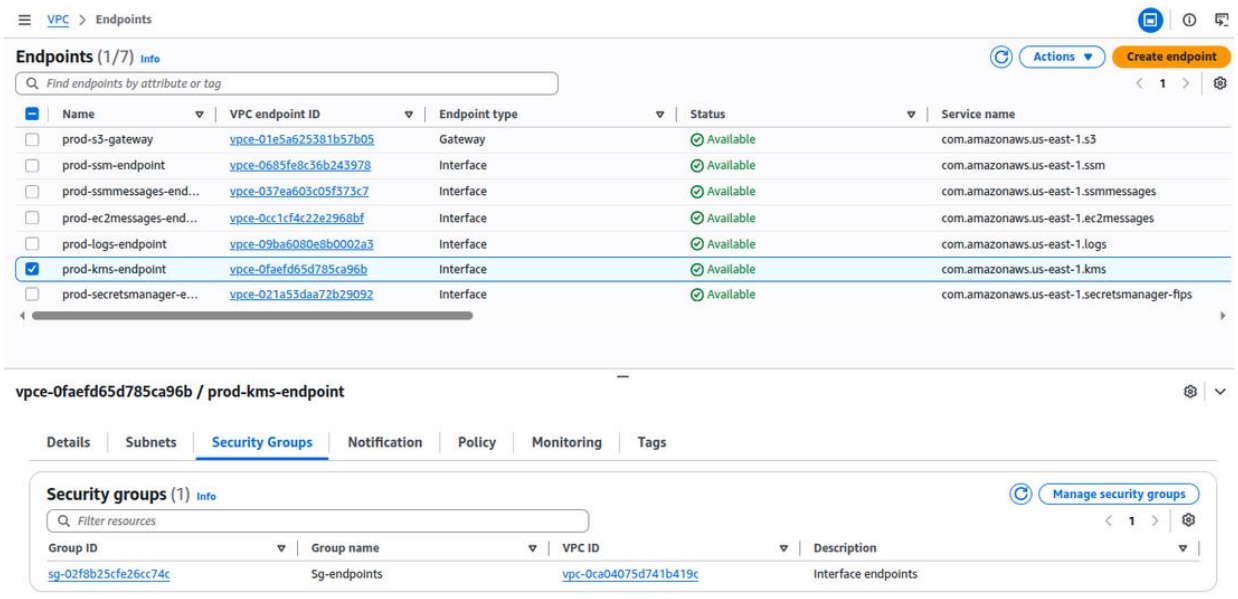


Figure 63. prod-kms-endpoint: Security Groups tab shows Sg-endpoints.

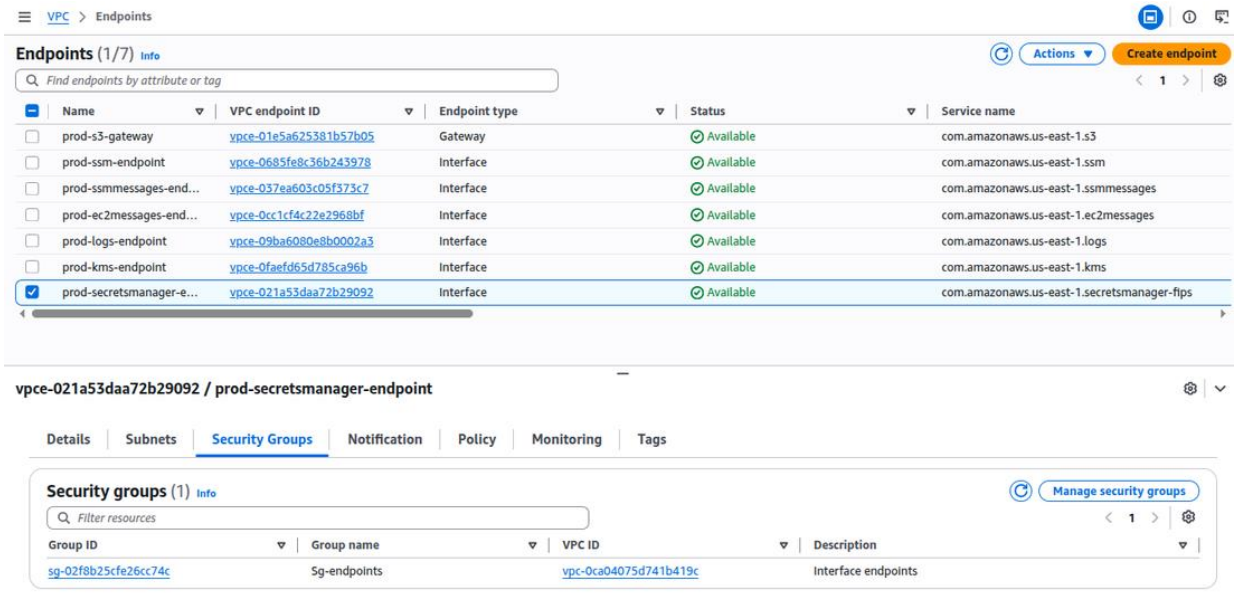


Figure 64. prod-secretsmanager-endpoint (service secretsmanager-fips): Security Groups tab shows Sg-endpoints.

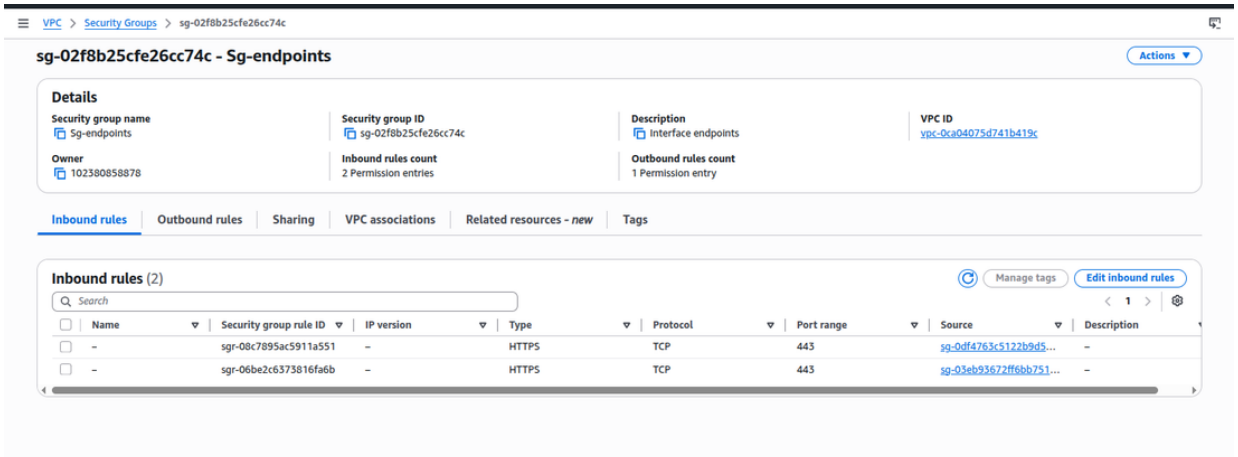


Figure 65. Sg-endpoints inbound rules: TCP 443 from two security groups.

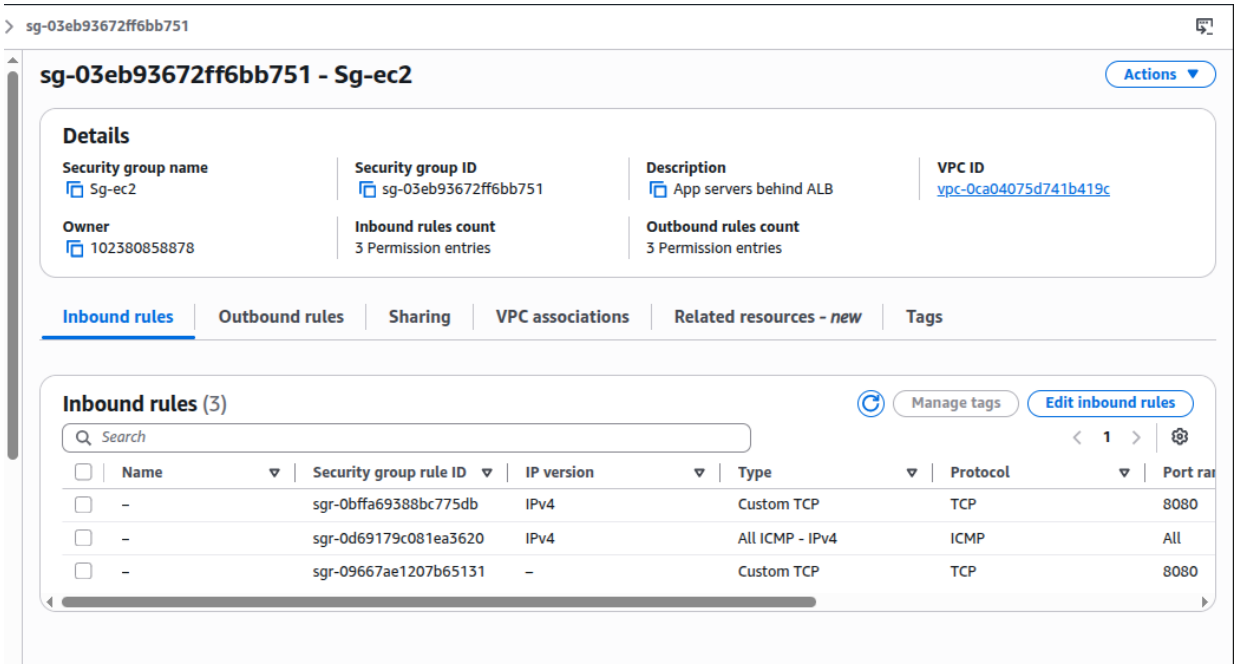


Figure 66. Sg-ec2 inbound rules: three rules, Source column cut off.

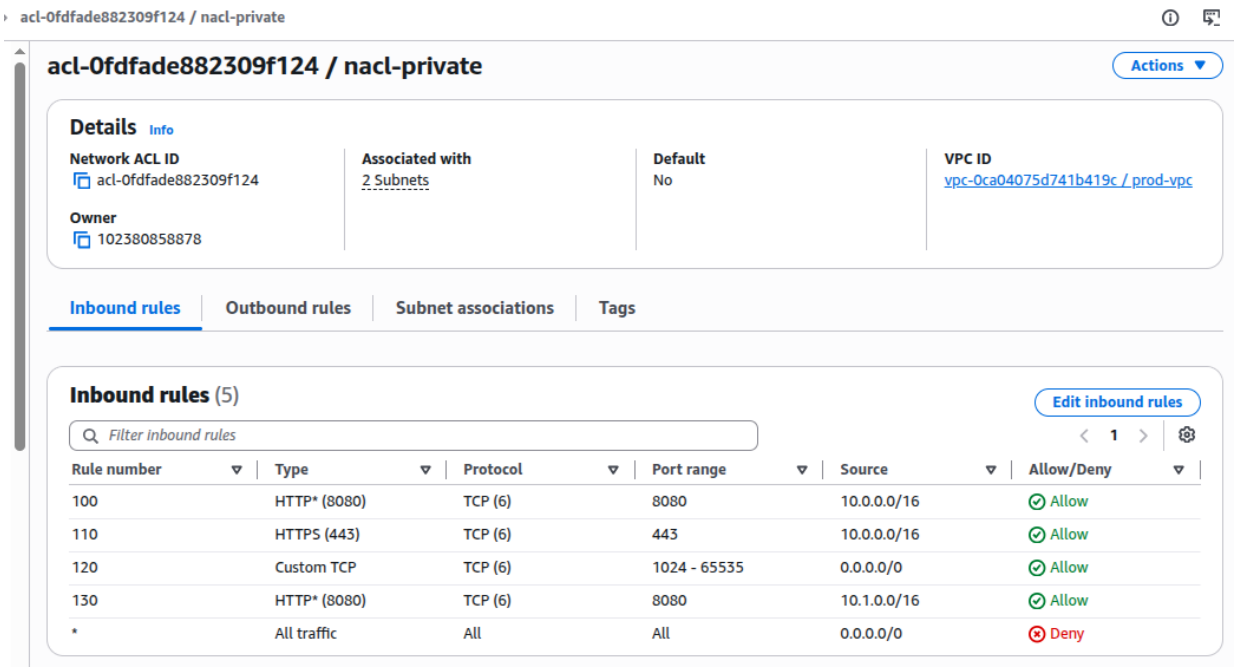


Figure 67. nacl-private inbound rules: rule 130, TCP 8080 from 10.1.0.0/16 added.

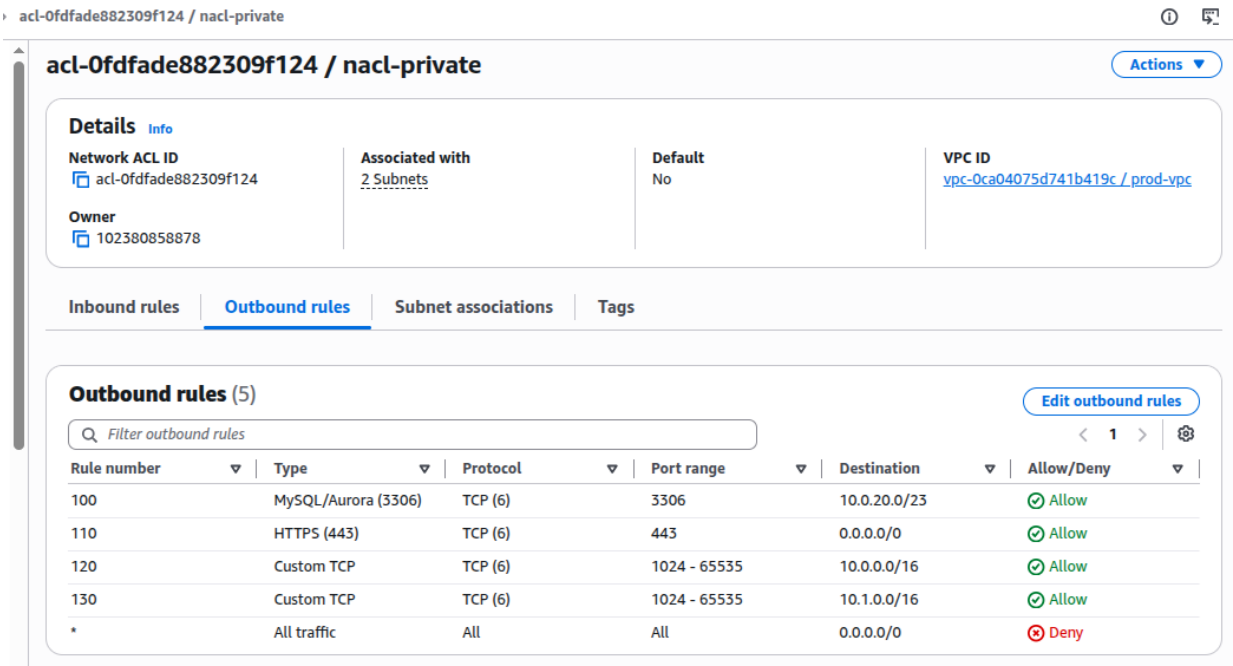


Figure 68. nacl-private outbound rules: rule 130, TCP 1024-65535 to 10.1.0.0/16 added.

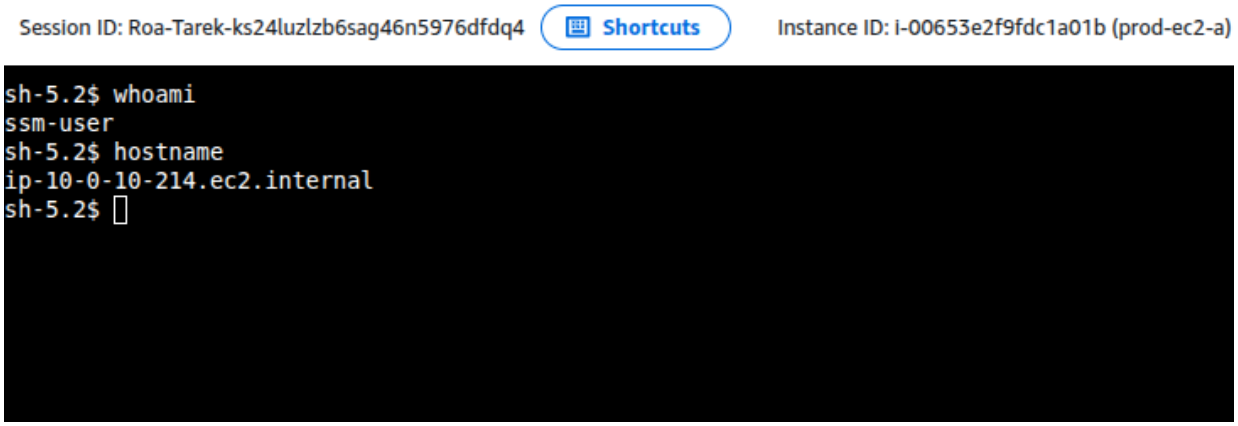


Figure 69. Session Manager shell on prod-ec2-a after the rule changes (whoami, hostname).

8. T19 to T21: Account-level services

T19: CloudTrail

Trail project-trail is logging, multi-region, with log file validation enabled and delivery to S3 bucket aws-cloudtrail-logs-102380858878-152d4262. SSE-KMS, SNS and CloudWatch Logs are off to save cost. A log file (805 B, 17:37) appears under AWSLogs/102380858878/CloudTrail/us-east-1/2026/10/19. Event history shows the CreateRole event by Roa-Tarek at 12:27:15. Result: passed.

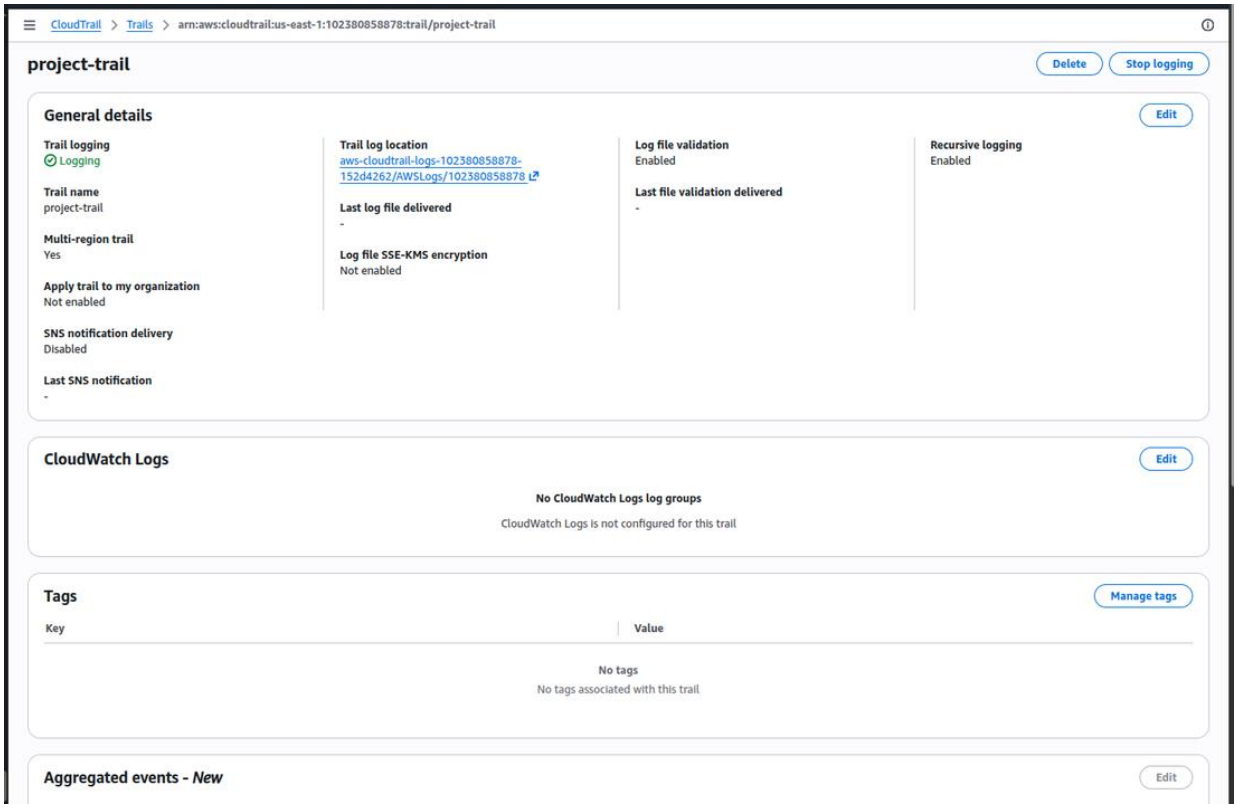


Figure 50. project-trail details: Logging, multi-region Yes, log file validation Enabled.

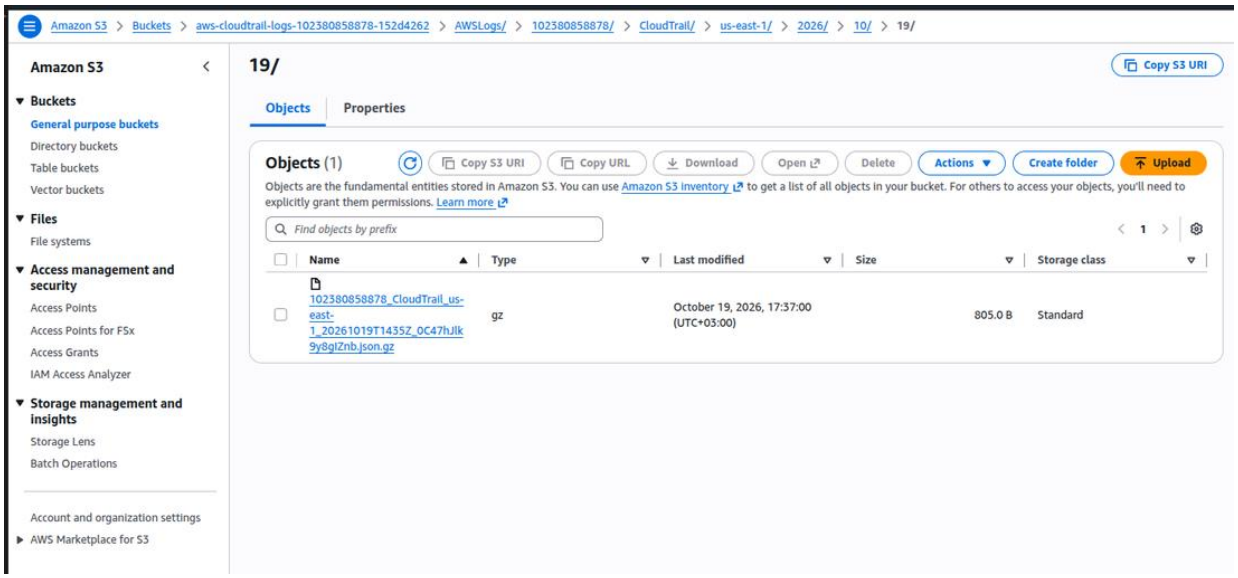


Figure 51. S3 bucket path for 19 October 2026 containing a CloudTrail log file.

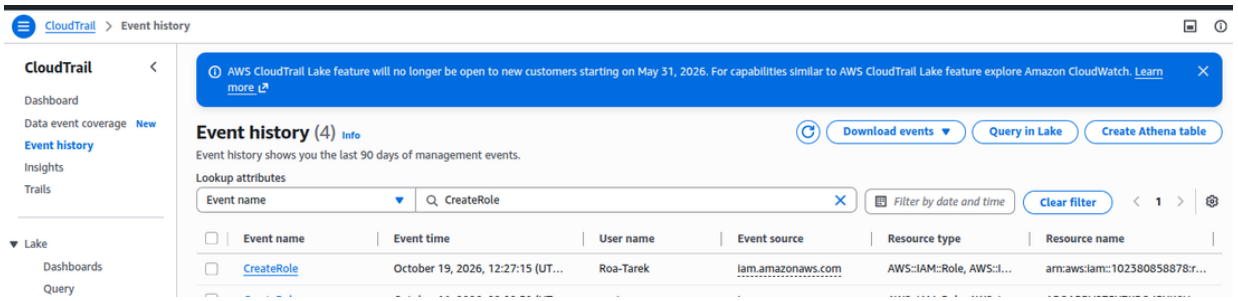


Figure 52. Event history filtered on CreateRole.

T20: GuardDuty

GuardDuty is enabled and sample findings were generated. The findings list is prefixed [SAMPLE], and the open finding uses placeholder instance i-99999999. The summary shows 479 total findings (21 critical, 181 high, 199 medium, 78 low). These are sample data, not real threats. Result: passed.

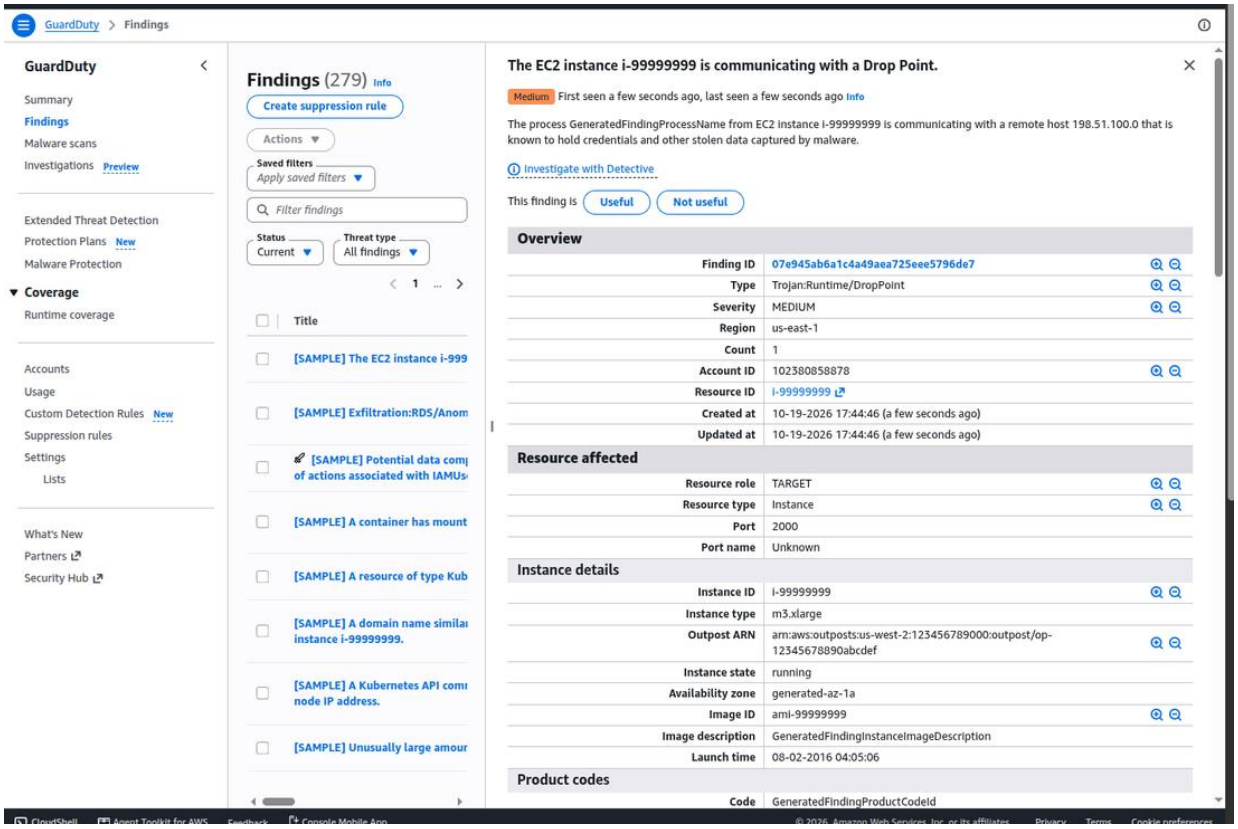


Figure 53. GuardDuty findings list with one sample finding open.

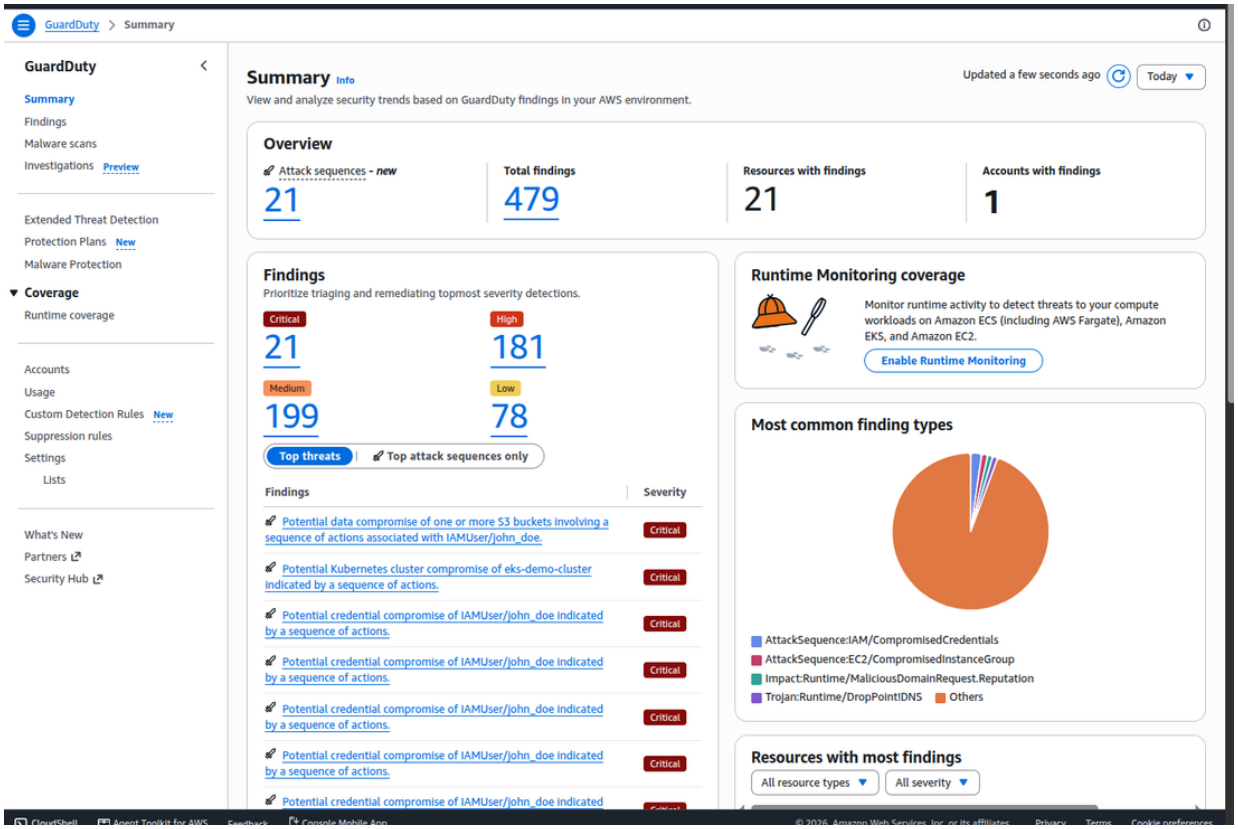


Figure 54. GuardDuty summary: 479 total findings, severity breakdown.

T21: Security Hub

Security Hub is enabled (Figure 55). The Threat summary shows the same 21/181/199/78 counts and sample account 111122223333 (Figure 56). The Threats page filtered to Critical lists 21 sample findings (Figure 57). Both required standards are enabled: AWS Foundational Security Best Practices v1.0.0 and CIS AWS Foundations Benchmark v1.2.0. Their scores show 0 percent because the score was still being generated (about 7 more minutes) (Figure 58). Result: enabled; a later screenshot with scores is still needed.

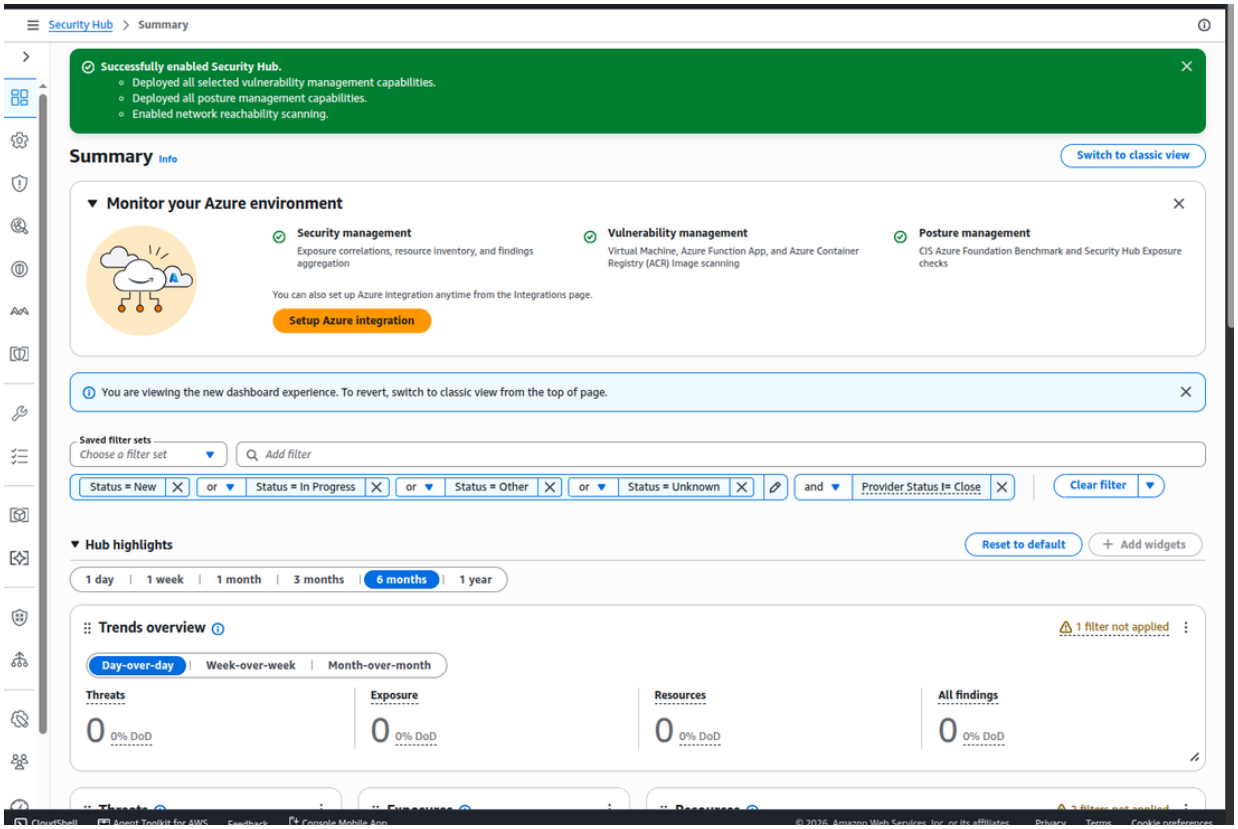


Figure 55. Security Hub enabled confirmation.

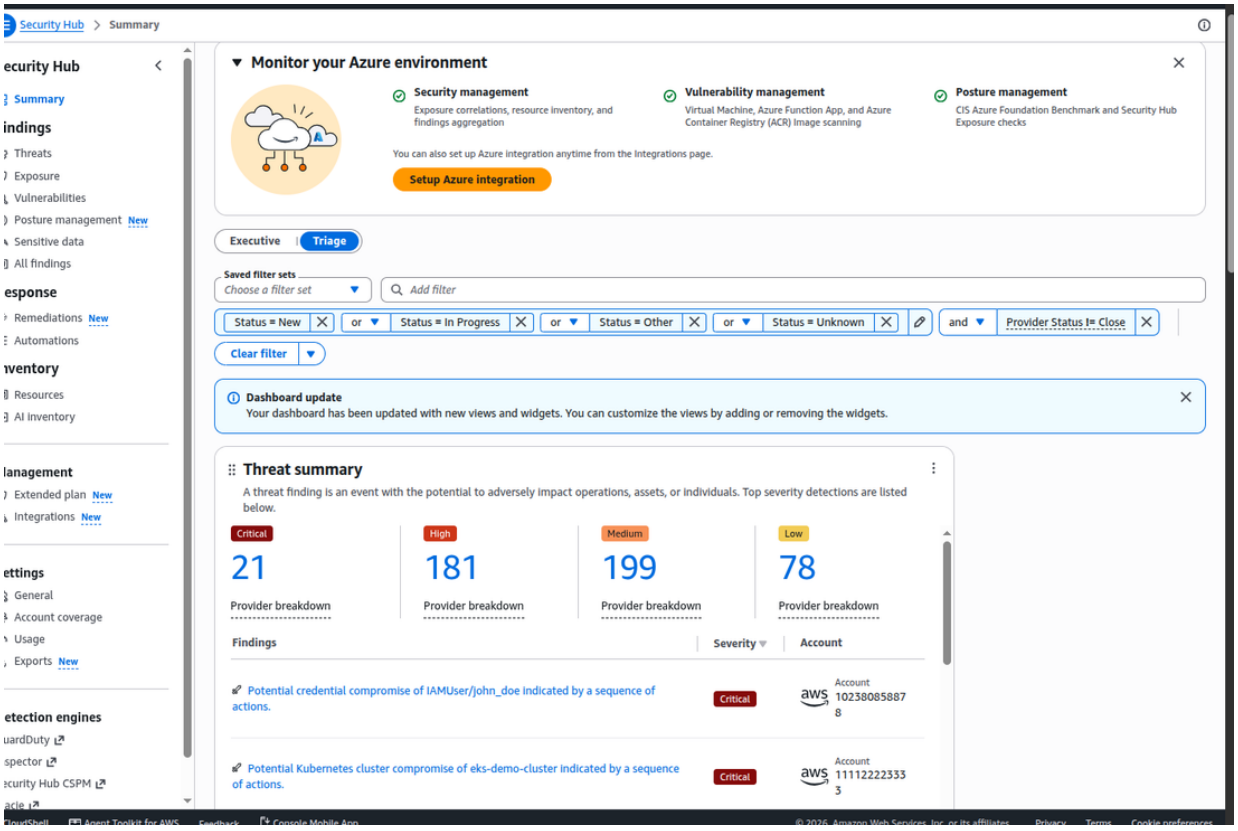


Figure 56. Security Hub summary: Threat summary with sample findings.

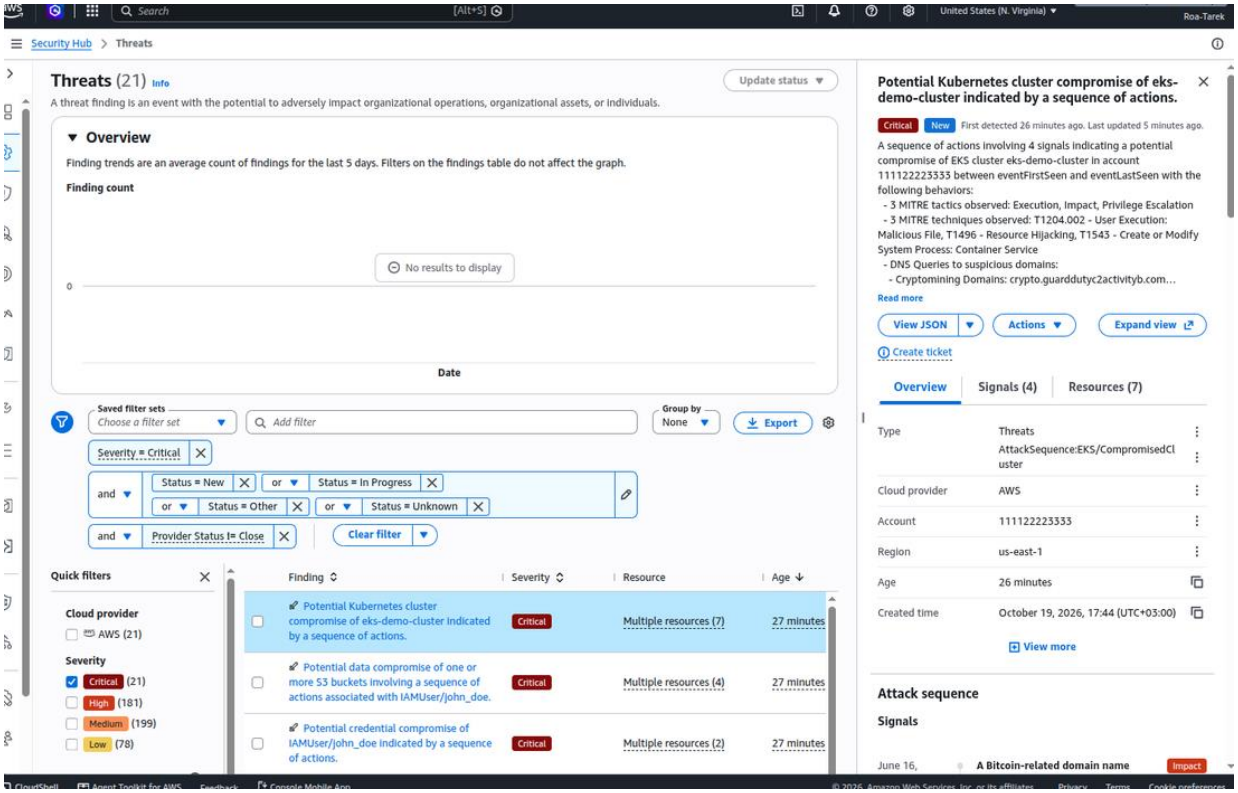


Figure 57. Security Hub Threats page filtered to Critical, with a finding open.

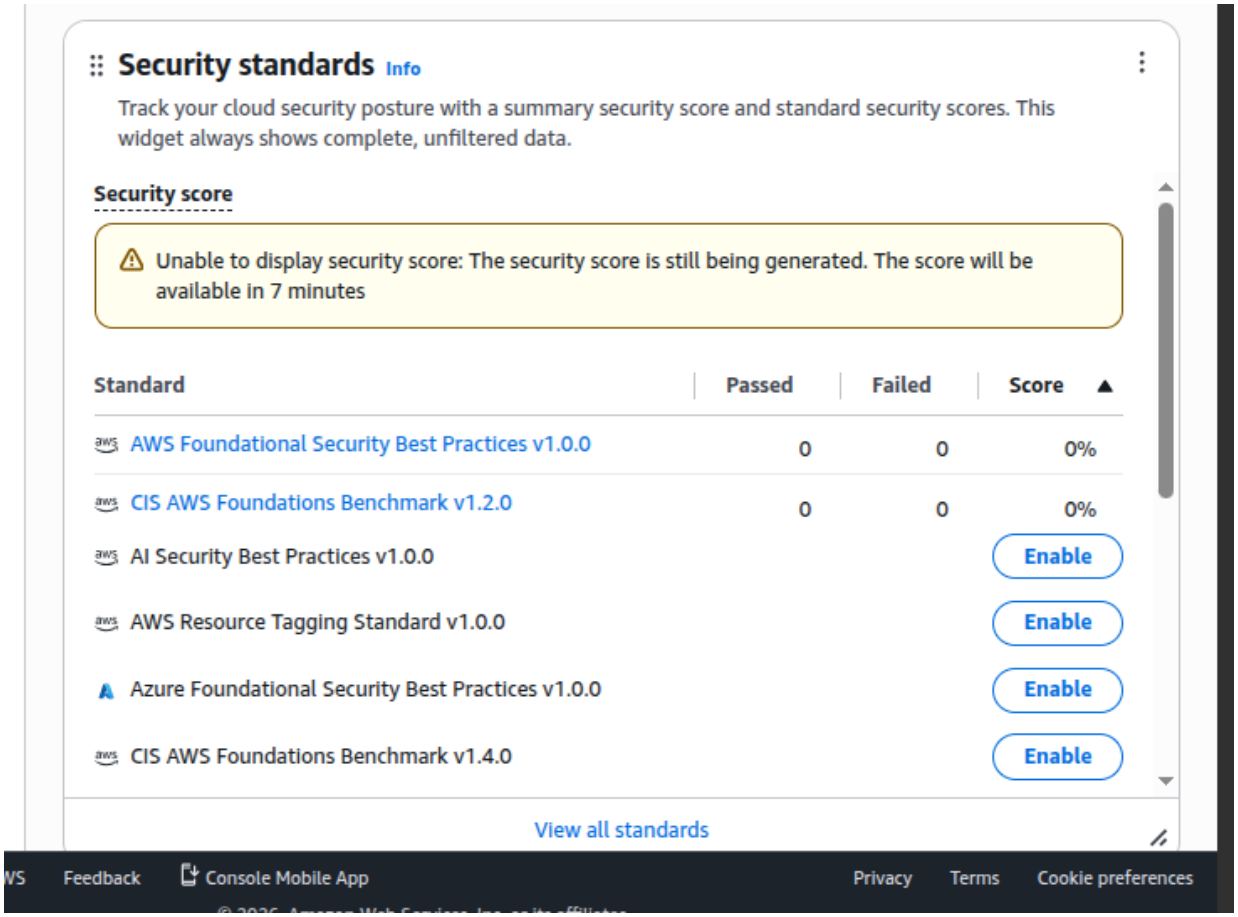


Figure 58. Security standards widget: two standards enabled, security score still generating.

9. T22 and T23: PrivateLink

Purpose. Expose the prod app privately to shared-vpc without peering, using an internal Network Load Balancer, an endpoint service and an interface endpoint (T22), then prove it works from shared-bastion (T23).

What was built (T22).

- Target group tg-privatelink (TCP 8080, instance targets) with prod-ec2-a and prod-ec2-b, both Healthy (Figures 72, 79).
- Internal Network Load Balancer prod-nlb, Active, created 22 October 2026 at 11:07, in the two private subnets (Figure 71). Cross-zone load balancing is enabled (Figure 78).
- prod-nlb has its own security group, Sg-nlb (description PrivateLink NLB, 1 inbound and 1 outbound rule), and Enforce inbound rules on PrivateLink traffic is On (Figure 91). The rules inside Sg-nlb are not shown.
- Endpoint service prod-privatelink-service (vpce-svc-0bc7e87e74432c019): Available, acceptance required, prod-nlb attached in both AZs (Figure 73).
- Consumer endpoint in shared-vpc: created Pending acceptance (Figures 74-75), accepted on the service (Figure 76), then Available in shared-private-a at 10.1.10.45 (Figure 77). That endpoint no longer appears in the later endpoint list, which shows shared-privatelink-endpoint-2 (created 12:16) Available instead (Figure 83). The final test uses the second endpoint.
- Supporting rules: Sg-shared-endpoint (shared-vpc) allows TCP 8080 from Sg-shared-bastion (Figure 81) and is attached to the endpoint (Figure 84). Sg-shared-bastion outbound allows 8080 to 10.0.0.0/16 and 10.1.0.0/16, ICMP to 10.0.0.0/16 and 443 to 0.0.0.0/0 (Figure 82). An earlier version had port 8088 to 10.1.0.0/16, a typo that was corrected (Figure 80). Sg-ec2 now has six inbound rules, including 8080 from 10.1.0.0/16 (Peering test from shared-vpc), 8080 from 10.0.10.0/23 (PrivateLink NLB), ICMP from 10.1.0.0/16, and 8080 from 10.100.0.0/22 and 10.0.10.0/24 without descriptions (Figure 70).
- nacl-private gained rules 130 (10.1.0.0/16) and 140 (10.100.0.0/22) in both directions: inbound 8080 and outbound 1024-65535 (Figures 86-87). The shared-vpc subnets use the default NACL, which allows all traffic (Figures 88-89).

Result (T23). Passed. From a Session Manager shell on shared-bastion, a TCP check to the endpoint address 10.1.10.24:8080 returned OPEN, and curl to the endpoint DNS name returned HTTP 200 with the page Hello from EC2 in prod-private-a. The name resolved to 10.1.10.24 and 10.1.11.81. Both runs succeeded; the two responses have different ETag and Last-Modified values, which suggests the NLB reached two different instances (Figure 90).

Notes. The bastion registered with SSM, which closes the earlier gap. The endpoint list now includes shared-ssm-endpoint, shared-ssmmessages-endpoint and shared-ec2messages-endpoint, which the original plan did not have (Figure 83). The 8088 typo cost no evidence but shows the troubleshooting path.

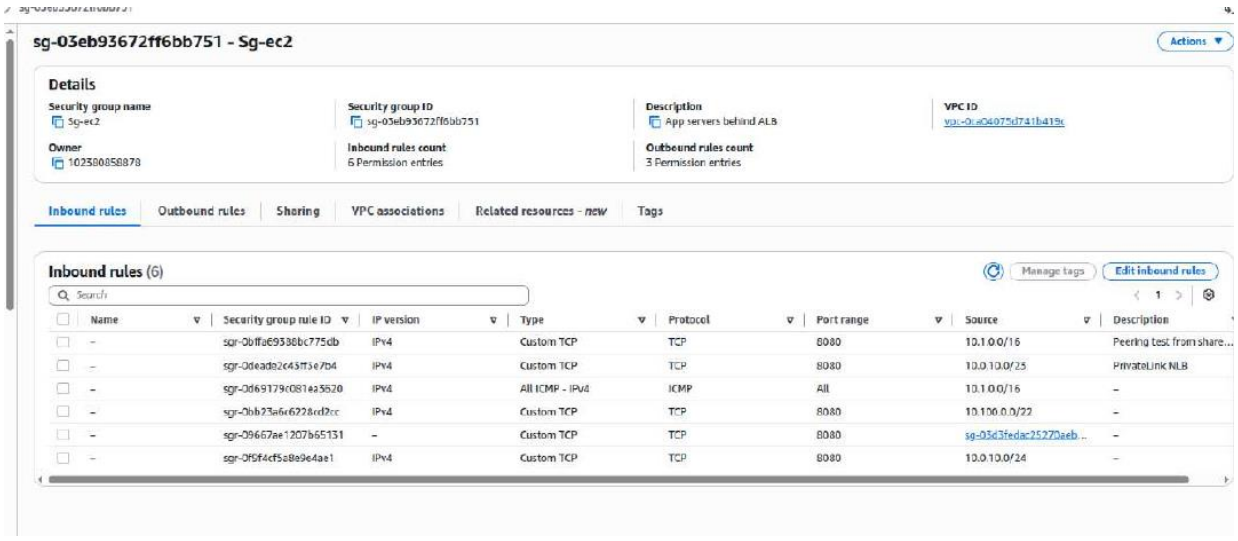


Figure 70. Sg-ec2 inbound rules: six rules, with sources visible.

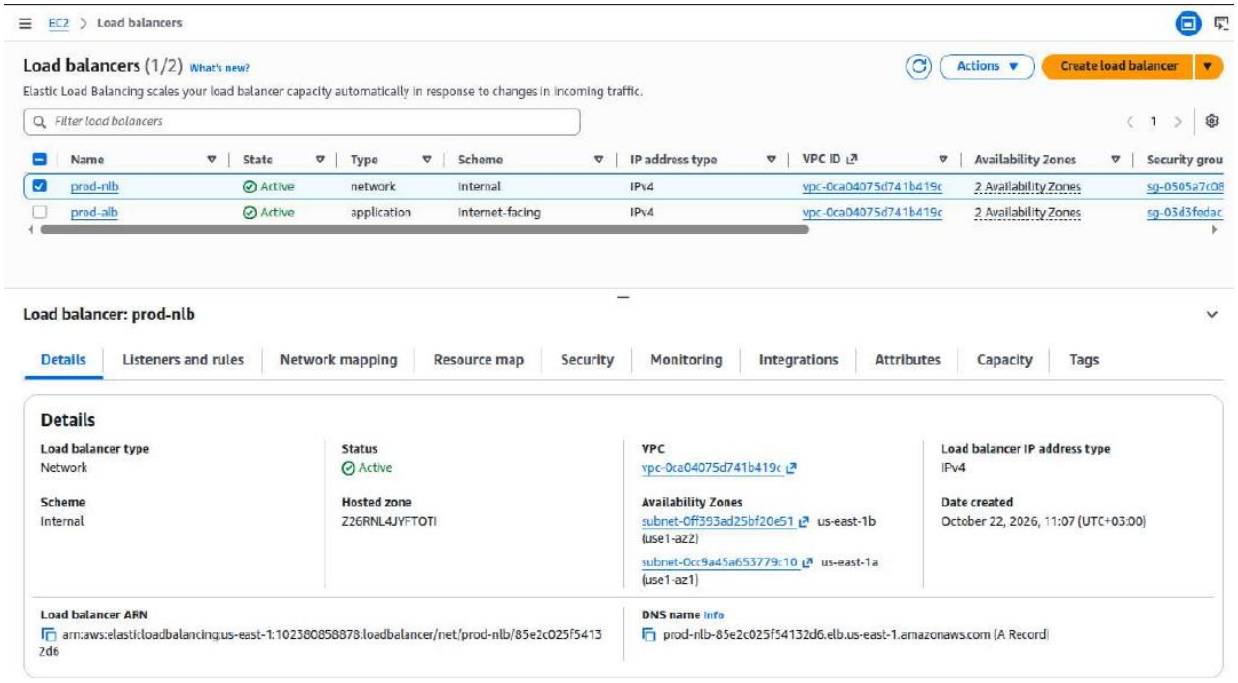


Figure 71. Load balancers: prod-nlb (network, internal, Active) and prod-alb.

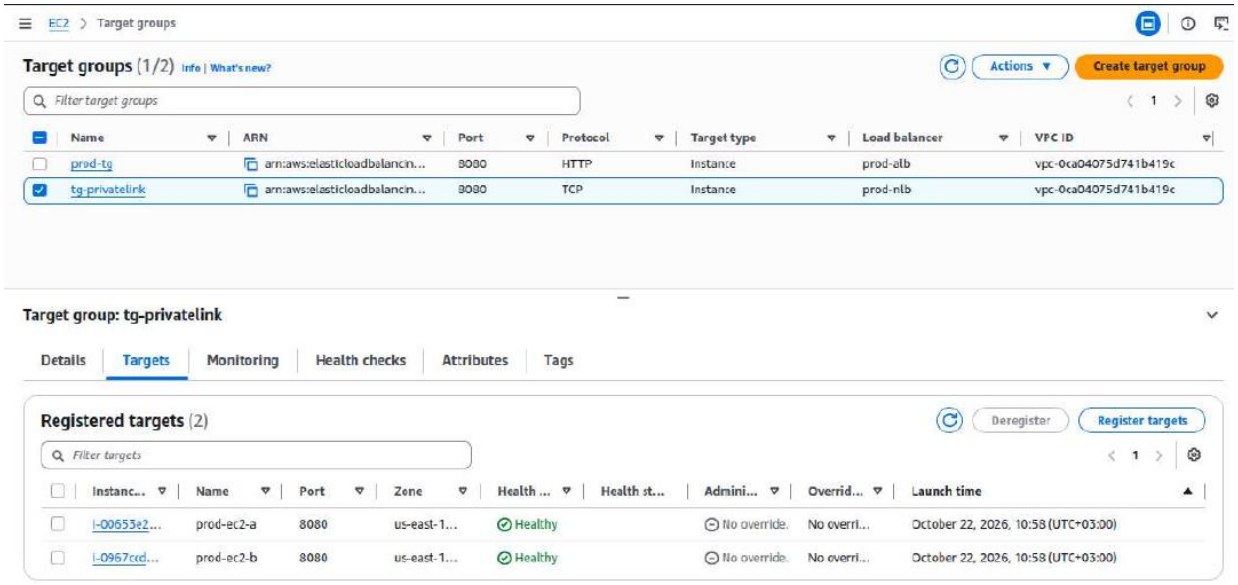


Figure 72. Target group tg-privatelink (TCP 8080): prod-ec2-a and prod-ec2-b Healthy.

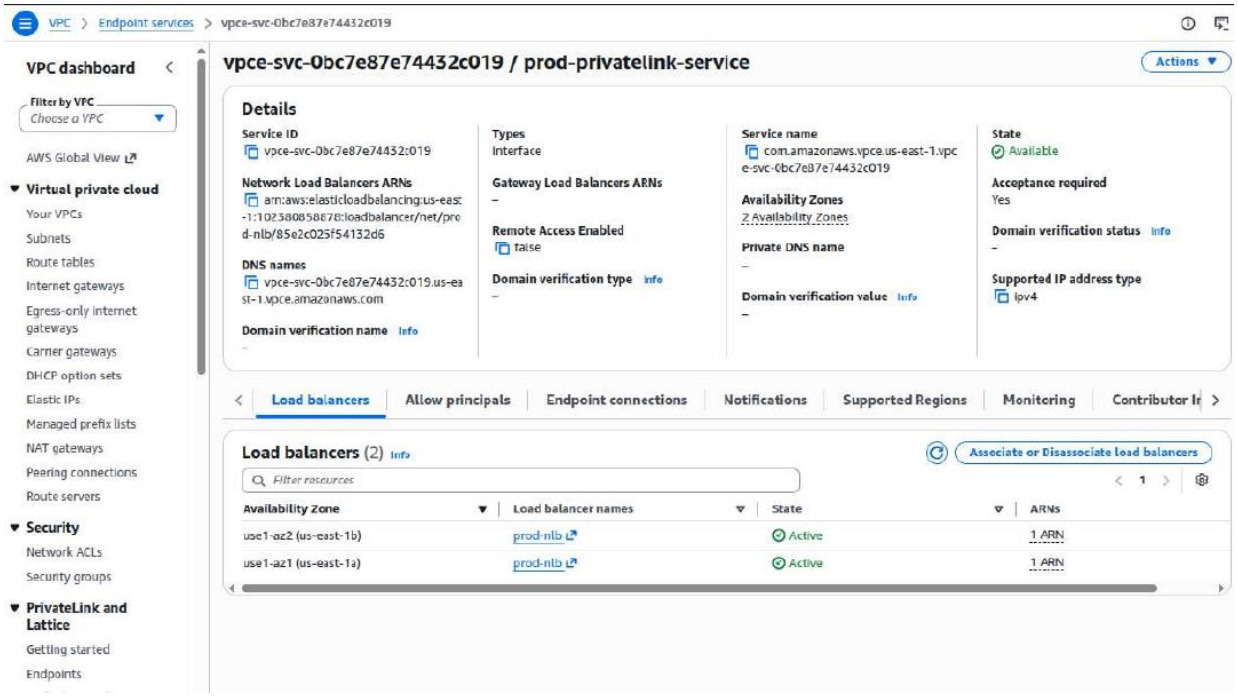


Figure 73. Endpoint service prod-privatelink-service: Available, acceptance required, prod-nlb attached.

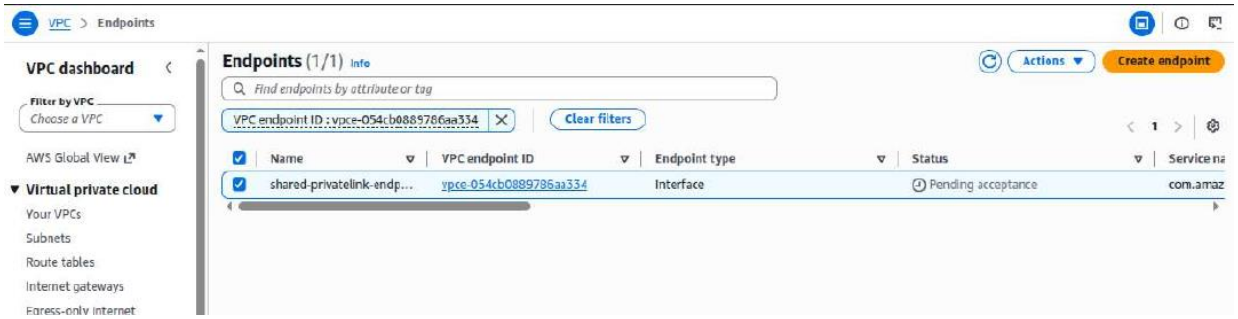


Figure 74. Endpoints list: shared-privatelink-endpoint, Pending acceptance.

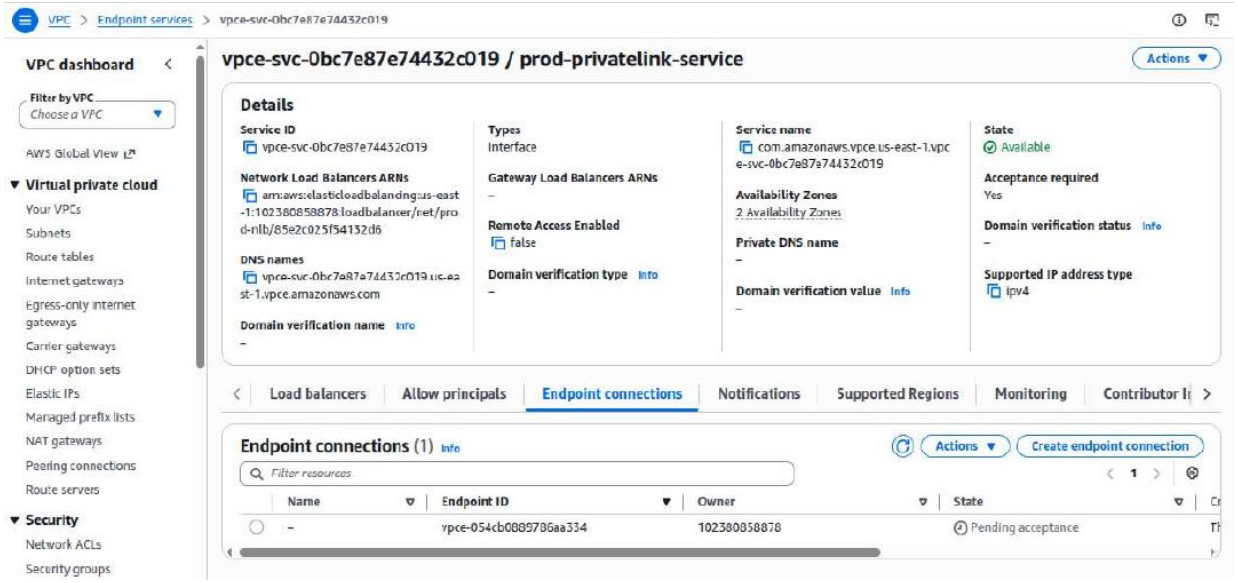


Figure 75. Endpoint service, Endpoint connections tab: Pending acceptance.

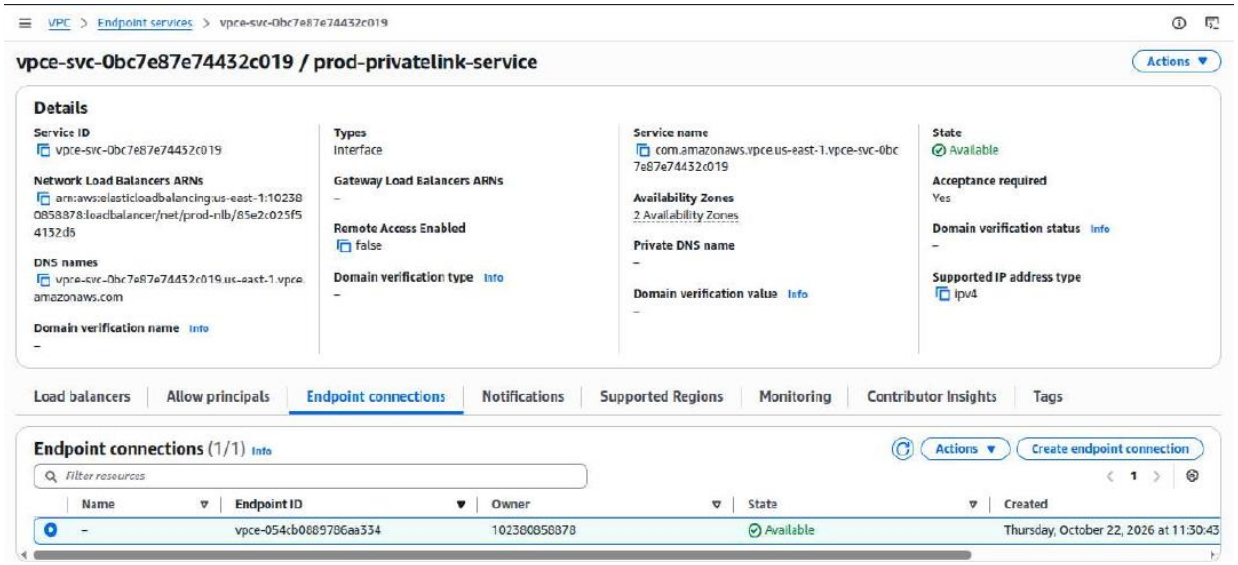


Figure 76. Endpoint service, Endpoint connections tab: connection Available.

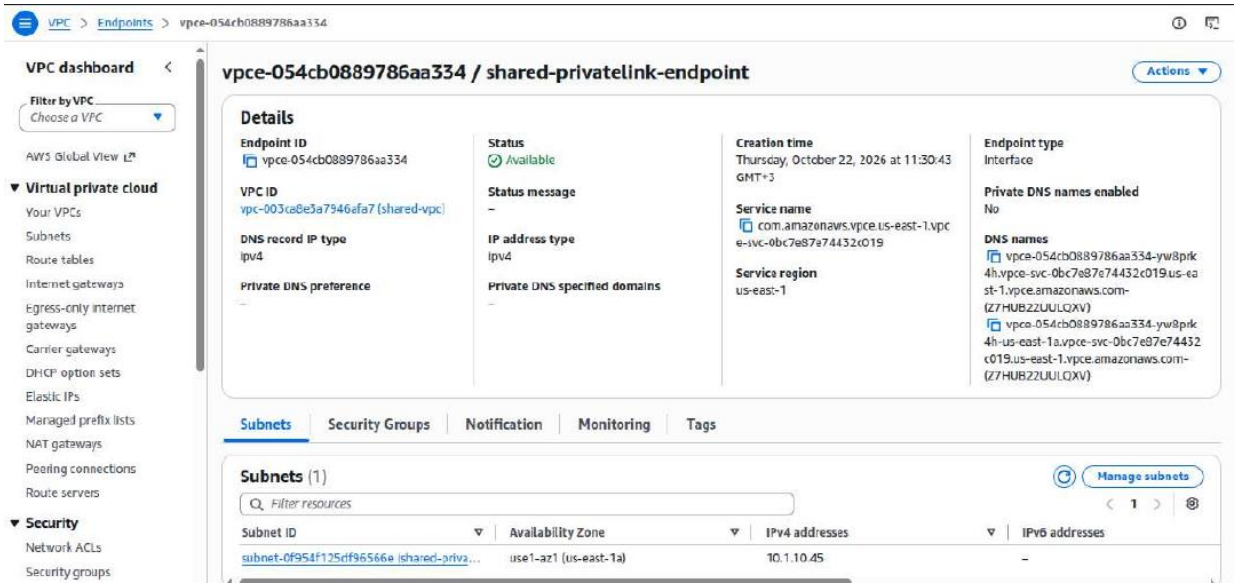


Figure 77. shared-privatelink-endpoint: Available, shared-vpc, subnet shared-private-a, IP 10.1.10.45, DNS names.

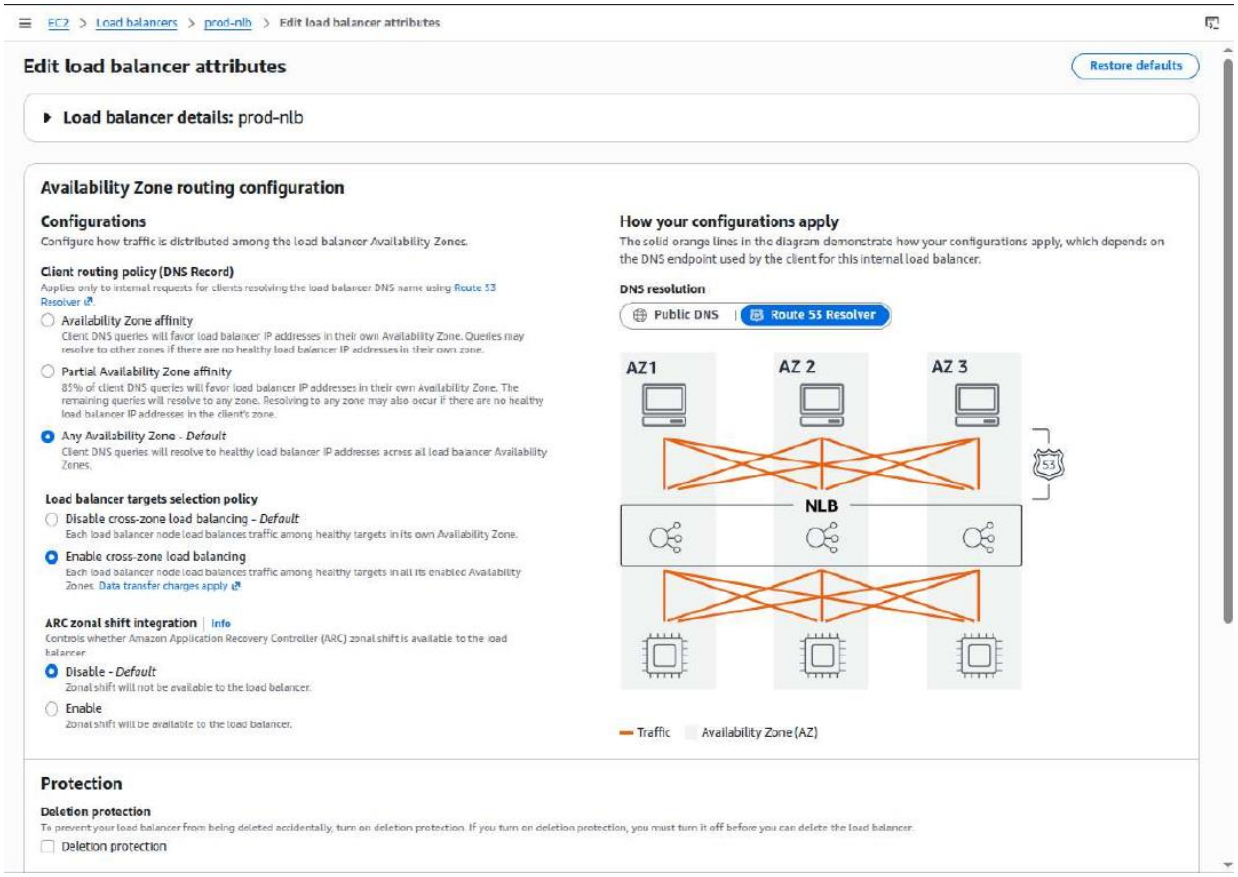


Figure 78. prod-nlb attributes: cross-zone load balancing enabled.

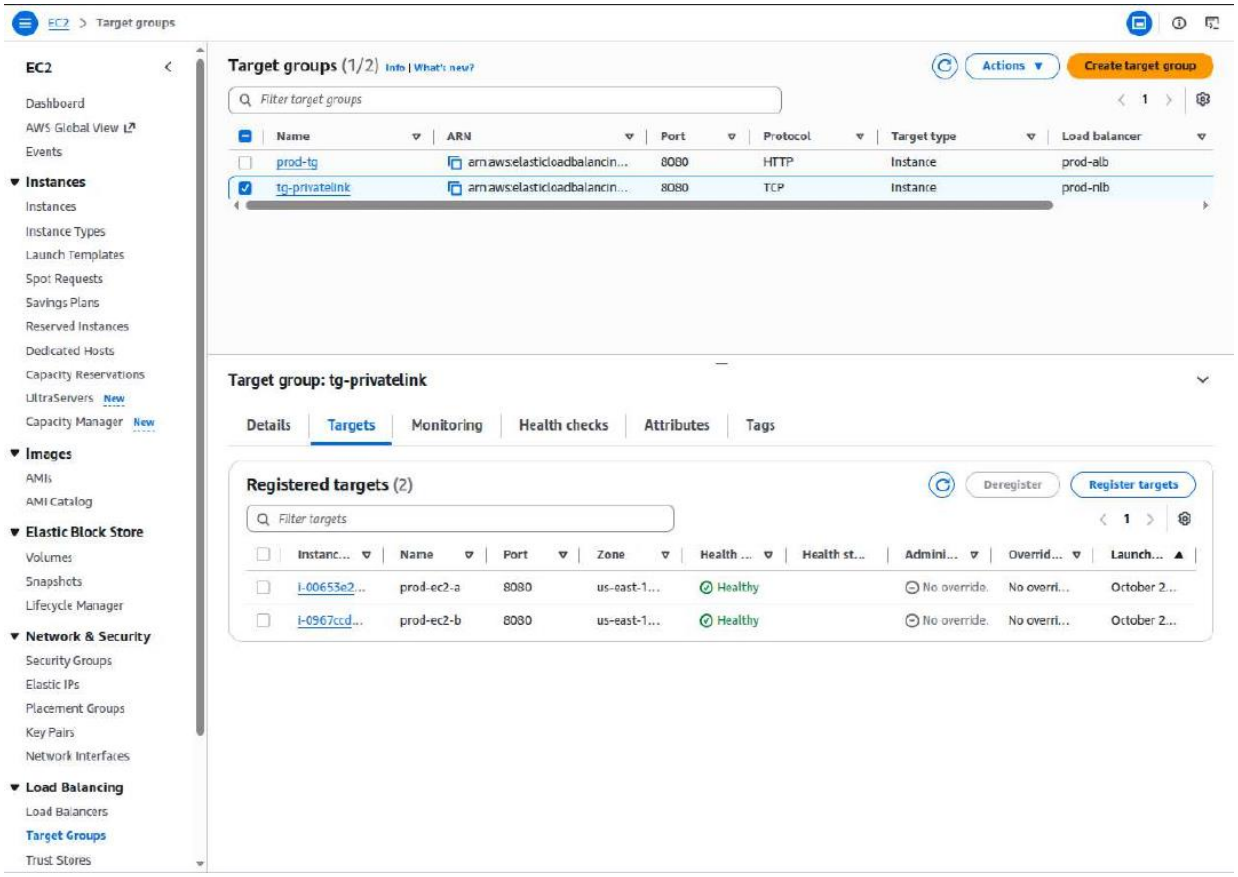


Figure 79. tg-privatelink, Targets tab: both targets Healthy.

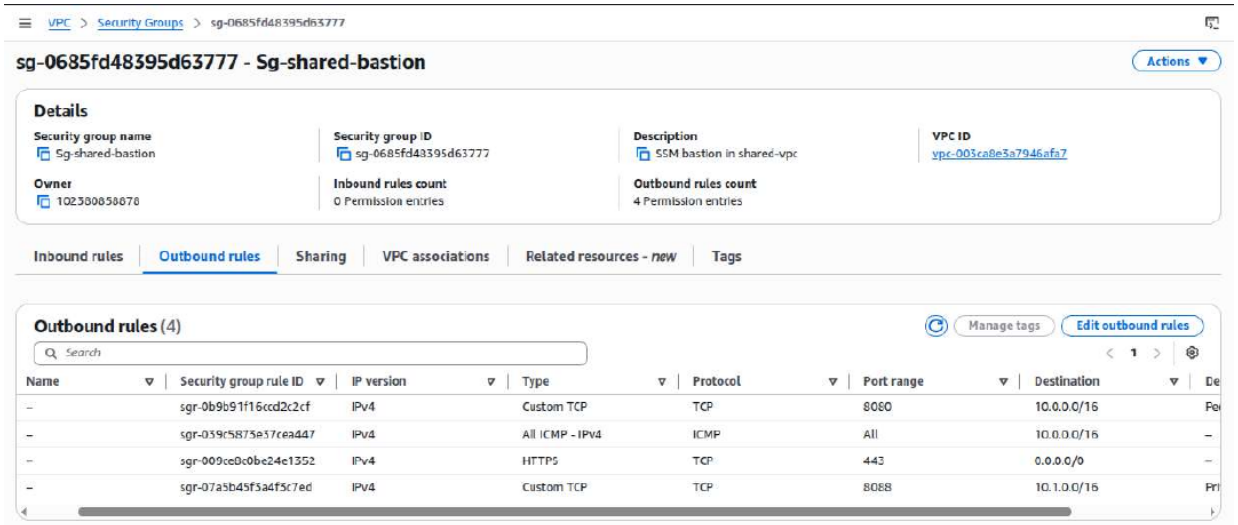


Figure 80. Sg-shared-bastion outbound rules with the 8088 typo.

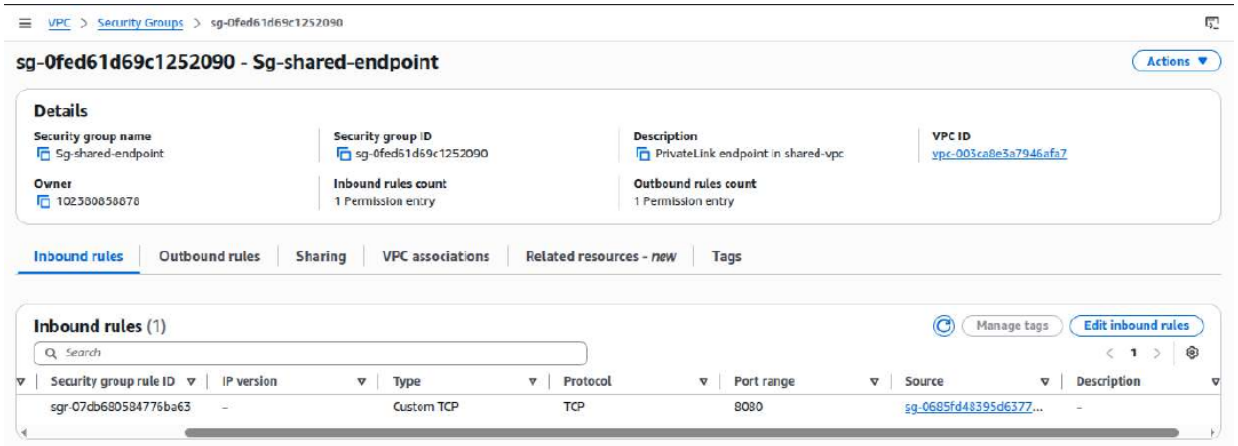


Figure 81. Sg-shared-endpoint inbound: TCP 8080 from Sg-shared-bastion.

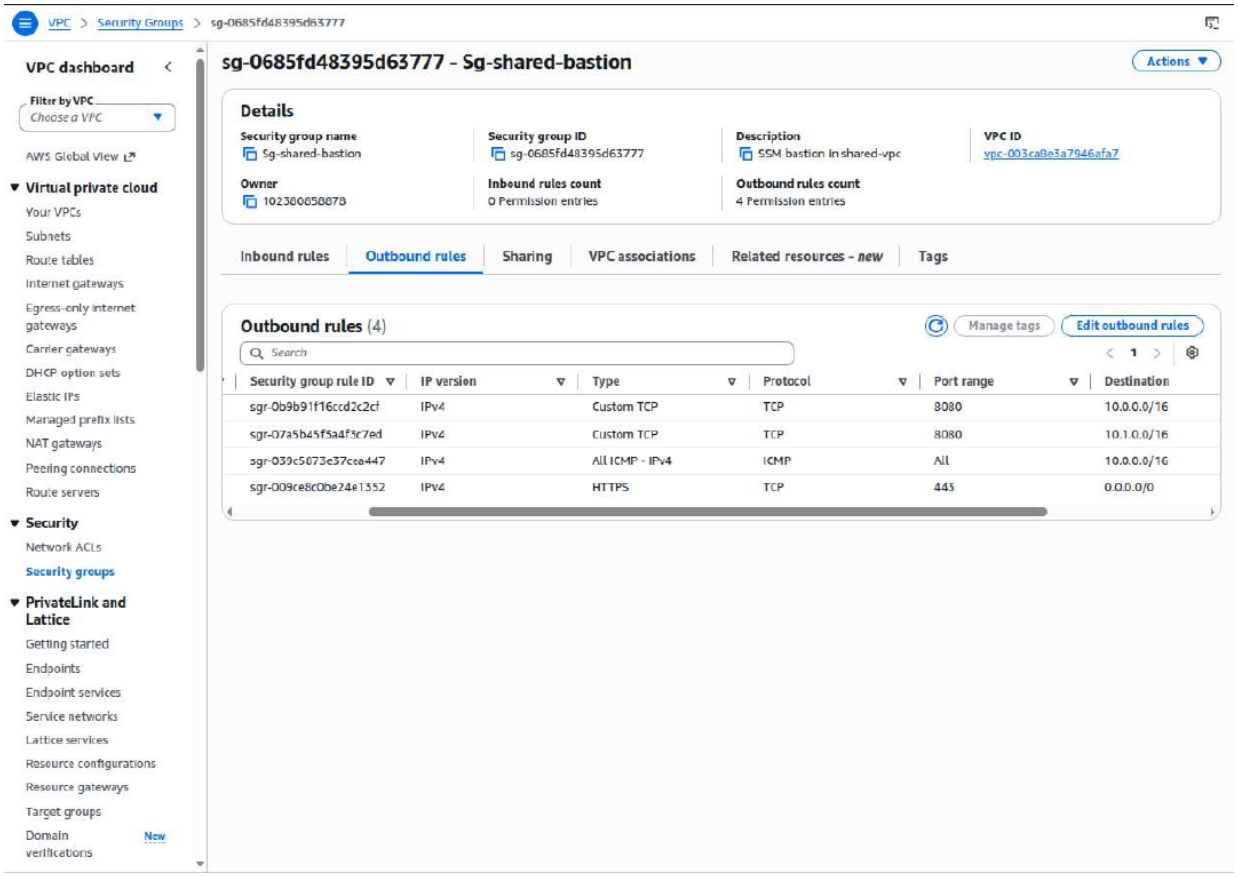


Figure 82. Sg-shared-bastion outbound rules after the fix: 8080 to 10.0.0.0/16 and 10.1.0.0/16.

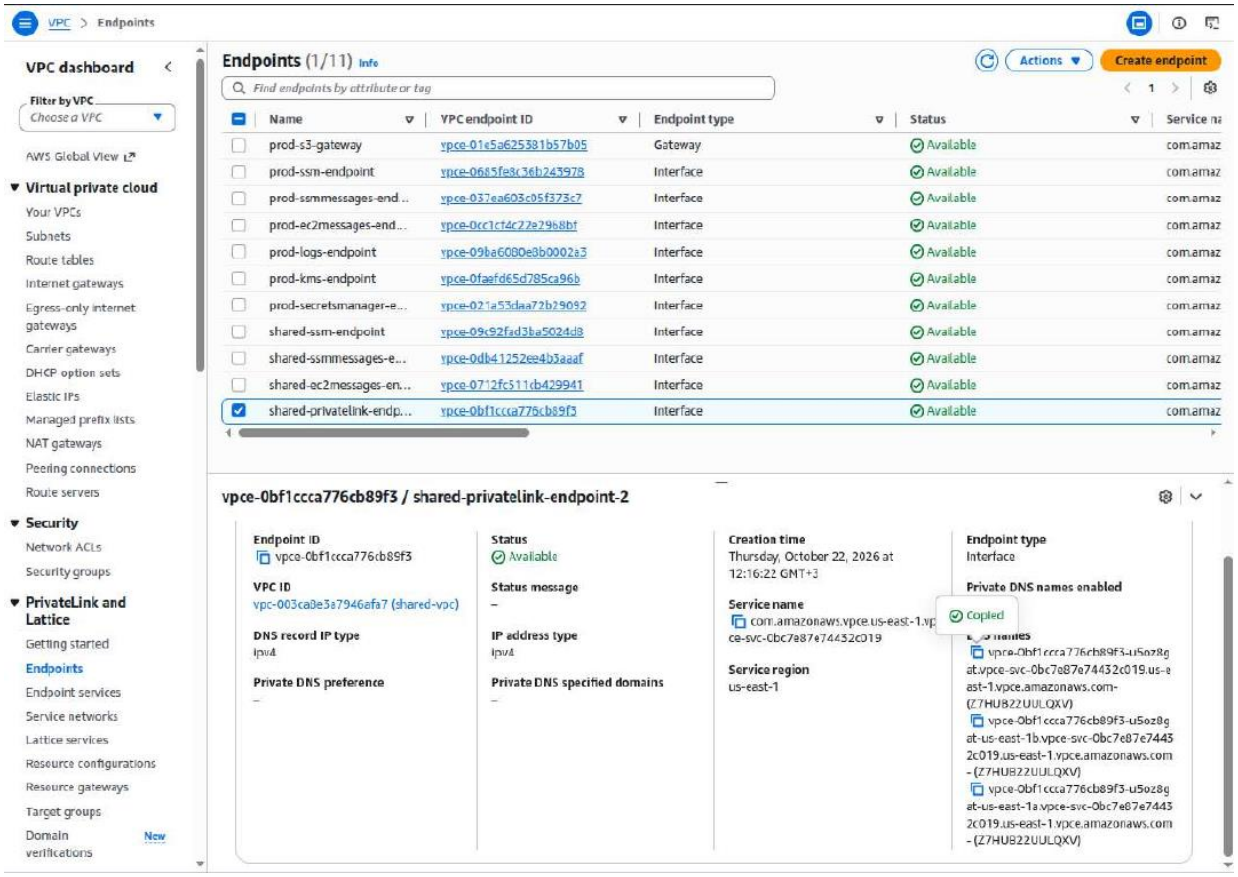


Figure 83. Endpoints list (11): shared SSM endpoints and shared-privatelink-endpoint-2, all Available.

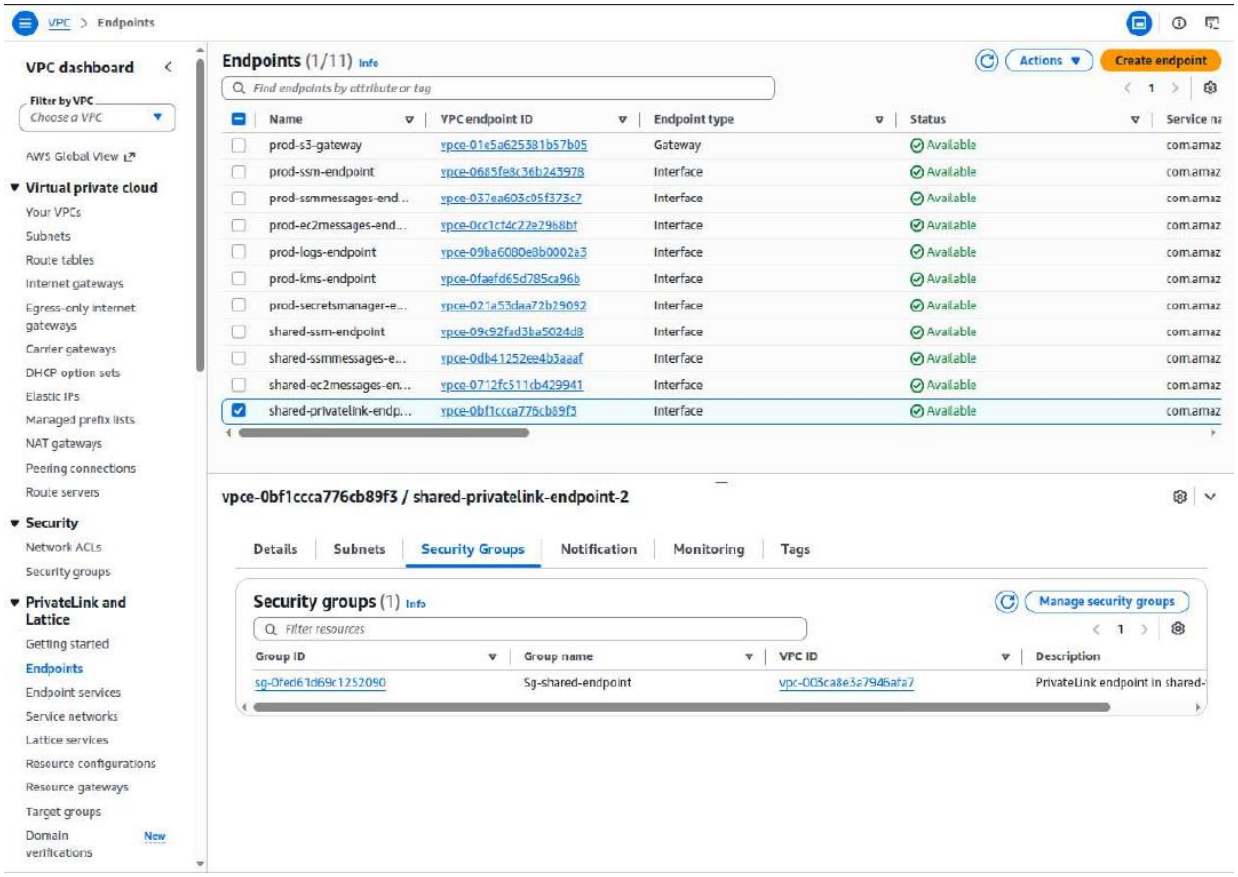


Figure 84. shared-privatelink-endpoint-2, Security Groups tab: Sg-shared-endpoint.

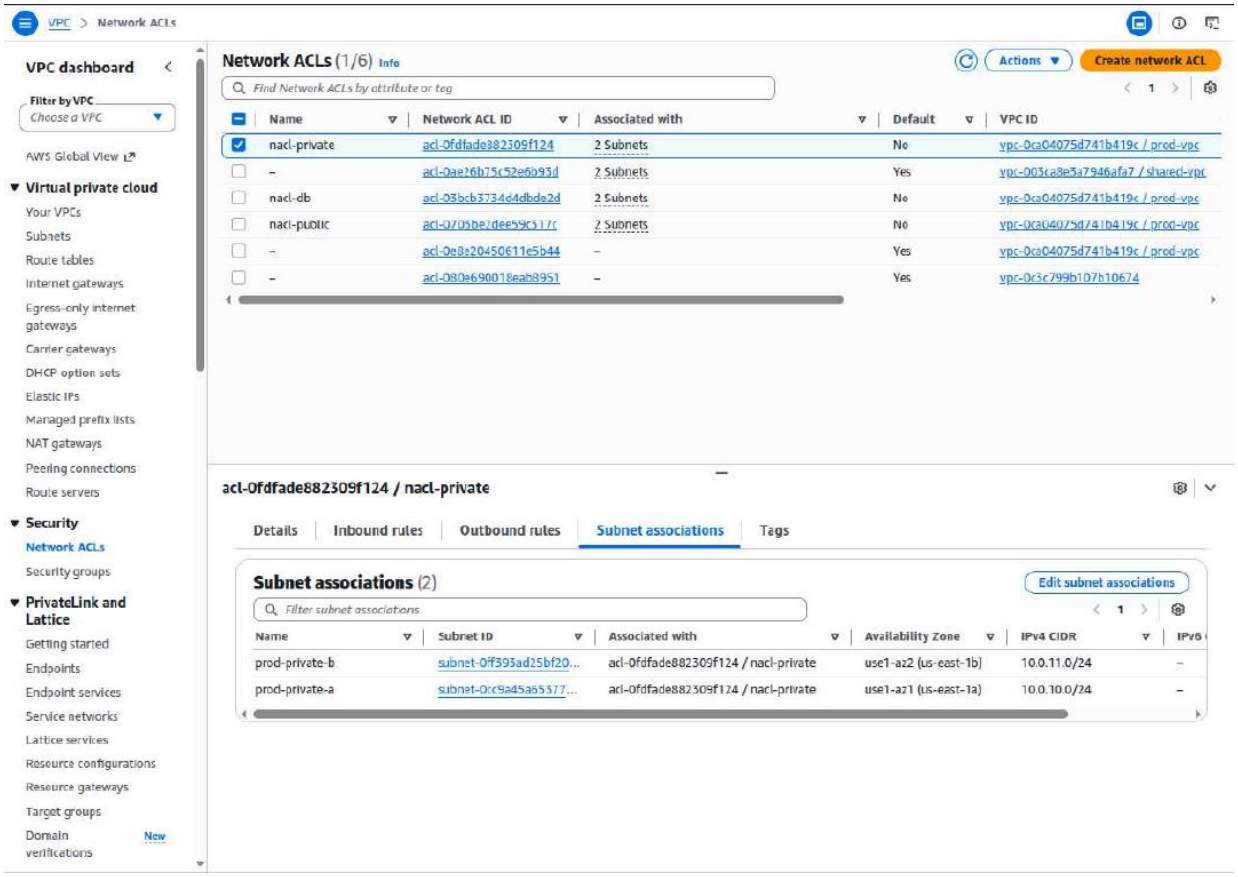


Figure 85. nacl-private subnet associations: prod-private-a and prod-private-b.

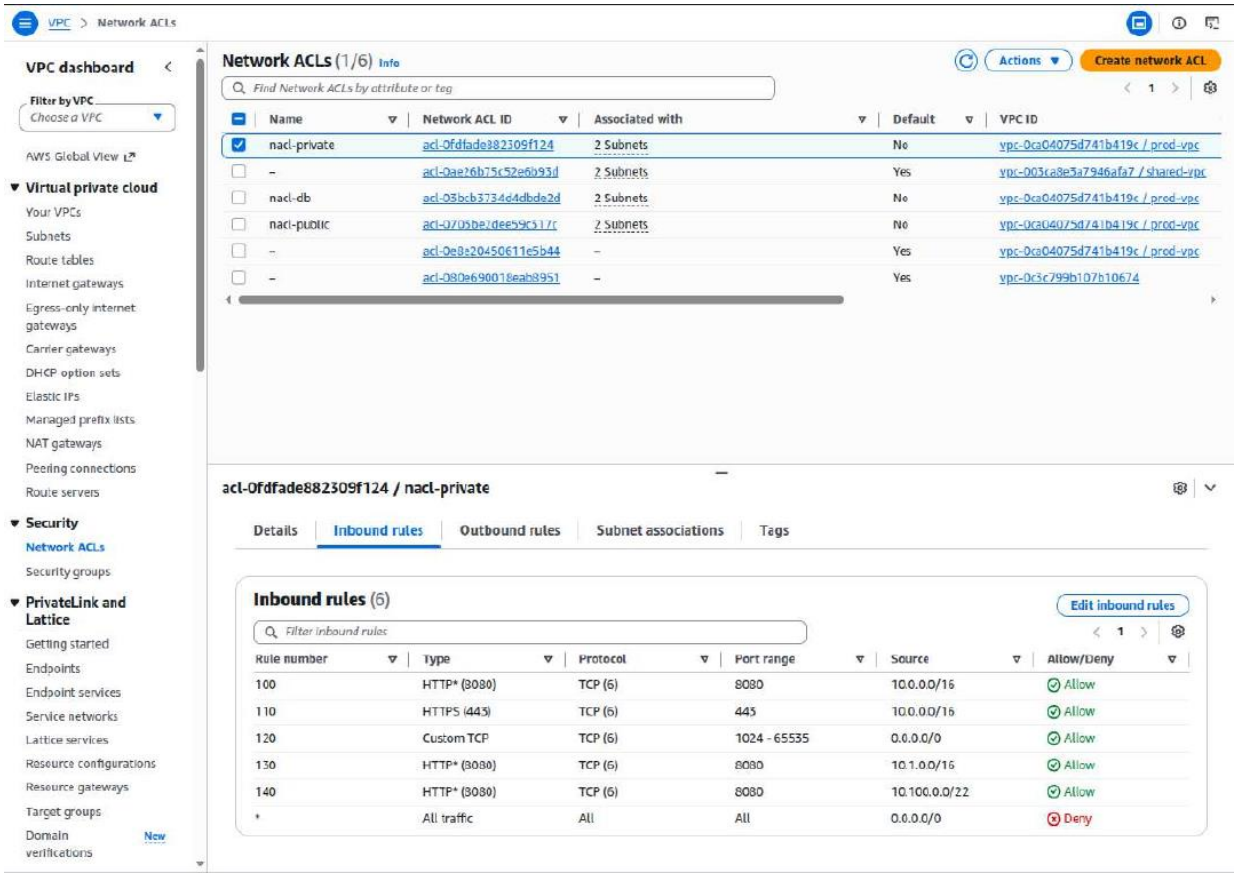


Figure 86. nacl-private inbound rules: 8080 from 10.1.0.0/16 (130) and 10.100.0.0/22 (140).

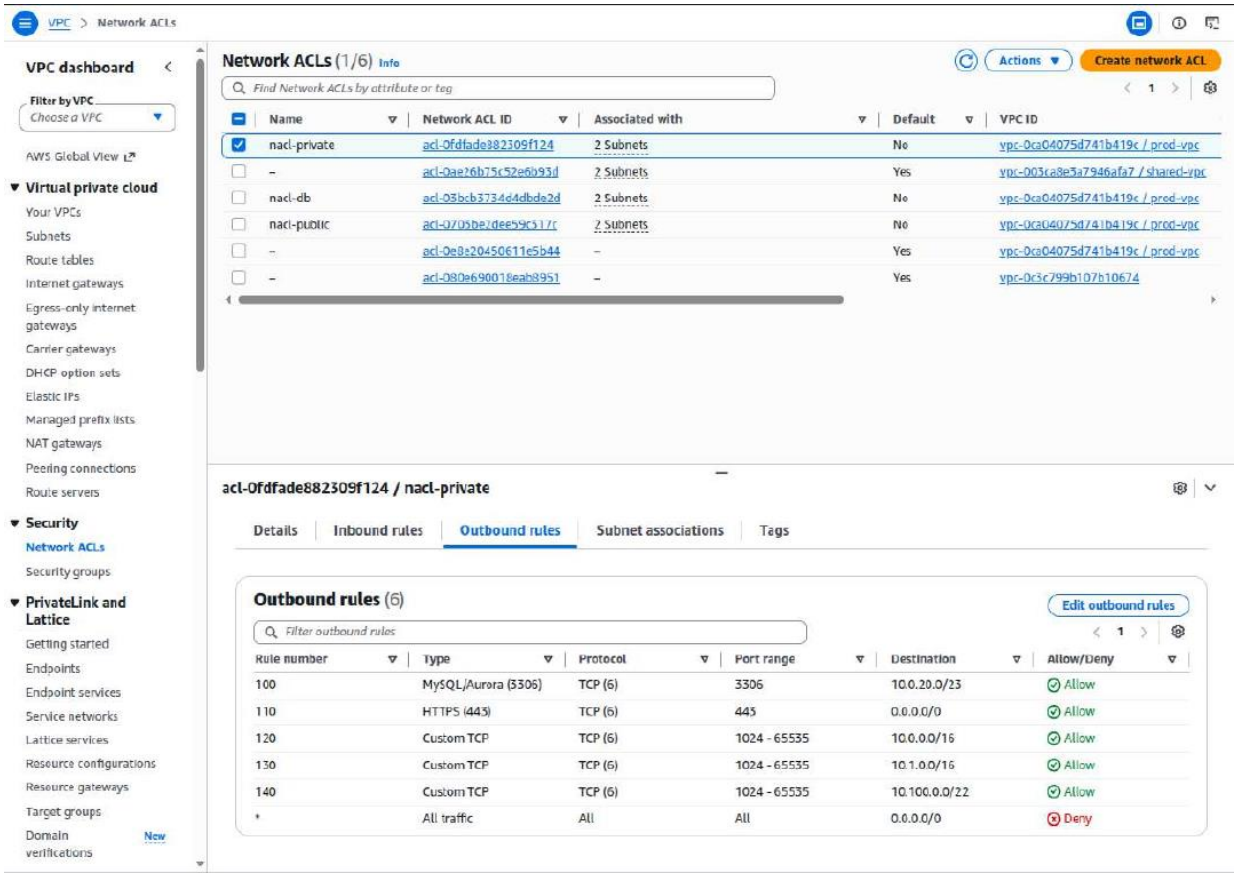


Figure 87. nacl-private outbound rules: 1024-65535 to 10.1.0.0/16 (130) and 10.100.0.0/22 (140).

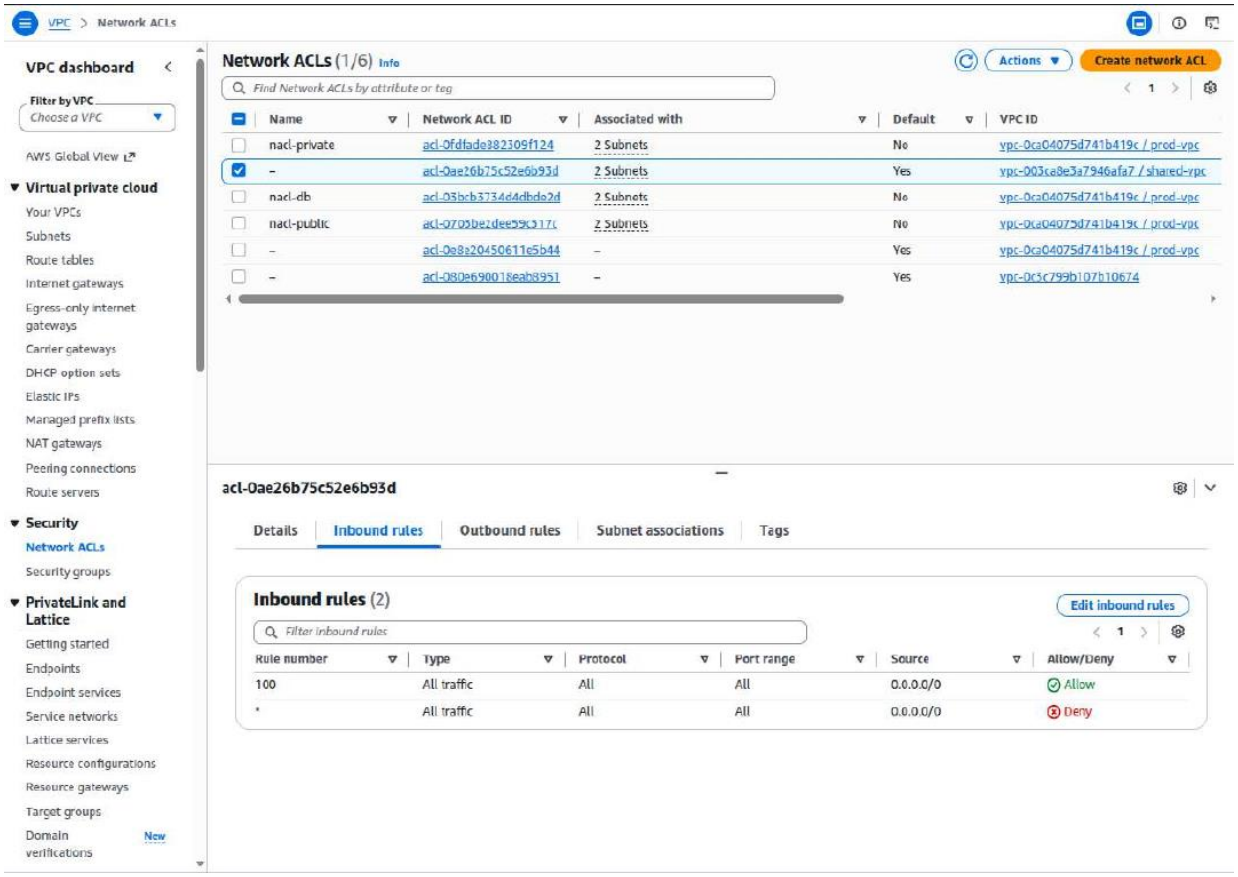


Figure 88. shared-vpc default NACL inbound rules: allow all.

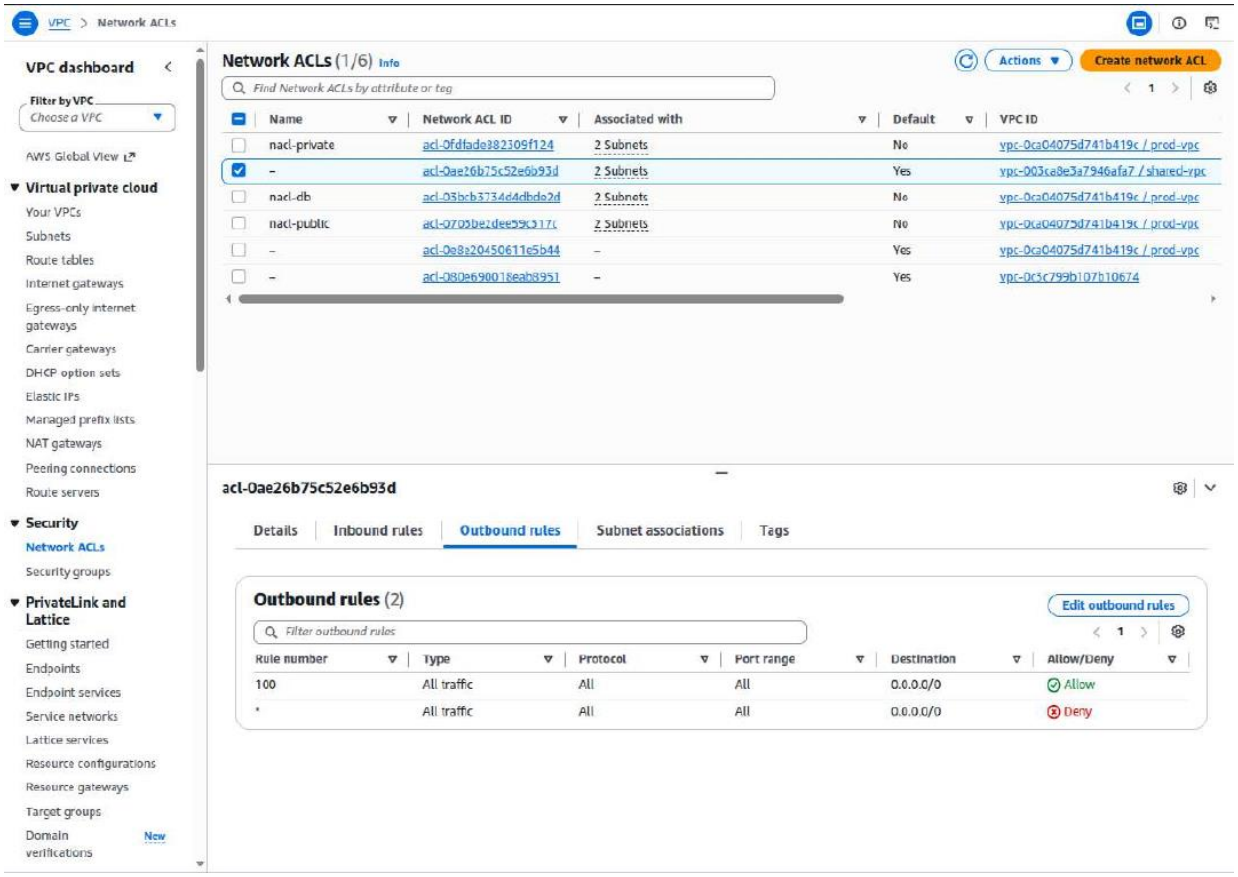


Figure 89. shared-vpc default NACL outbound rules: allow all.

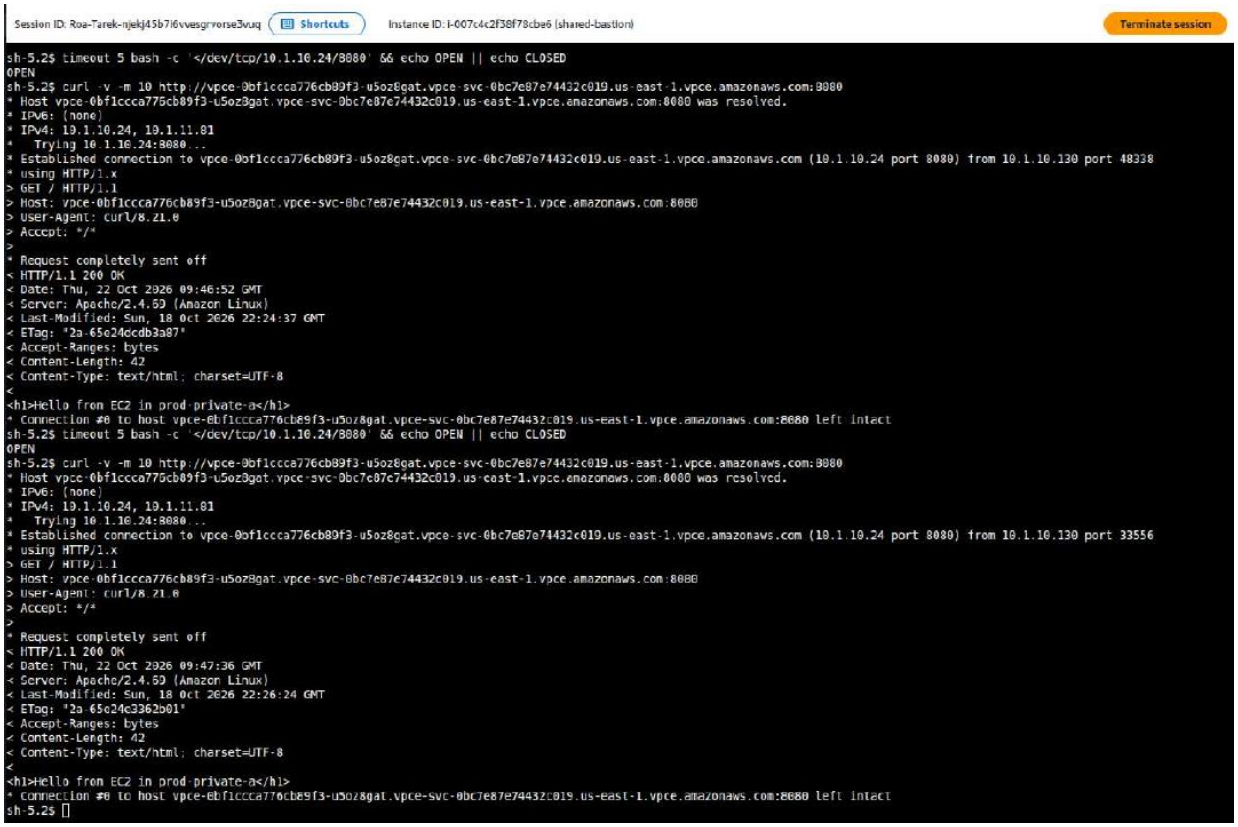


Figure 90. Test 23: shared-bastion session, TCP check OPEN and curl through the endpoint DNS name returns HTTP 200.

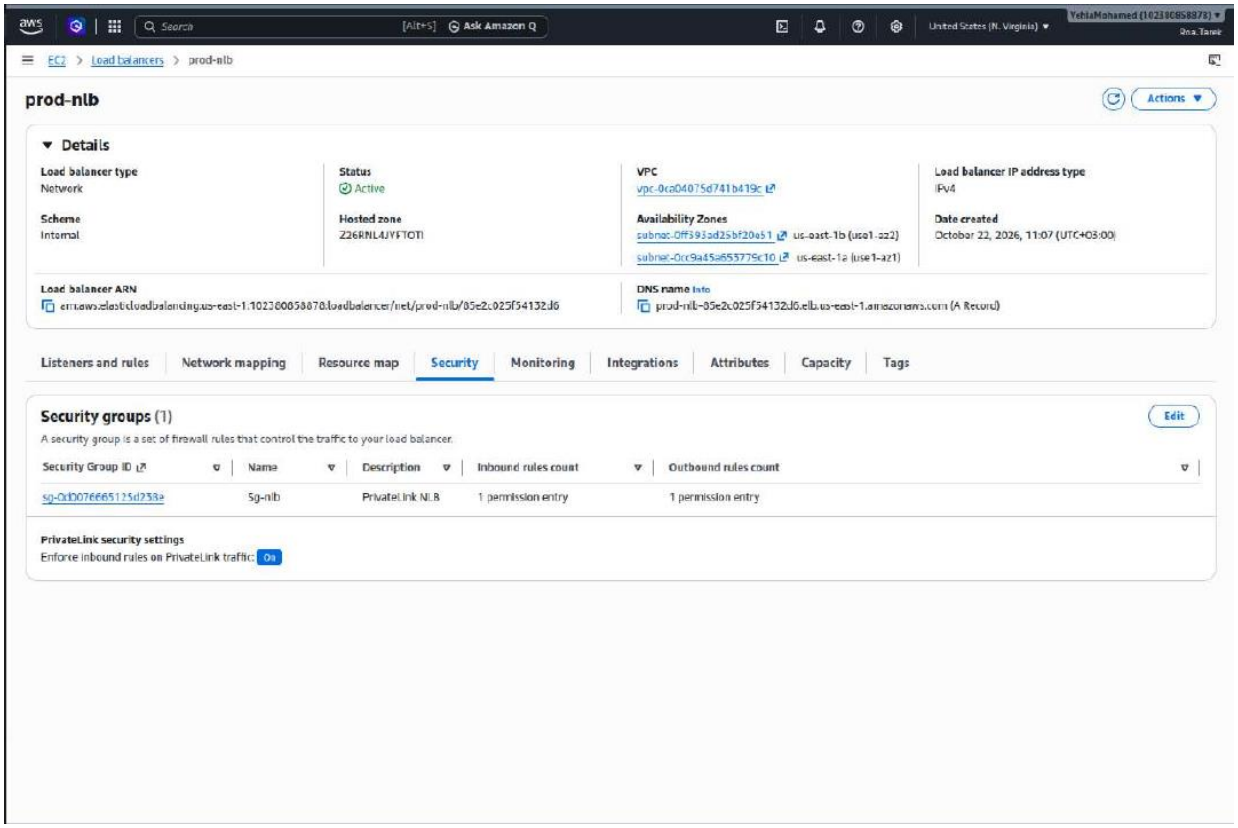


Figure 91. prod-nlb Security tab: Sg-nlb attached, Enforce inbound rules on PrivateLink traffic On.

10. T24: AWS Direct Connect concepts

Purpose. A concepts document only. AWS Direct Connect cannot be deployed in this lab because it needs physical colocation, so nothing was built. The Site-to-Site VPN and Client VPN work in T34 to T39 covers the hybrid connectivity that is actually demonstrated. The full source document is T24_AWS_Direct_Connect_Concepts.docx; its content is reproduced below with an added diagram.

10.1 What it is

AWS Direct Connect (DX) is a private, dedicated network link between an on-premises site and AWS. Traffic travels over the provider's physical circuit and does not cross the public internet. This gives more consistent latency and throughput than an internet-based VPN, and data transfer out is billed at a lower rate than regular internet egress.

10.2 Connection types

Type	Speeds	Who provisions it
Dedicated connection	1, 10 or 100 Gbps (400 Gbps in selected locations)	Requested directly from AWS, one physical port per customer
Hosted connection	Lower speeds, starting at 50 Mbps	An AWS Direct Connect Partner provisions it on the partner's own port

Dedicated connections suit large, steady workloads. Hosted connections suit smaller sites that do not need a full port.

10.3 Setup process

- Request a connection in the Direct Connect console: choose the DX location, port speed and name.
- AWS issues a LOA-CFA (Letter of Authorization and Connecting Facility Assignment). It authorizes the cross-connect and names the AWS port.
- The customer, or the colocation or network provider, gives the LOA-CFA to the facility, which builds the cross-connect between the customer's router and the AWS port.
- Once the link is up, create one or more virtual interfaces (VIFs) on it.
- Configure BGP on the customer router and attach the VIF to the AWS side (virtual private gateway, Direct Connect gateway or transit gateway).
- Test connectivity, then add monitoring and a backup path.

10.4 Virtual interfaces

A single physical connection carries several logical VIFs, each one a VLAN with its own BGP session.

VIF type	Reaches	Typical use
Private VIF	One VPC through its virtual private gateway, or many VPCs through a Direct Connect gateway	Private access to workloads using private IPs
Public VIF	AWS public services (S3, DynamoDB and others) using public IPs	Bulk data transfer to public endpoints without the internet
Transit VIF	A Transit Gateway through a Direct Connect gateway	Many VPCs and accounts through one attachment

10.5 Resilience and scale

- LAG (link aggregation group): bundles up to 4 connections of the same speed at the same location into one logical link. It adds bandwidth and survives the loss of one member link.

- Direct Connect gateway: a global resource that connects a VIF to VPCs in multiple Regions and accounts, so one connection reaches many VPCs.
- Redundant connections: AWS recommends connections at two or more locations for high availability. Maximum-resilience designs use separate devices at separate locations.
- Failover: BGP with BFD detects failures quickly. A Site-to-Site VPN is a common backup path.
- Encryption: DX is private but not encrypted by default. Options are MACsec on supported 10 and 100 Gbps dedicated connections, or an IPsec VPN over a public or transit VIF.

10.6 Direct Connect compared with VPN

	Direct Connect	Site-to-Site VPN
Path	Dedicated private circuit	Encrypted tunnels over the internet
Latency	Consistent and low	Variable
Throughput	Up to 100 Gbps per connection	Up to about 1.25 Gbps per tunnel
Encryption	Not by default (MACsec or VPN over DX optional)	IPsec built in
Time to provision	Days to weeks, needs colocation or a partner	Minutes
Cost	Port-hour charges, cross-connect fees, data transfer out	Hourly connection fee plus data transfer

In practice the two are combined: DX is the primary path and the VPN is the encrypted backup.

10.7 Why it is not deployed here

- It requires physical colocation or a partner circuit, which this lab does not have.
- Port-hour charges start as soon as the connection is available, which does not fit the project budget.
- The Site-to-Site VPN and Client VPN in T34 to T39 demonstrate the same hybrid-connectivity pattern at a small, deletable cost.

10.8 How it would fit this design

The transit gateway is already the hub for the VPCs, so one transit VIF would give on-premises users access to both VPCs without a separate attachment per VPC (Figure 92). The existing security controls would still apply:

- Security groups on the app tier (Sg-ec2, Sg-rds) would need rules for the on-premises CIDR (192.168.0.0/16 in this project).
- nacl-private and nacl-db would need matching inbound and outbound rules. Rule #50 stays reserved for the IR deny.
- A Site-to-Site VPN over the same transit gateway would be the backup path.

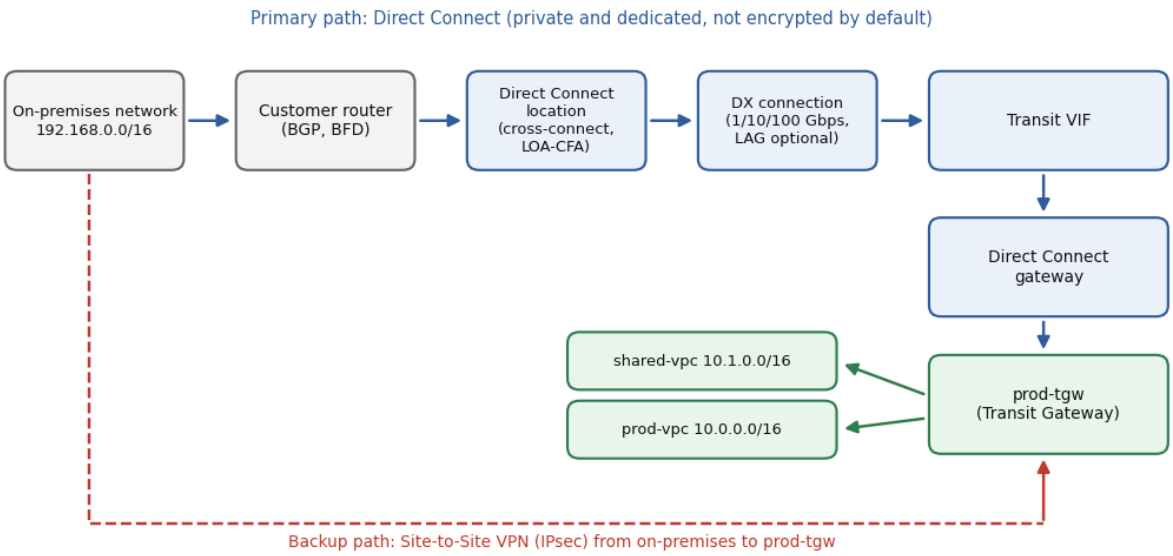


Figure 92. Direct Connect path to the transit gateway with the Site-to-Site VPN as backup (concept only, not deployed).

Result. Completed as a concepts document. No AWS resources were created. Source: AWS Direct Connect User Guide and pricing page; check current prices and speeds there before quoting numbers in the final report (T67).

11. Resource inventory for cleanup (T26 and T68)

Yehia is doing the cleanup. This table lists what Roa created or changed, so nothing is missed. Take a before-screenshot of anything that bills before deleting it. Delete in the order below, because AWS blocks deletion while a resource is still in use.

Order	Resource	Billing	Notes
1	Consumer endpoint shared-privatelink-endpoint-2 (shared-vpc)	Hourly	Delete first so no connection remains on the service.
2	Endpoint service prod-privatelink-service	None	If blocked, reject the connection first.
3	Network Load Balancer prod-nlb	Hourly	Deletion protection is off. Leave prod-alb alone until T68.
4	Target group tg-privatelink	None	Leave prod-tg alone until T68.
5	Interface endpoints prod-logs-endpoint, prod-kms-endpoint, prod-secretsmanager-endpoint	Hourly per AZ	The plan deletes these in T26. They were created by Shrouk (T29).
6	Interface endpoints shared-ssm-endpoint, shared-ssmmessages-endpoint, shared-ec2messages-endpoint	Hourly per AZ	Not in the original plan. They keep shared-bastion on SSM, so delete only when no more tests need the bastion.
7	Security groups Sg-nlb and Sg-shared-endpoint	None	Wait 5 to 10 minutes after the endpoints and NLB are gone, then retry if AWS says they are in use.
T68	Kept until the final report: S3 gateway endpoint and prod SSM, SSM messages and EC2 messages endpoints, prod-alb, prod-tg,	Hourly (NAT, ALB, RDS, endpoints)	Deleted in the team cleanup, after T67.

	EC2s, RDS, NAT Gateway		
T68	Security groups Sg-alb, Sg-ec2, Sg-rds, Sg-nat, Sg-bastion, Sg-endpoints, Sg-shared-bastion, and the leftover launch-wizard groups	None	After EC2, ALB, RDS and endpoints are gone.
T68	NACLs nacl-public, nacl-private, nacl-db	None	Subnets must be moved to the default NACL or deleted first. nacl-private also holds Yehia's rule #50 deny.
T68	IAM role ec2-ssm-role	None	After the EC2 instances are terminated.
T68	CloudTrail project-trail and S3 bucket aws-cloudtrail-logs-102380858878-152d4262	Small (S3)	Take a screenshot, empty the bucket, delete the bucket, then the trail.
T68	Security Hub, GuardDuty, AWS Config recorder	Config bills per item	Disable Security Hub first, then stop the Config recorder, then disable GuardDuty. The 30-day trials are free.
T68	Reachability Analyzer saved paths (5)	None	Optional tidying.

Not on this list: Yehia's own resources (Lambda, EventBridge, SNS, flow logs bucket, snapshots, Traffic Mirror) and Mariam's edge resources (CloudFront, WAF, Route 53, S3), which their owners list in their own handovers.

SECTION 3 — WORKSTREAM 3 (TASKS T27 – T41) — FULL UNABRIDGED DOCUMENTATION & ALL 66 FIGURES

Shrouk (with Yehia) — VPC Peering, 10 VPC Endpoints, Transit Gateway, Site-to-Site VPN & Dual-Method Client VPN

Connectivity & Hybrid Engineers: Shrouk Tarek Negeda & Yehia Mohamed

Part 3A: Shrouk's Complete Connectivity & Hybrid Report (Tasks T27 – T41 — All 31 Pages & 57 Screenshots)

Connectivity & Hybrid Report

Secure AWS Network Architecture Project – Tasks T27 to T41

Prepared by: Shrouk Tarek Negeda

Role: Connectivity & Hybrid

Region: us-east-1 (N. Virginia)

Date: October 21–29, 2026

1. Executive Summary

This report documents tasks T27–T41 (Connectivity & Hybrid) of the Secure AWS Network Architecture project, executed in us-east-1 by Shrouk. It covers VPC Peering, routing, VPC Endpoints, connectivity testing, Transit Gateway, Site-to-Site VPN and Client VPN, followed by the cleanup of all cost-incurring resources.

Outcome: peering, endpoints, SSM access, S3 gateway access and Transit Gateway routing were deployed and verified. The Site-to-Site VPN was created and verified as configured (tunnels Down by design, because the customer gateway is simulated). The Client VPN connection was established in the first attempt but the reachability test failed; a second run in the rebuilt lab environment (October 29) succeeded: ping, HTTP 200 and browser access to the private production server through the tunnel (Section 8).

Key resources

Resource Identifier

prod-vpc vpc-0ca04075d741b419c (10.0.0.0/16)

shared-vpc vpc-003ca8e3a7946afa7 (10.1.0.0/16)

VPC Peering pcx-0d04890e8e329cd6b (prod-shared-peering)

S3 Gateway Endpoint vpce-01e5a625381b57b05

Transit Gateway tgw-0c89610214e95caf7 (prod-tgw, ASN 64512)

Virtual Private Gateway vgw-07fce58d0fe0550b7 (prod-vgw)

Site-to-Site VPN vpn-0ce8c4e95b3394a29 (prod-vpn, static 192.168.0.0/16)

Client VPN endpoint cvpn-endpoint-0a432f08d863be3f5 (client CIDR 10.100.0.0/22)

2. T27 – VPC Peering

A peering connection named prod-shared-peering was created with prod-vpc as requester and shared-vpc as accepter (same account, same Region), then accepted. Its status became Active.

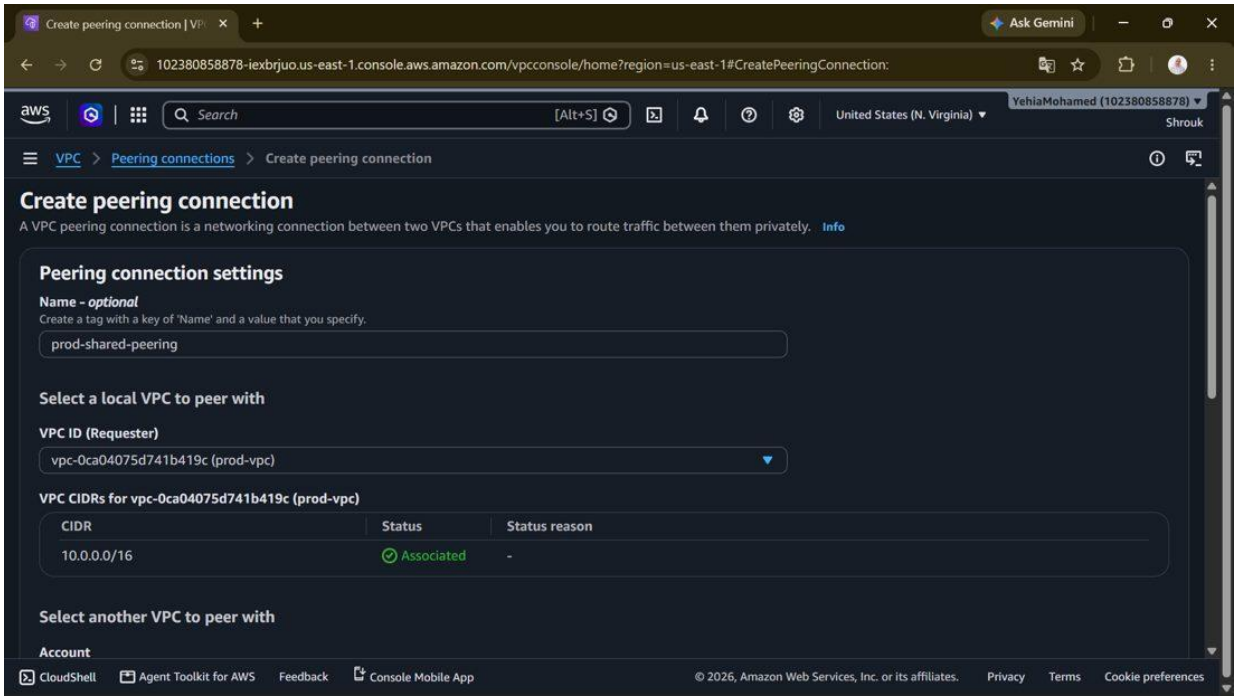


Figure: Peering settings: requester prod-vpc (10.0.0.0/16).

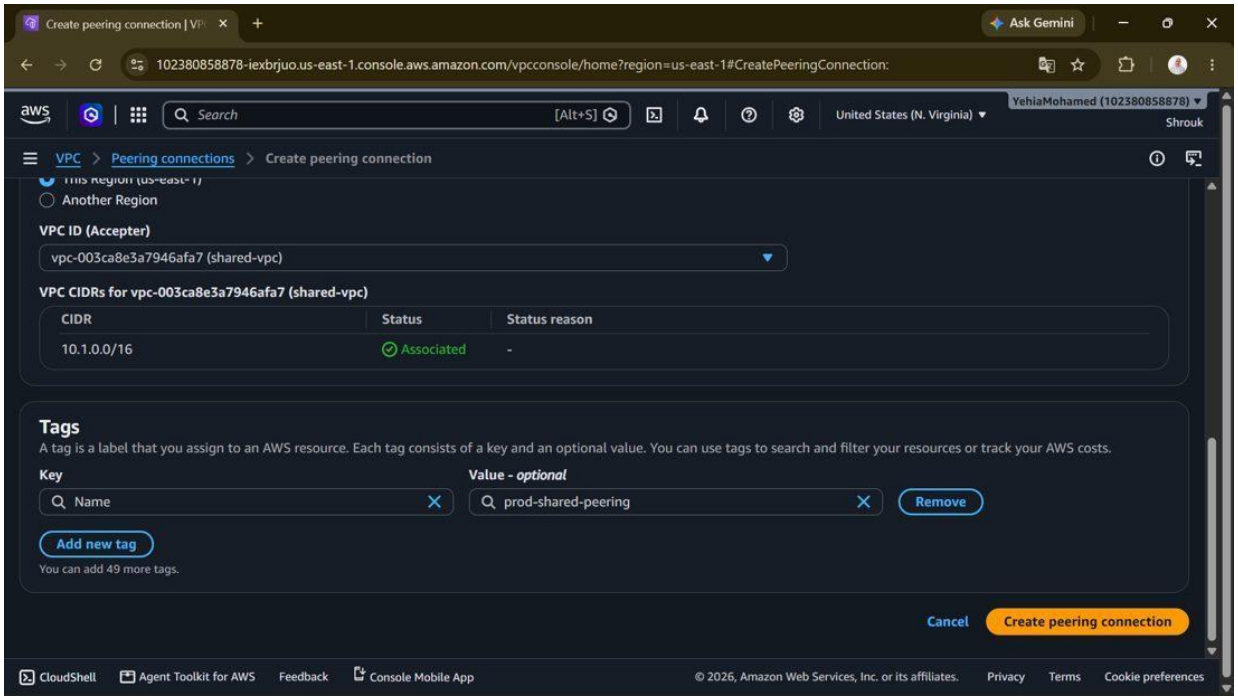


Figure: Accepter shared-vpc (10.1.0.0/16) and Name tag.

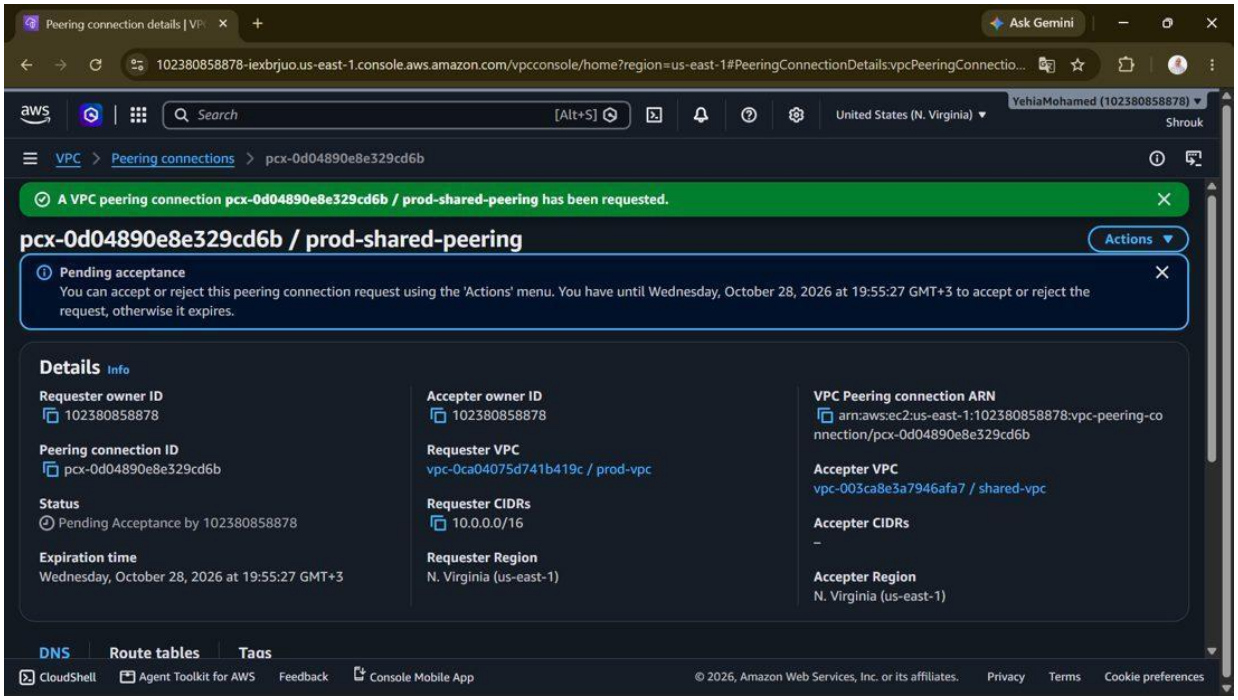


Figure: Request created: Pending acceptance.

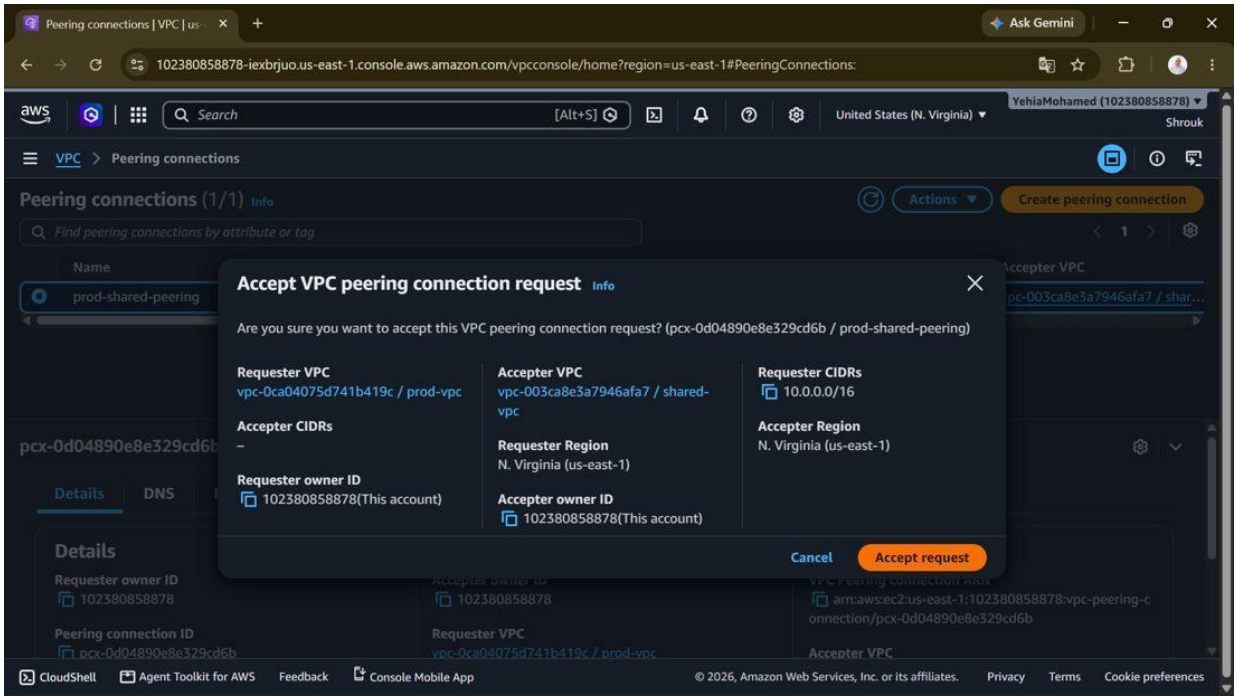


Figure: Accept VPC peering connection request.

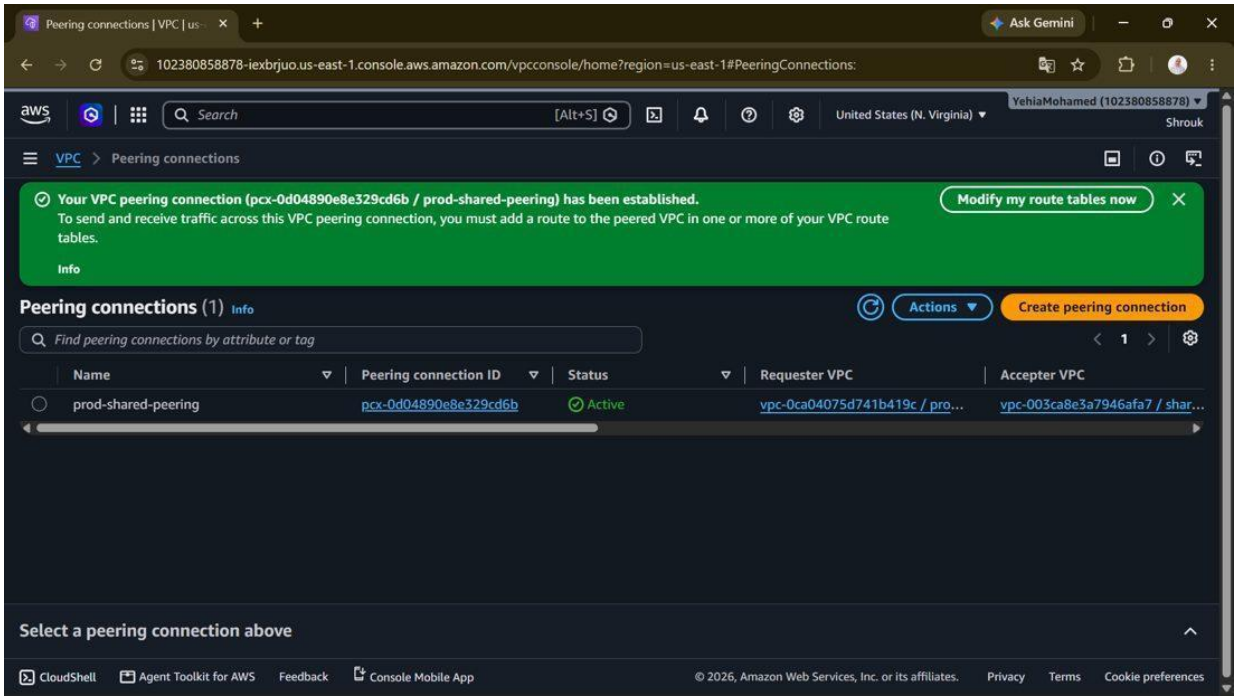


Figure: Peering connection status: Active.

3. T28 – Route Tables for Peering

Routes were added so traffic can cross the peering: 10.1.0.0/16 → pcx in prod-vpc (rt-private and rt-public) and 10.0.0.0/16 → pcx in rt-shared. rt-db was intentionally left unchanged because the database tier has no route outside the VPC.

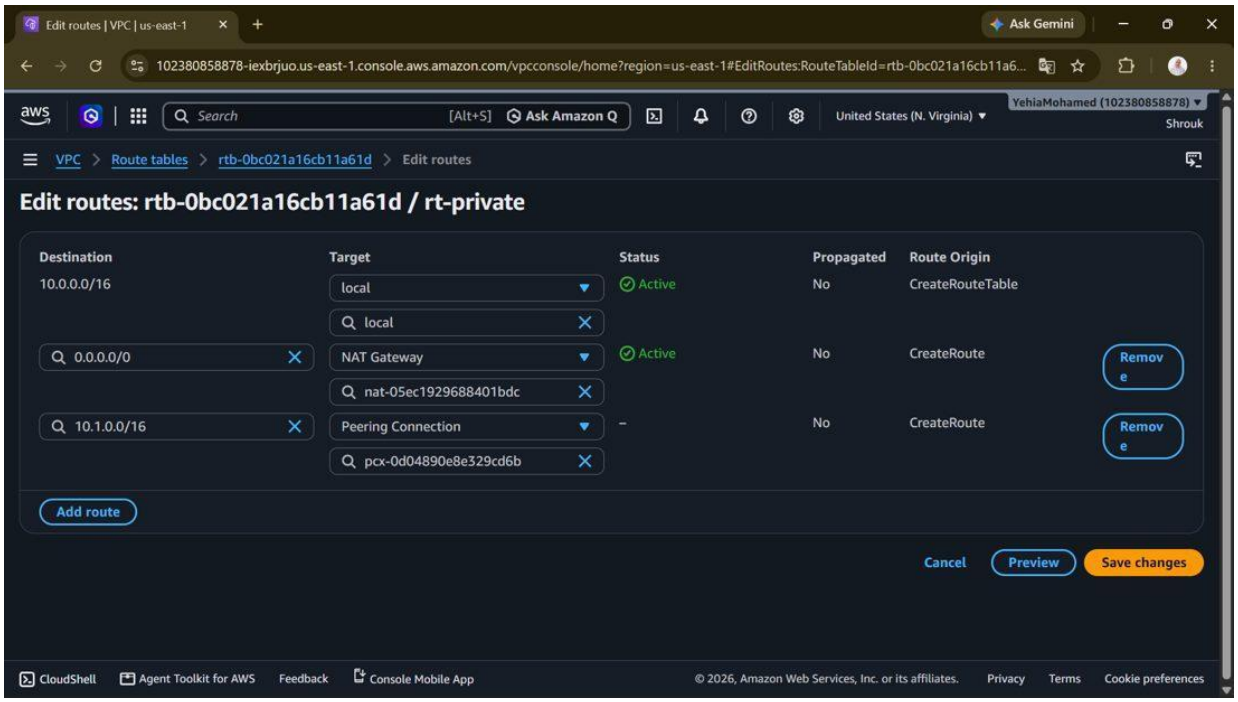


Figure: rt-private: adding 10.1.0.0/16 → peering connection.

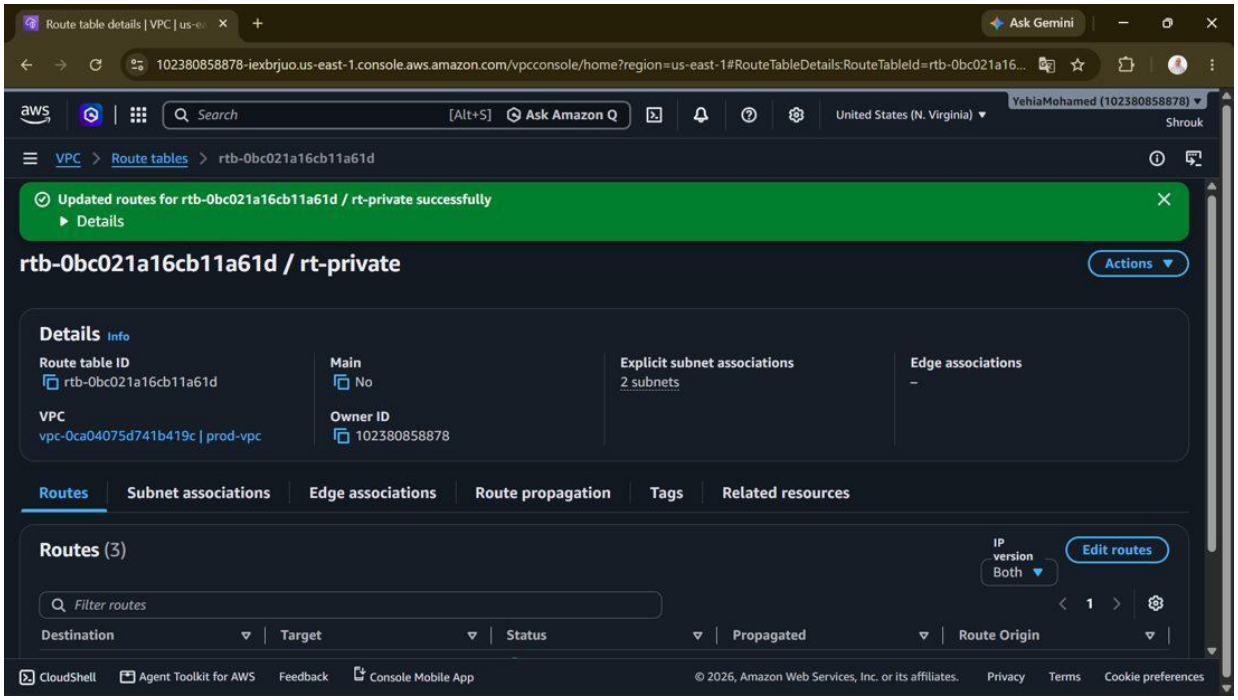


Figure: rt-private: routes updated successfully.

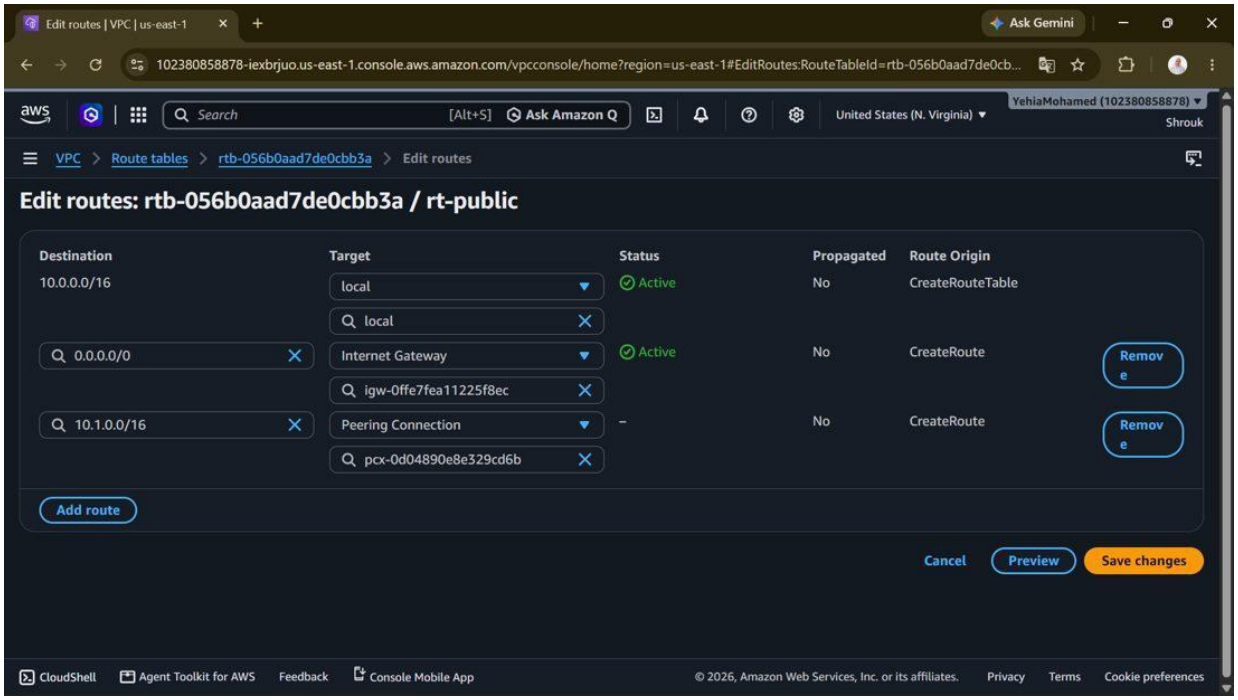


Figure: rt-public: adding the peering route.

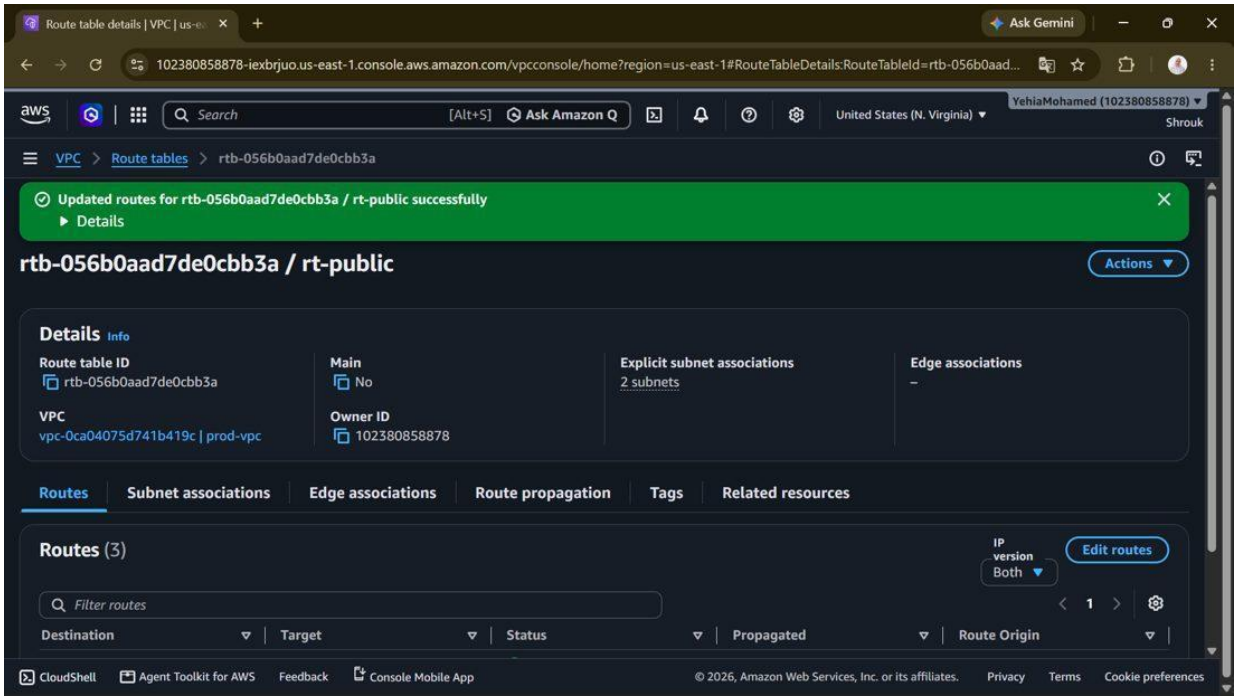


Figure: rt-public: routes updated successfully.

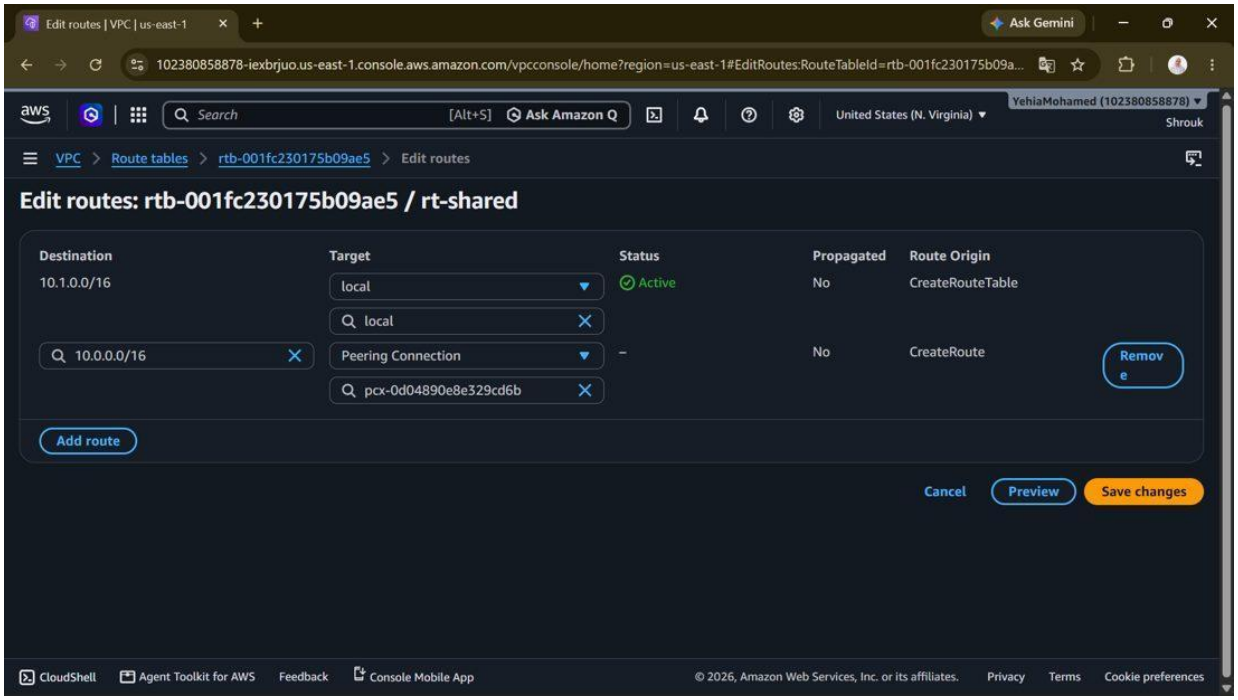


Figure: rt-shared: adding 10.0.0.0/16 → peering connection.

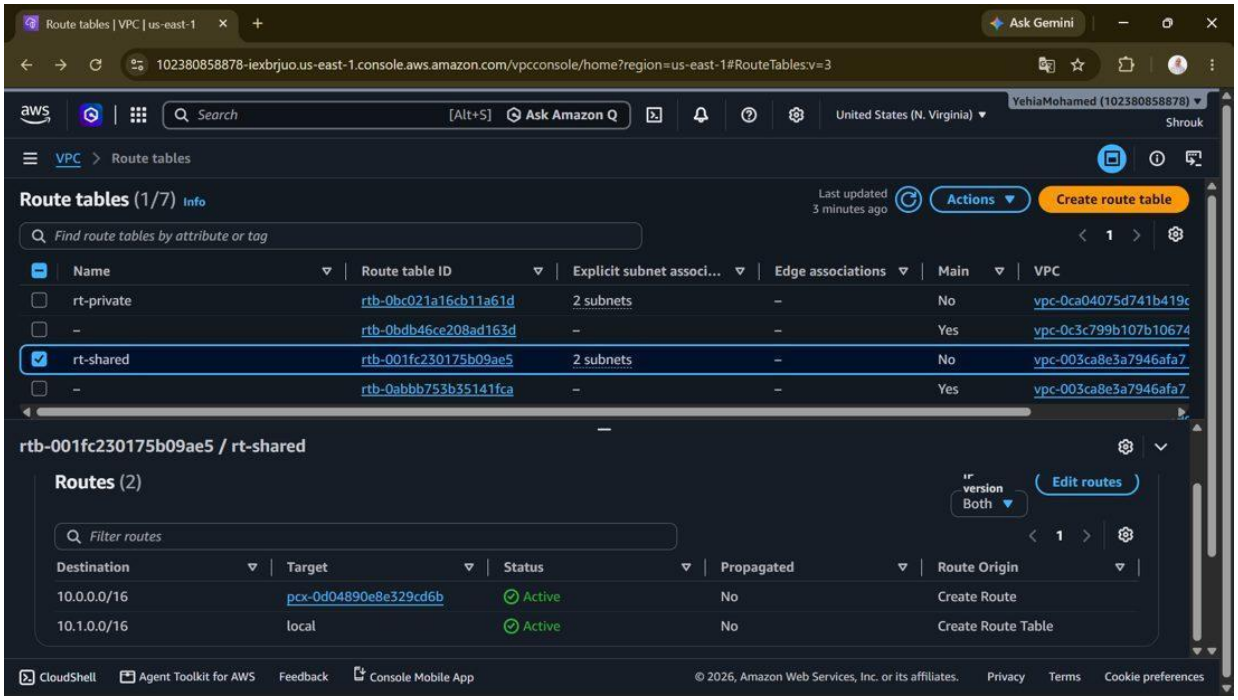


Figure: rt-shared routes: 10.0.0.0/16 → pcx (Active) and local.

4. T29 – VPC Endpoints

4.1 S3 Gateway Endpoint

A Gateway endpoint for com.amazonaws.us-east-1.s3 was created in prod-vpc and associated with rt-private, rt-db and rt-public (the Main route table has no subnets, so it was skipped). Policy: Full access. Gateway endpoints have no hourly cost.

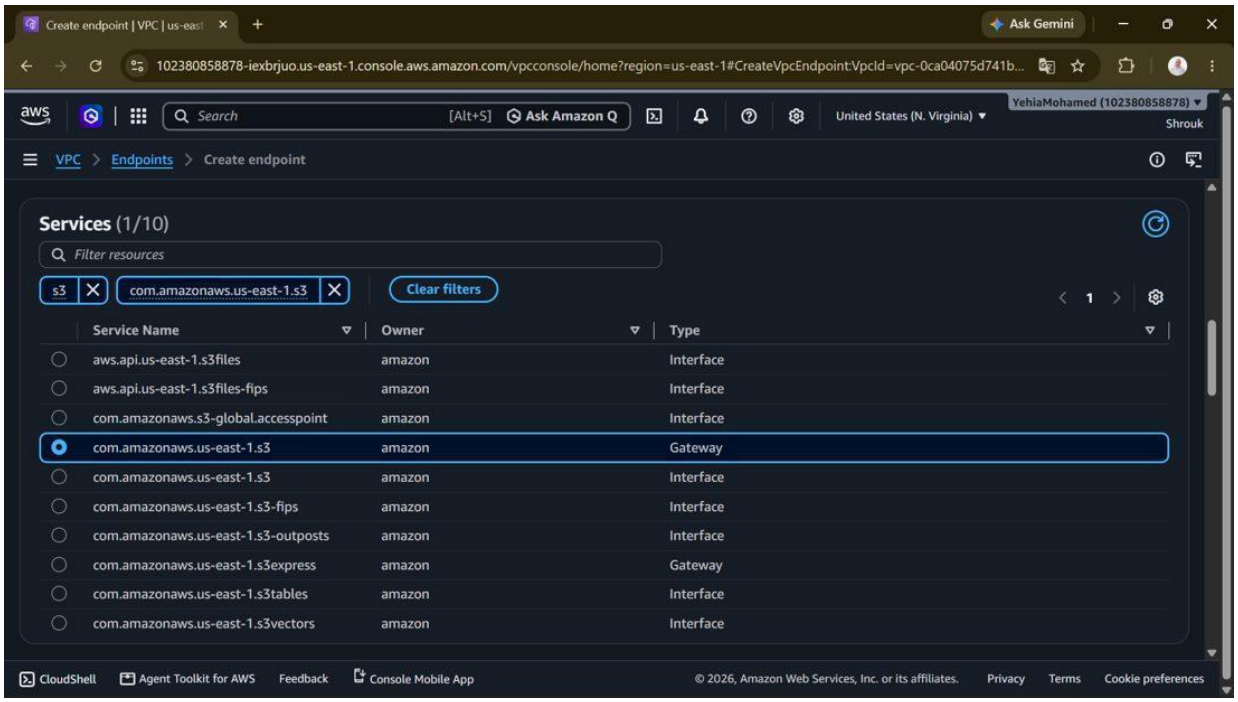


Figure: Selecting the S3 service of type Gateway (not Interface, not s3express).

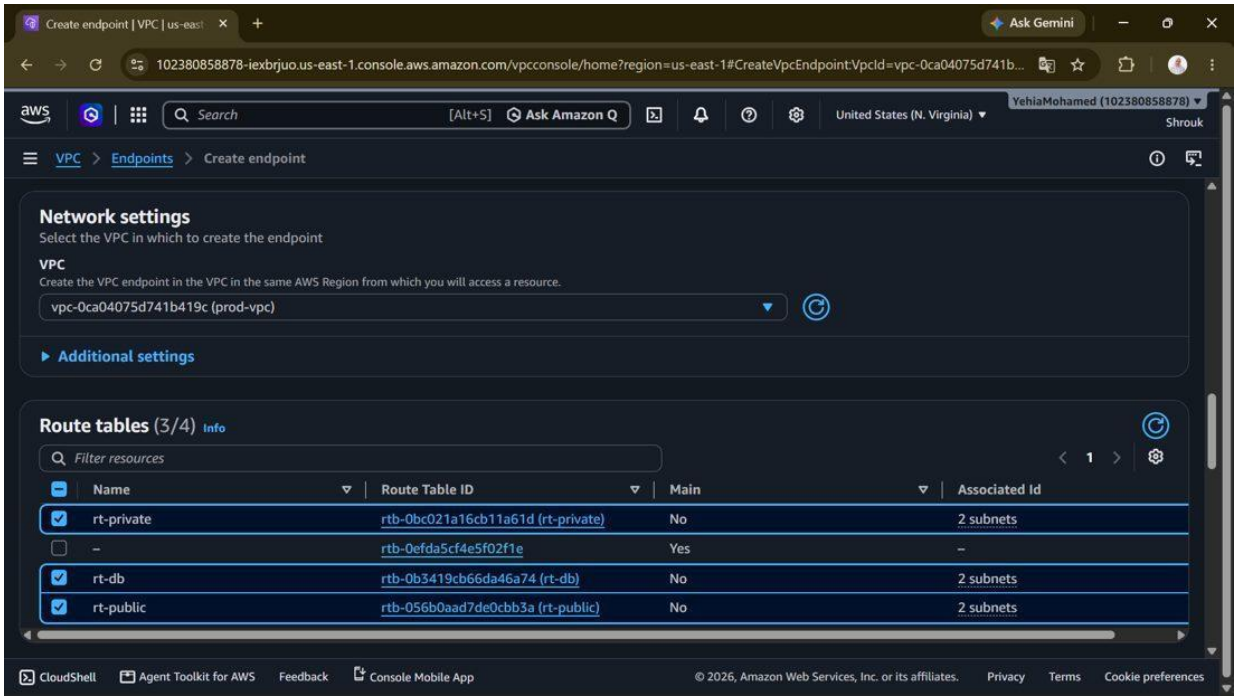


Figure: Route tables selected: rt-private, rt-db, rt-public.

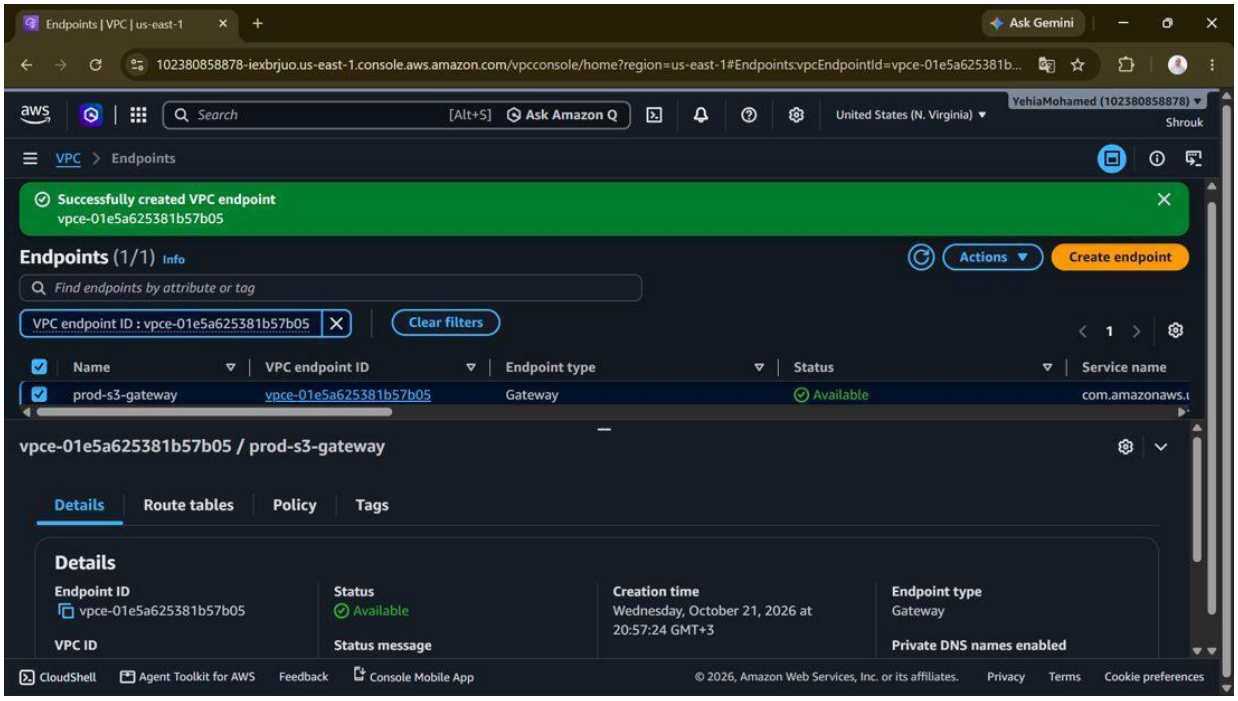


Figure: prod-s3-gateway created: Available.

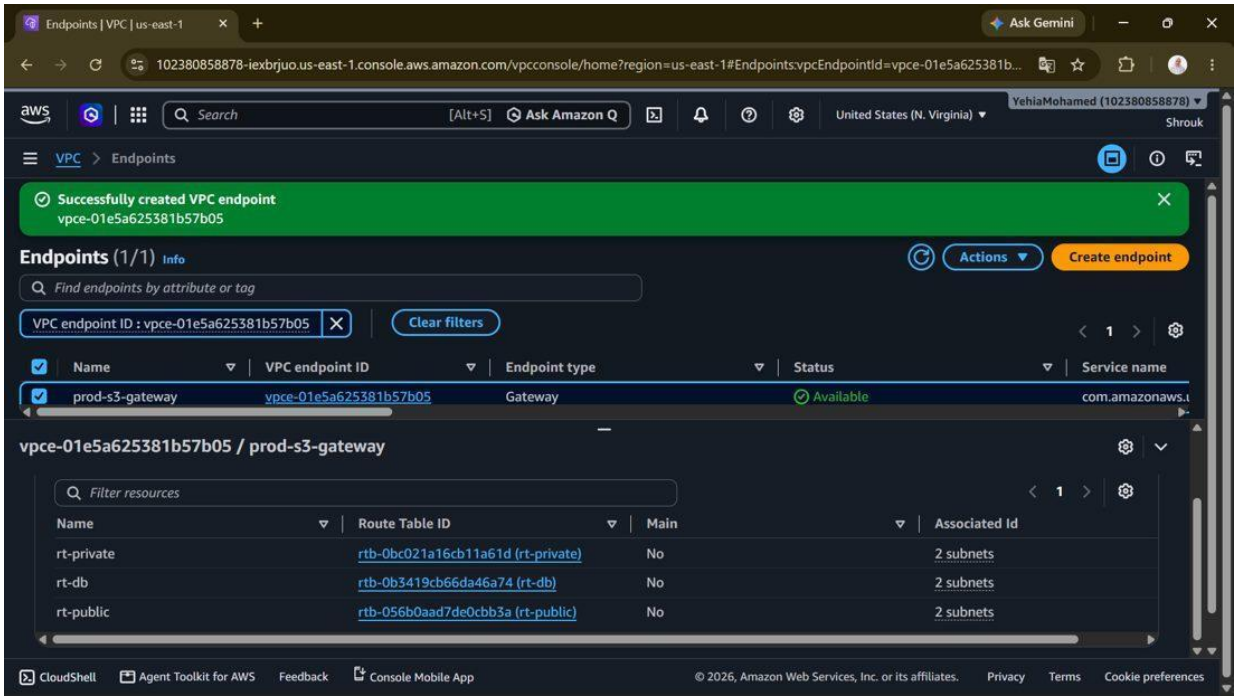


Figure: Route tables tab of the endpoint.

4.2 Interface Endpoints in prod-vpc

Six Interface endpoints (ssm, ssmmessages, ec2messages, logs, kms, secretsmanager)

were created in prod-vpc, in prod-private-a (us-east-1a) and prod-private-b (us-east-1b), with

Private DNS enabled, the Sg-endpoints security group and Full access policy.

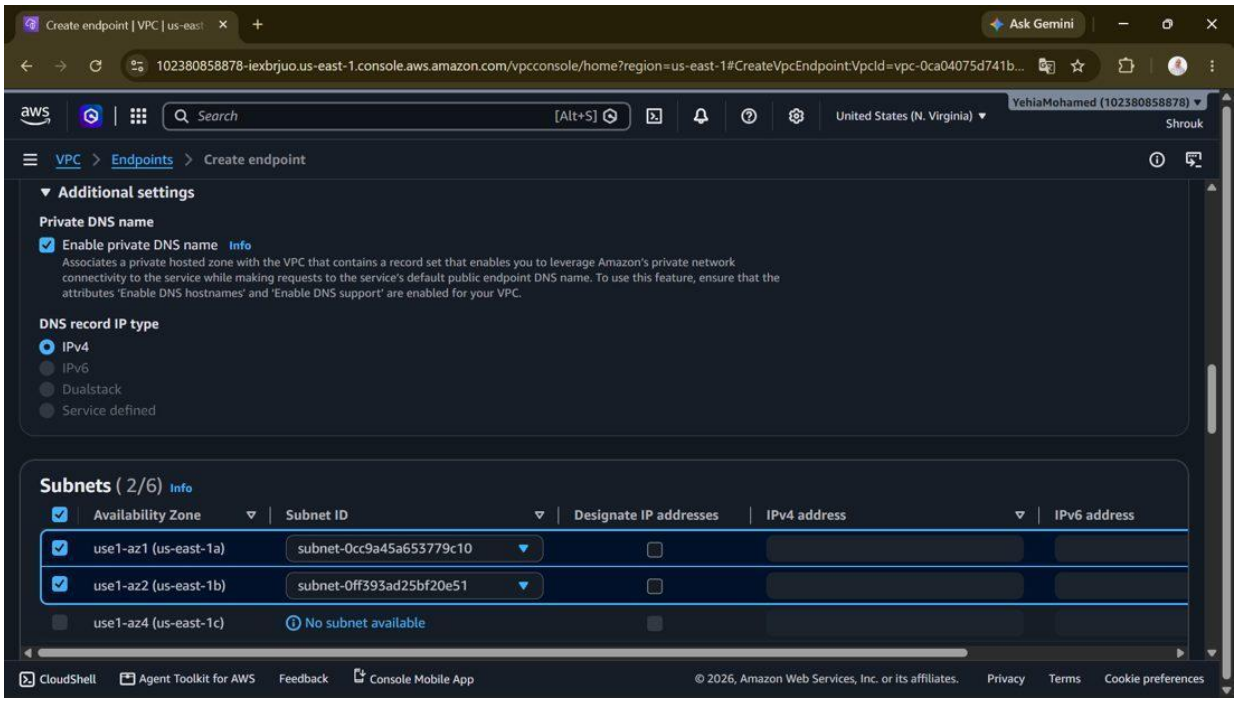


Figure: Example (ssmmessages): Private DNS enabled and the two private subnets.

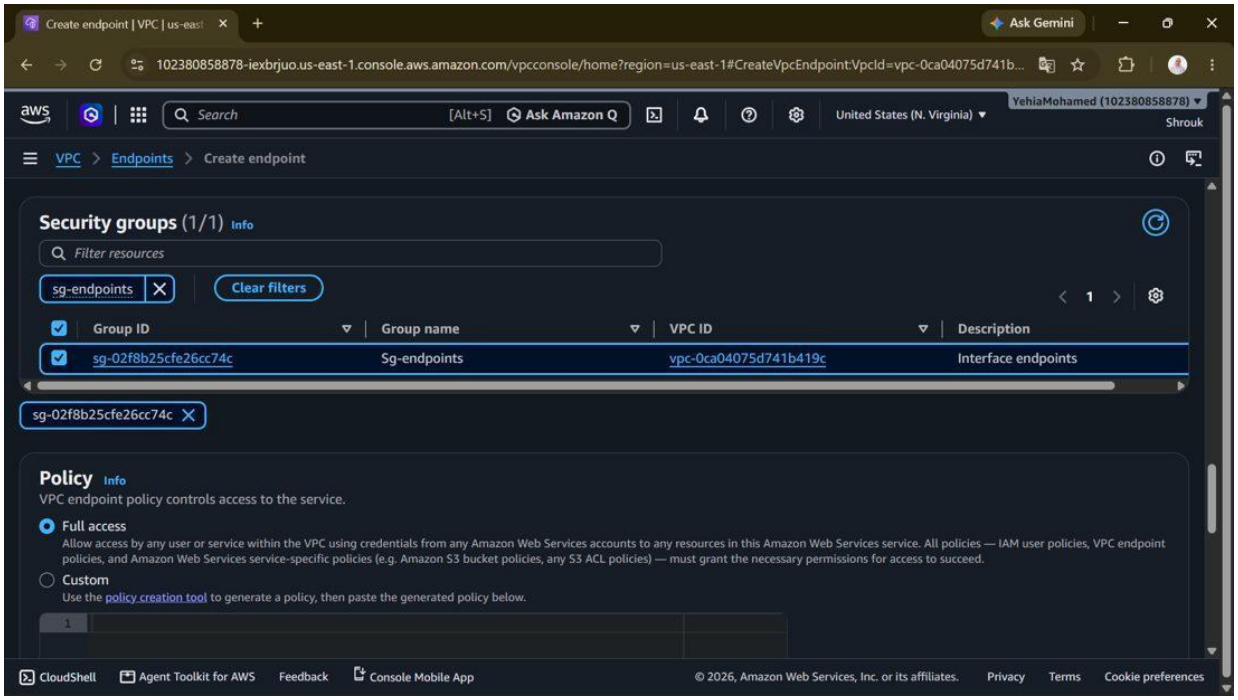


Figure: Example: security group Sg-endpoints selected.

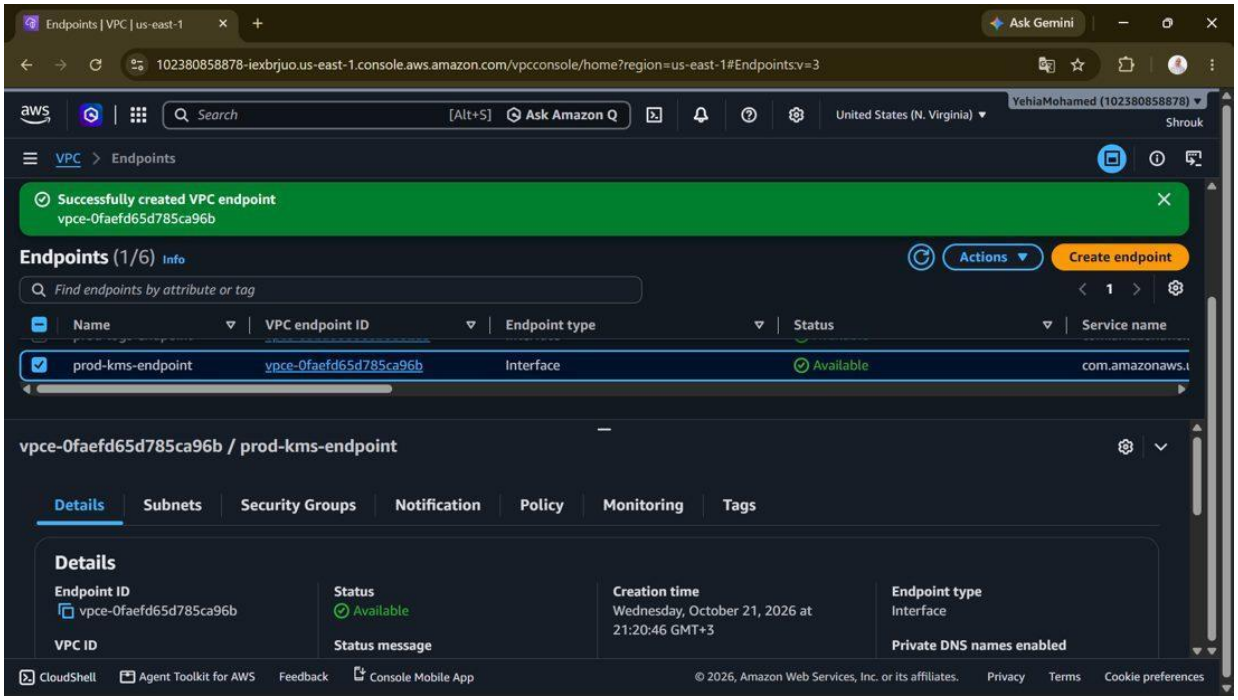


Figure: kms endpoint created: Available.

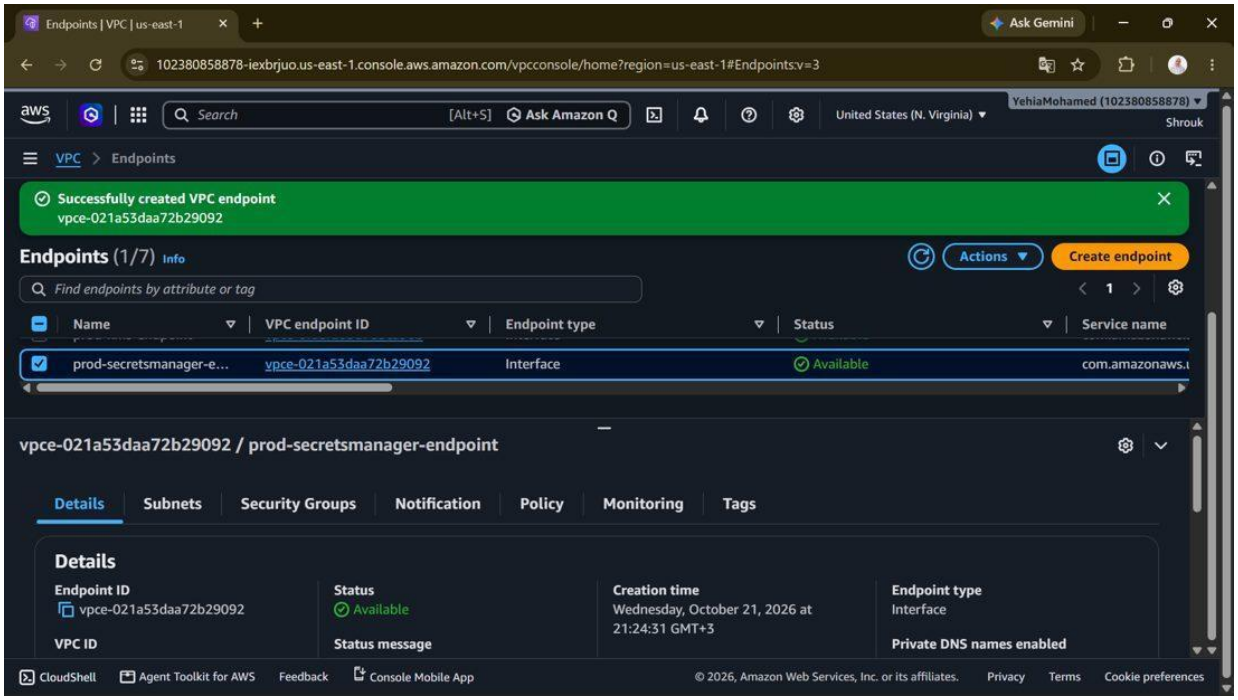


Figure: secretsmanager endpoint created: Available.

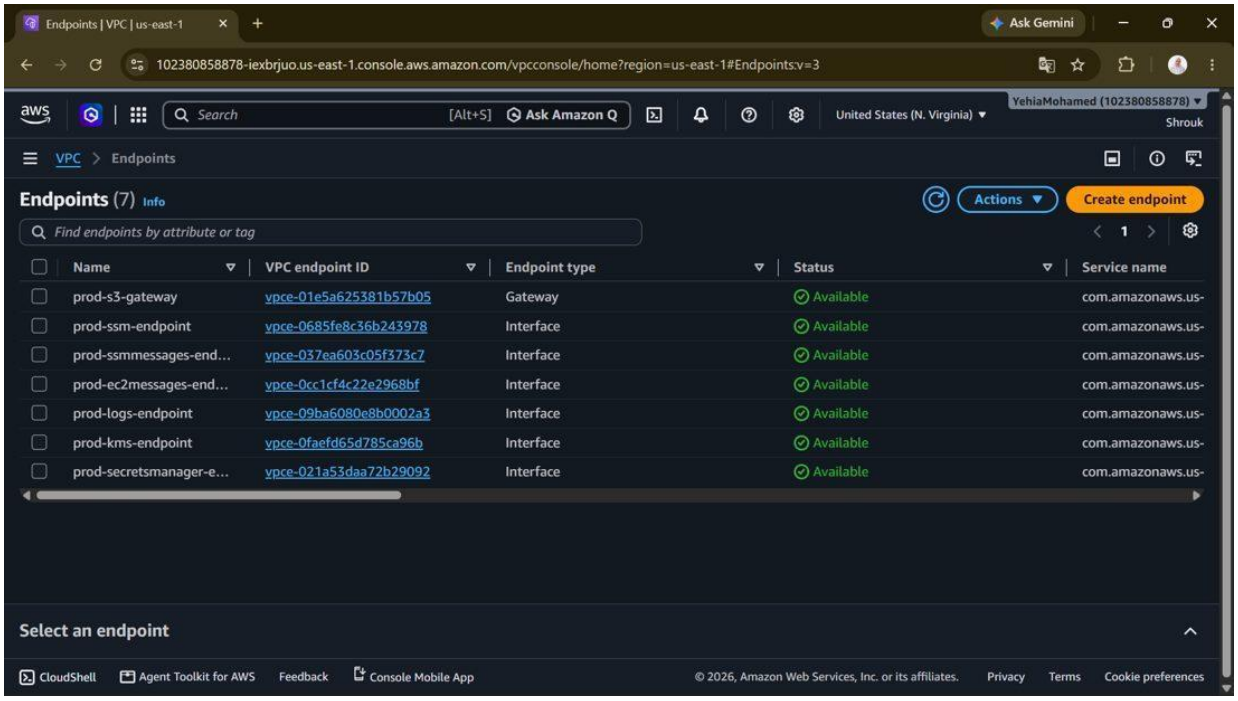


Figure: All 7 endpoints in prod-vpc (1 Gateway + 6 Interface): Available.

4.3 SSM Endpoints in shared-vpc

shared-vpc has no NAT or IGW, so the shared-bastion could not register with Systems Manager. A security group named shared-endpoints-sg (HTTPS 443 from 10.1.0.0/16) was created, and three Interface endpoints (ssm, ssmmessages, ec2messages) were created in shared-private-a/b.

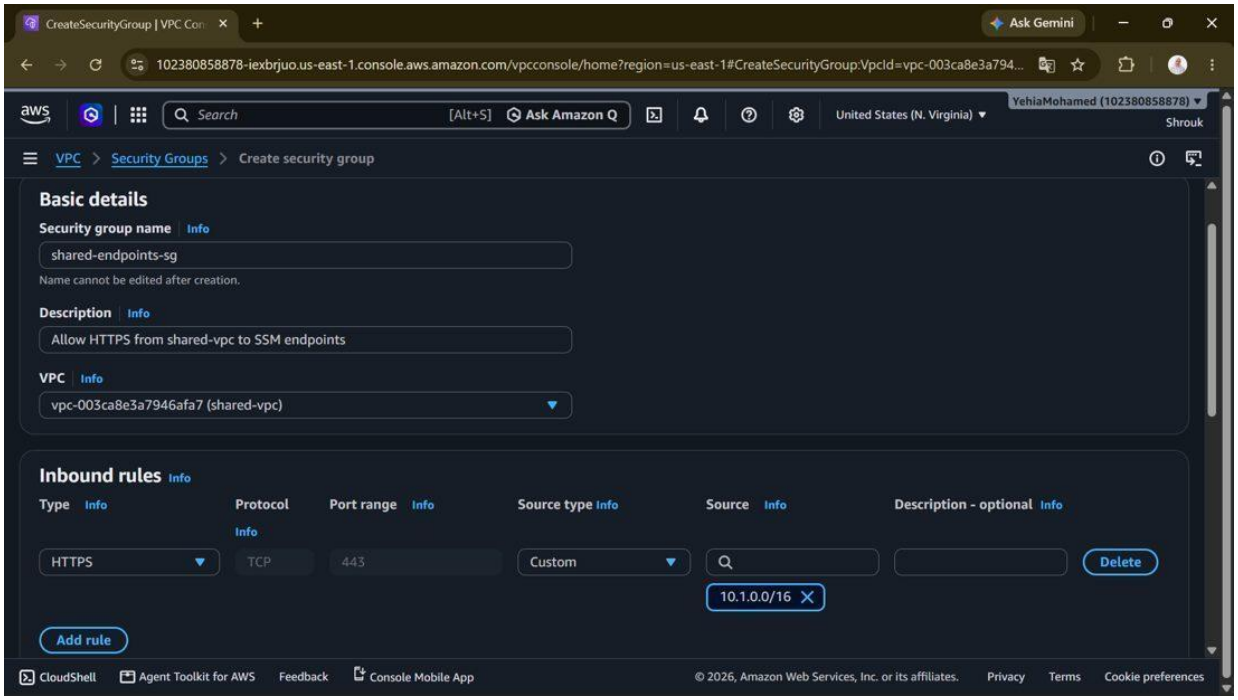


Figure: Creating shared-endpoints-sg in shared-vpc.

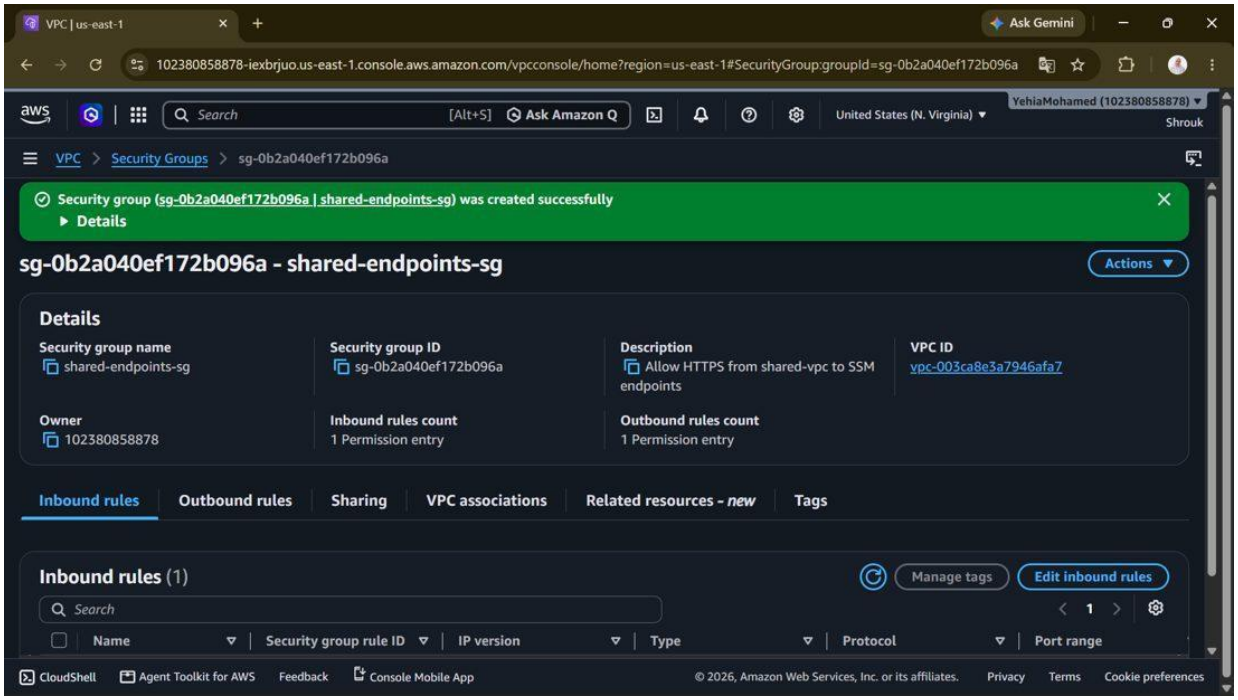


Figure: shared-endpoints-sg created.

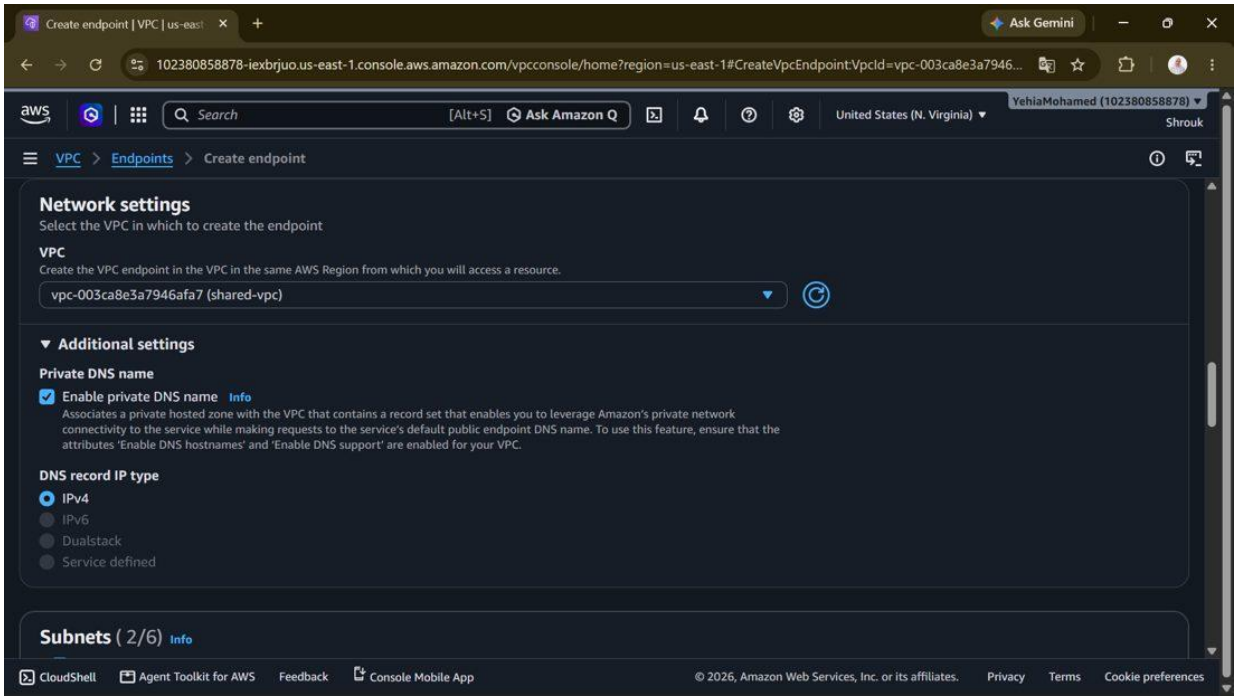


Figure: shared-ssm-endpoint: VPC shared-vpc with Private DNS enabled.

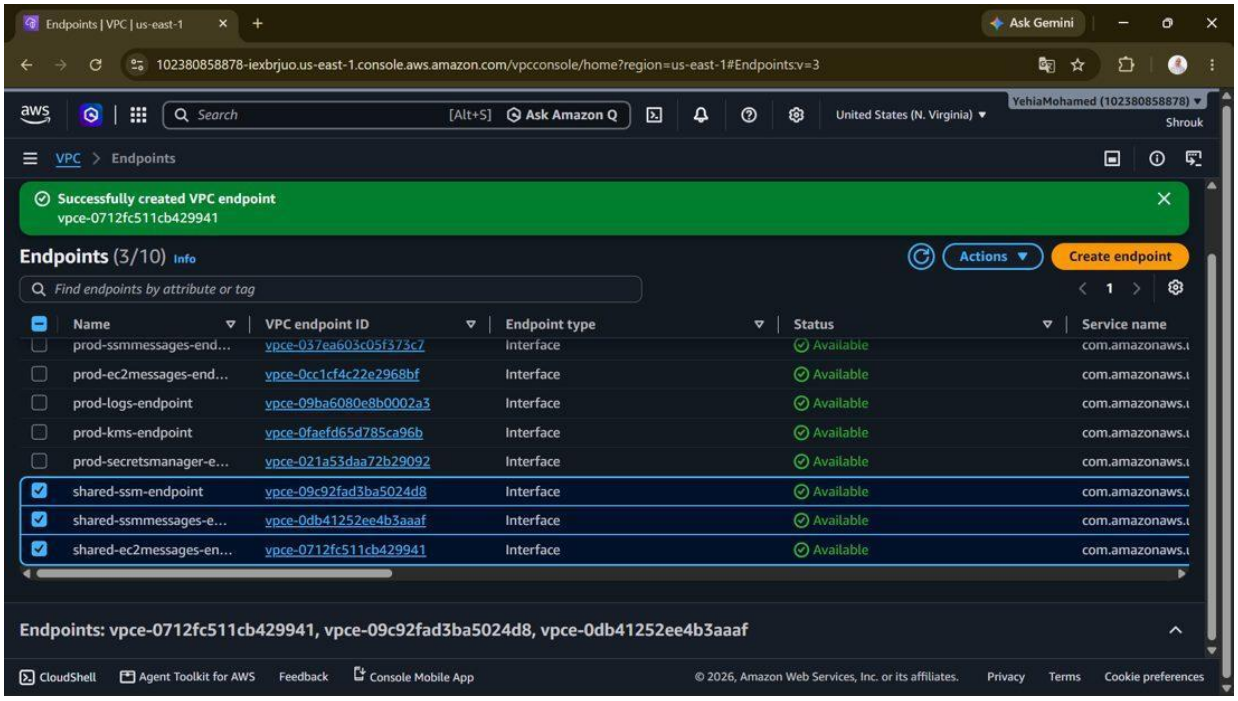


Figure: All 10 endpoints Available (7 in prod-vpc + 3 in shared-vpc).

5. T30 – Peering, Endpoint and S3 Connectivity Tests

The EC2 instances and the shared-bastion were started and registered in Fleet Manager (Online). Tests were then run from the bastion over the peering and from prod-ec2-a through the endpoints.

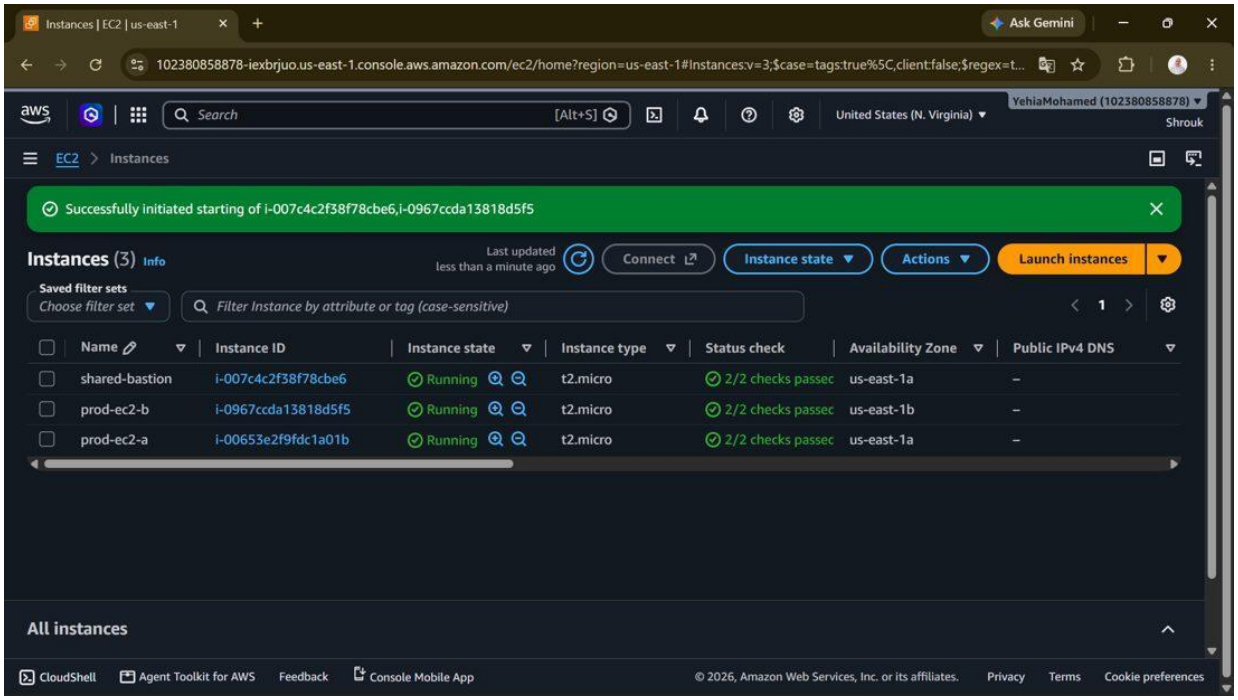


Figure: All three instances running (2/2 status checks).

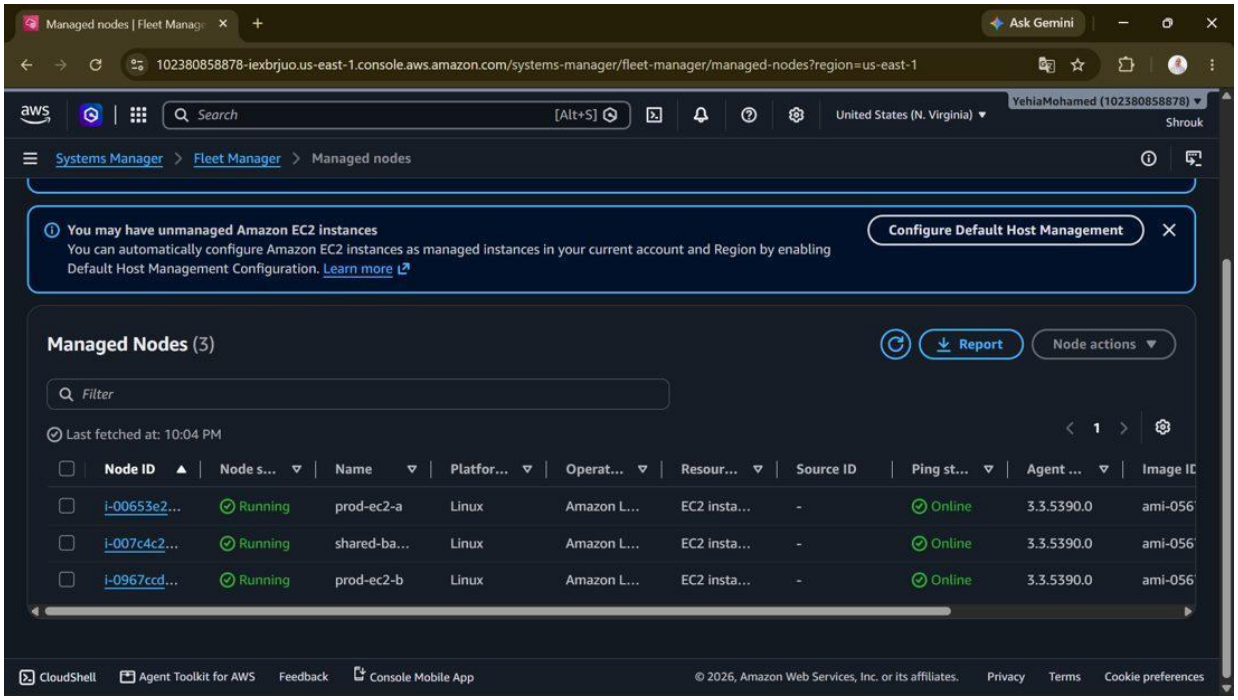


Figure: Fleet Manager: prod-ec2-a, prod-ec2-b and shared-bastion are Online (SSM works through the

endpoints).

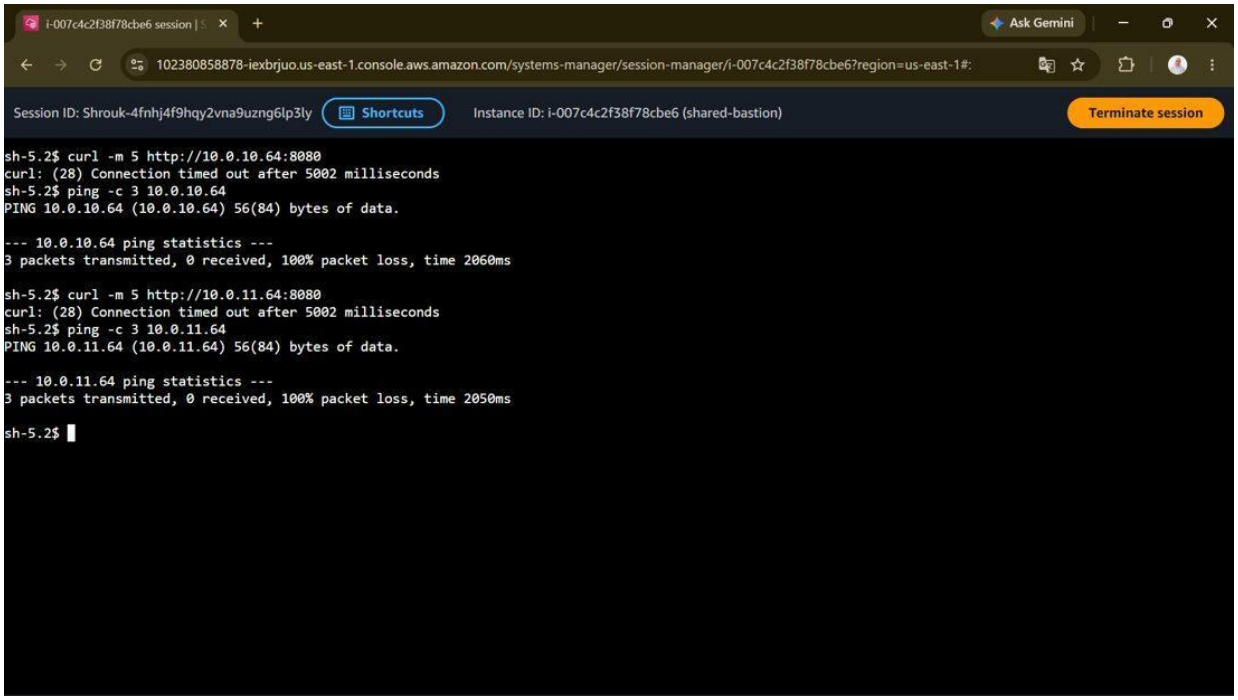


Figure: First attempt: curl from the bastion timed out (root cause in Section 9, issue 2).

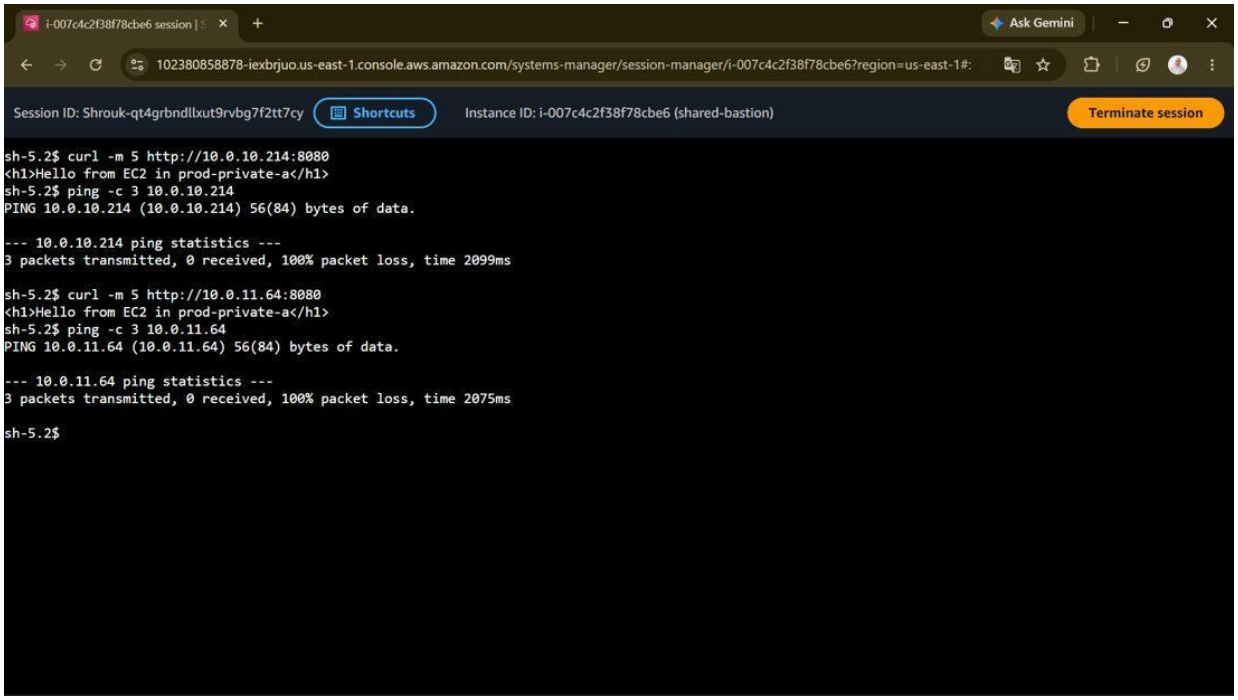


Figure: After the fix: curl returns the application page through the peering. ICMP ping is blocked by design.

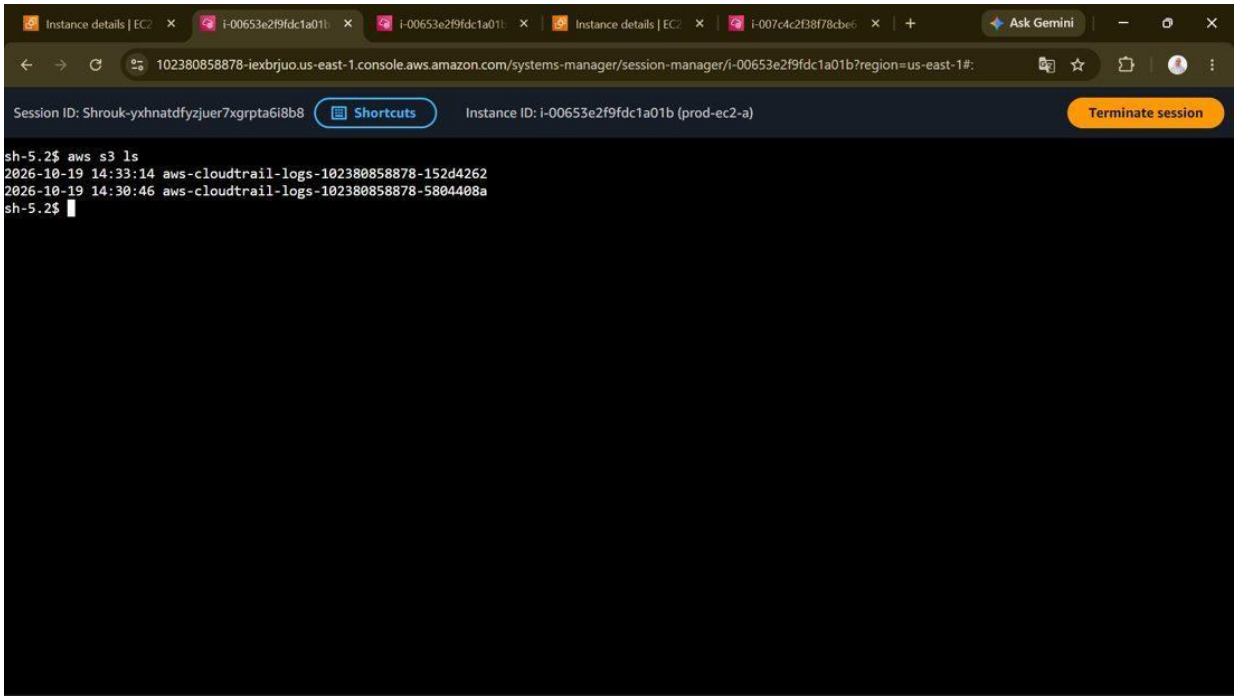


Figure: From prod-ec2-a: aws s3 ls succeeds with no public IP.

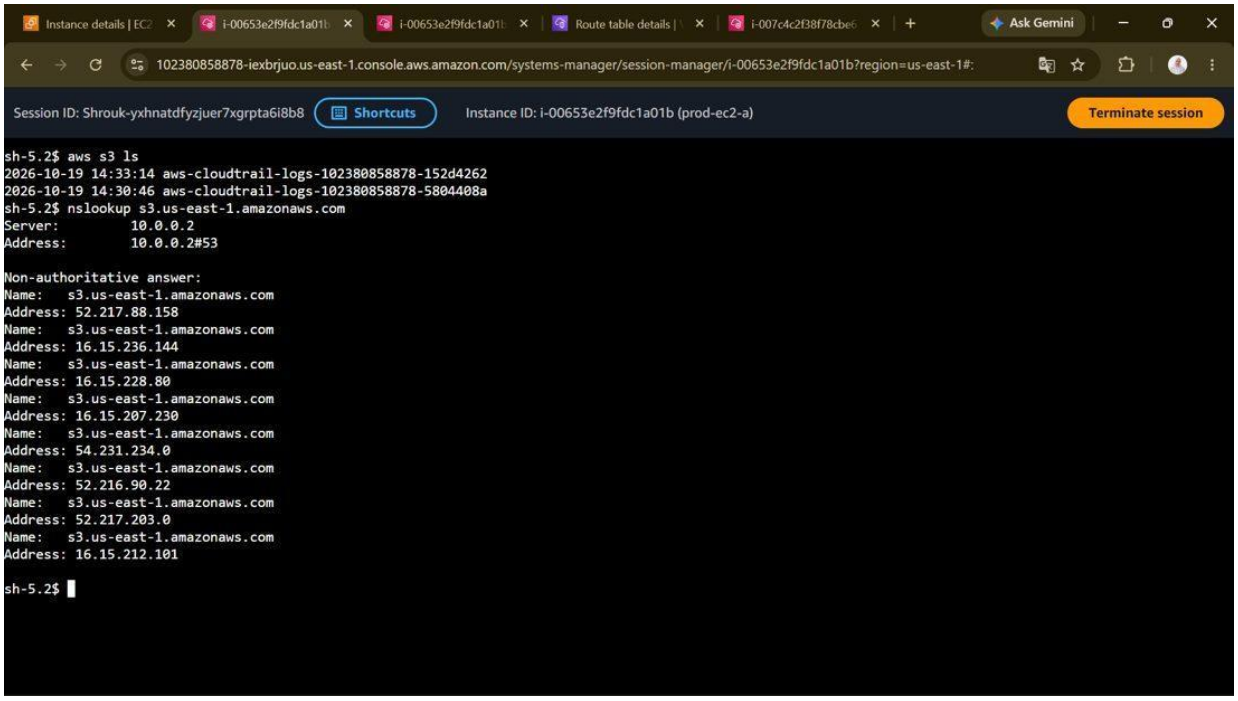


Figure: nslookup of the S3 endpoint from prod-ec2-a (the gateway endpoint works at routing level, not DNS).

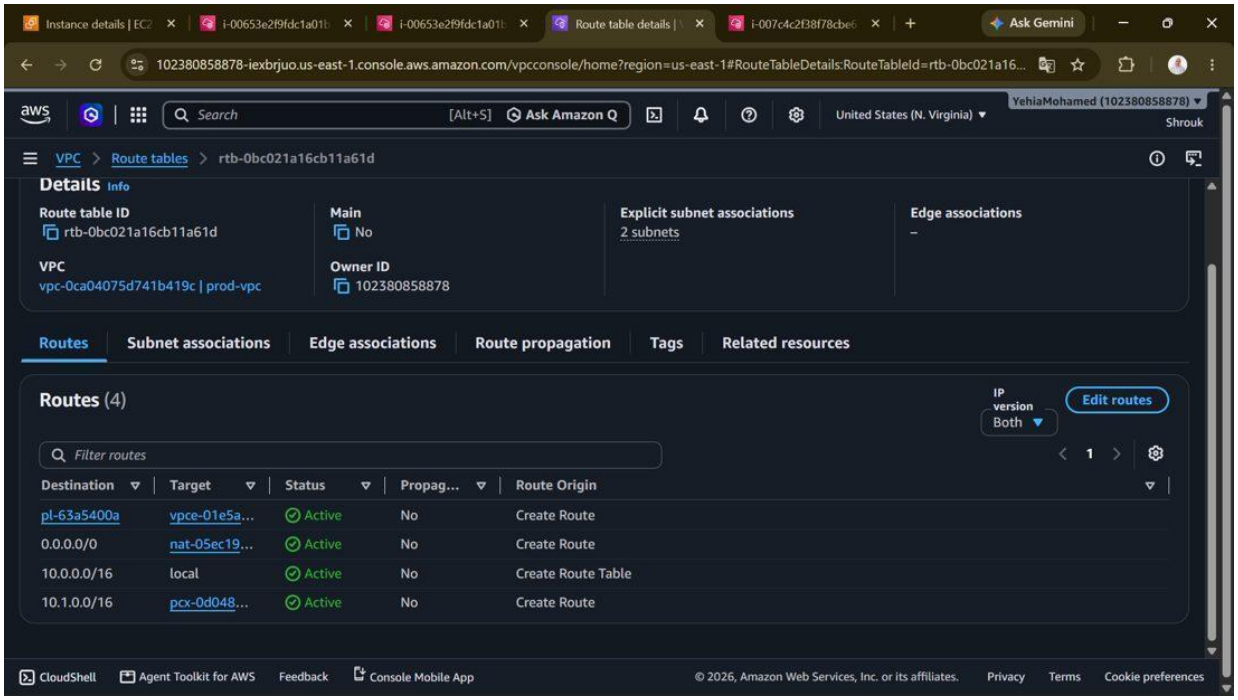


Figure: rt-private: the S3 prefix list (pl-63a5400a) routes to the gateway endpoint, which is more specific than

0.0.0.0/0 → NAT, so S3 traffic bypasses the NAT.

5.1 Re-verification in the rebuilt environment (October 29)

The lab environment was later rebuilt with new resource IDs (prod-vpc vpc-0e2a24823e55813b8, peering pcx-078460e96c5a00631). The peering routes, the endpoints and the bastion-to-prod test were repeated there and documented in the shared lab documentation prepared by Yehia Mohamed; the evidence is included here for completeness.

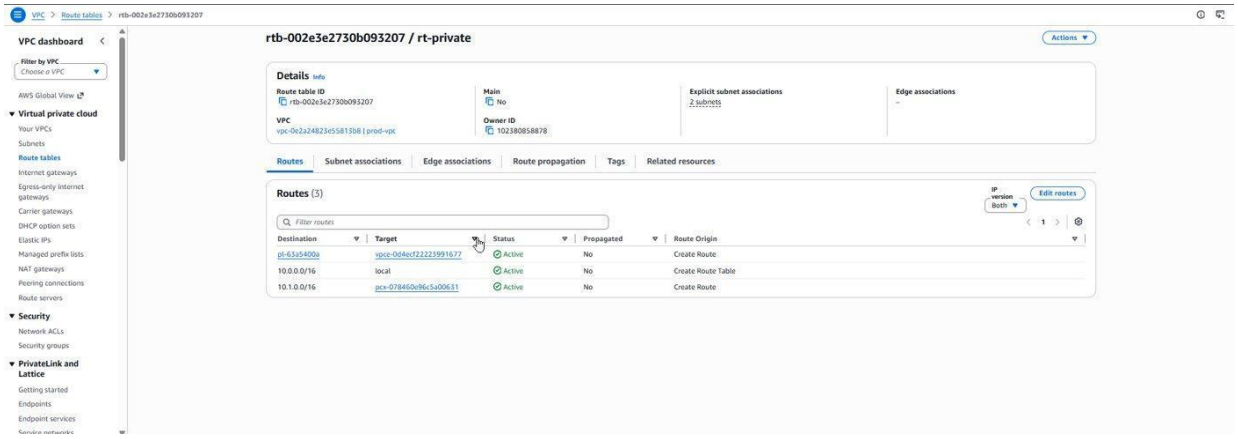


Figure: Rebuilt environment: rt-private routes (S3 prefix list and peering).

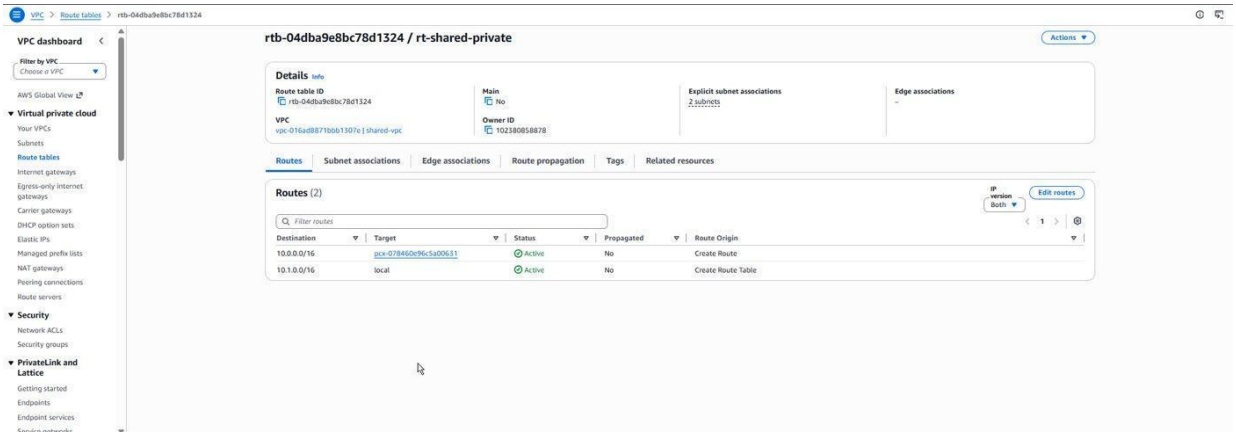


Figure: Rebuilt environment: rt-shared-private routes 10.0.0.0/16 to the peering.

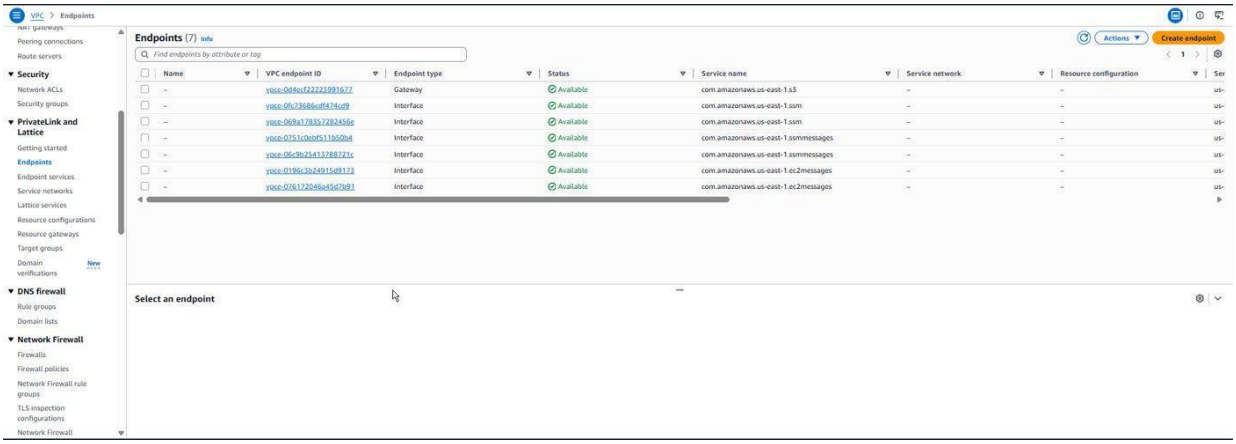


Figure: Rebuilt environment: VPC endpoints Available.

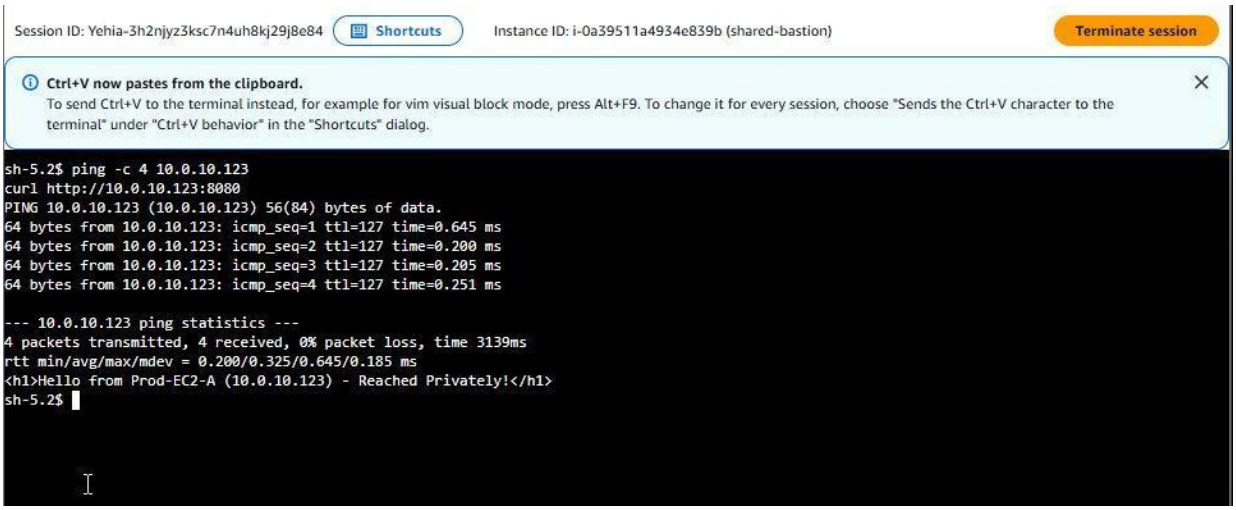


Figure: Rebuilt environment: SSM session on shared-bastion, ping 0% loss and curl to prod-ec2-a

(10.0.10.123:8080) over the peering.

6. T31–T33 – Transit Gateway

prod-tgw (ASN 64512) was created with two VPC attachments (prod-vpc and shared-vpc, in the private subnets). The default TGW route table associated both attachments and propagated 10.0.0.0/16 and 10.1.0.0/16. Because the same CIDRs already routed through the peering, the peering routes were replaced with the Transit Gateway in rt-private and rt-

shared (a route table cannot hold two routes for the same CIDR), and the test was repeated.

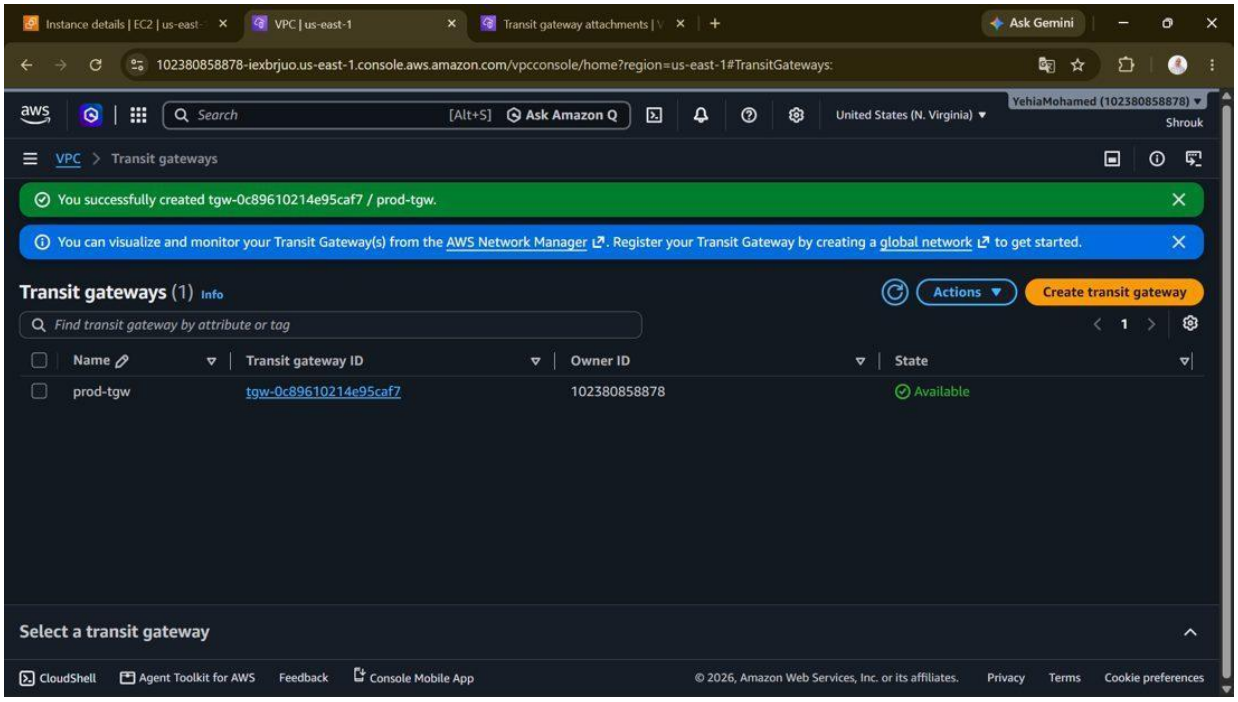


Figure: prod-tgw created: Available.

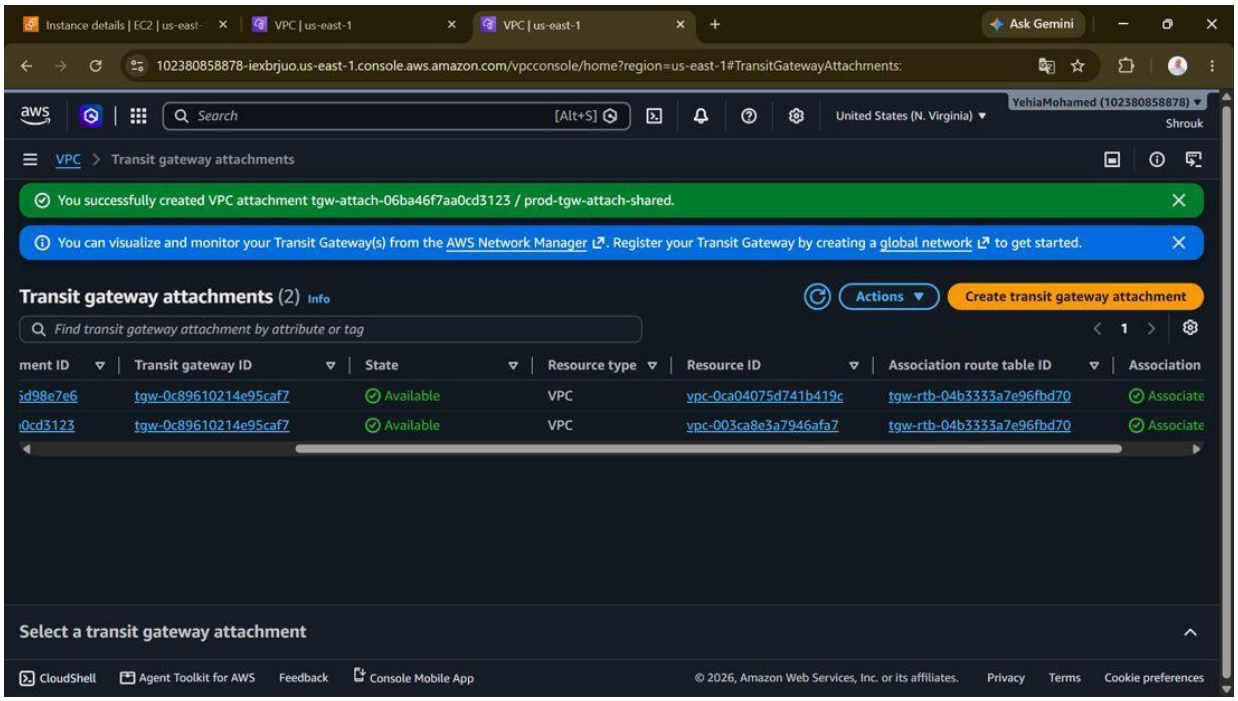


Figure: Two VPC attachments Available and associated with the TGW route table.

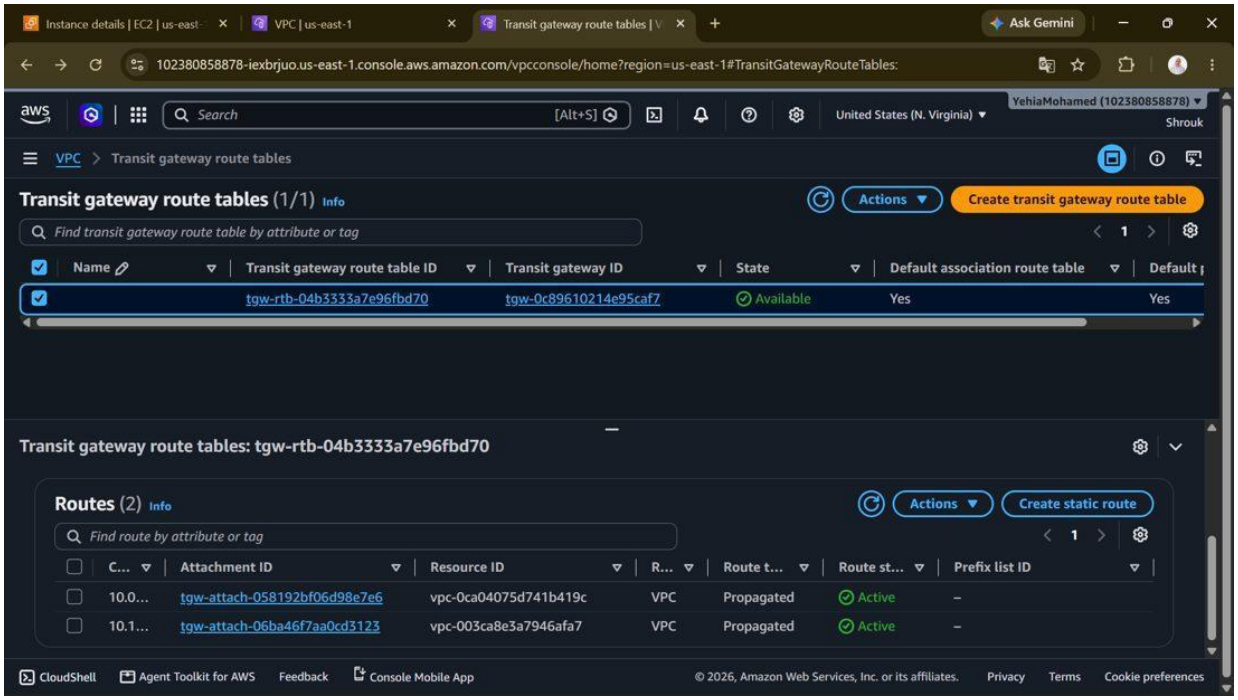


Figure: TGW route table: both CIDRs propagated and Active.

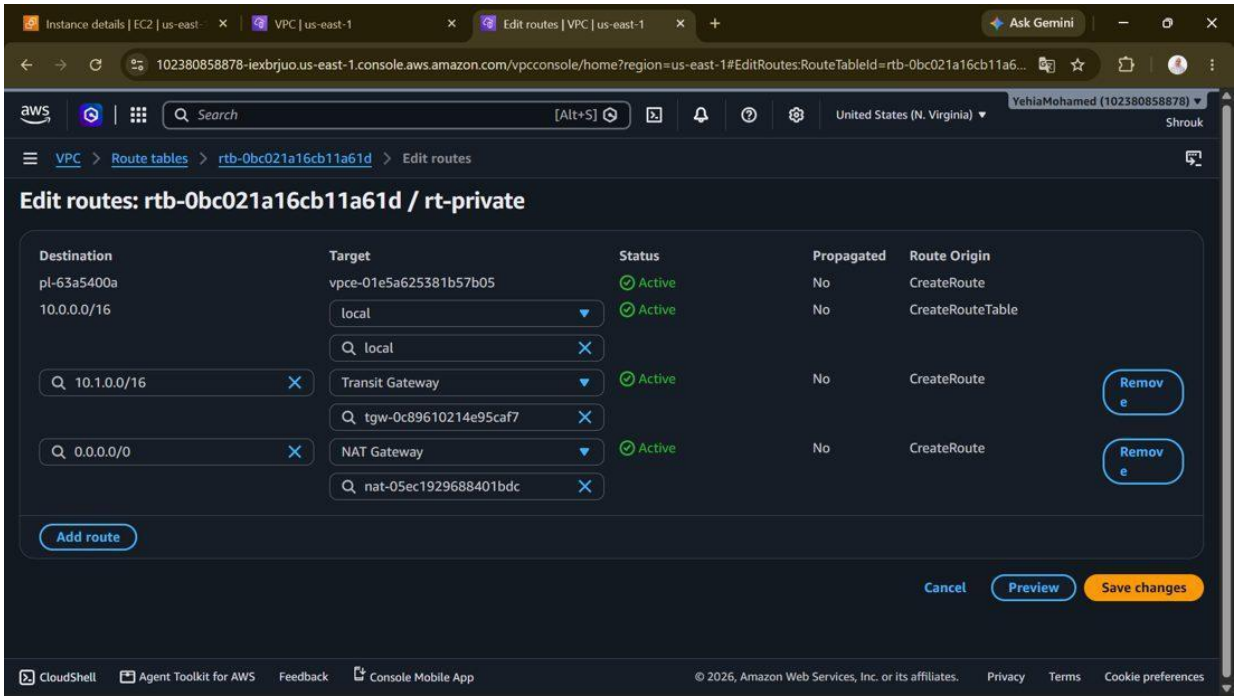


Figure: rt-private: 10.1.0.0/16 now targets the Transit Gateway.

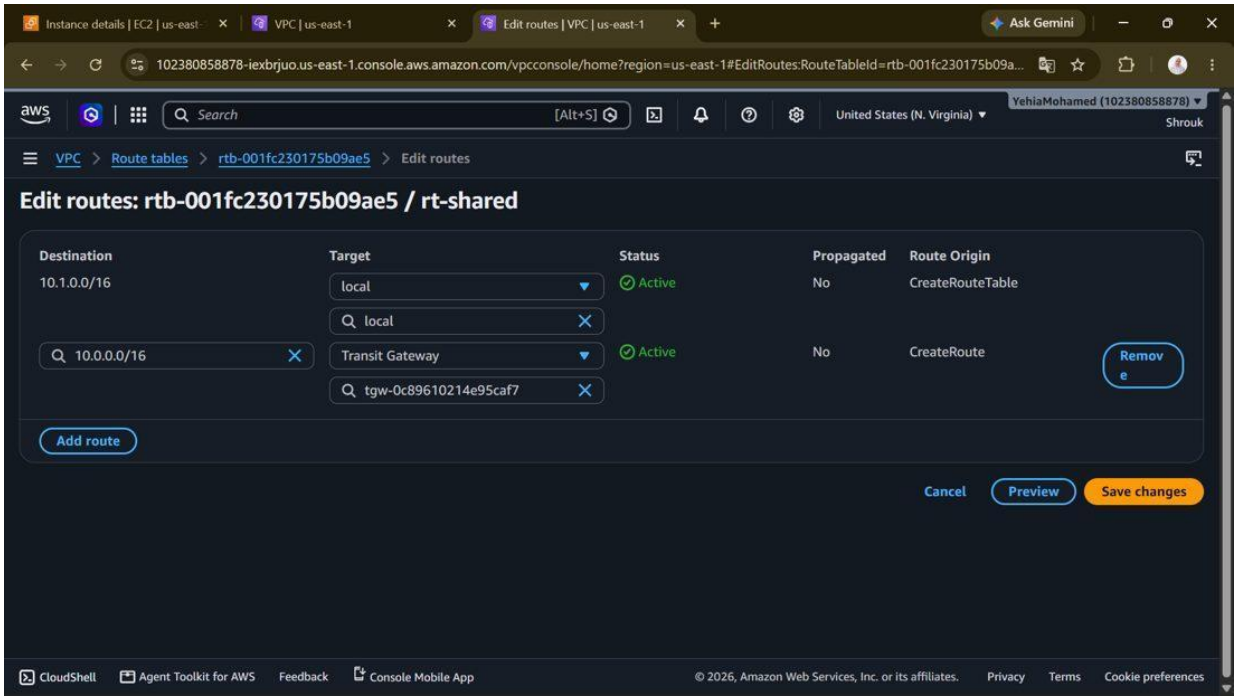


Figure: rt-shared: 10.0.0.0/16 now targets the Transit Gateway.

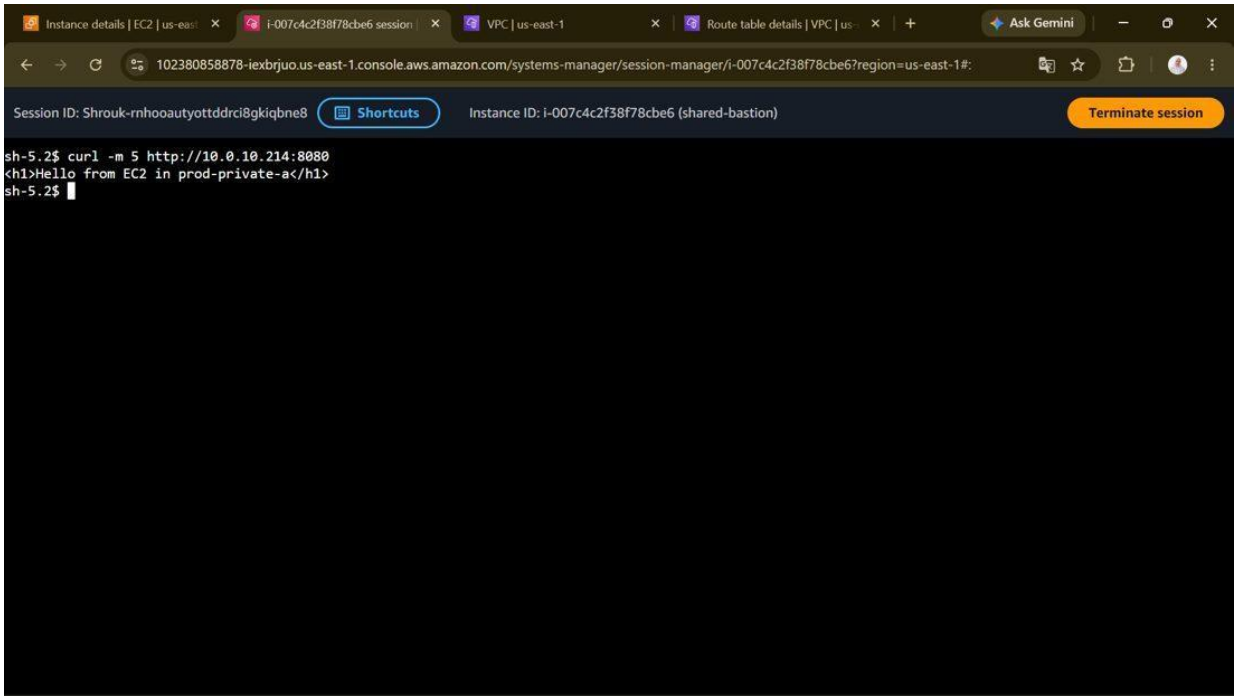


Figure: curl from the bastion to prod-ec2-a succeeds with the peering routes replaced by the TGW. Path: bastion (10.1.10.130) → rt-shared → TGW → prod attachment → prod-ec2-a.

7. T34–T36 – Site-to-Site VPN

A Virtual Private Gateway (prod-vgw) was attached to prod-vpc. A Customer Gateway (onprem-cgw) was created with the documentation address 203.0.113.10 and ASN 65000 to simulate an on-premises device. The VPN connection prod-vpn uses static routing for 192.168.0.0/16. Because no real device answers at the customer gateway, both tunnels are Down by design; the deliverable here is the correct configuration, which was captured before

everything was deleted.

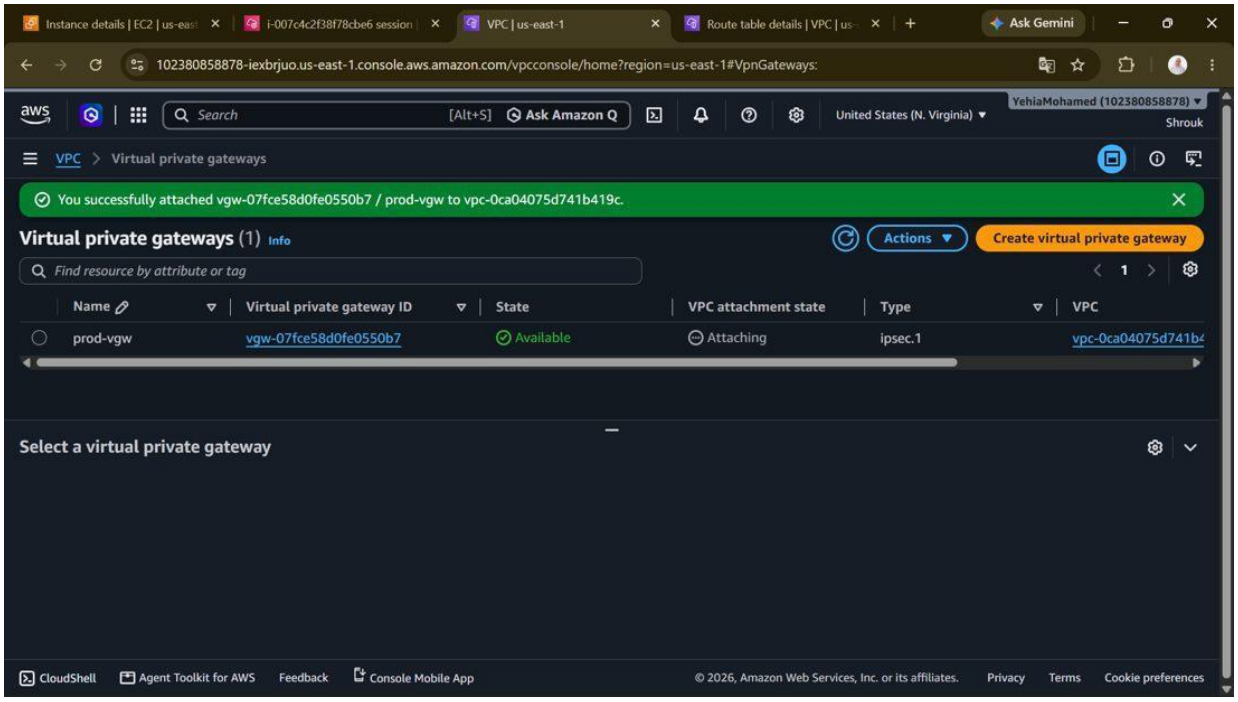


Figure: prod-vgw attached to prod-vpc.

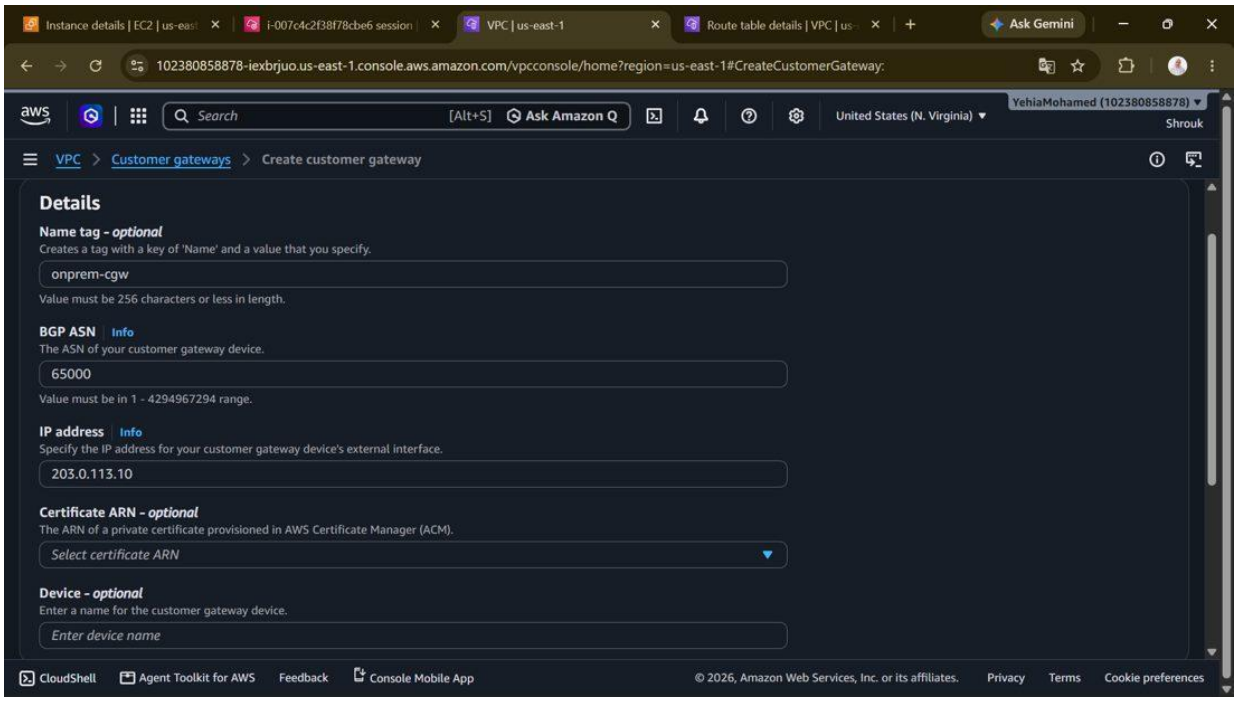


Figure: Creating the customer gateway onprem-cgw (203.0.113.10, ASN 65000).

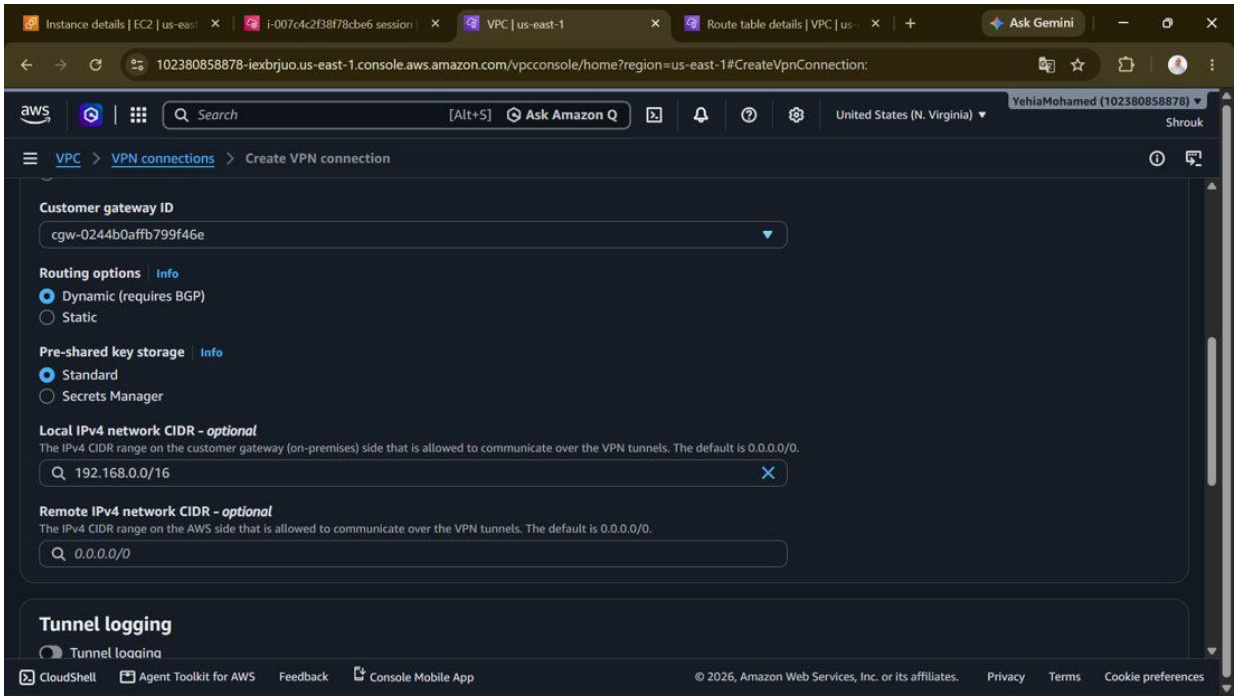


Figure: VPN connection settings: static routing, local CIDR 192.168.0.0/16.

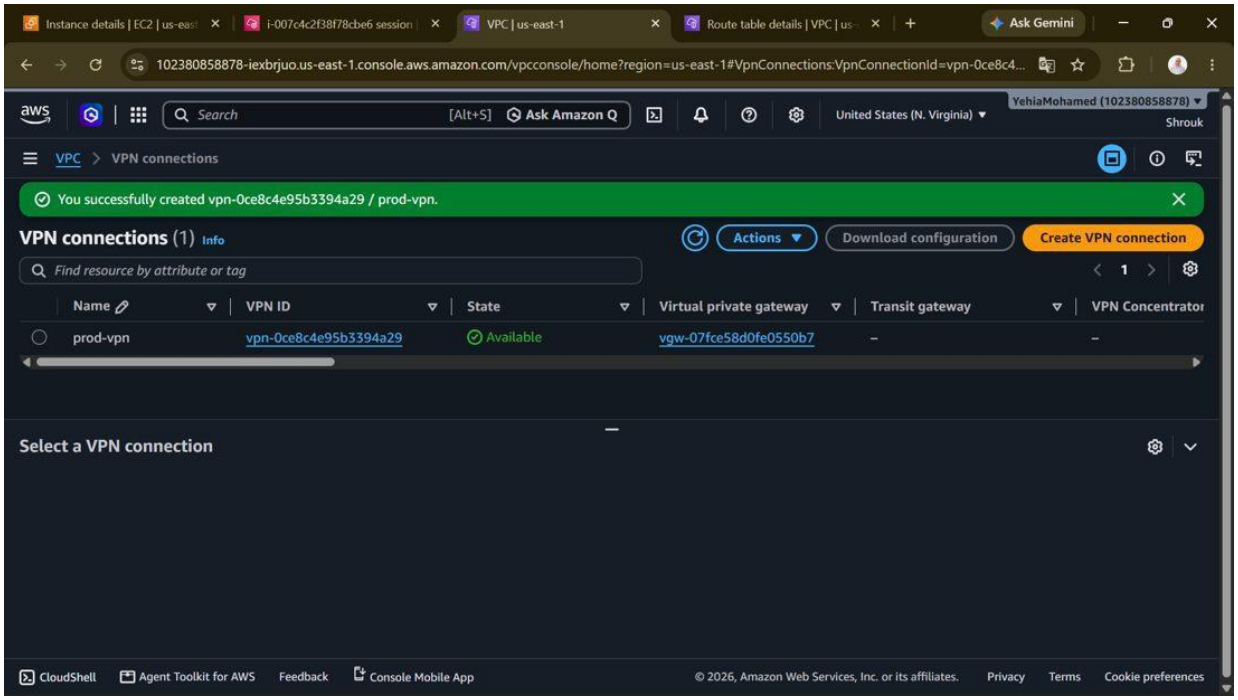


Figure: prod-vpn created: Available (attached to the VGW).

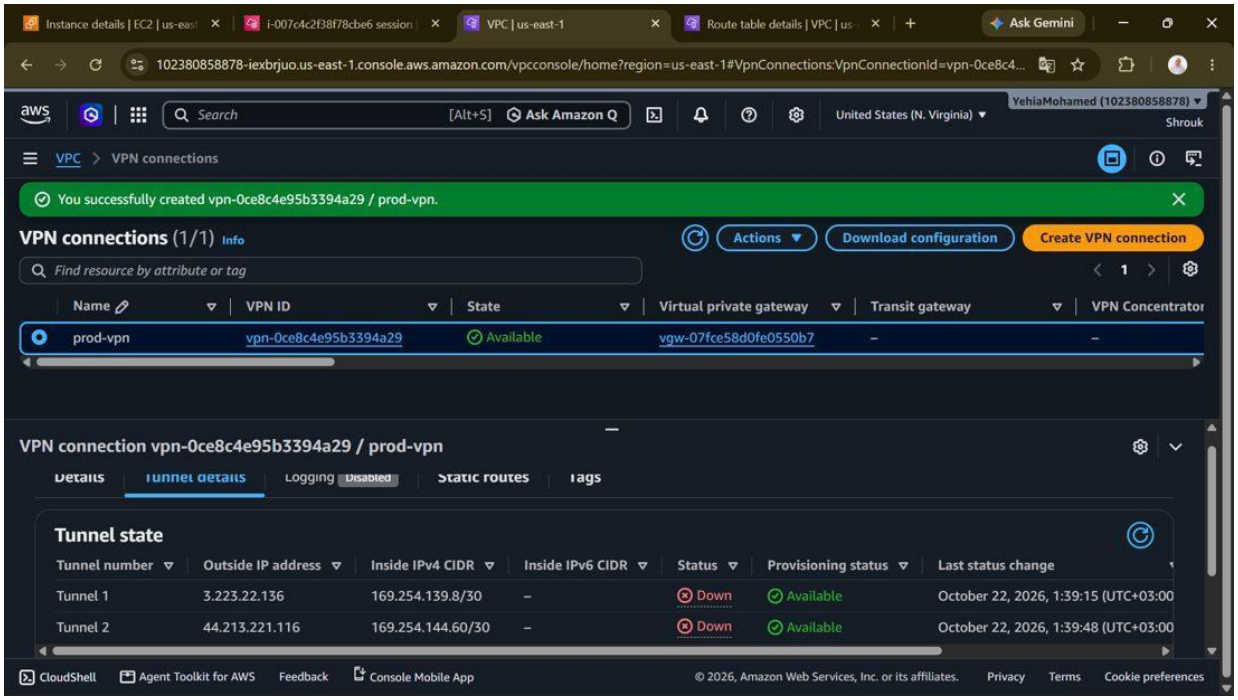


Figure: T35 – Tunnel details: both tunnels are Down (expected: the customer gateway is simulated) while

provisioning status is Available.

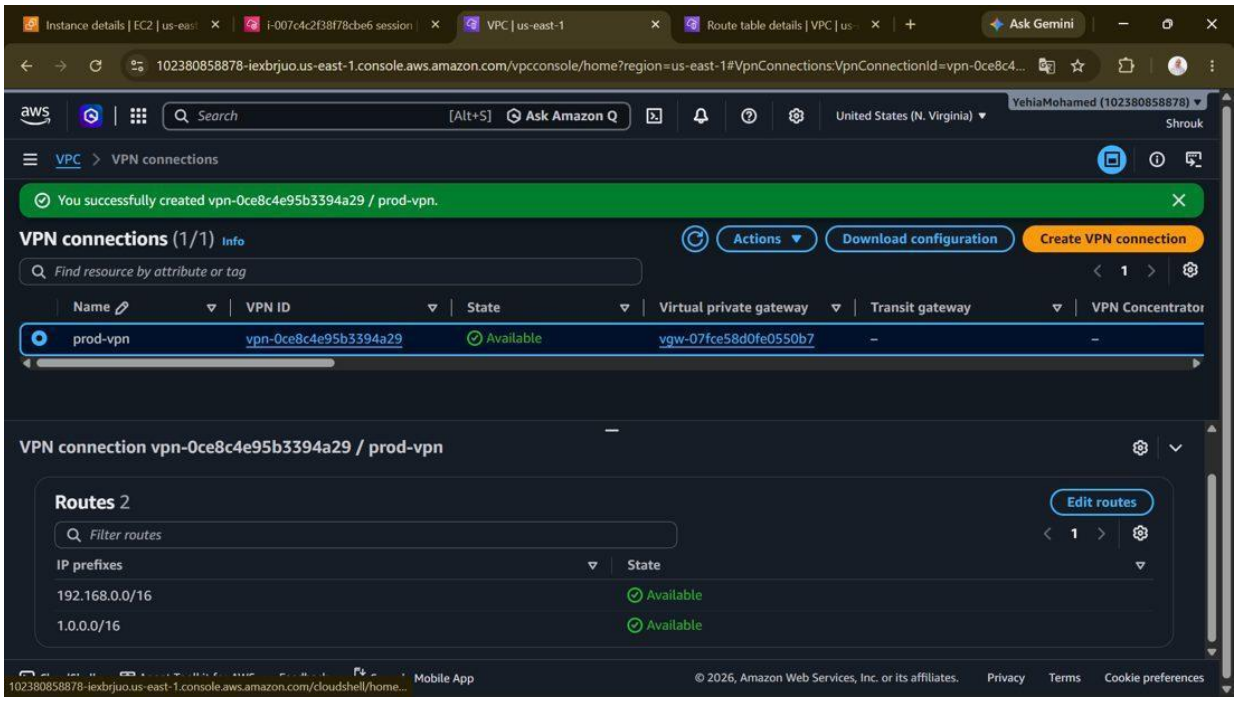


Figure: T35 – Static routes of the VPN connection.

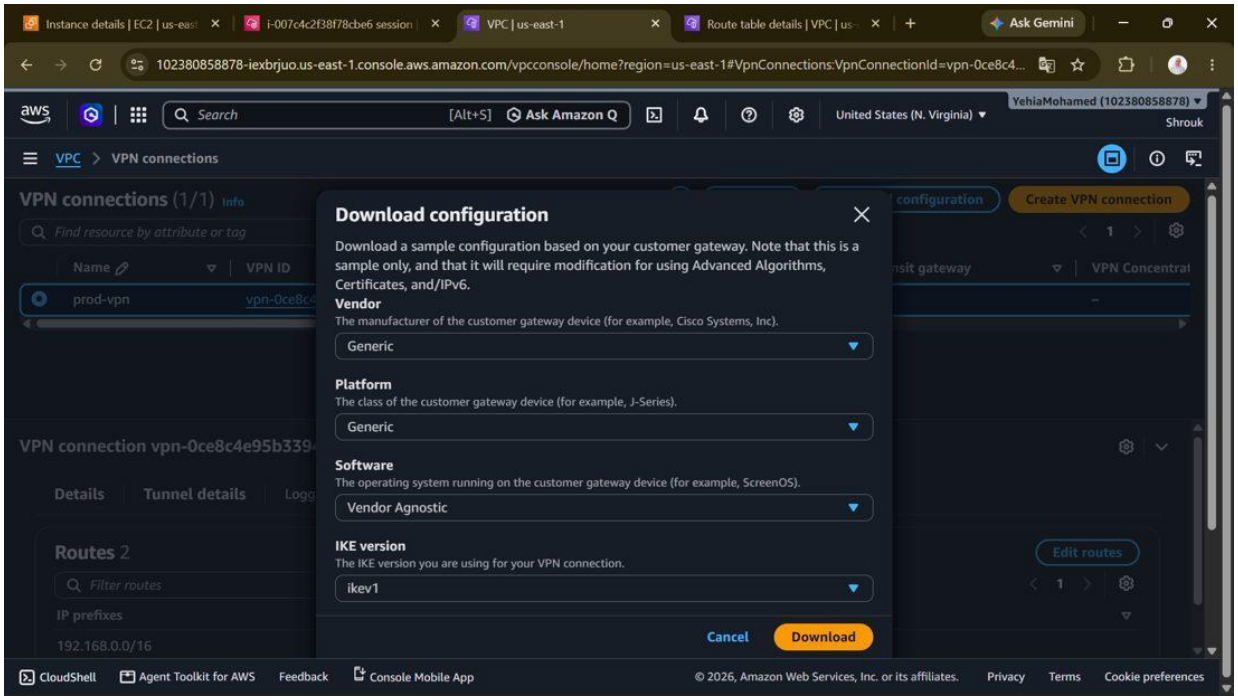


Figure: T35 – Download configuration dialog (Generic / Generic / Vendor Agnostic / IKEv1).

The downloaded configuration file (vpn-0ce8c4e95b3394a29.txt) confirms the tunnel parameters below. The pre-shared keys it contains are intentionally omitted from this report.

Parameter Tunnel 1 Tunnel 2

AWS outside IP 3.223.22.136 44.213.221.116

Customer gateway outside IP 203.0.113.10 203.0.113.10

Inside CIDR (customer / AWS) 169.254.139.10 /

169.254.139.9

169.254.144.62 /

169.254.144.61

IKE / IPsec IKEv1, PSK, AES-128-CBC,

SHA1, DH group 2

IKEv1, PSK, AES-128-CBC,

SHA1, DH group 2

Static route next hop 169.254.139.9 169.254.144.61

Note: the Static routes tab listed 192.168.0.0/16 (the route requested by the task) and also a second prefix, 1.0.0.0/16; the origin of the second prefix was not determined.

Deletion order: VPN connection, then detach and delete the VGW (the VGW only deletes after detaching completes), then the customer gateway.

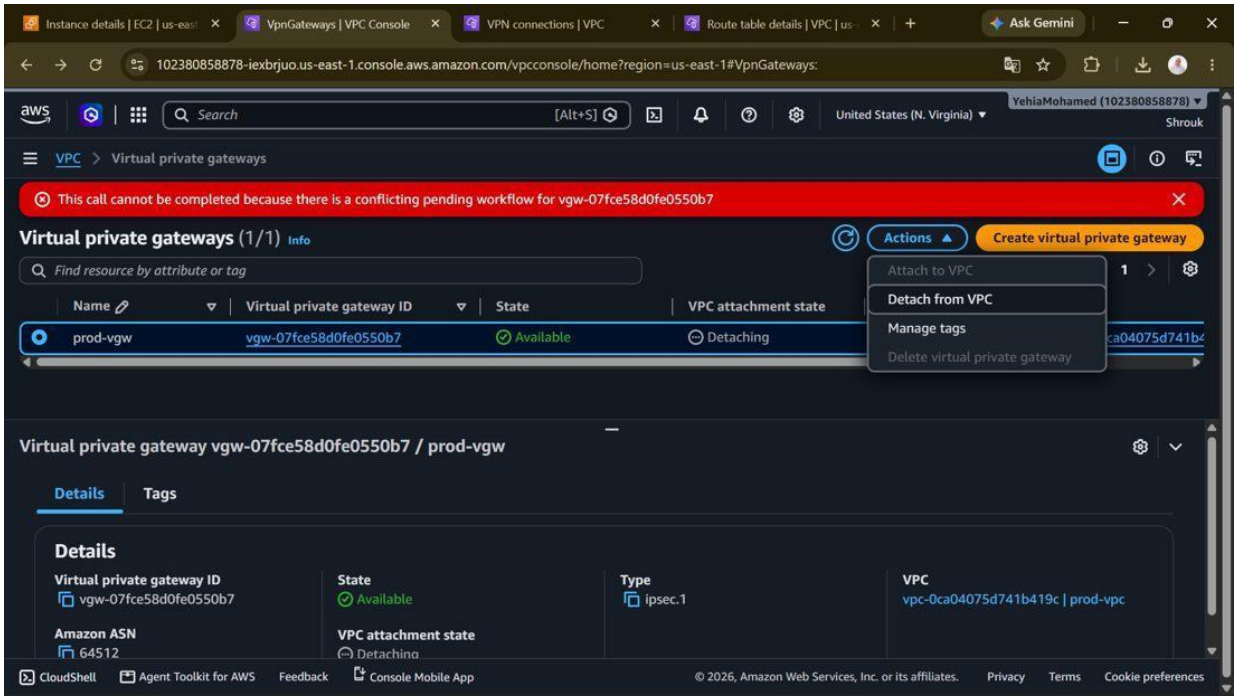


Figure: Detaching the VGW: delete stays disabled until the detachment completes.

8. T37–T39 – Client VPN

8.1 First attempt (October 22)

A mutual-authentication Client VPN endpoint was created. Server and client certificates were generated with easy-rsa in CloudShell and imported into ACM (the server certificate needed a domain-style common name, server.domain.tld). Settings: client CIDR 10.100.0.0/22 (no overlap with 10.0.0.0/16), UDP, full tunnel, association with prod-private-a, security group Sg-ec2, and an authorization rule for 10.0.0.0/16 (Active). Required rules were opened for the client range in Sg-ec2 and nacl-private.

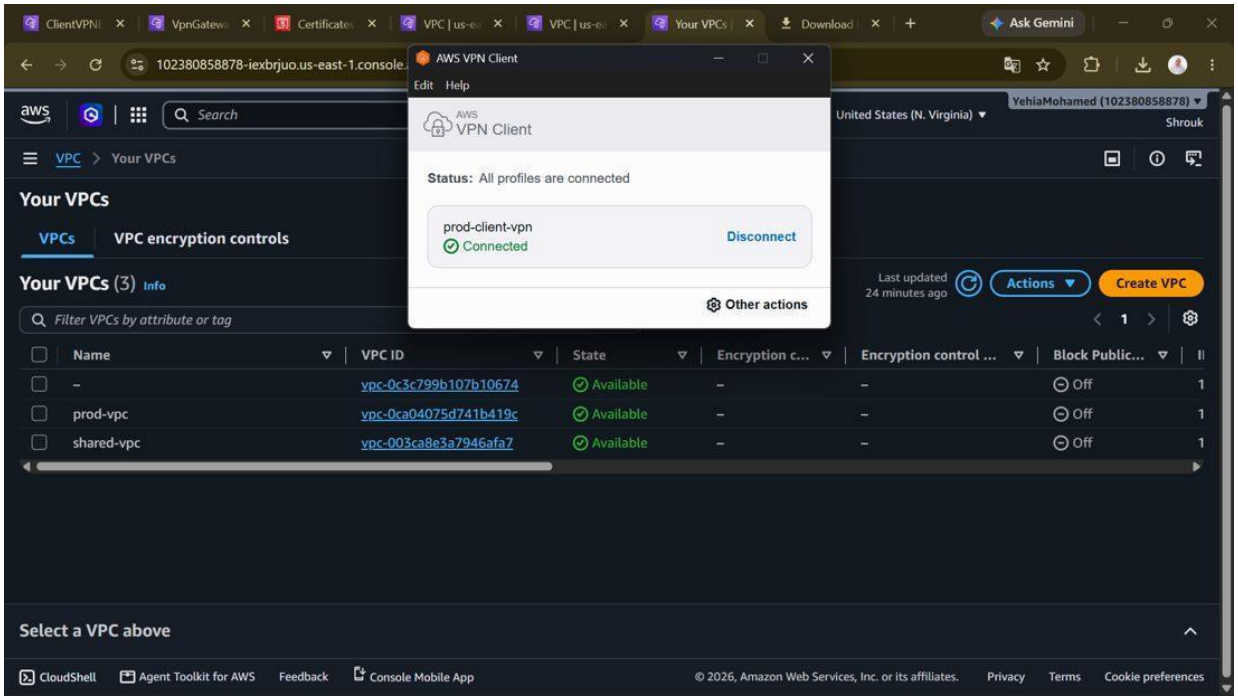


Figure: AWS VPN Client: profile prod-client-vpn is Connected (first attempt).

Result: the endpoint recorded six successful client connections (client IPs from 10.100.0.x) and traffic left through the AWS TAP adapter, but the reachability test to 10.0.10.214:8080 timed out although the application answered from inside AWS. Security groups, network ACLs, the authorization rule, the endpoint route table and instance state were checked; the cause was not isolated in this attempt.

8.2 TLS handshake failure after the rebuild (October 28)

After the environment was rebuilt, an OpenVPN test showed the server certificate verified successfully (VERIFY OK, CN=server.domain.tld), followed by a 60-second silence and "TLS key negotiation failed". Inspection showed the cause: the endpoint trusted a client certificate issued by CN=yes, while the client certificate placed in the .ovpn was issued by CN=Easy-RSA CA. Two certificate sets from different CAs were mixed, so the client was rejected silently. Contributing factors found in the same investigation: a truncated </key> tag after manual editing, the Quick start defaults (default security group and a public subnet instead of prod-private-a), a TCP test against a UDP endpoint, and 50–75% packet loss on the first mobile hotspot (ping to 8.8.8.8).

8.3 Successful run (October 29)

The Client VPN was rebuilt from a clean state (documented in the shared lab documentation prepared by Yehia Mohamed): one Easy-RSA CA issued both certificates (server.vpn.local and client1.vpn.local), both were imported to ACM, and the endpoint cvpn-endpoint-053c30ff1d8369afa was created with client CIDR 10.100.0.0/22, TCP port 443, split tunnel, mutual TLS, security group sg-0a0ccfd5557de97db (a self-referencing rule was added),

association with prod-private-a and an authorization rule for 10.0.0.0/16. The exported .ovpn received the client certificate and key inline, a "app." prefix on the remote host name, and verify-x509-name for the server. Which of these differences from the first attempt was decisive was not tested individually.

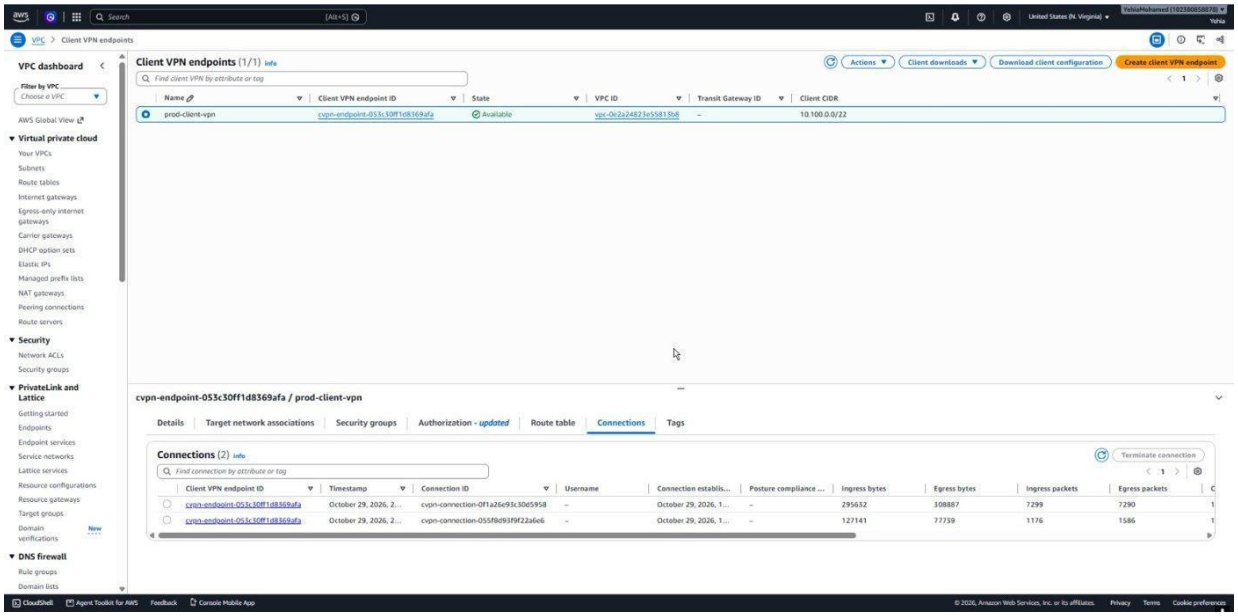


Figure: Client VPN endpoint prod-client-vpn: Available, with two connections listed.

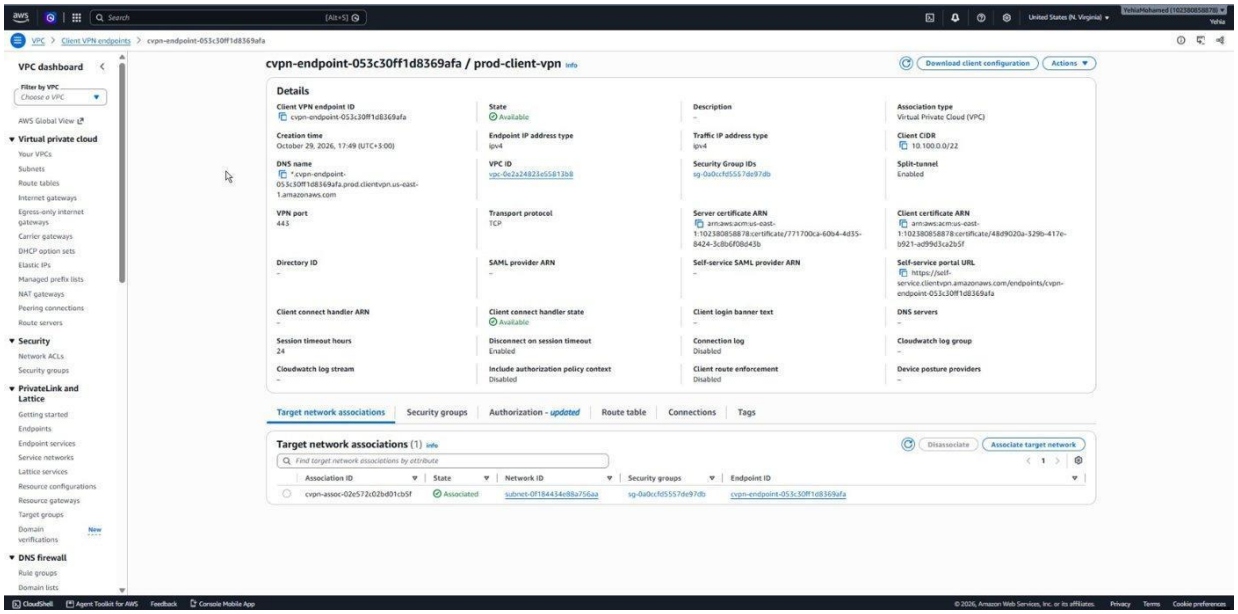


Figure: Endpoint details (TCP 443, split tunnel, security group) and the target network association: Associated.

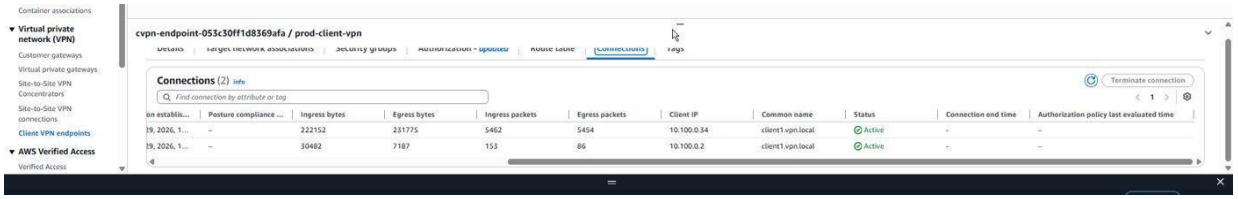


Figure: Connections tab: two client sessions from the 10.100.0.0/22 pool.

Verification from the Windows workstation over a stable mobile hotspot: ping 10.0.10.123

returned 4/4 replies (0% loss, average about 305 ms); Invoke-WebRequest to http://10.0.10.123:8080 returned StatusCode 200 with the private page; and the browser displayed the same page.

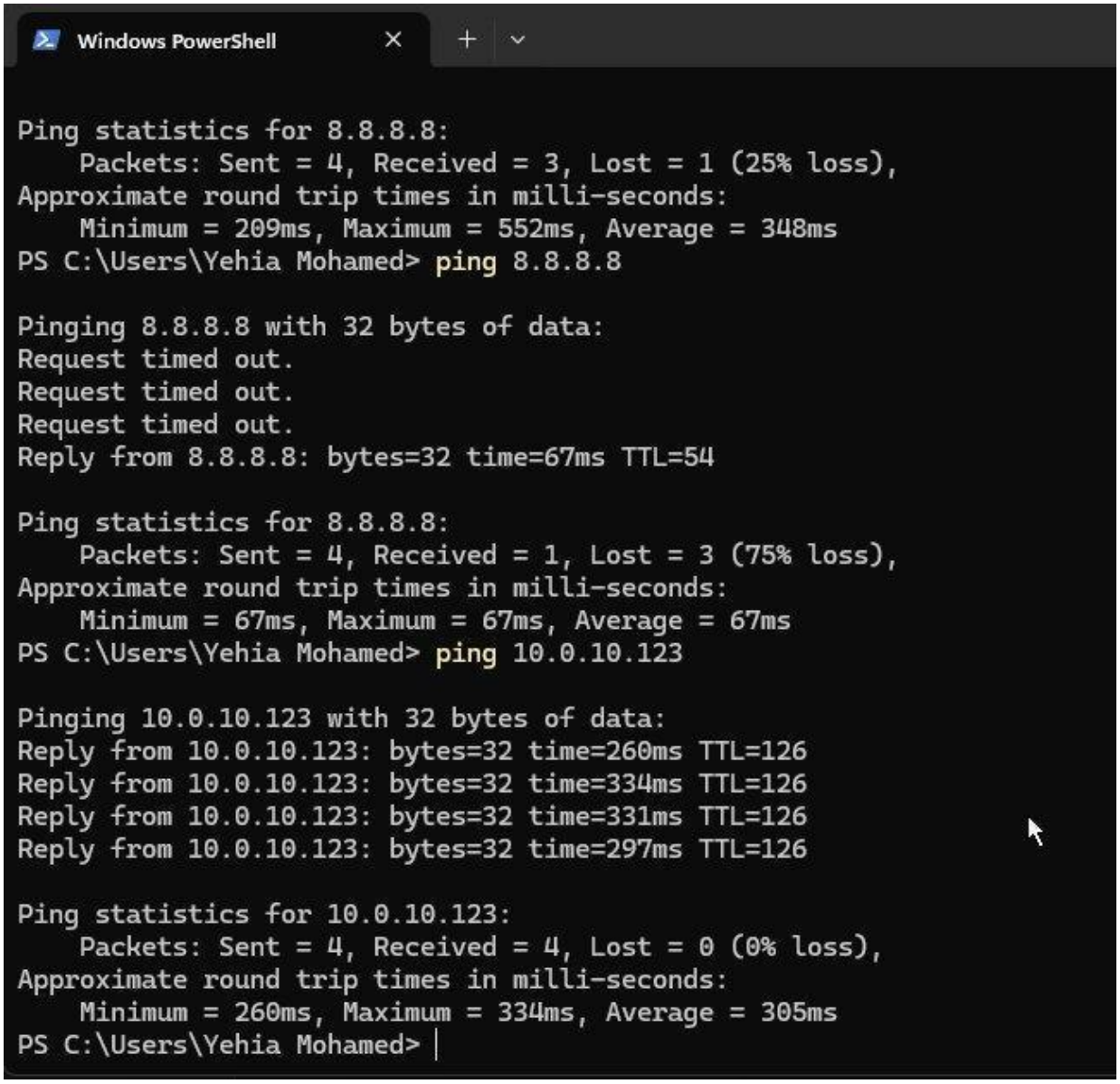


Figure: PowerShell: unstable first carrier (75% loss to 8.8.8.8), then ping to 10.0.10.123 with 0% loss over the tunnel.

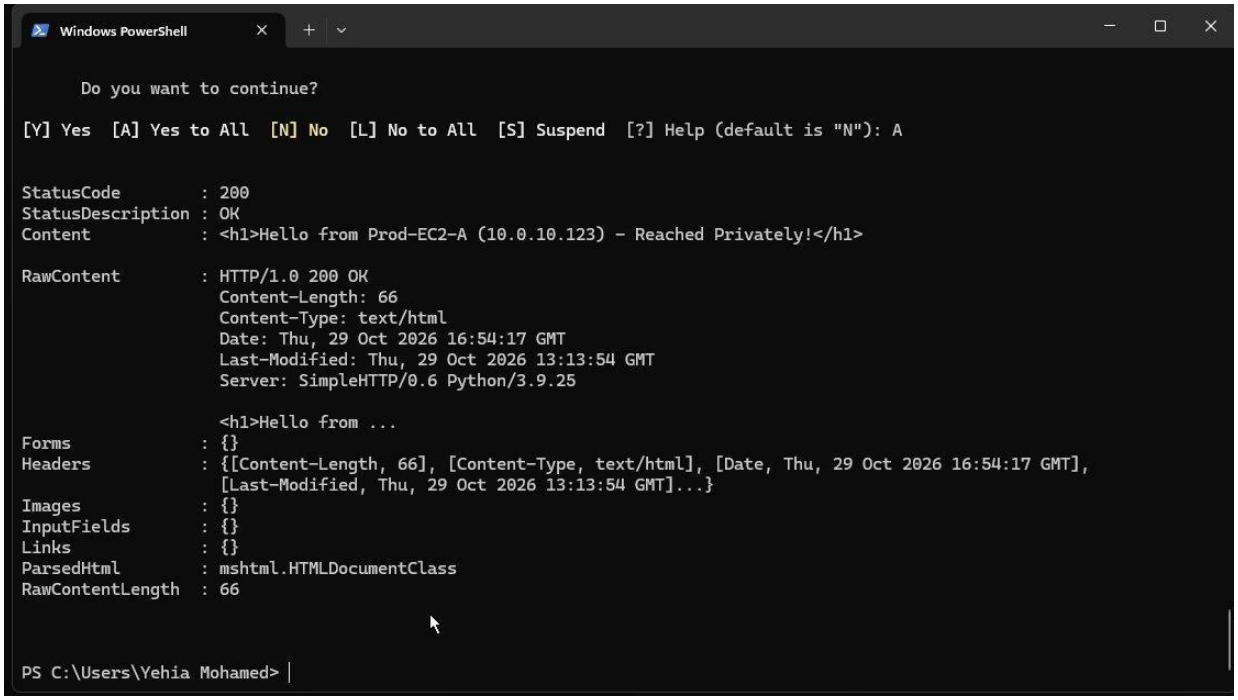


Figure: PowerShell: HTTP request to 10.0.10.123:8080 returns StatusCode 200.

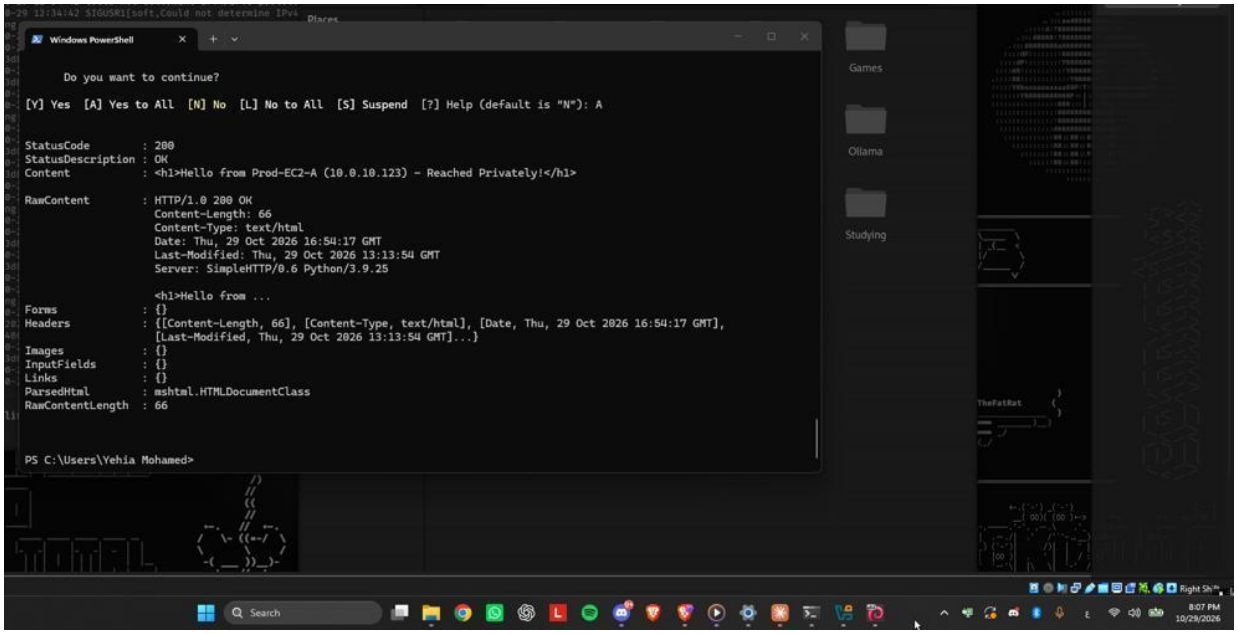


Figure: Full Windows Desktop Proof — Active OpenVPN Tray Icon & PowerShell HTTP 200 Verification —

Complete desktop capture showing the active OpenVPN GUI connection indicator in the Windows system tray alongside the PowerShell terminal receiving HTTP 200 OK from the private EC2 server 10.0.10.123.

Figure: Browser access to http://10.0.10.123:8080 through the Client VPN.

8.4 Teardown (T39)

After the evidence was captured, a dependency-ordered teardown script was executed: all Client VPN target networks were disassociated and the script waited for the disassociation to finish before deleting the endpoint (billing is per hour of association); then VPC endpoints, EC2 instances, peering connections, ACM certificates and the lab VPCs were removed. The local .ovpn files, which contain the client private key, should also be deleted.

9. Issues Encountered and Resolutions

Issue Resolution

1 Security group name rejected
for starting with "sg-"
Used shared-endpoints-sg; AWS reserves the sg- prefix for IDs.

2 Bastion could not reach prod EC2 (curl timeout)
The bastion security group only allowed outbound 443. Added outbound TCP 8080 to 10.0.0.0/16; the test then succeeded. (Inbound rules for 10.1.0.0/16 on Sg-ec2 and nacl-private were added by the security owner.)

Issue Resolution

3 Peering and TGW routes for the same CIDR
A route table cannot hold two routes for one CIDR, so the peering routes were replaced by the TGW in both VPCs for the test and documented.

4 Wrong console page opened (VPN Concentrator)
Cancelled without creating anything and used Site-to-Site VPN connections instead.

5	Client VPN rejected the server certificate	The common name "server" is not a domain. Issued a new certificate with CN server.domain.tld and re-imported it to ACM.
6	VGW could not be deleted immediately	"Conflicting pending workflow": the detach was still in progress. Waited until the attachment state was Detached, then deleted.
7	Client VPN reachability test timed out (first attempt)	Not isolated at the time. The second run succeeded after a clean rebuild (TCP 443, split tunnel, self-referencing security group rule, one CA for both certificates, association with prod-private-a); see Section 8.3.
8	TLS key negotiation failed (October 28)	The client certificate in the .ovpn was issued by CN=Easy-RSA CA while the endpoint trusted CN=yes. Fixed by generating server and client certificates from one CA.
9	OpenVPN error: Endtag </key> missing	A manual edit truncated the key block; the .ovpn was regenerated with a script that appends the certificate and key blocks.

10. Cost Control and Cleanup (T40)

Services that bill by the hour were deployed, tested, documented and deleted in the same working session: Transit Gateway with two attachments (about $0.05/hour per attachment), Site-to-Site VPN (about $0.05/hour) and Client VPN (about $0.10/hour per association). The deletion order was: Client VPN association and endpoint, VPN connection, VGW and customer gateway, then TGW attachments, and finally the Transit Gateway. After the successful Client VPN run, a full teardown script also removed the VPC endpoints, EC2 instances, peering connections, ACM certificates and the lab VPCs (Section 8.4).

11. Conclusion

- Peering, routing and endpoints were configured and verified with screenshots.

• SSM, S3 gateway access and Transit Gateway routing were tested successfully.

• The Site-to-Site VPN configuration was documented (tunnels Down by design).

• The Client VPN reachability issue of the first attempt was resolved in a clean second run (ping, HTTP 200 and browser access to the private server), with the handshake root cause documented.

• All hourly-billed hybrid resources were deleted after testing.

Part 3B: Complete Dual-Method Client VPN & Peering Verification Report (All 9 High-Resolution Screenshots)

```
AWS Cloud Networking & Hybrid Connectivity Lab Report
Comprehensive Technical Documentation: VPC Peering, PrivateLink Endpoints, and Dual-Method AWS Client VPN Verification (Simulated VPC Node & Local Endpoint)
```

Executive Summary: This technical report documents the end-to-end architecture, implementation, troubleshooting, and verification of a multi-VPC hybrid networking environment in AWS (us-east-1). It covers private cross-VPC routing via VPC Peering, zero-NAT instance management via AWS PrivateLink VPC Endpoints, and mutual-TLS remote access using AWS Client VPN validated via both an in-cloud simulated on-premises router and a local Windows workstation.

1. Lab Architecture & Provisioned Infrastructure Inventory

To establish a realistic enterprise network topology while maintaining strict cost efficiency, three isolated Virtual Private Clouds (VPCs) were deployed in the N. Virginia (us-east-1) region without expensive NAT Gateways. Private connectivity and instance administration were achieved entirely through VPC Interface/Gateway Endpoints and encrypted VPN tunnels.

Resource Name / Role	AWS Resource ID	CIDR / IP / Port	Architectural Function
Production VPC (prod-vpc)	vpc-0e2a24823e55813b8	10.0.0.0/16	Hosts private application workloads and Client VPN target network
Prod Private Subnet A	subnet-prod-private-a	10.0.10.0/24	Isolated private subnet hosting prod-ec2-a and VPC interface endpoints
Prod Private Route Table	rtb-002e3e2730b093207	rt-private	Routes 10.1.0.0/16 to Peering and S3 traffic to Gateway Endpoint
Prod App Server (prod-ec2-a)	i-099ff51cc7ad35914	10.0.10.123 (TCP 80/8080)	Private Python HTTP web server verifying cross-VPC and VPN reachability
Shared Services VPC (shared-vpc)	vpc-016ad8871bbb1307e	10.1.0.0/16	Central management VPC peered with Production VPC
Shared Private Route Table	rtb-04dba9e8bc78d1324	rt-shared-private	Routes 10.0.0.0/16 traffic across pcx-078460e96c5a00631
Shared SSM Bastion Host	i-0a39511a4934e839b	10.1.10.101	Private bastion accessed via SSM Session Manager for peering tests
Simulated On-Premises VPC	vpc-0fd9f599f88c3b238	192.168.0.0/16	External network environment simulating a remote client/site

On-Prem Simulation Router	i-08d8e7589d049bf62	192.168.1.222 / 3.237.191.143	Simulated remote node used for Method 1 Client VPN tunnel verification
VPC Peering Connection	pcx-078460e96c5a00631	10.0.0.0/16 <-> 10.1.0.0/16	Direct private routing between prod-vpc and shared-vpc
AWS Client VPN Endpoint	cvpn-endpoint-053c30ff1d8369afa	10.100.0.0/22 (TCP 443)	Mutual-TLS OpenVPN endpoint associated with prod-private-a

2. Part 1: VPC Peering & VPC Endpoints Implementation (Tasks T27 – T30)

2.1 Establishing Cross-VPC Peering & Bidirectional Routing (T27 & T28)

A VPC Peering connection (pcx-078460e96c5a00631) was provisioned and accepted between the Production VPC (vpc-0e2a24823e55813b8, CIDR 10.0.0.0/16) and the Shared Services VPC (vpc-016ad8871bbb1307e, CIDR 10.1.0.0/16). Because VPC peering is non-transitive and requires explicit symmetric routing, both private route tables were updated:

- **Production Private Route Table (rtb-002e3e2730b093207):** Added a static route directing destination CIDR 10.1.0.0/16 to target pcx-078460e96c5a00631, alongside the S3 Gateway prefix list route (pl-63a5400a -> vpce-0d4ecf22223991677).
- **Shared Services Private Route Table (rtb-04dba9e8bc78d1324):** Added a return static route directing destination CIDR 10.0.0.0/16 to target pcx-078460e96c5a00631.

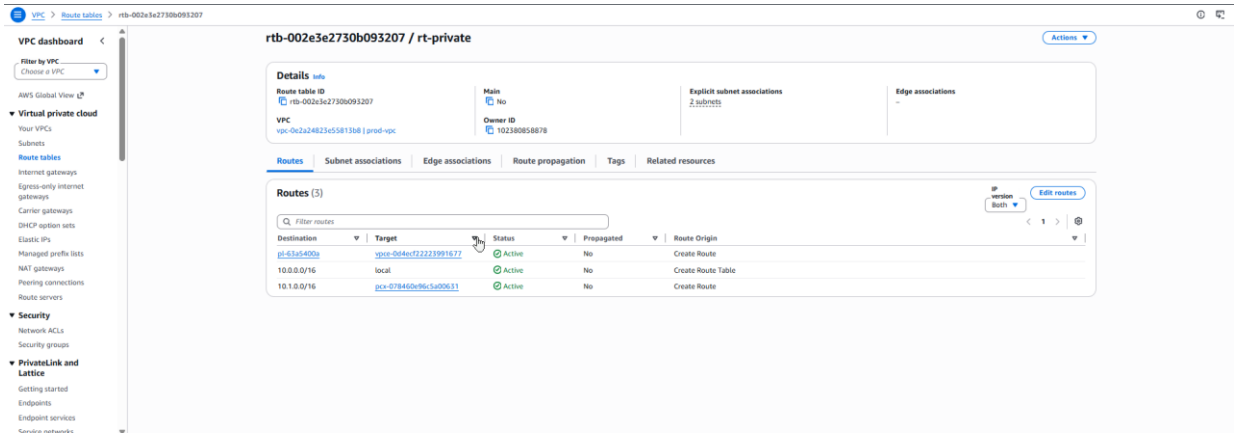


Figure 1: Production Private Route Table (rtb-002e3e2730b093207) — Active route directing 10.1.0.0/16 traffic across VPC Peering connection pcx-078460e96c5a00631 and S3 traffic via Gateway Endpoint vpce-0d4ecf22223991677.

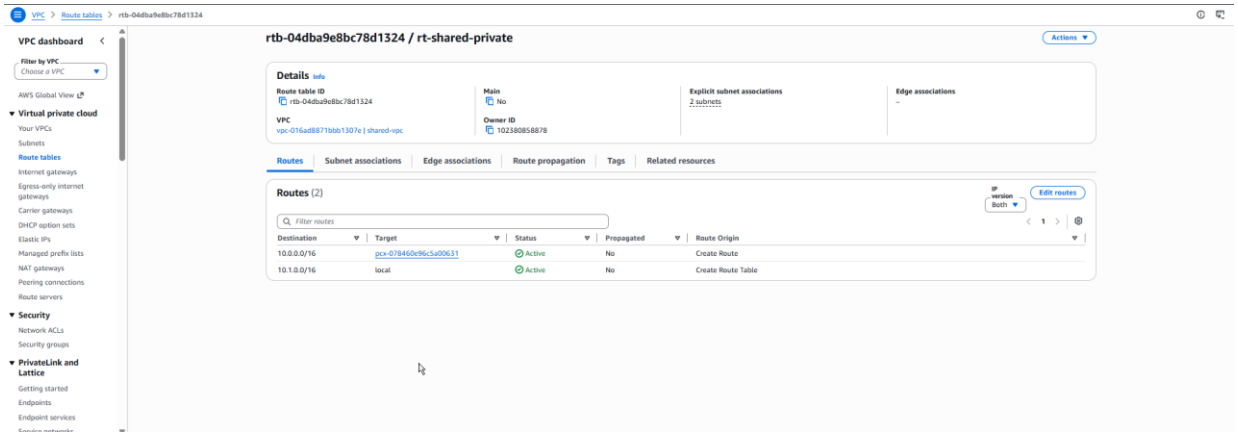


Figure 2: Shared Services Private Route Table (rtb-04dba9e8bc78d1324) — Active symmetric route directing 10.0.0.0/16 traffic back to prod-vpc via pcx-078460e96c5a00631.

2.2 PrivateLink Interface & Gateway VPC Endpoints (T29)

To enable AWS Systems Manager (SSM) Session Manager connectivity to completely isolated private EC2 instances without traversing the public internet or incurring NAT Gateway hourly charges, seven VPC Endpoints were deployed across prod-vpc and shared-vpc:

- **1x S3 Gateway Endpoint (vpce-0d4ecf22223991677):** Provides private routing to Amazon S3 (`com.amazonaws.us-east-1.s3`) for package repositories and artifact access.
- **6x AWS PrivateLink Interface Endpoints:** Three interface endpoints (`ssm`, `ssmmessages`, and `ec2messages`) attached to `prod-private-a` in `prod-vpc`, and three corresponding interface endpoints attached to `shared-private-a` in `shared-vpc` with Private DNS enabled and HTTPS (TCP 443) allowed.

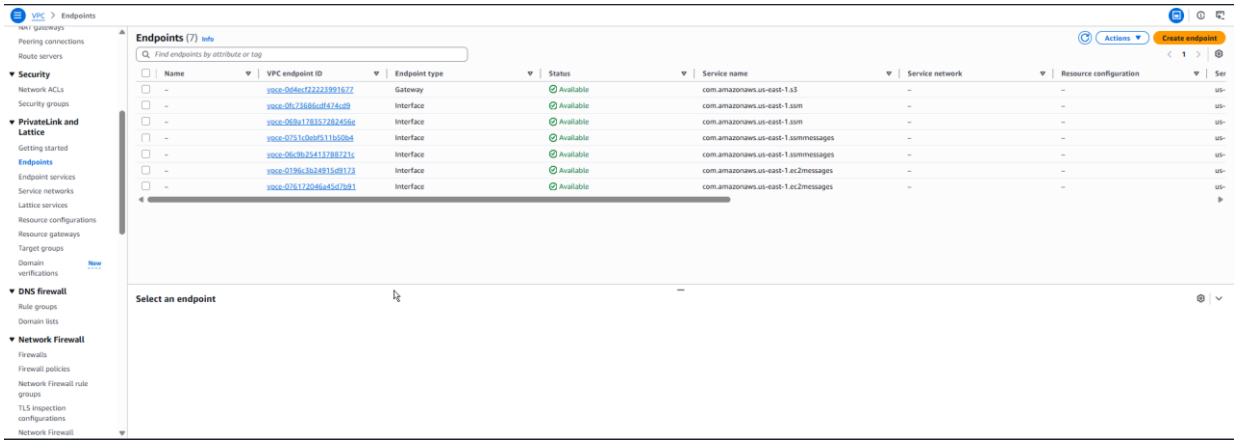


Figure 3: Provisioned VPC Endpoints in us-east-1 — All seven VPC Endpoints (1 Gateway Endpoint for S3 and 6 Interface Endpoints for SSM) showing Status = Available.

2.3 End-to-End Peering & PrivateLink Verification (T30)

Using the SSM Interface Endpoints, an interactive browser-based shell session was established into `shared-bastion` (`i-0a39511a4934e839b`, Private IP `10.1.10.101`). From inside `shared-vpc`, both ICMP reachability and HTTP application layer access to `prod-ec2-a` (`10.0.10.123:8080` in `prod-vpc`) were tested and verified with 0% packet loss.

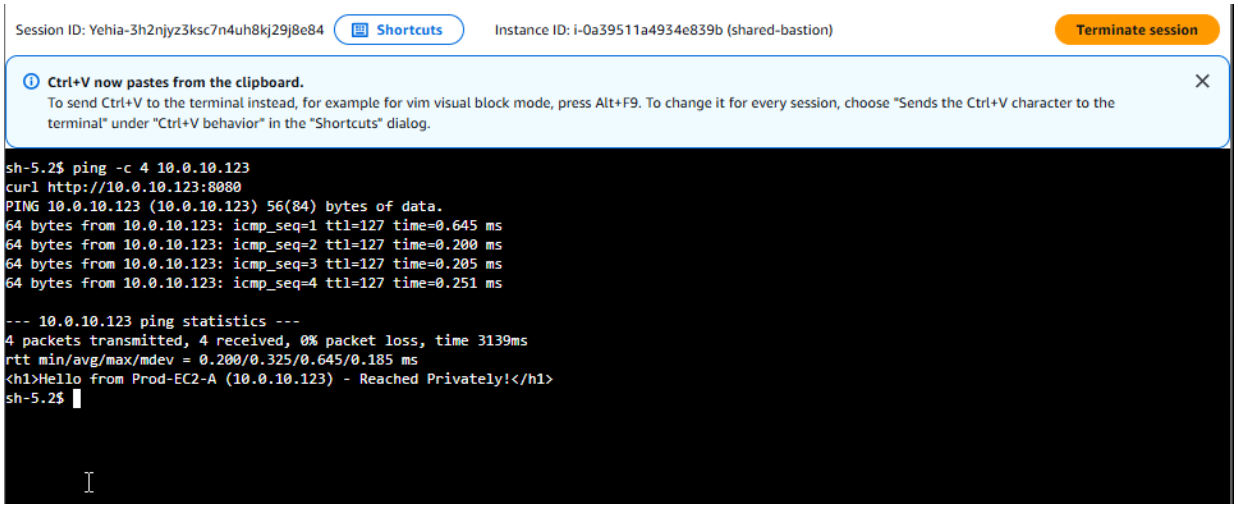


Figure 4: Cross-VPC Connectivity Verification via SSM Session Manager — Successful ICMP ping (0% packet loss, ~0.2ms latency) and HTTP curl response ('<h1>Hello from Prod-EC2-A (10.0.10.123) - Reached Privately!</h1>') from shared-bastion to prod-ec2-a.

3. Part 2: AWS Client VPN Architecture, Obstacles & Engineering Solutions (T37 – T39)

3.1 Mutual-TLS Certificate Generation & Endpoint Configuration

AWS Client VPN (`cvpn-endpoint-053c30ff1d8369afa`) was configured with Mutual Authentication (Certificate-based) using OpenVPN Easy-RSA v3. A dedicated Certificate Authority (`Easy-RSA CA`), a server certificate (`server.vpn.local`), and a client certificate (`client1.vpn.local`) were generated from the same PKI hierarchy and imported into AWS Certificate Manager (ACM).

The endpoint was provisioned with Client CIDR `10.100.0.0/22`, associated with `prod-private-a`, and configured with an ingress authorization rule permitting all clients access to `10.0.0.0/16`.

3.2 Technical Obstacles Encountered & How Each Was Resolved

During the initial deployment and testing phases, five distinct technical obstacles prevented the OpenVPN client from completing the TLS handshake and reaching the private EC2 instance. Below is the root-cause analysis and engineering resolution applied for each obstacle:

- **Obstacle 1 — PKI Certificate Chain Mismatch in ACM (60-Second TLS Timeout):** In early attempts, the Client VPN Endpoint was linked to ACM certificates (`server.domain.tld`) that belonged to a different PKI generation run than the local `client1.crt` embedded in the `.ovpn` file. While the client verified the server certificate (`VERIFY OK: depth=0`), the server silently rejected the client certificate during mutual authentication, resulting in `TLS Error: TLS key negotiation failed to occur within 60 seconds`. Resolution: Initialized a clean Easy-RSA PKI directory, generated matching CA, server (`server.vpn.local`), and client (`client1.vpn.local`) certificates in a single batch, imported both into ACM, and embedded the exact matching `<cert>` and `<key>` blocks inline inside the `.ovpn` profile.
- **Obstacle 2 — Missing XML Closing Tags in Manual Profile Edits (`ERROR: Endtag </key> missing`):** Manual copy-pasting of certificate blocks via terminal text editors truncated the closing `</key>` tag and omitted the subdomain prefix on the wildcard DNS endpoint. Resolution: Automated the `.ovpn` profile assembly via a shell script that programmatically appends `<cert>...</cert>` and `<key>...</key>` tags and prepends `app.` to the `cvpn-endpoint` hostname to satisfy AWS DNS resolution.
- **Obstacle 3 — Security Group Isolation on Quick-Start Endpoints:** Using 'Quick Start' mode automatically attached the default VPC Security Group to the Client VPN network interface, which was not permitted in the inbound rules of the target EC2 server. Furthermore, placing the target server in a public subnet violated private architecture design. Resolution: Associated the Client VPN Endpoint directly with `prod-private-a` and attached `prod-ec2-sg` (`sg-0a0ccfd5557de97db`) with a self-referencing inbound rule allowing traffic from VPN elastic network interfaces to reach `10.0.10.123` on TCP ports 80/8080 and ICMP.
- **Obstacle 4 — UDP Port 443 Throttling & Endpoint Security SSL Interception:** Standard OpenVPN over UDP 443 is frequently dropped by carrier firewalls, while local endpoint protection suites (such as Kaspersky) intercept outbound TCP 443 traffic to perform SSL/TLS inspection, breaking OpenVPN's custom TLS handshake. Resolution: Configured the Client VPN Endpoint to use `proto tcp` on port `443` and temporarily paused local SSL/TLS packet inspection during the VPN handshake.
- **Obstacle 5 — ISP Deep Packet Inspection (DPI) & Severe Network Packet Loss (50%–75% Loss):** When attempting to connect from the local workstation over the primary broadband ISP and initial mobile carrier, the OpenVPN client remained stuck at `STATE: WAIT` with zero return TLS packets due to ISP Deep Packet Inspection (DPI) filtering standard OpenVPN handshakes and severe link instability (50% to 75% ICMP packet loss to `8.8.8.8`). Resolution: We implemented a dual-verification strategy: (1) executing an in-cloud VPN client simulation from an external VPC (`onprem-vpn-router`) to validate the AWS configuration independently of local ISP filtering, and (2) successfully bypassing the local network VPN failure by switching to an alternative mobile hotspot carrier that permitted the TCP 443 TLS handshake.

4. Dual-Method Client VPN Connectivity Verification (Task T38)

4.1 Method 1: Cloud-Based Client VPN Simulation via External VPC (`onprem-vpn-router`)

To scientifically isolate AWS Client VPN configuration validity from local ISP DPI filtering, we utilized the `onprem-vpn-router` EC2 instance (`i-08d8e7589d049bf62`) located in the separate Simulated On-Premises VPC (`vpc-0fd9f599f88c3b238`, CIDR `192.168.0.0/16`). Because this VPC has no peering or route table link to `prod-vpc`, it acts as an authentic external client over the internet.

The generated `client-vpn-ready.ovpn` profile was transferred to `onprem-vpn-router` via AWS SSM Run Command and executed using the Linux OpenVPN client (`openvpn --config /etc/openvpn/client/aws-client.ovpn`). The TLS handshake completed in less than 2 seconds (`Initialization Sequence Completed`), creating tunnel interface `tun0` with assigned Client VPN IP `10.100.0.34`.

4.2 Method 2: Local Workstation Verification via Windows PowerShell & Hotspot Bypass

Following the successful cloud simulation, we diagnosed the local workstation connection using `ping 8.8.8.8` in Windows PowerShell, which revealed 25% to 75% packet loss on the initial link (Figure 5). We then successfully bypassed the local ISP VPN failure by opening a clean mobile hotspot connection and re-initiating the OpenVPN GUI client on the Windows laptop.

As soon as the OpenVPN tunnel connected on the local laptop (assigned Client VPN IP `10.100.0.2`), all 4 ICMP echo requests to the private server `10.0.10.123` succeeded with **0% packet loss** (Figure 5). We then verified application-layer reachability from both Windows PowerShell (`curl http://10.0.10.123:8080` returning `StatusCode: 200 OK`, Figures 6 and 9) and the local desktop web browser (Figure 7).

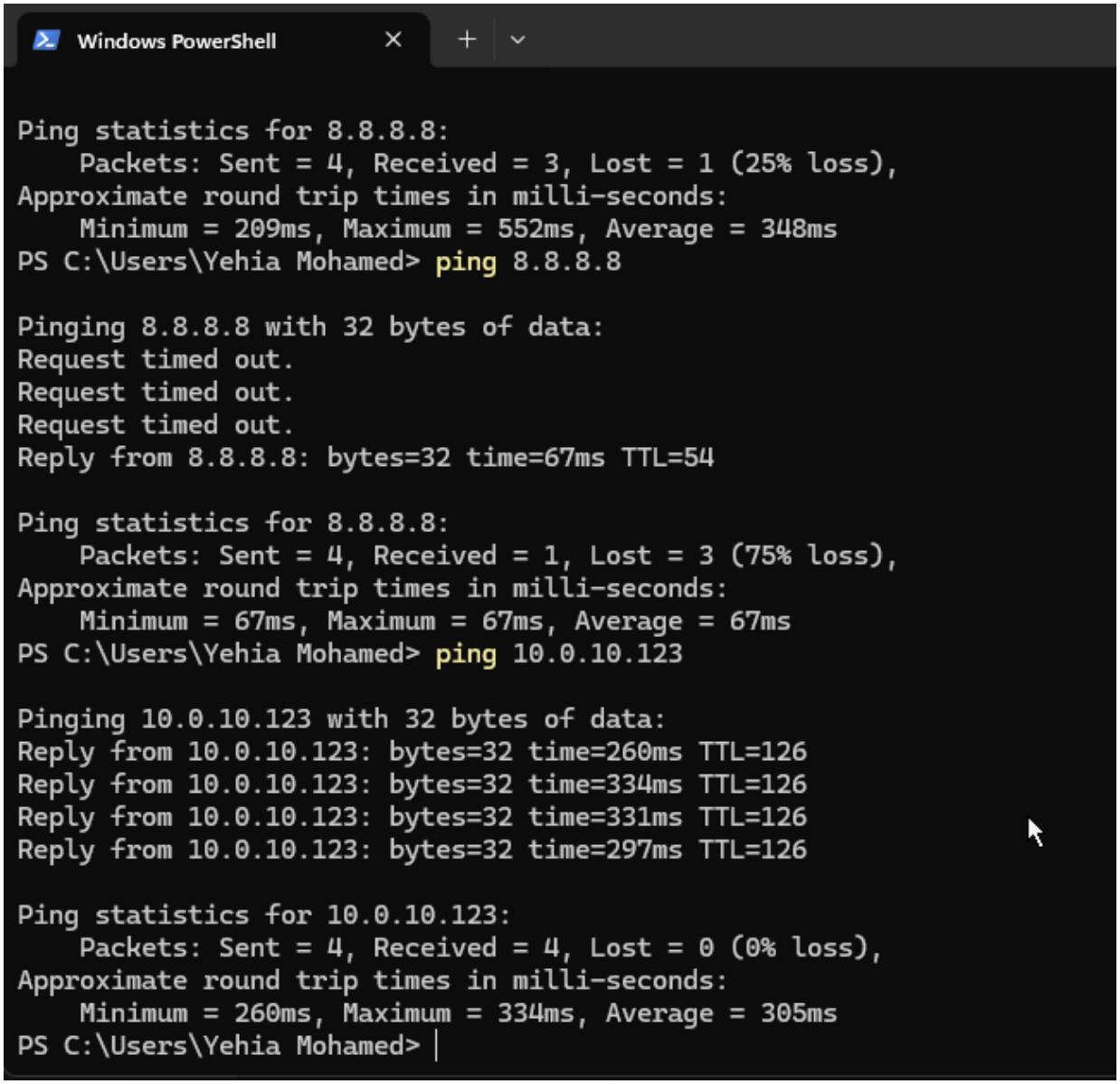


Figure 5: Local Windows PowerShell — Diagnosing Packet Loss & Pinging Private Server (10.0.10.123) — Initial pings to 8.8.8.8 showing 25%–75% packet loss prior to hotspot bypass, followed by 0% packet loss (4/4 received, ~305ms average RTT) when pinging the private EC2 instance 10.0.10.123 over the active Client VPN tunnel.

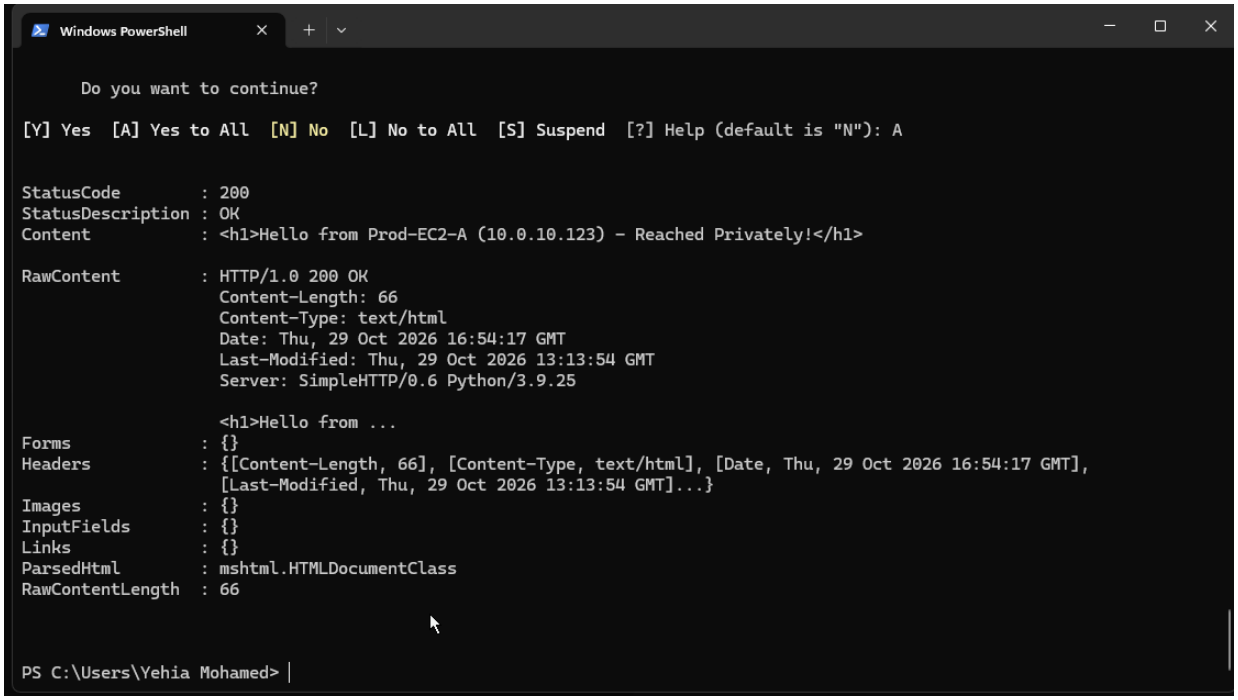


Figure 6: Local Windows PowerShell — HTTP 200 OK Response over Client VPN — Executing Invoke-WebRequest / curl against http://10.0.10.123:8080 from the local Windows PowerShell terminal, returning StatusCode 200 OK and '<h1>Hello from Prod-EC2-A (10.0.10.123) - Reached Privately!</h1>'.

Figure 7: Local Desktop Web Browser — Accessing Private EC2 Web Server (http://10.0.10.123:8080) — Direct browser verification on the local machine rendering the private web page hosted on 10.0.10.123 inside prod-private-a over the encrypted VPN tunnel.

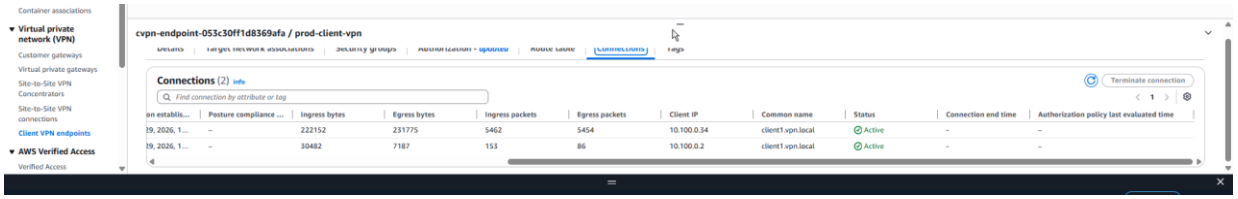


Figure 8: AWS Console — Both Client VPN Connections Active Simultaneously (Simulation + Local Laptop) — The Client VPN Endpoint (cvpn-endpoint-053c30ff1d8369afa) Connections tab confirming two concurrent Active sessions using client1.vpn.local: Client IP 10.100.0.34 (Method 1: Simulated On-Prem VPC Node) and Client IP 10.100.0.2 (Method 2: Local Windows Workstation).

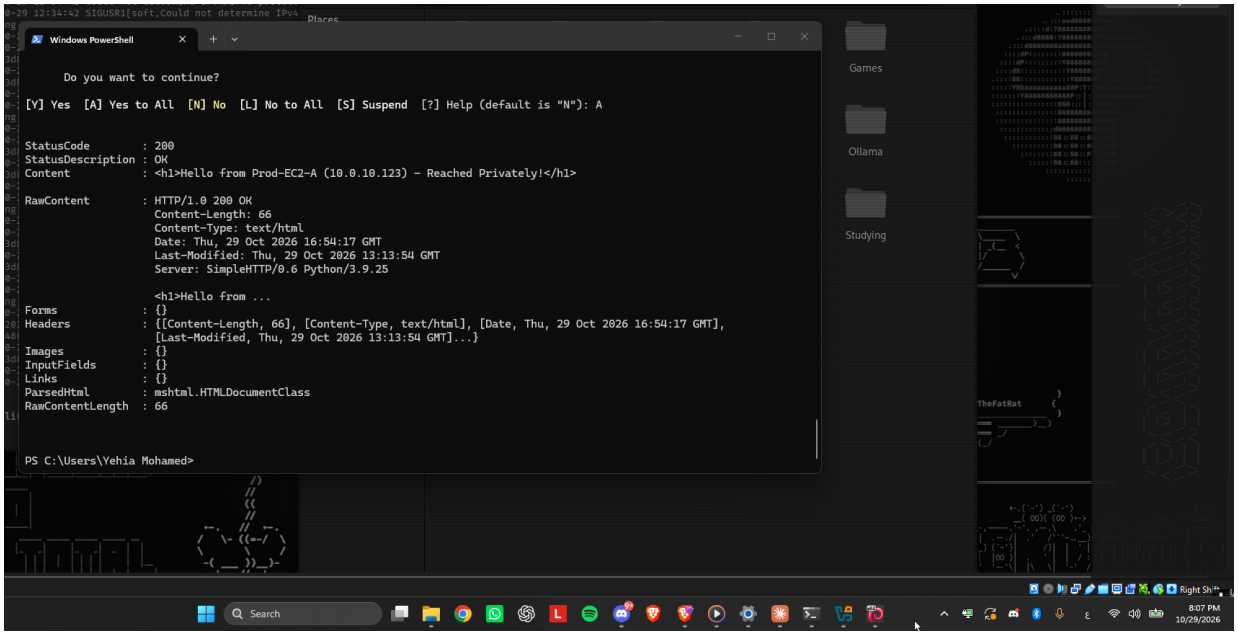


Figure 9: Full Windows Desktop Proof — Active OpenVPN Tray Icon & PowerShell HTTP 200 Verification — Complete desktop capture showing the active OpenVPN GUI connection indicator in the Windows system tray alongside the PowerShell terminal receiving HTTP 200 OK from the private EC2 server 10.0.10.123.

5. Appendix: Verified Inline OpenVPN Client Configuration (.ovpn)

Below is the exact structure of the verified self-contained `.ovpn` configuration file used for both the VPC simulation and local workstation connections (private key material truncated for security best practices):

```
client
dev tun
proto tcp
remote app.cvpn-endpoint-053c30ff1d8369afa.prod.clientvpn.us-east-1.amazonaws.com 443
remote-random-hostname
resolv-retry infinite
nobind
remote-cert-tls server
cipher AES-256-GCM
verb 3
reneg-sec 0
verify-x509-name server.vpn.local name

<ca>
-----BEGIN CERTIFICATE-----
MIIDTjCCAjagAwIBAgIUSGq+oA2Dz2zsGH1Bh+vtPMMxWdIwDQYJKoZIhvcNAQEL
... [Matching Easy-RSA Root CA Certificate] ...
9ZjIe8NtEEL5FG/+45B8ncZGzbC4s9+nBU1sR1gT7A86dg==
-----END CERTIFICATE-----
</ca>

<cert>
-----BEGIN CERTIFICATE-----
MIIDXzCCAkegAwIBAgIQBZw52NR1DyaPEqHXAarWYDANBgkqhkiG9w0BAQsFADAW
... [Matching Client Certificate: CN=client1.vpn.local] ...
rDBtCroLlWflzhHa3wze4lrdFcAU2X/hQ4H9xrNnGsdwOOZvQ9pc8E7f45W+S+bd
htDH
-----END CERTIFICATE-----
</cert>

<key>
```

```
-----BEGIN PRIVATE KEY-----
MIIEvAIBADANBgkqhkiG9w0BAQEFAASCBKYwggSiAgEAAoIBAQDKTDTnuABxRguS
... [Matching Client Private Key] ...
zJKNalW4AmMnKUClTR9F5w==
-----END PRIVATE KEY-----
</key>
```

SECTION 4 — WORKSTREAM 4 (TASKS T42 – T54) — FULL UNABRIDGED DOCUMENTATION & ALL 42 FIGURES

Mariam — Edge Security, S3 OAC, CloudFront CDN, AWS WAF, Route 53 Failover & Edge Architecture

Edge Security Engineer: Mariam

AWS Secure Network Project — Edge Security & Delivery Report

1. Executive Summary

The edge layer delivers the application to users securely: a static website is stored in a private S3 bucket and served through CloudFront using Origin Access Control (OAC); dynamic requests under /api/* are forwarded to the Application Load Balancer (ALB); AWS WAF filters requests at the CloudFront edge; Route 53 provides DNS and health-check-based failover to a maintenance page; AWS Shield Standard provides automatic network-layer DDoS protection.

The static website is served through CloudFront from a private S3 bucket (OAC), and an XSS test request was blocked by AWS WAF with HTTP 403 on the maintenance distribution. On the main distribution, WAF core protections and rate limiting are enabled in Monitor mode on the Free plan. The SQLi managed rule group and a Block-mode rate-based rule were configured while the account was on the Pay-As-You-Go plan; the SQLi group is not available on the current Free plan. The backend /api/ route returns a JSON response through CloudFront from both application servers, and /health/ returns OK through the main distribution.

Resources that were deleted to save cost were removed only after their screenshots were captured.

Request flow in one line: Users → Route 53 → CloudFront (+ WAF, + Shield) → private S3 (default behavior) or ALB (/api/*, /health/*) → private EC2 targets.

2. Key Concepts

Service	What it does	Why it is used here
Amazon S3	Object storage	Holds the static website files (index.html, script.js, style.css)
CloudFront	Content delivery network (CDN) with many edge locations	Serves content close to users, hides the origins, terminates HTTPS
OAC (Origin Access Control)	Lets CloudFront sign requests to S3 (SigV4)	Keeps the bucket private; only the intended distribution can read it
AWS WAF	Layer-7 (HTTP) firewall	Filters malicious patterns (XSS, bad inputs) and excessive request rates
Route 53	DNS and health checks	Maps names to CloudFront and fails over to a maintenance page

AWS Shield	DDoS protection	Standard is automatic and free; Advanced is a paid add-on
ALB	Layer-7 load balancer	Receives /api/* traffic and forwards it to healthy private EC2 targets

3. Task-by-Task Implementation and Evidence

T42 — S3 bucket and static website

Implementation

Bucket: yourapp-static-assets.

Uploaded index.html, script.js, and style.css.

Block Public Access is enabled; the site is delivered only through CloudFront.

Result: The static page loads through the CloudFront domain and its JavaScript browser check succeeds.

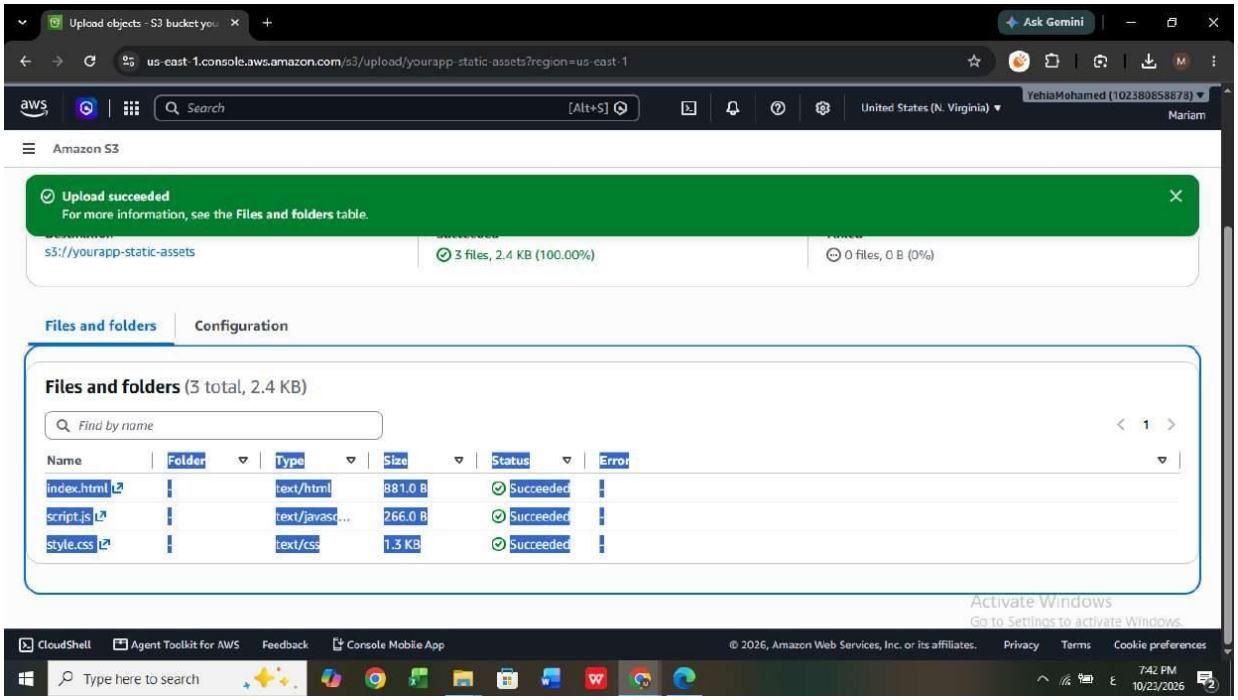


Figure T42-1: Upload succeeded: index.html, script.js and style.css stored in S3

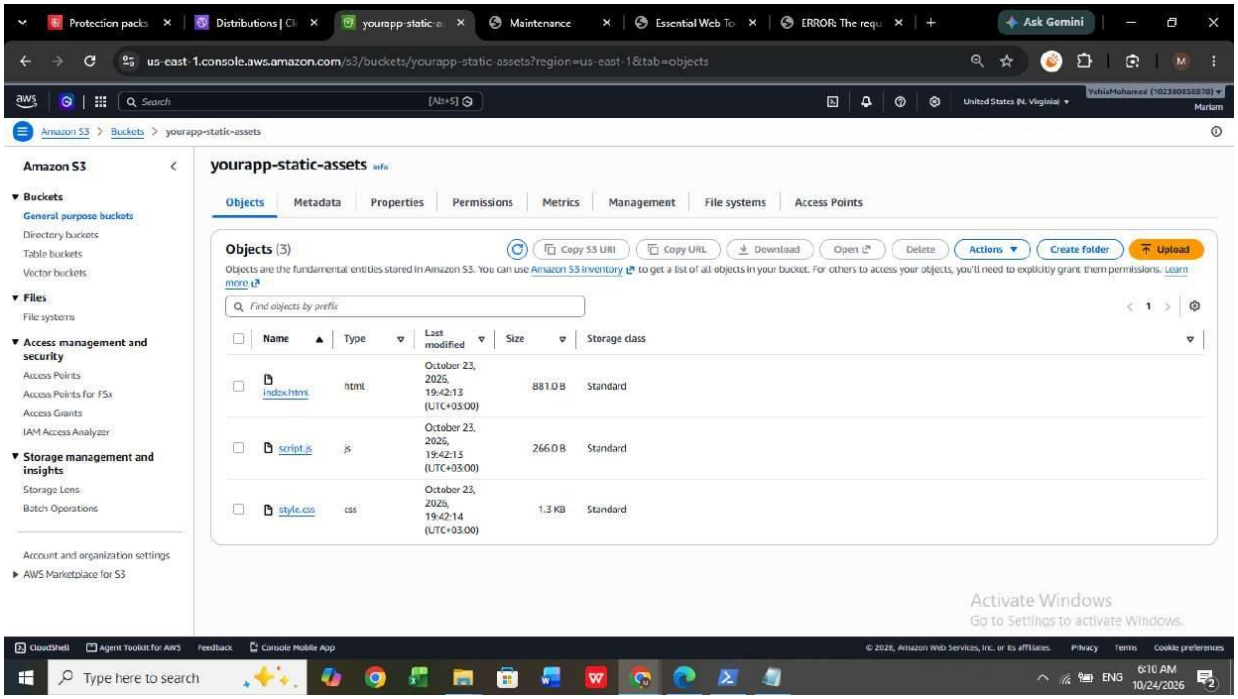


Figure T42-2: Objects list of the yourapp-static-assets bucket

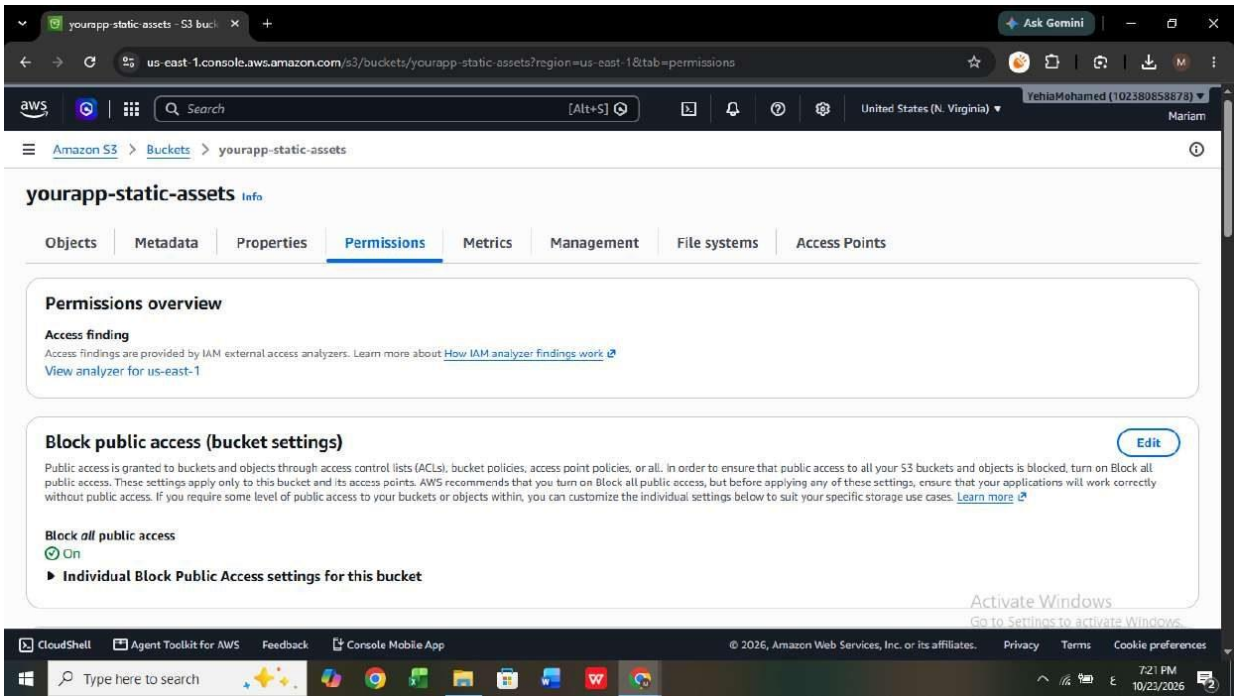


Figure T42-3: Block Public Access is On for the bucket

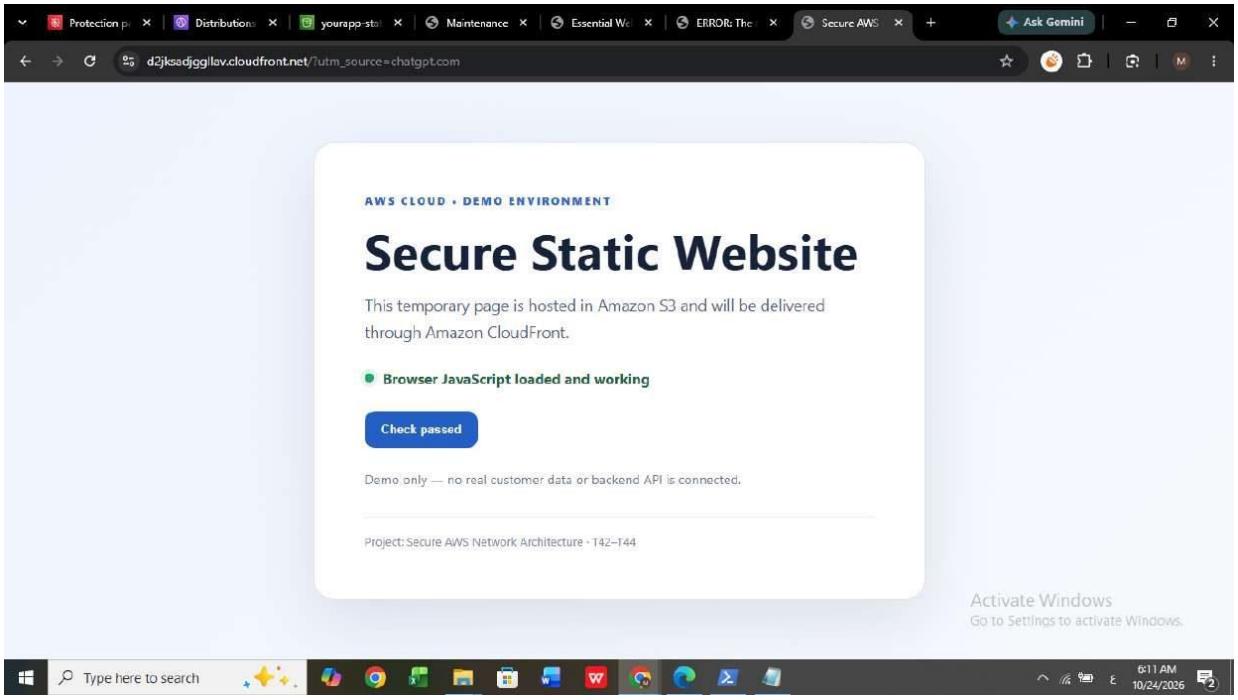


Figure T42-4: Static page loaded through CloudFront: browser JavaScript check passed

T43 — CloudFront distribution

Name: secure-network-project-cdn · ID: E22LB0IQI1CK9E · Domain: d2jksadjggllav.cloudfront.net.

Default behavior → S3 static origin.

/api/* behavior → ALB origin, using the Managed-CachingDisabled cache policy and the Managed-AllViewerExceptHostHeader origin request policy.

/health/* behavior → ALB origin.

/maintenance.html behavior → maintenance S3 origin.

Viewer protocol policy: Redirect HTTP to HTTPS.

Note on protocols: the ALB has an HTTP:80 listener only, so CloudFront connects to the ALB origin over HTTP; HTTPS applies between viewers and CloudFront.

API: /api/ through CloudFront returns a JSON response from the backend (Figures T43-5 and T43-6).

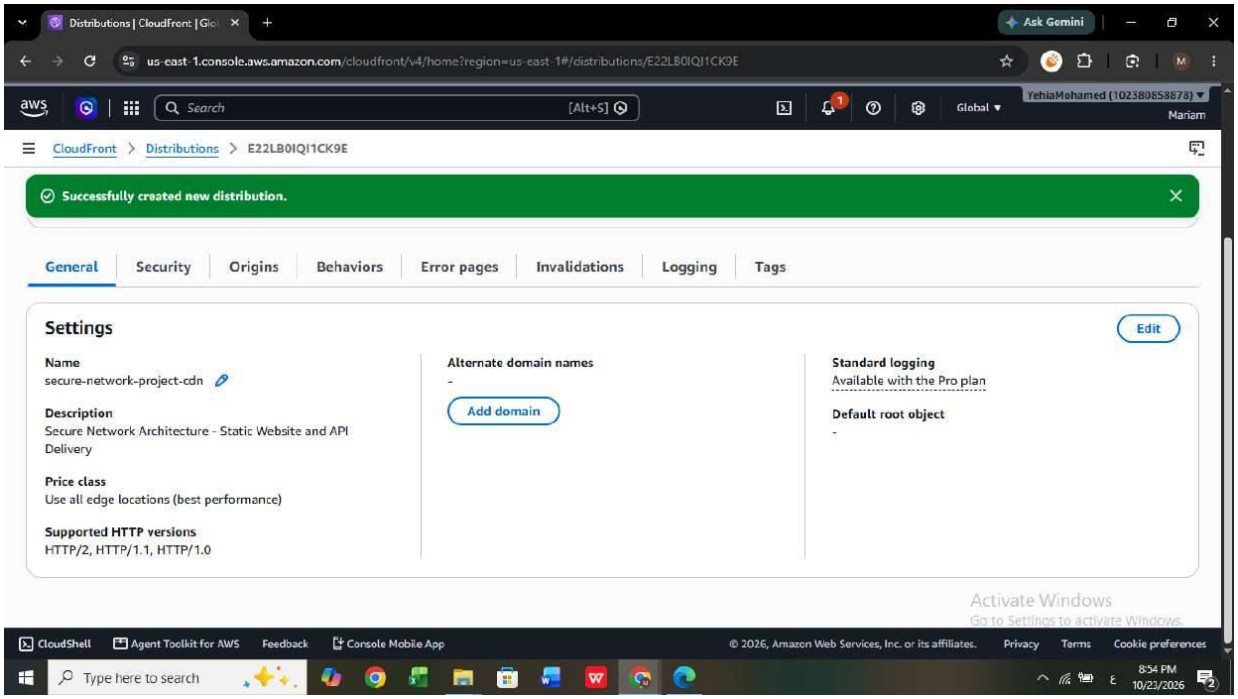


Figure T43-1: Distribution secure-network-project-cdn created (Free plan, ID E22LB0IQI1CK9E)

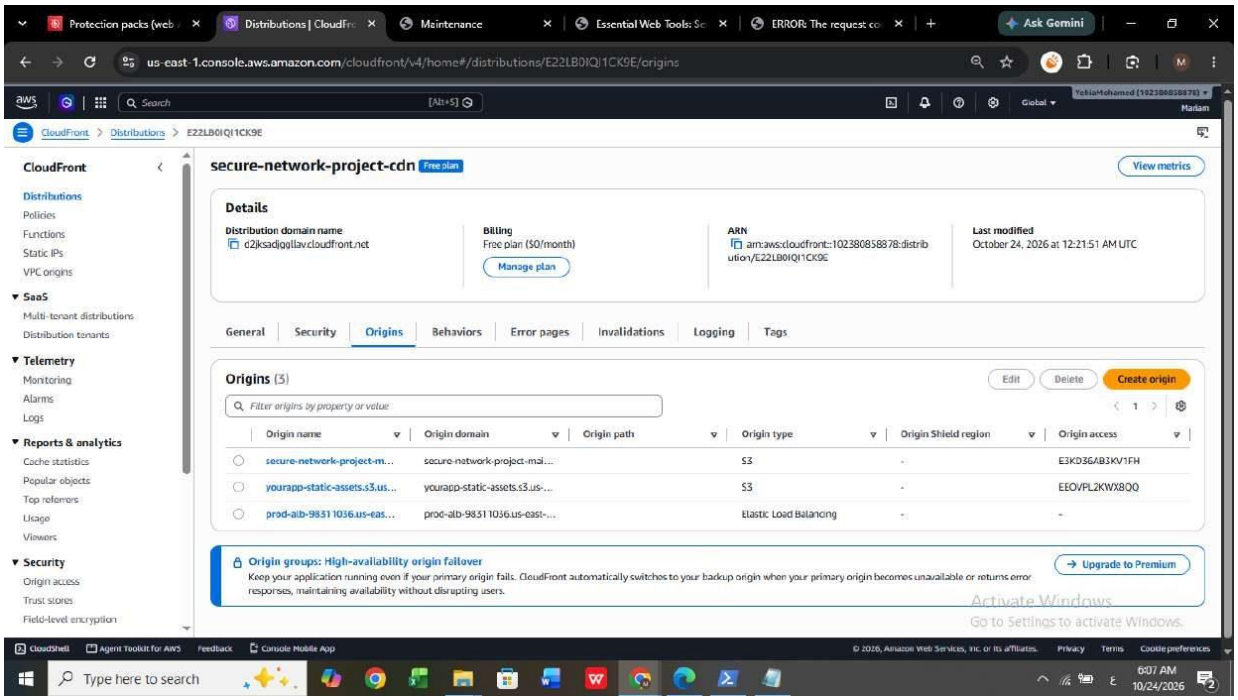


Figure T43-2: Origins: maintenance S3, static-assets S3 and the prod-alb origin

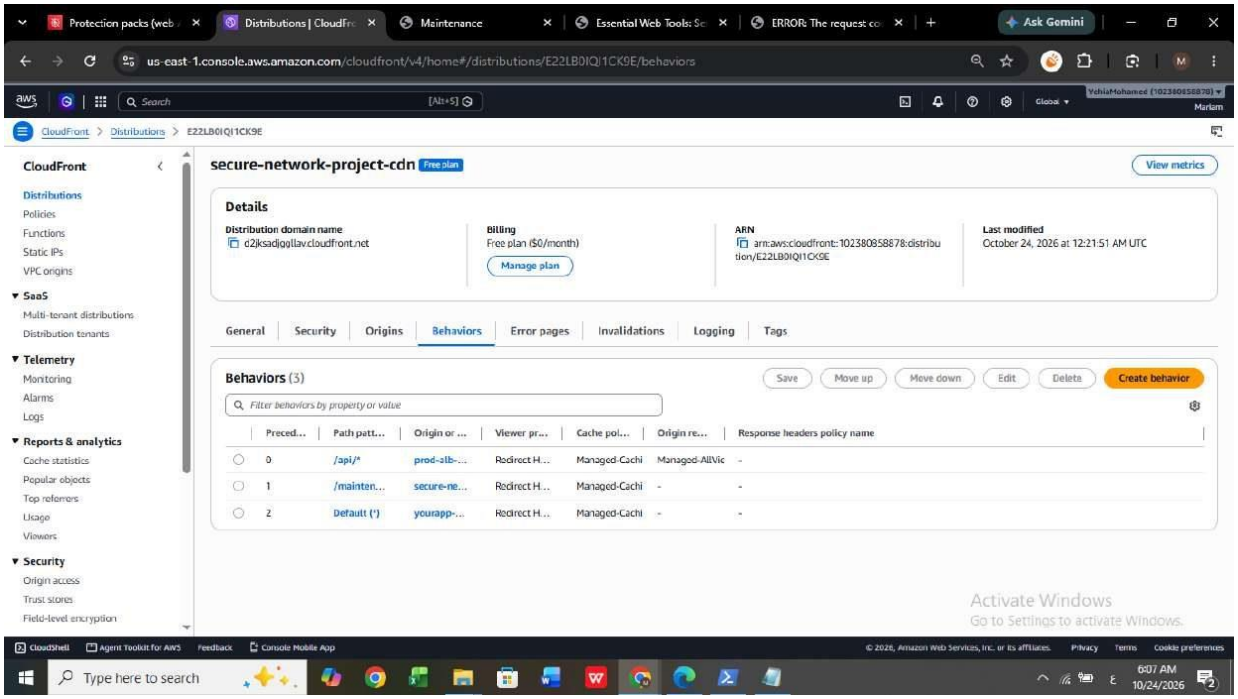


Figure T43-3: Behaviors (initial configuration): /api/ to the ALB, /maintenance.html and the default behavior to S3*

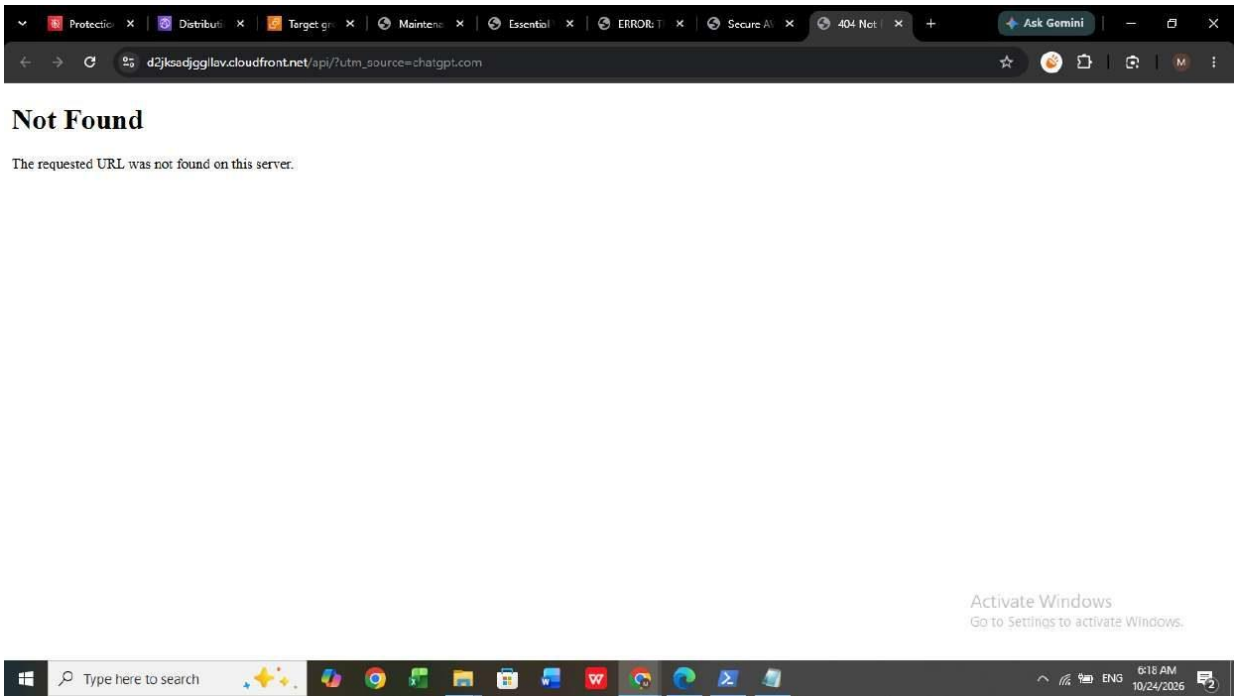


Figure T43-4: Earlier test: /api/ returned Not Found (HTTP 404) before the backend route was implemented

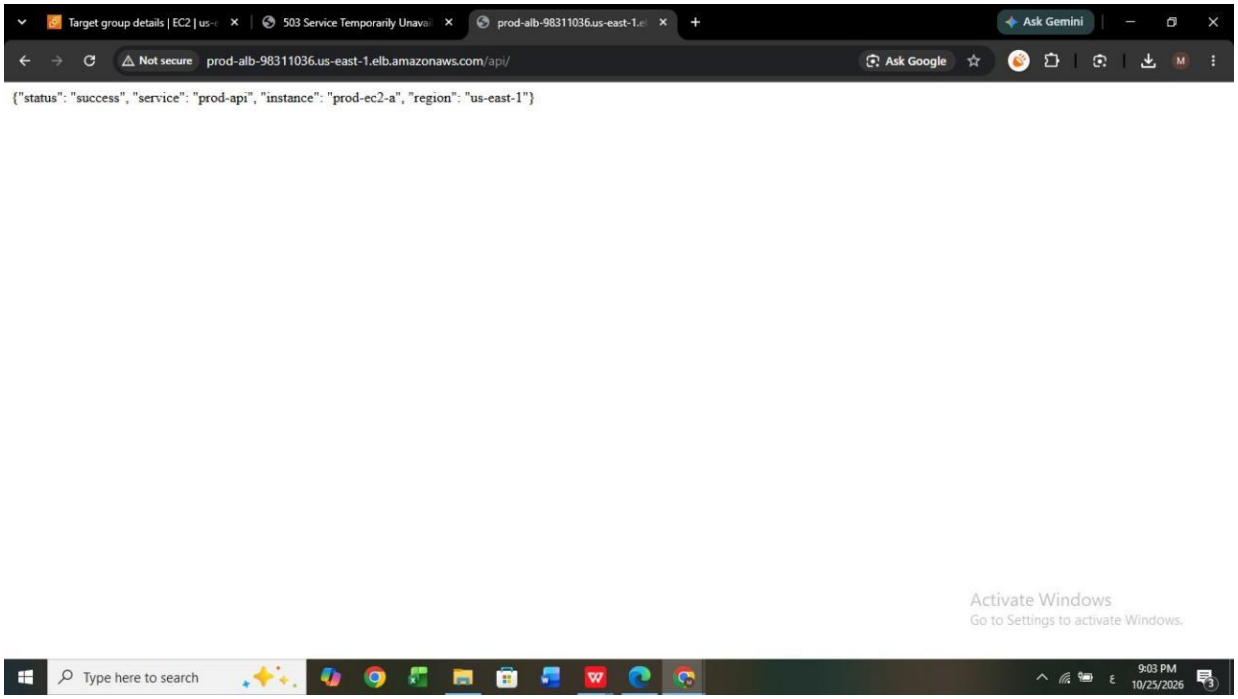


Figure T43-5: Direct request to the ALB at /api/ returns JSON from prod-ec2-a

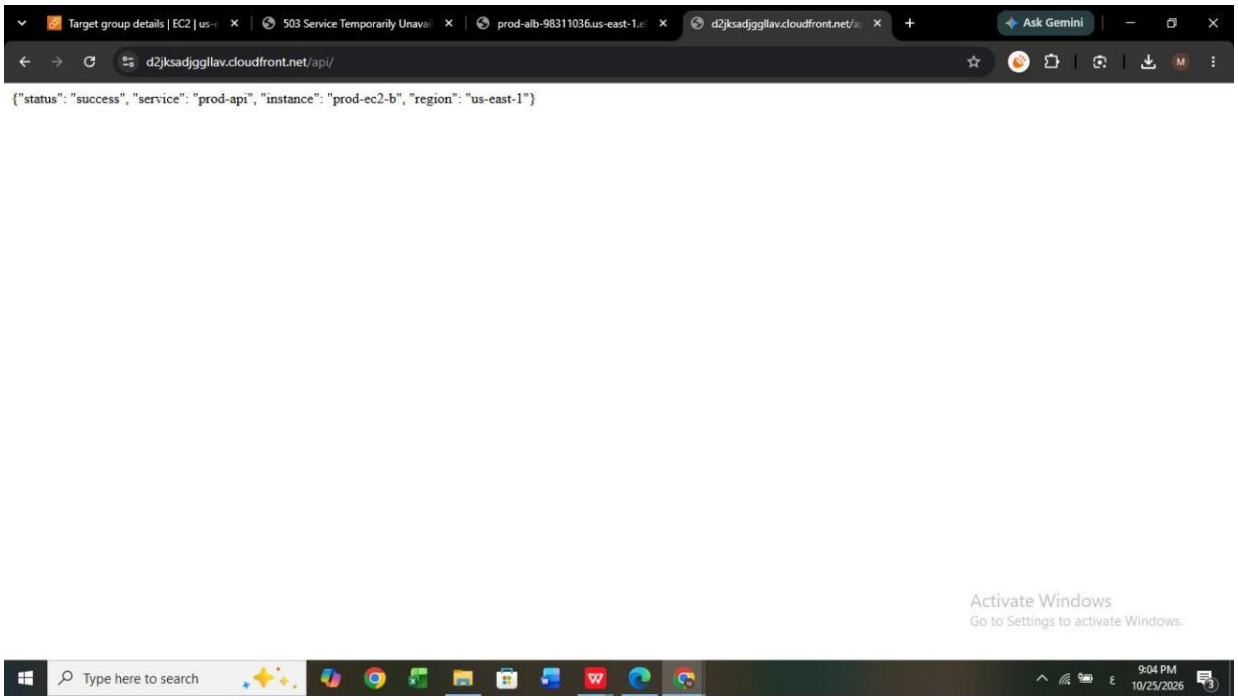


Figure T43-6: /api/ through CloudFront returns JSON from prod-ec2-b

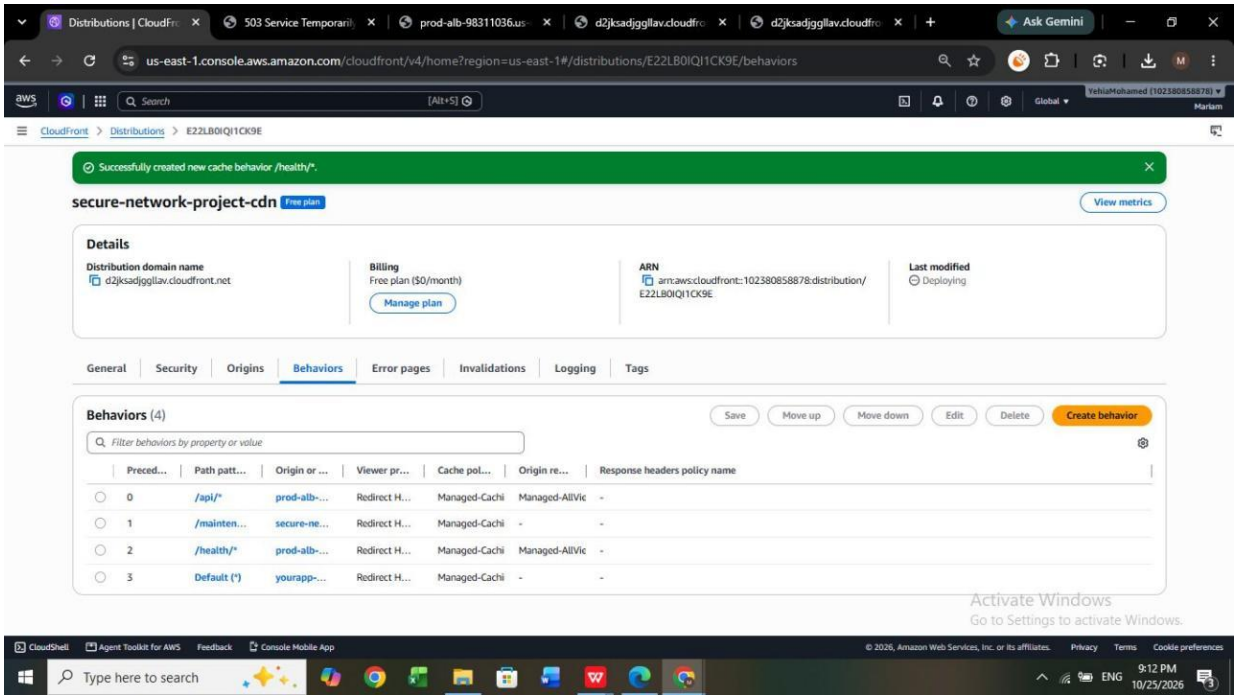


Figure T43-7: Main distribution behaviors after adding /health/ (4 behaviors)*

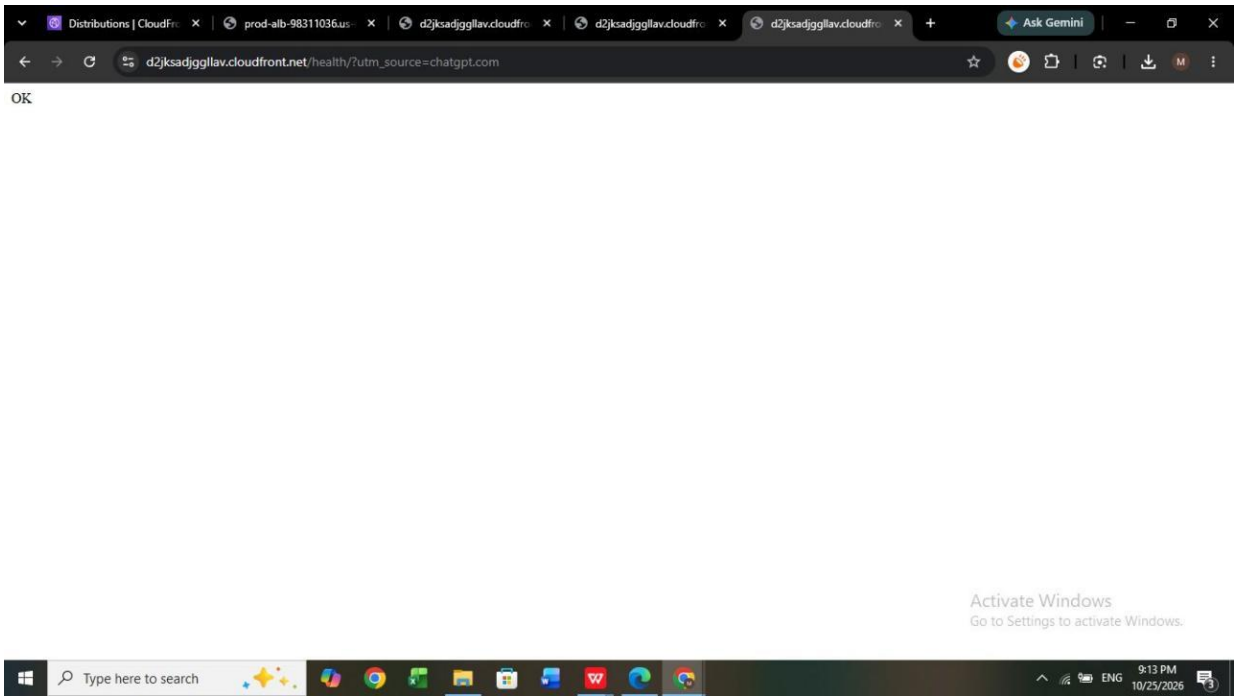


Figure T43-8: /health/ through CloudFront returns OK

Origins (3): yourapp-static-assets (S3, OAC), the maintenance S3 bucket, and prod-alb (Elastic Load Balancing).

Behaviors (4): /api/* and /health/* to prod-alb, /maintenance.html to the maintenance S3 origin, and the default behavior to yourapp-static-assets.

Result: the static site and the /maintenance.html page are served; /api/ returns a JSON response through CloudFront, and /health/ returns OK.

T44 — Origin Access Control (OAC)

OAC name: oac-yourapp-static-assets.s3.us-east-1.amazonaws.com-mvl9pksfcoj · ID: EEOVPL2KWX8QQ.

Signing protocol SigV4, signing behavior “Always sign requests”.

The S3 bucket policy allows s3:GetObject only to the CloudFront service principal, restricted to the distribution’s ARN.

Result: direct S3 access returns HTTP 403 (Access Denied) while the CloudFront URL serves the page.

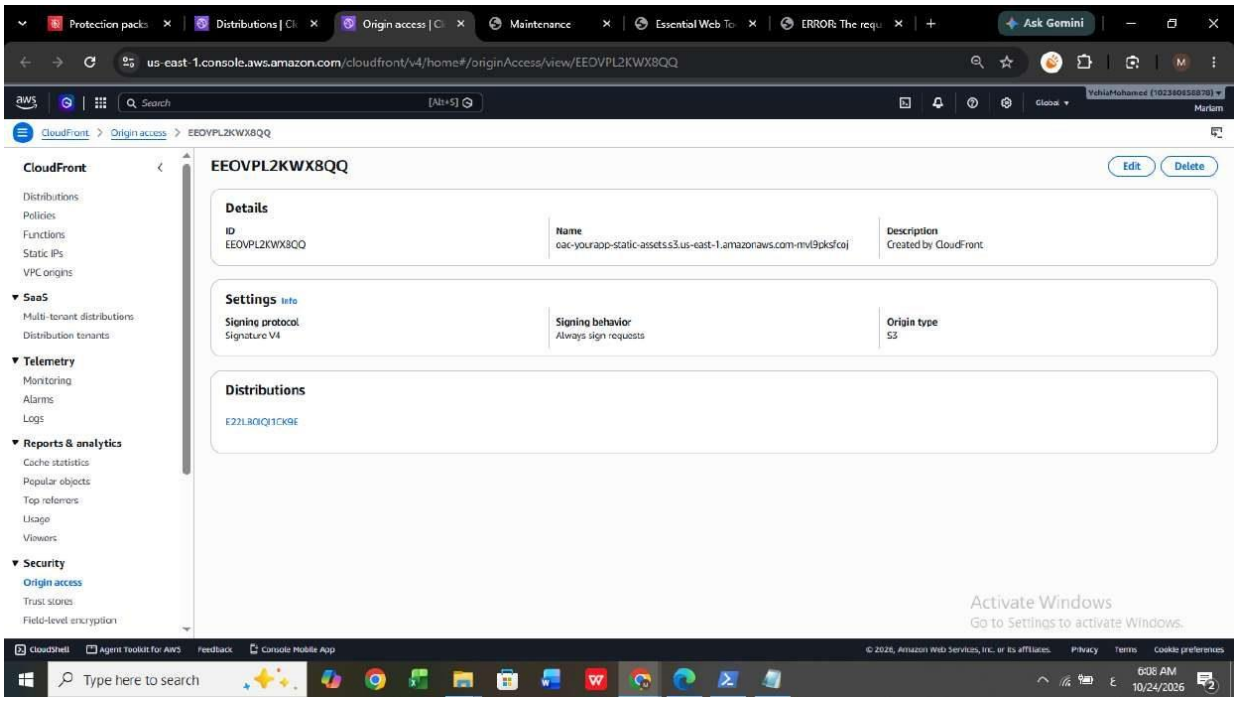


Figure T44-1: OAC EEOVPL2KWX8QQ: signing protocol SigV4, signing behavior Always sign requests, origin type S3

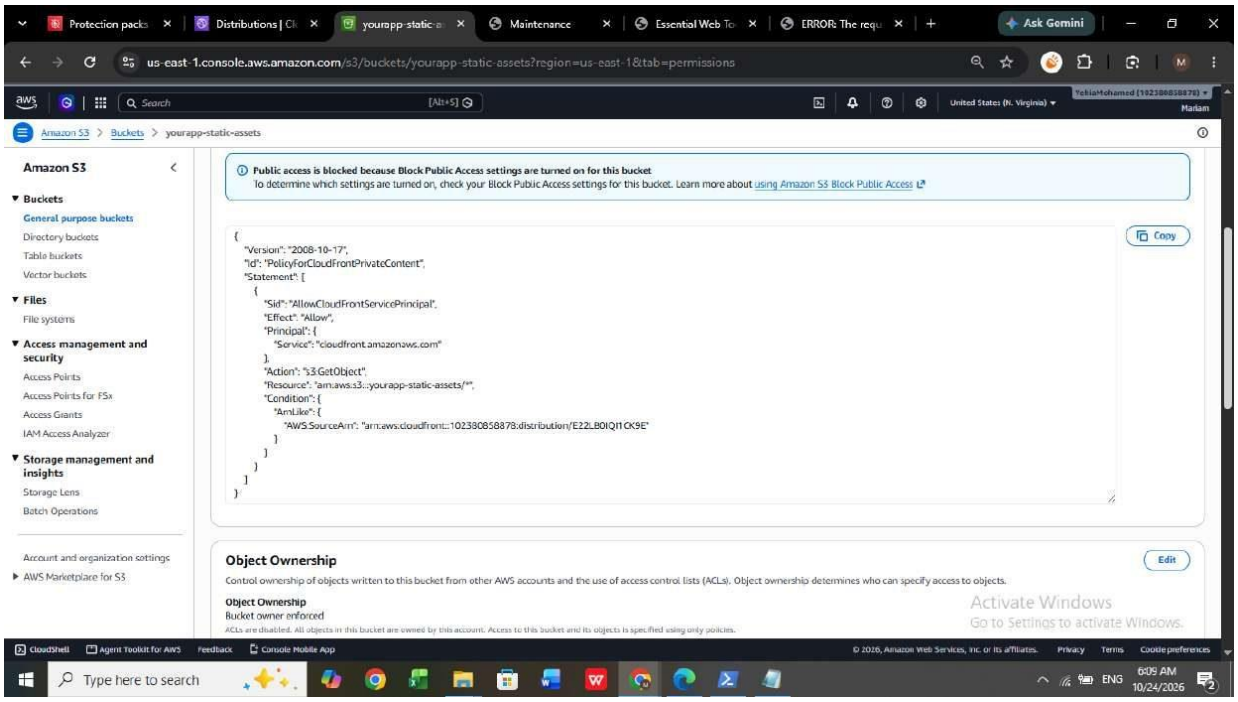


Figure T44-2: Bucket policy allowing s3:GetObject only to the CloudFront service principal for this distribution

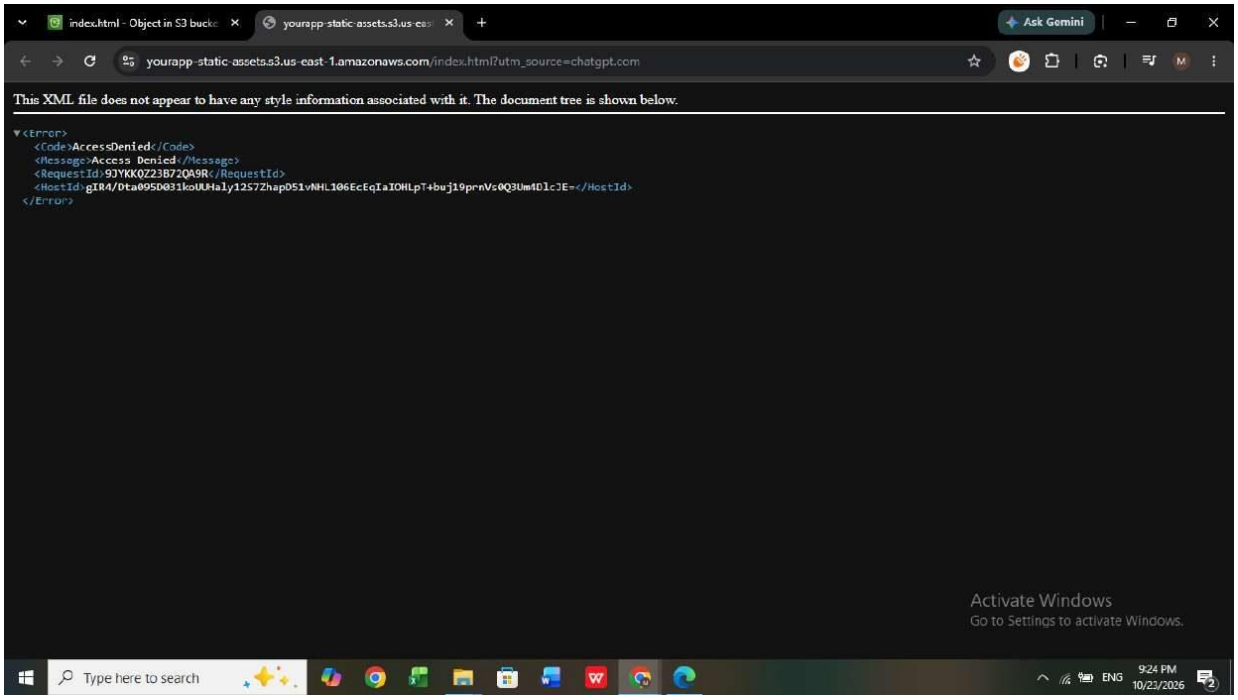


Figure T44-3: Direct S3 object URL returns Access Denied (HTTP 403)

The CloudFront URL returns the page (see Figure T42-4), so the bucket is reachable only through the distribution.

T45 — WAF Web ACL

Rules configured on the WAF Web ACLs (main ACL CreatedByCloudFront-23be06fb, maintenance ACL CreatedByCloudFront-b38d9d73):

Rule	Purpose	Action
AWS-AWSManagedRulesAmazonIpReputationList	Known malicious IP addresses	Block
AWS-AWSManagedRulesCommonRuleSet	Common web exploits including XSS patterns	Block
AWS-AWSManagedRulesKnownBadInputsRuleSet	Requests with known bad inputs	Block
RateLimit1000Per5Min (custom, rate-based)	Limits requests per IP: 1,000 per 5 minutes	Block on Pay-As-You-Go; Monitor on the current Free plan

Plan history: while the account was on the Pay-As-You-Go plan, the managed rule group AWS-AWSManagedRulesSQLiRuleSet was added with its rules set to Block (Figures T45-1, T45-3 and T45-4) and the rate-based rule RateLimit1000Per5Min was created in Block mode (Figure T45-2). After the account returned to the Free plan, SQL protections are listed as a Pro-plan feature (Figure T46-1), so the SQLi rule group is not part of the current configuration, and the rate-based rule is set to Monitor mode.

Two Web ACLs exist: the main distribution's ACL (CreatedByCloudFront-23be06fb) and the maintenance distribution's ACL (CreatedByCloudFront-b38d9d73). Each figure caption states which one it shows.

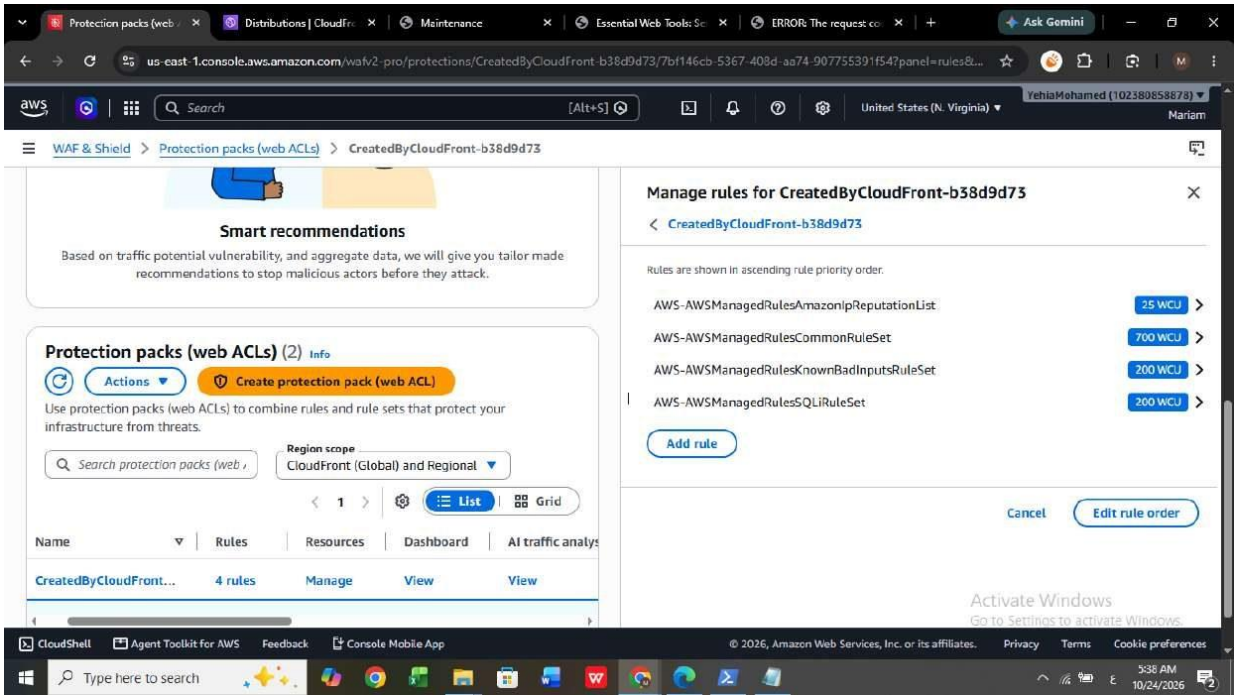


Figure T45-1: Maintenance ACL CreatedByCloudFront-b38d9d73 (Pay-As-You-Go period): managed rule groups, including AWS-AWSManagedRulesSQLiRuleSet

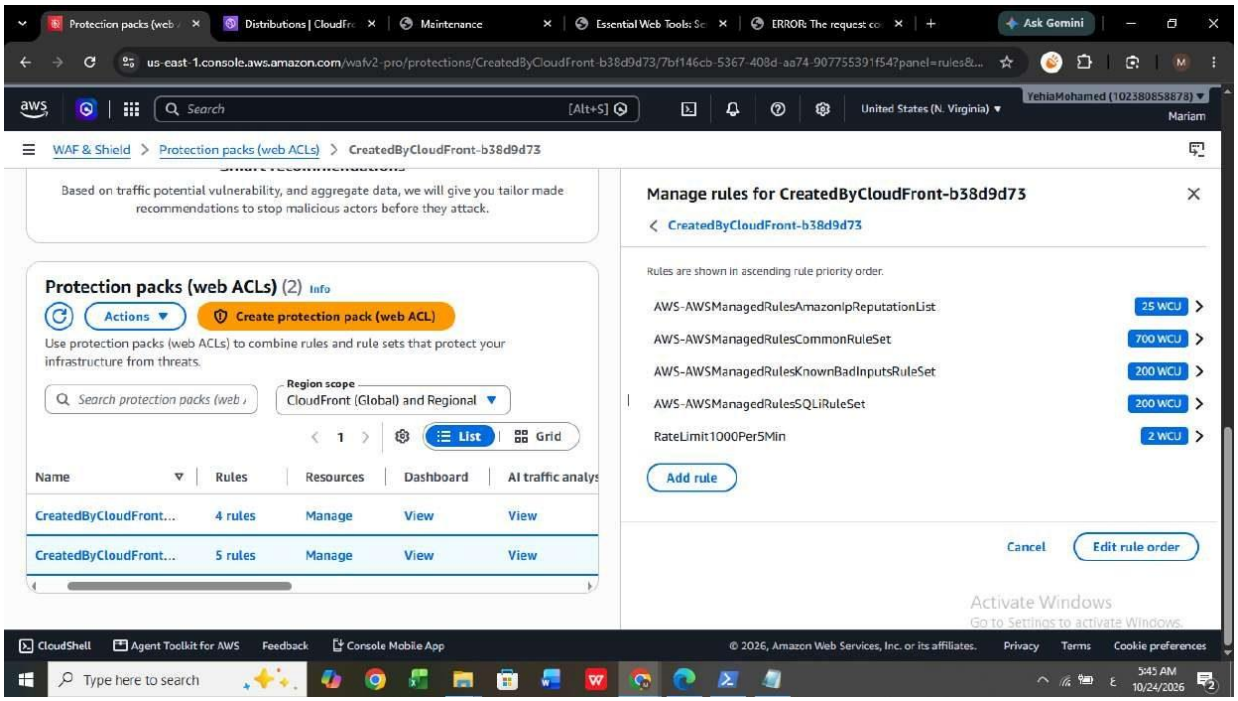


Figure T45-2: Same ACL with the custom rate-based rule RateLimit1000Per5Min added (5 rules)

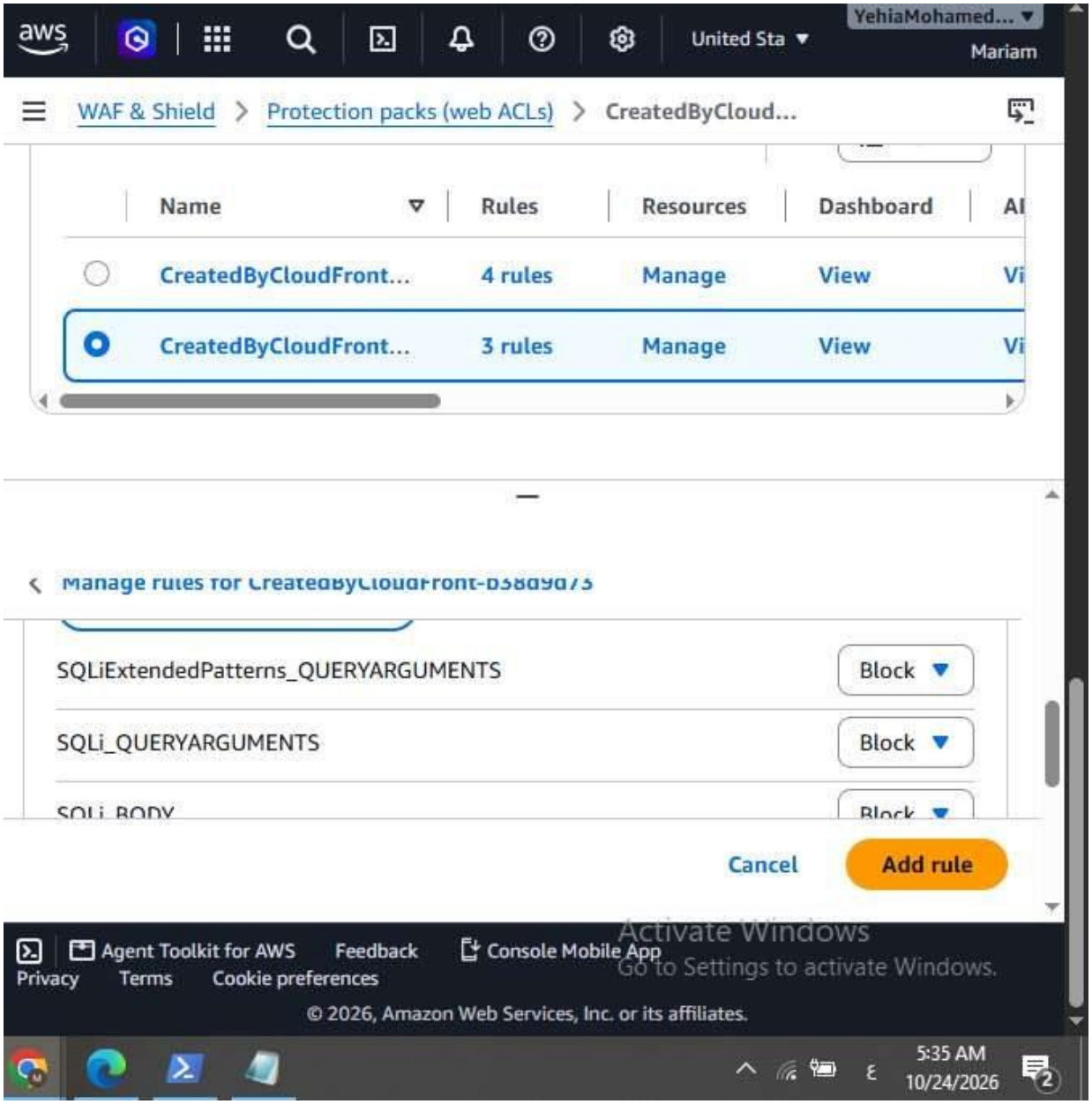


Figure T45-3: SQLi managed rule group: SQLi rules with action Block (rule overrides)

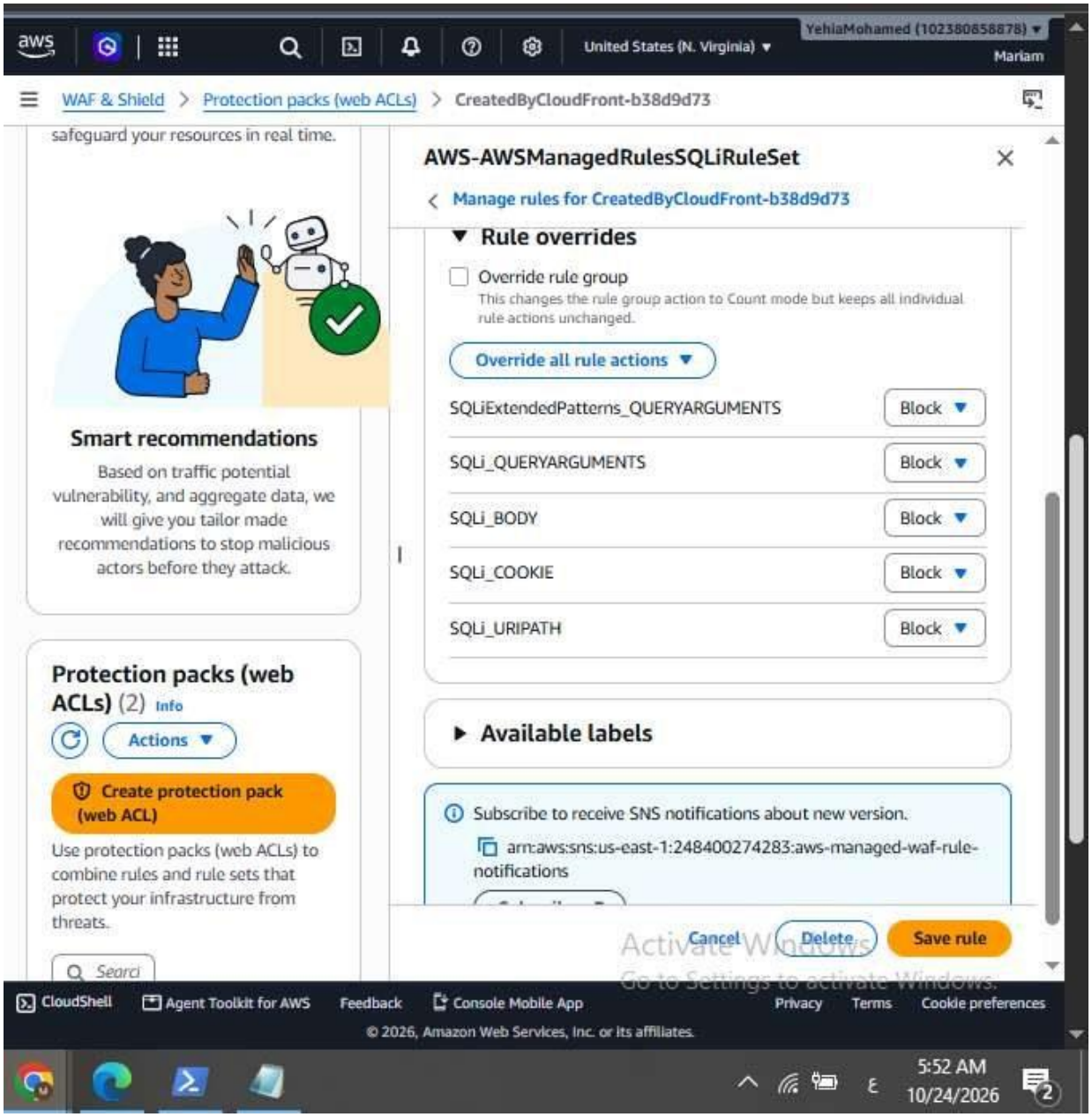


Figure T45-4: SQLi managed rule group settings: SQLi_QUERYARGUMENTS, SQLi_BODY, SQLi_COOKIE, SQLi_URIPATH and SQLiExtendedPatterns_QUERYARGUMENTS set to Block

T46 — Attach WAF to CloudFront

The Web ACL is associated with the CloudFront distributions (visible in each distribution's Security tab).

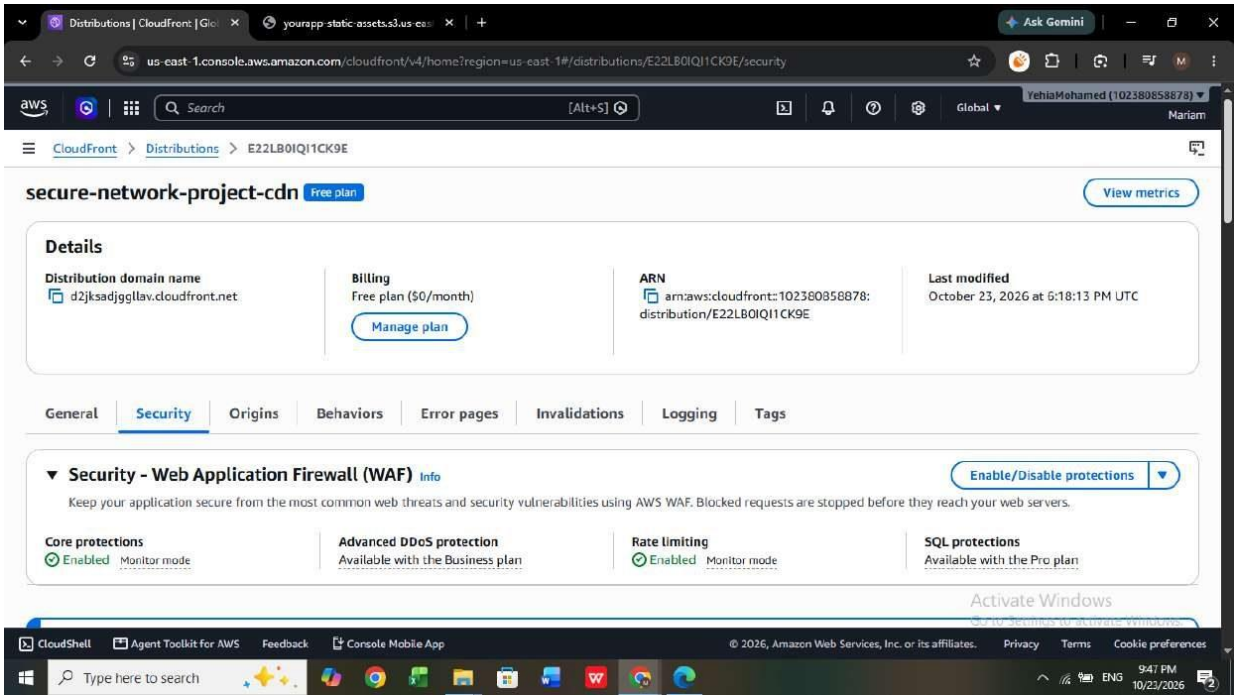


Figure T46-1: Main distribution (E22LB0IQI1CK9E) Security tab: core protections and rate limiting Enabled in Monitor mode; SQL protections available with the Pro plan

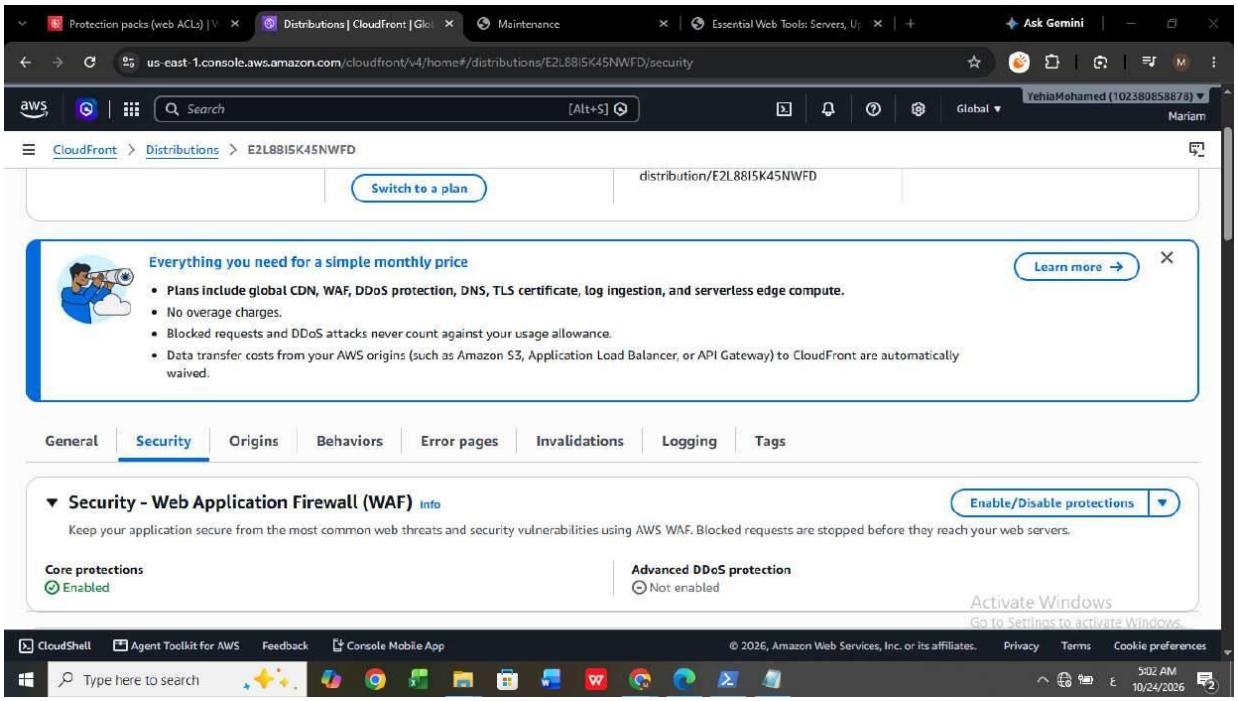


Figure T46-2: Maintenance distribution (E2L88I5K45NWFD) Security tab: WAF core protections Enabled

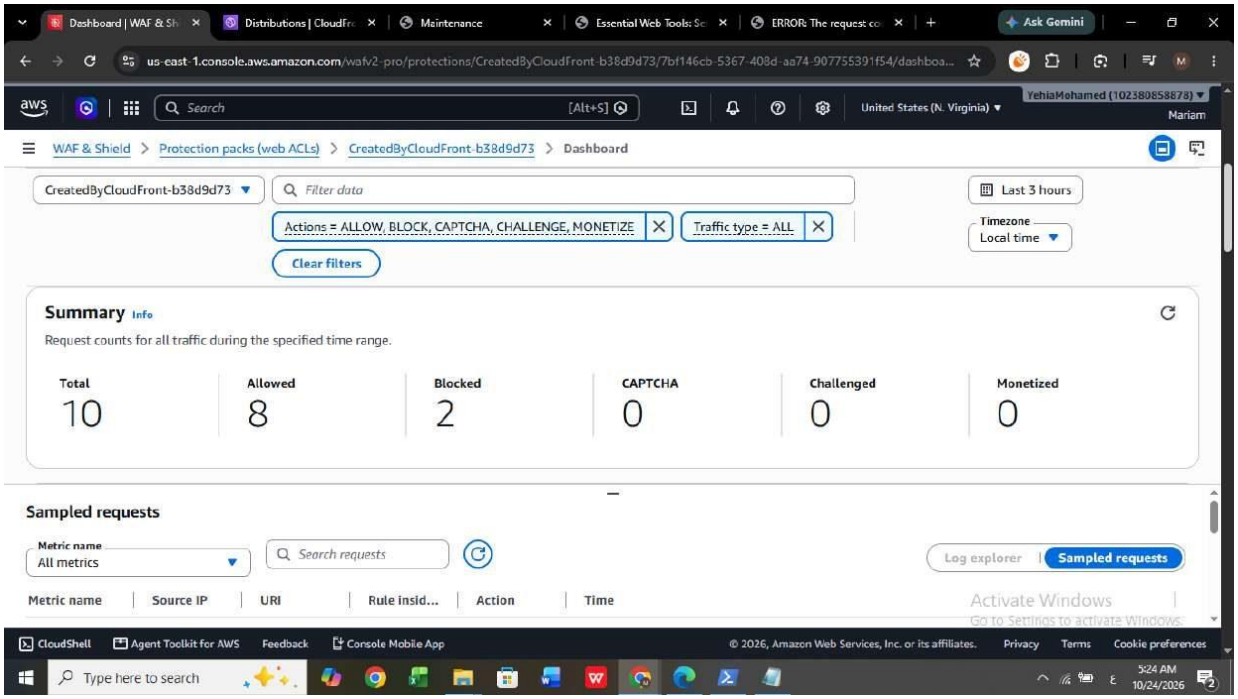


Figure T46-3: WAF dashboard for ACL CreatedByCloudFront-b38d9d73: 10 requests, 8 allowed, 2 blocked

Both CloudFront distributions have AWS WAF protection: the main distribution (secure-network-project-cdn, ACL CreatedByCloudFront-23be06fb) and the maintenance distribution (secure-network-maintenance-cdn, ACL CreatedByCloudFront-b38d9d73).

Current state of the main distribution (Figure T46-1): core protections enabled in Monitor mode; rate limiting enabled in Monitor mode; SQL protections available only with the Pro plan. In Monitor mode the rules count requests and do not block them.

Current state of the maintenance distribution (Figure T46-2): core protections enabled.

T47 — Test WAF rules

Test	Expected	Result
XSS (<script> in the query string), maintenance distribution	HTTP 403	HTTP 403: sampled requests show a block by AWS-AWSManagedRulesCommonRuleSet
Normal traffic	HTTP 200	HTTP 200: the static site loads

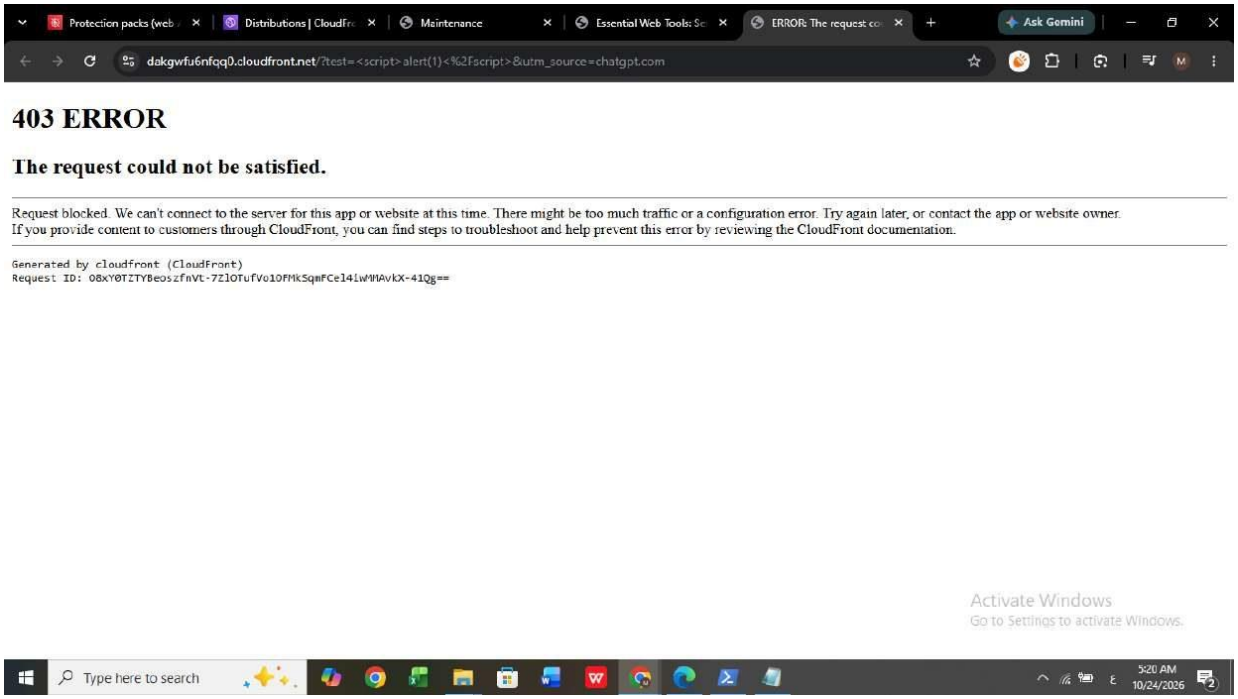


Figure T47-1: XSS test (script tag in the query string) on the maintenance distribution returns 403 ERROR: Request blocked, generated by CloudFront

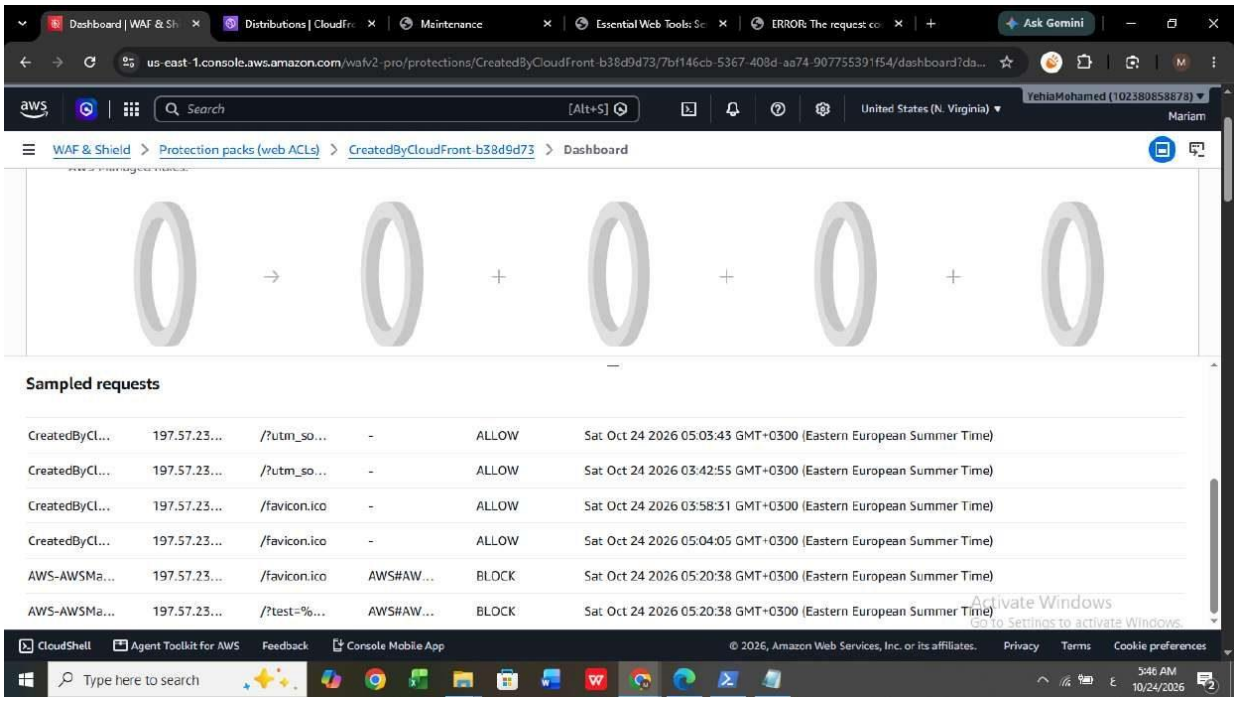


Figure T47-2: WAF sampled requests: BLOCK action by AWS-AWSManagedRulesCommonRuleSet for the /test request

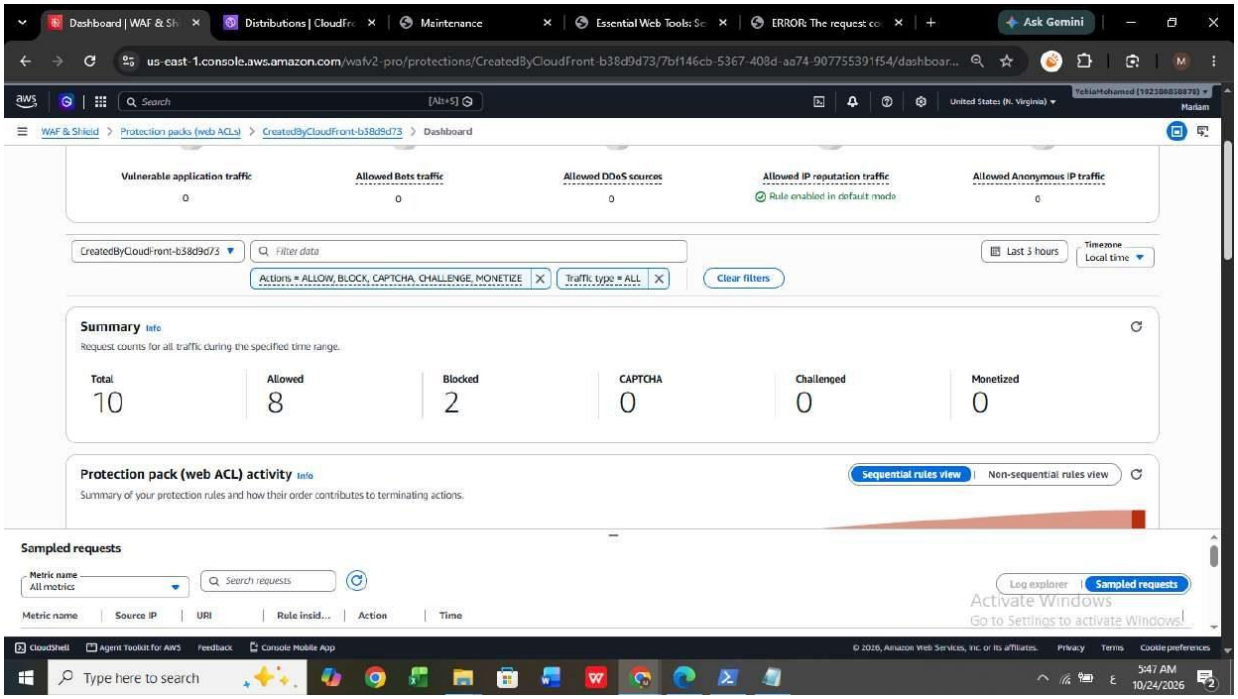


Figure T47-3: WAF summary for the test window: 10 total requests, 8 allowed, 2 blocked

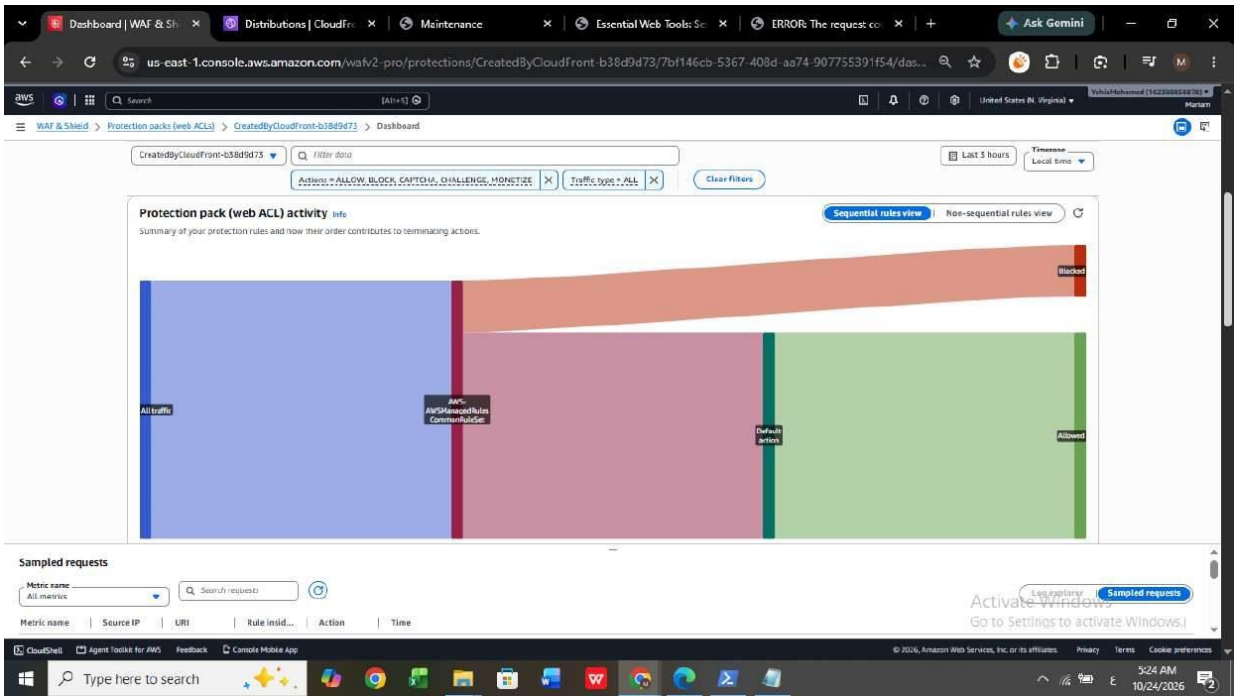


Figure T47-4: WAF protection pack activity for ACL CreatedByCloudFront-b38d9d73: traffic flow through the rules to allowed and blocked requests

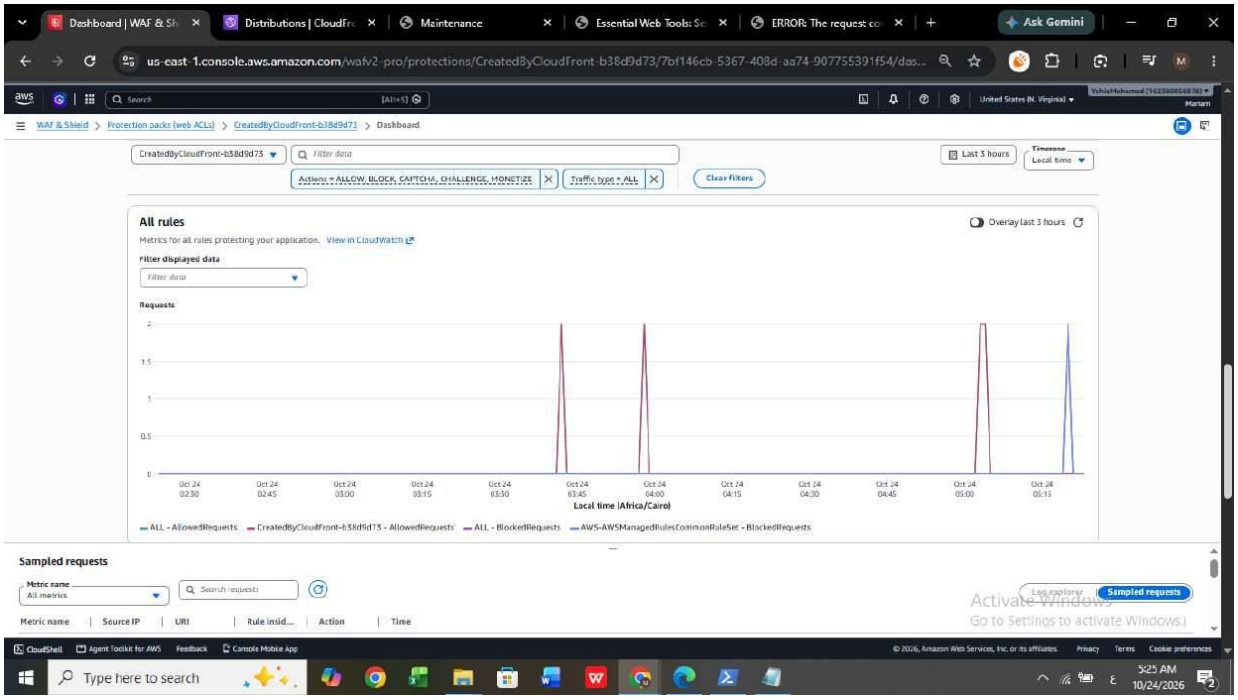


Figure T47-5: WAF rule metrics over time: allowed and blocked requests, including blocks by AWS-AWSManagedRulesCommonRuleSet

Test scope: the XSS request in Figures T47-1 to T47-5 was sent to the maintenance distribution (dakgwfu6nfqq0.cloudfront.net, ACL CreatedByCloudFront-b38d9d73). Normal traffic (HTTP 200) is shown in Figure T42-4. The main distribution currently runs its WAF protections in Monitor mode (Figure T46-1).

T48 — Route 53

Hosted zone: yourapp.com.

Alias records yourapp.com and static.yourapp.com pointing to the main CloudFront distribution.

Note: yourapp.com is a placeholder domain that is not publicly delegated to this hosted zone, so public DNS resolution of these names is not expected; testing uses the CloudFront domain and the hosted zone's name servers.

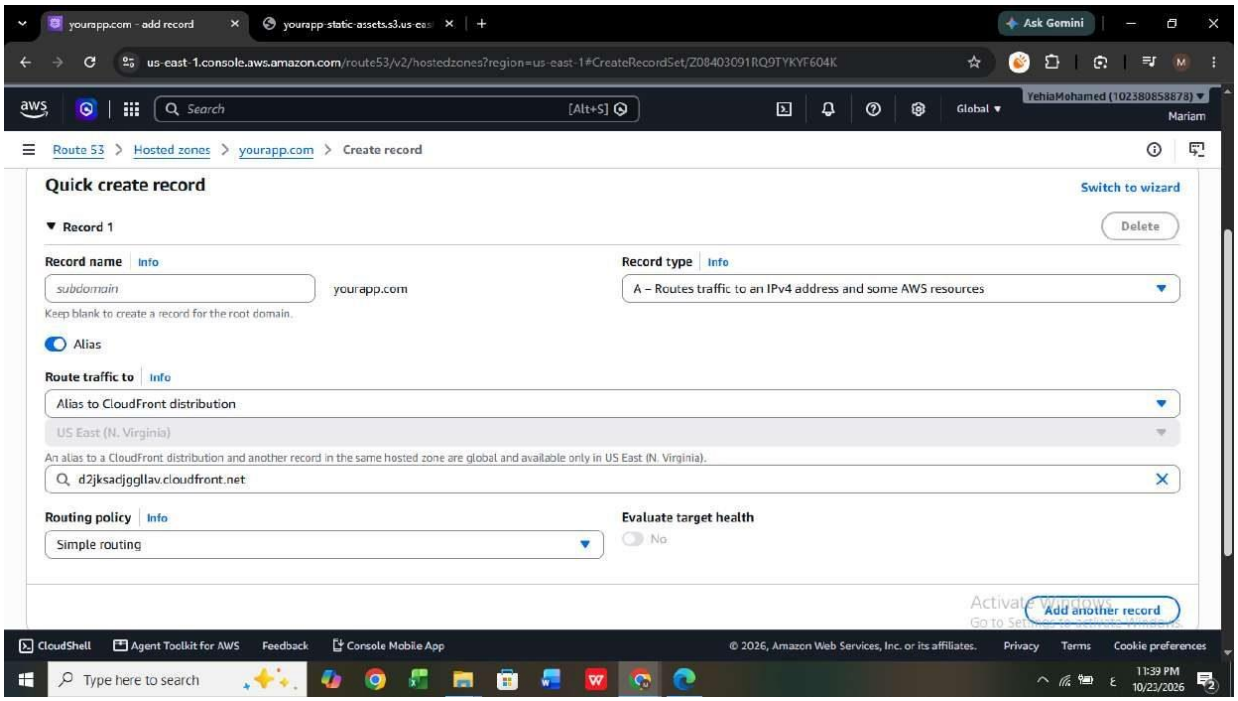


Figure T48-1: Quick create record: yourapp.com A (Alias to CloudFront distribution d2jksadjggllav.cloudfront.net)

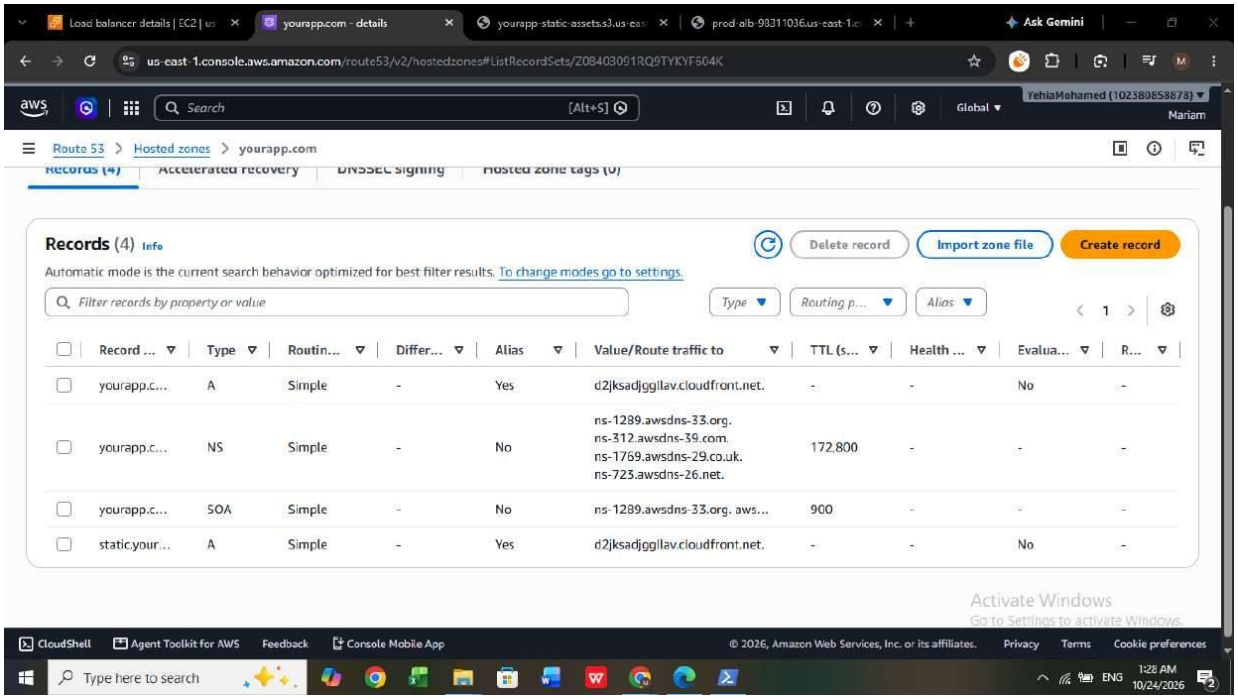


Figure T48-2: Hosted zone yourapp.com: A alias records for yourapp.com and static.yourapp.com, plus NS and SOA

T49 — Failover routing and health check

Health check project-alb-health-check: HTTP against the ALB DNS name prod-alb-98311036.us-east-1.elb.amazonaws.com, port 80, path /. Status: Healthy.

Failover records named failover.yourapp.com: **Primary** → the ALB with the health check; **Secondary** → the maintenance distribution secure-network-maintenance-cdn (ID E2L88I5K45NWFD, domain dakgwfu6nfqq0.cloudfront.net).

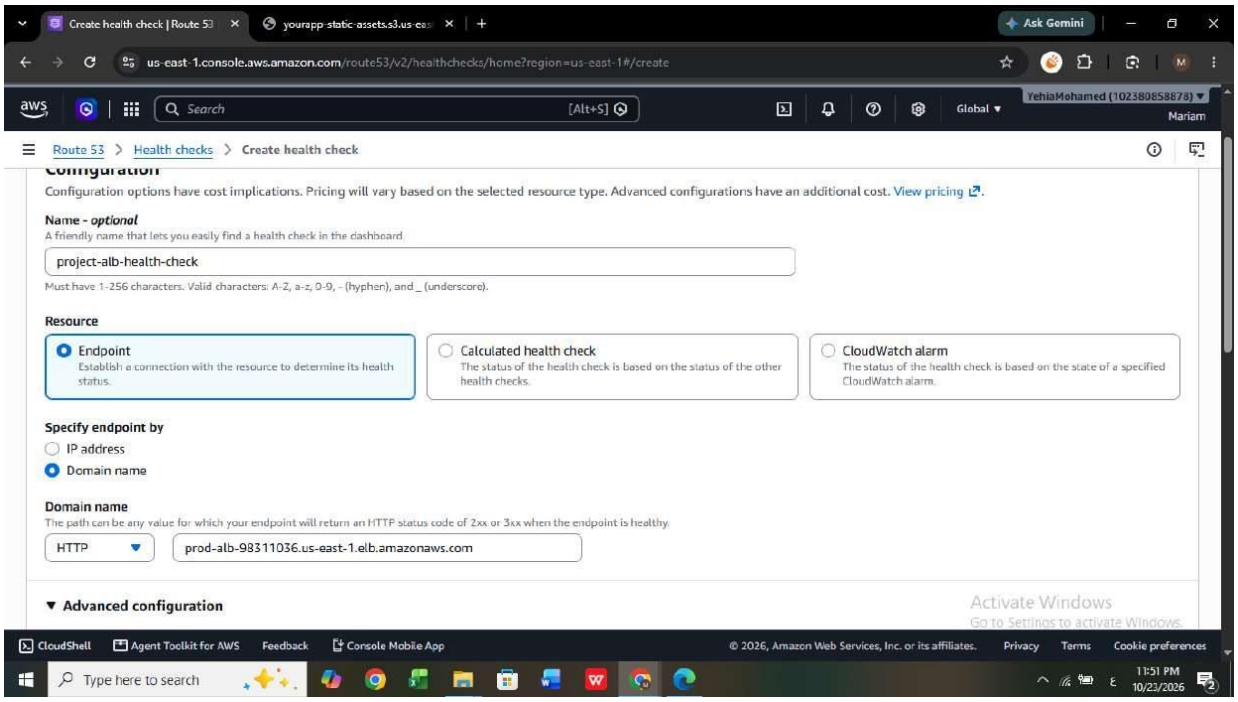


Figure T49-1: Create health check project-alb-health-check: Endpoint, domain name of prod-alb, protocol HTTP

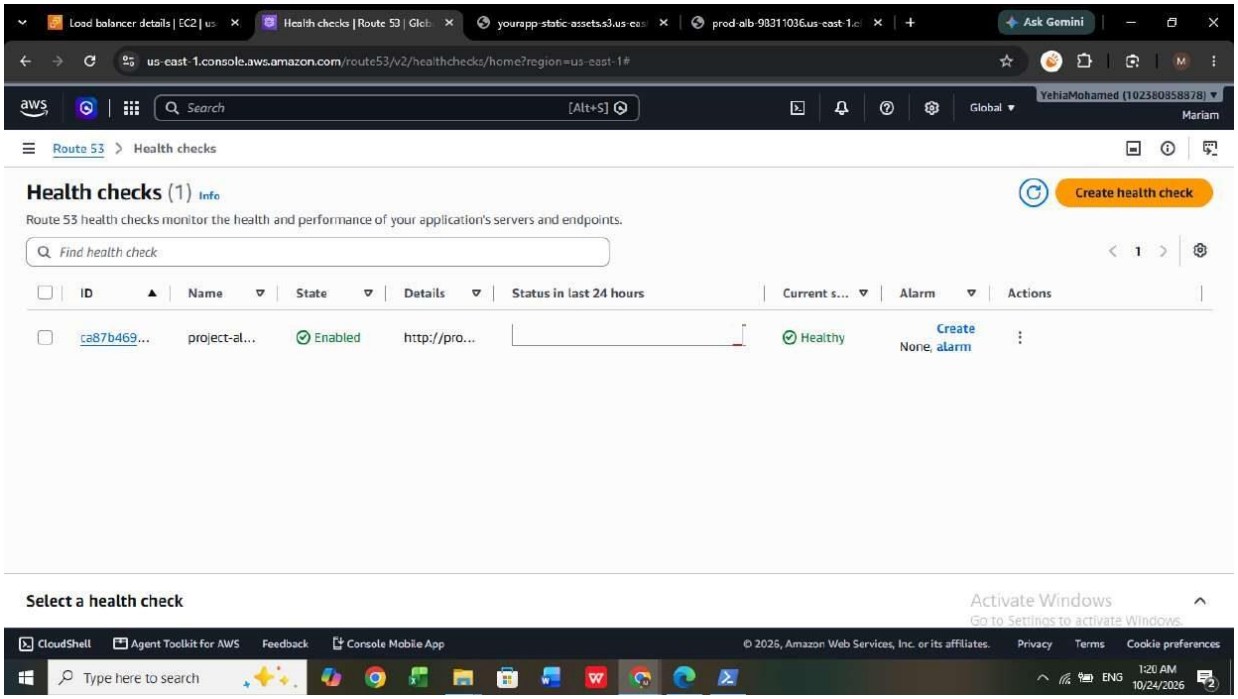


Figure T49-2: Health check status: Enabled and Healthy

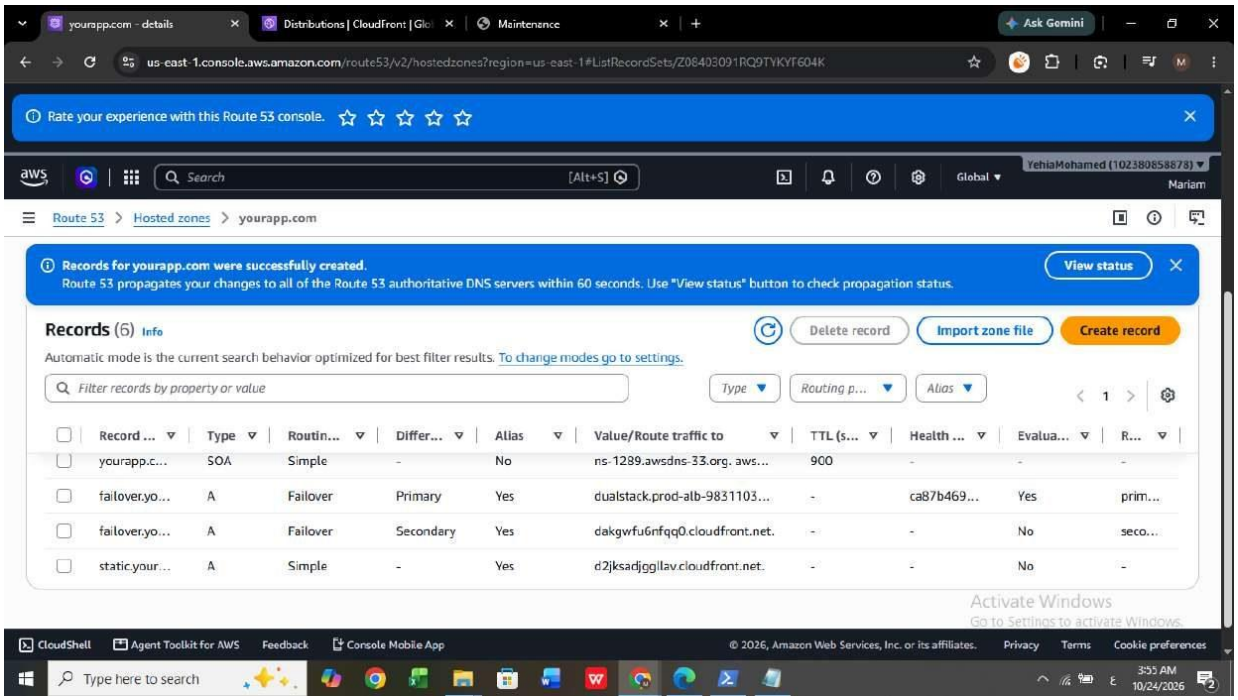


Figure T49-3: Records: failover.yourapp.com Primary (ALB, with health check) and Secondary (maintenance CloudFront)

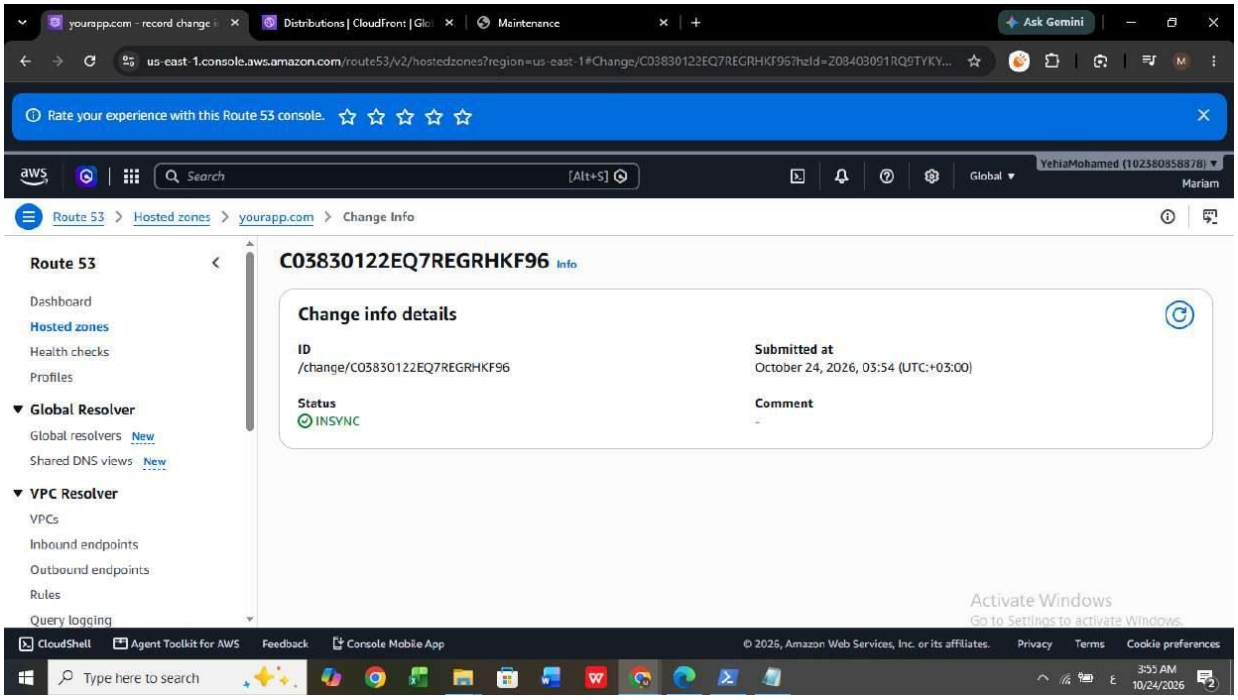


Figure T49-4: Failover record change status INSYNC

Maintenance page (the secondary target):

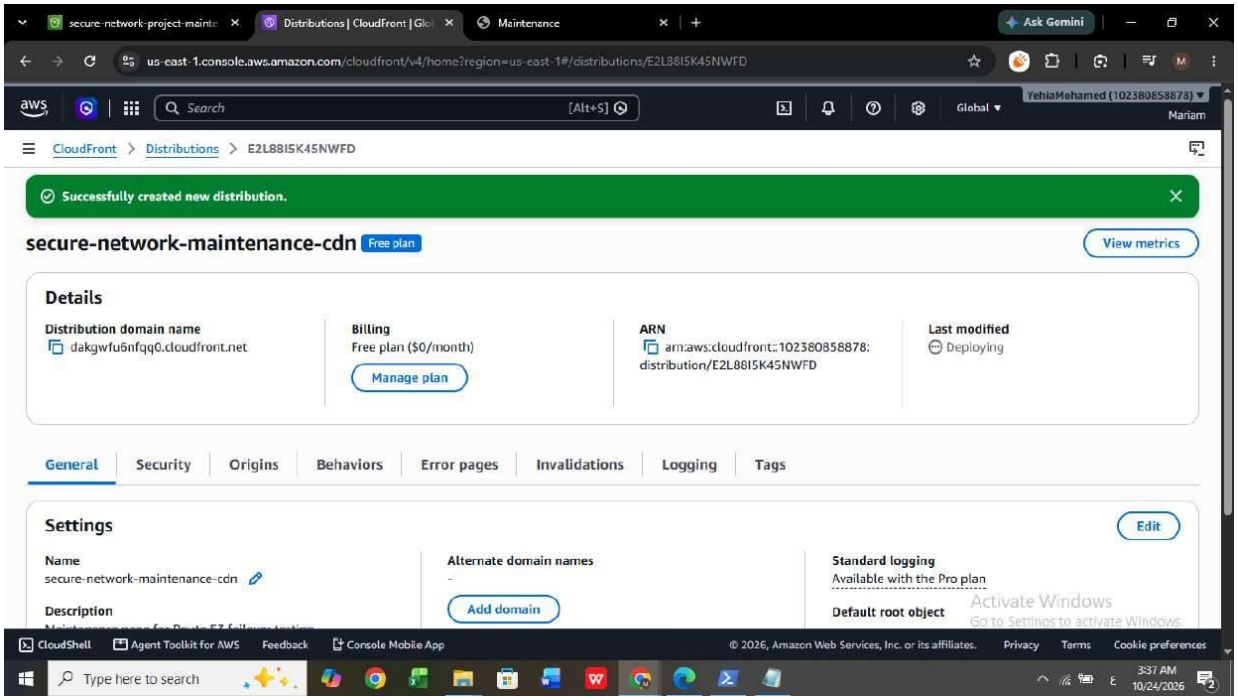


Figure T49-5: Distribution secure-network-maintenance-cdn created (Free plan)

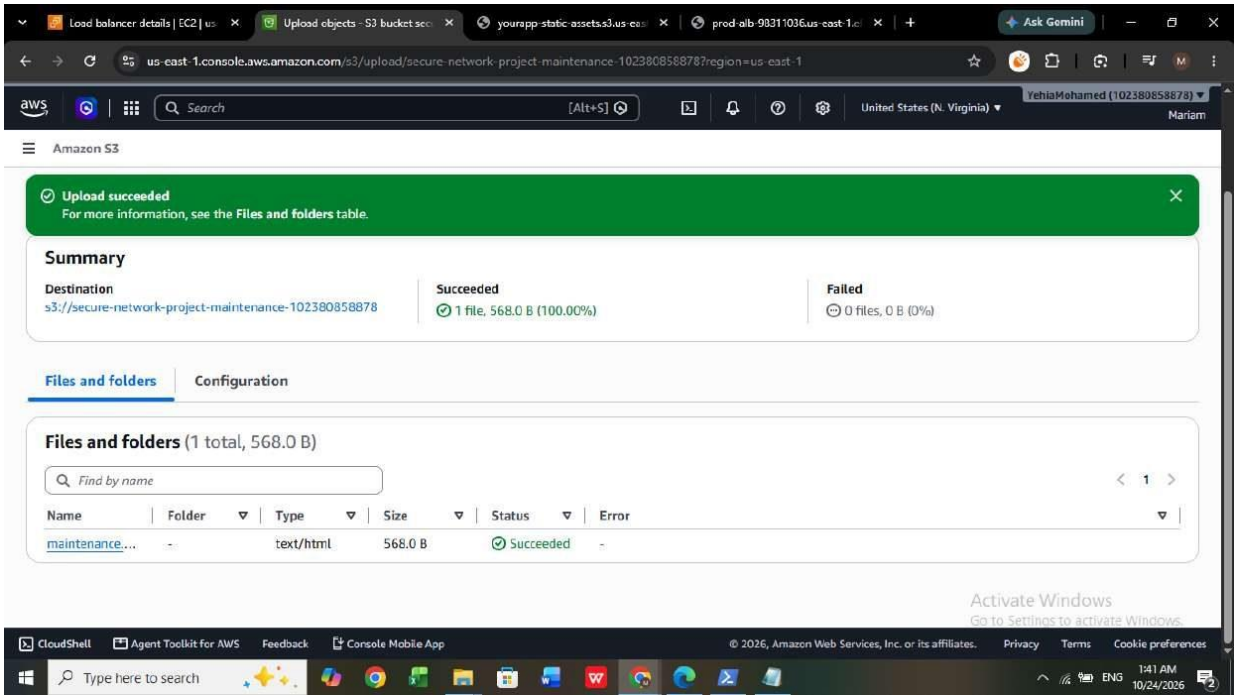


Figure T49-6: Maintenance page uploaded to its S3 bucket

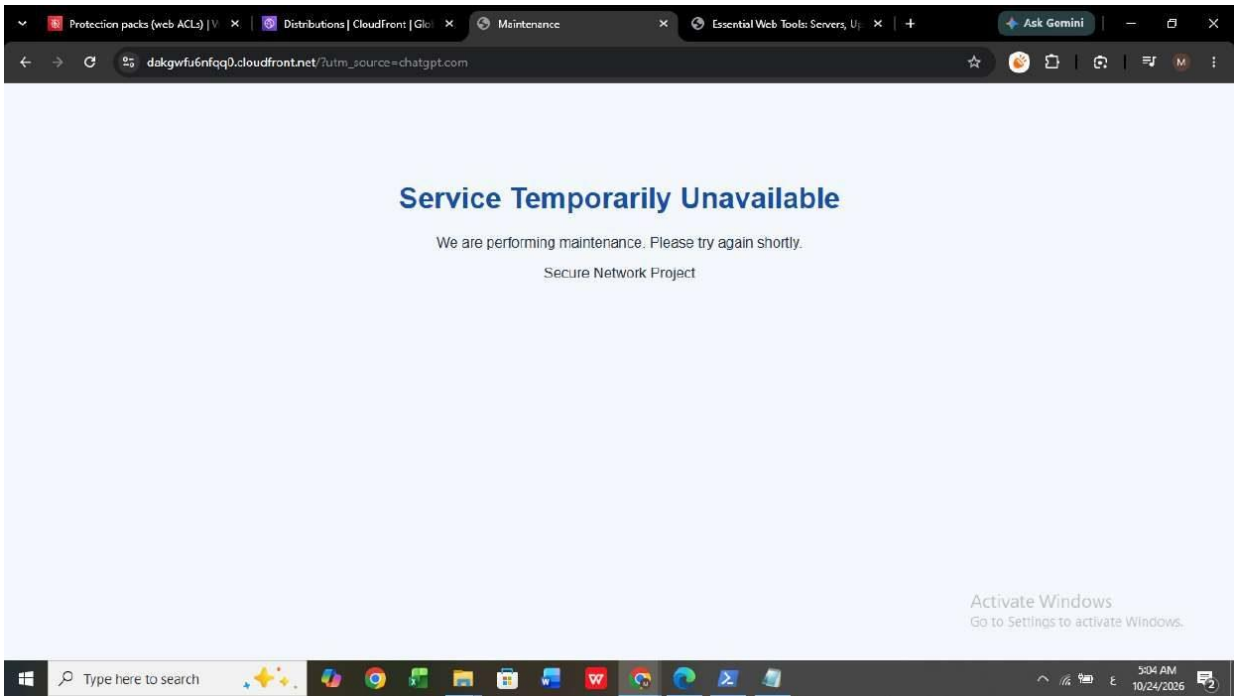


Figure T49-7: Maintenance page served by the maintenance distribution (dakgwfu6nfqq0.cloudfront.net)

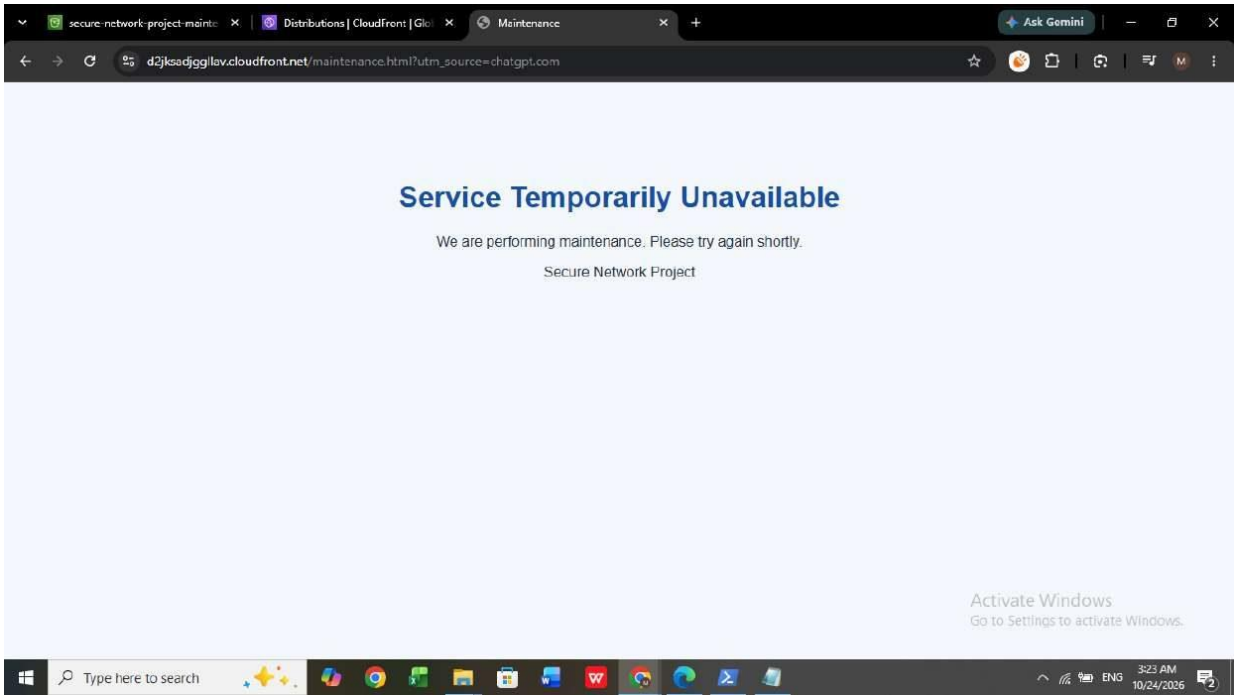


Figure T49-8: Maintenance page served through the main distribution at /maintenance.html

T50 — Failover test

The Route 53 records and the health check were verified with the Route 53 Test record tool for failover.yourapp.com and a direct request to the ALB: the health check project-alb-health-check reports Healthy, the Primary record is answered, and the application responds with HTTP 200 (Figures T50-1 to T50-4). The Secondary record points to the maintenance distribution secure-network-maintenance-cdn (Figures T49-3 and T49-5).

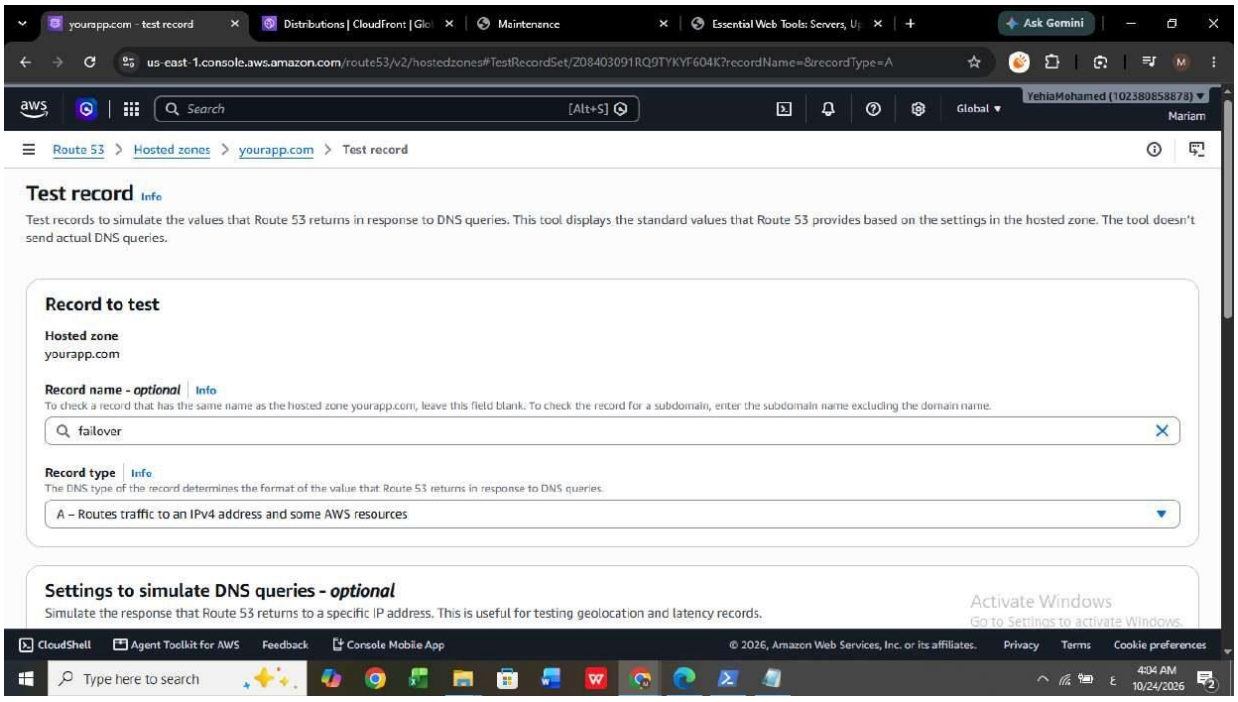


Figure T50-1: Test record setup for failover.yourapp.com (type A)

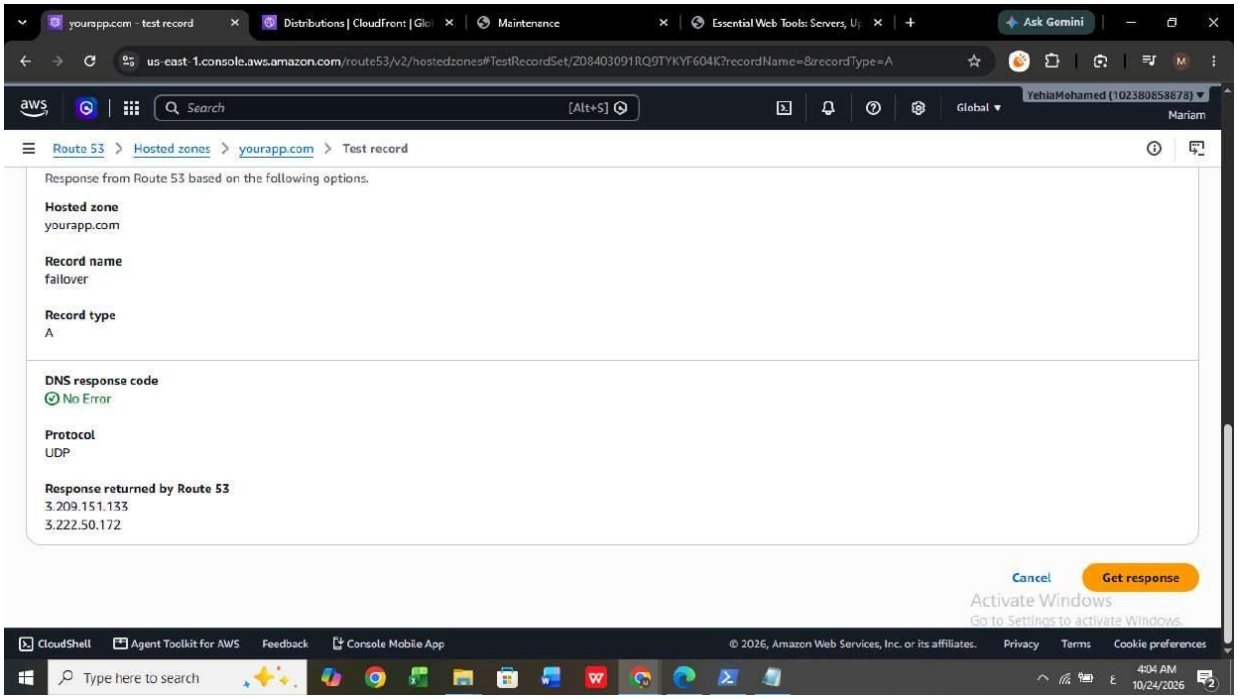


Figure T50-2: Route 53 response: DNS response code No Error, returning IP addresses (primary healthy)

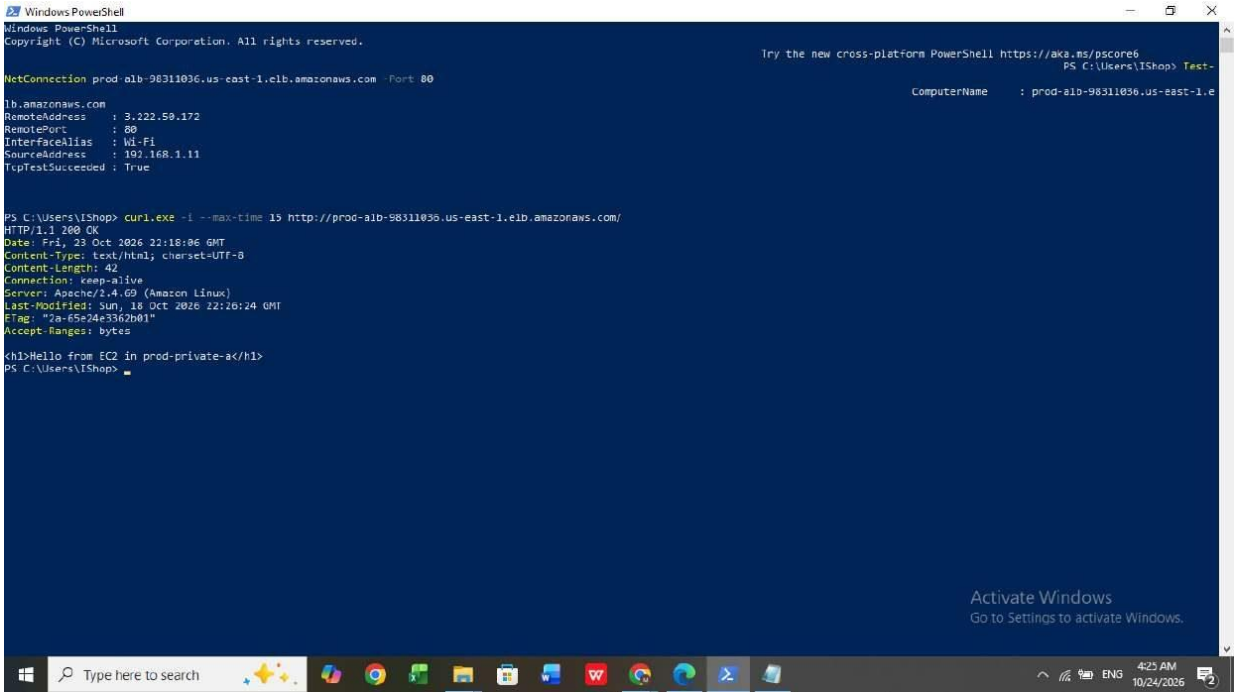


Figure T50-3: Direct request to the ALB returns HTTP 200 “Hello from EC2 in prod-private-a”

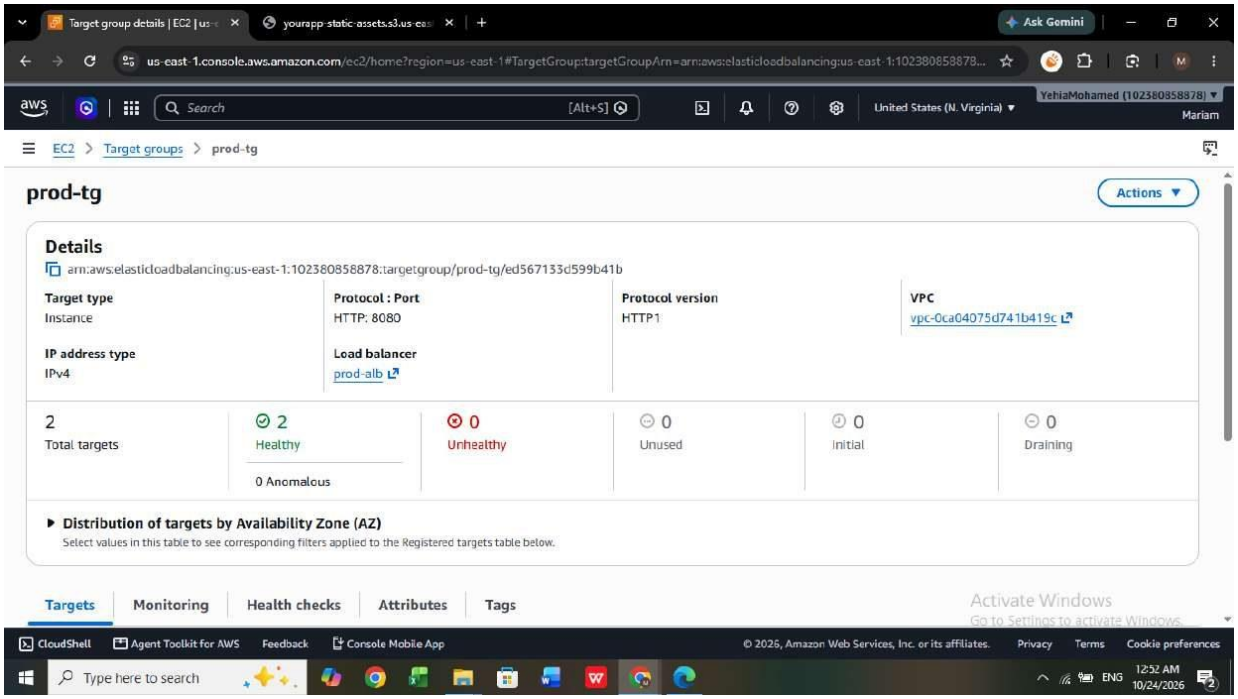


Figure T50-4: Target group prod-tg: 2 healthy targets

4. Security Documentation (T51–T53)

T51 — AWS Shield: Standard vs. Advanced

	Shield Standard	Shield Advanced
Cost	Included automatically for all AWS customers at no extra charge	Paid subscription, commonly listed at about USD 3,000 per month (verify current pricing and commitment terms before submission)
Setup	None — always on	Subscribe and register the protected resources
Protection	Common network and transport layer (L3/L4) DDoS attacks, e.g. SYN/UDP floods and reflection attacks	Enhanced detection and mitigation for L3/L4 plus application-layer (L7) attack support together with AWS WAF
Protected services	CloudFront, Route 53, and other AWS edge/network infrastructure; covers resources such as ALB and Elastic IP at the network layer	CloudFront, Route 53, Global Accelerator, ALB, Elastic IP and other supported resources, once registered
Human support	Standard AWS support only	Access to the Shield Response Team (SRT) under applicable AWS Support plan terms
Cost protection	No	Credits for scaling charges caused by DDoS attacks, subject to AWS terms
Visibility	Basic	Attack diagnostics and near-real-time metrics

Used in this project: Shield Standard only (automatic, nothing to configure). Shield Advanced is documented, not deployed.

When to consider Advanced: a business-critical, internet-facing service where downtime is very costly, a history of being targeted, a need for 24/7 expert response, or a need for protection against DDoS-driven cost spikes. For a training project or a low-risk site, Standard plus WAF rate limiting is usually sufficient.

T52 — CloudFront vs. AWS Global Accelerator

Aspect	Amazon CloudFront	AWS Global Accelerator
Main use case	Content delivery: websites, APIs, media, downloads	Improving availability and latency of applications over the AWS global network
Caching	Yes — caches content at edge locations	No — does not cache; it proxies traffic to the application
Protocols	HTTP/HTTPS (and WebSocket)	TCP and UDP (any application, including non-HTTP such as gaming, IoT, VoIP)
WAF integration	Direct: a Web ACL can be attached to the distribution	Not attached to the accelerator; WAF is applied on the endpoint behind it (e.g. ALB)
IP addresses	Many changing edge IPs; use DNS names	Two static anycast IP addresses (useful for allow-listing)
Failover	Origin failover and Route 53 integration	Health-check-based failover between regional endpoints, fast and without DNS caching delay
Pricing model	Pay per request and data transferred out	Fixed hourly fee per accelerator plus data-transfer premium fees
Other edge features	Edge functions, compression, signed URLs, OAC	Static IPs, traffic dials, client IP preservation on supported endpoints

Conclusion: for this project, an HTTP(S) website plus an API, CloudFront is the right fit because it provides caching, OAC for the private S3 bucket and direct WAF integration. Global Accelerator would be preferred for non-HTTP (TCP/UDP) workloads, or when clients need fixed IP addresses or very fast regional failover. They can also be combined. Neither service is universally better.

T53 — DDoS Mitigation Plan

Layers already in place: Shield Standard (automatic), CloudFront in front of the origins, WAF with managed rules and a rate-based rule, private S3 via OAC, Route 53 health check with failover to a maintenance page.

1. Detection

CloudWatch metrics to watch: CloudFront Requests, 4xxErrorRate, 5xxErrorRate; WAF BlockedRequests and AllowedRequests; ALB RequestCount, HTTPCode_ELB_5XX_Count, HealthyHostCount; Route 53 health check status.

WAF sampled requests show what is being blocked and by which rule.

Route 53 health-check failure is an early signal that the primary origin is under stress.

Already configured by the team (Yehia's workstream): CloudWatch alarms unhealthy-targets, high-network-out and high-cpu-ec2a, the SNS topic security-alerts with a confirmed email subscription, and the GuardDuty → EventBridge → Lambda auto-quarantine pipeline. *Recommended:* CloudFront and WAF specific alarms (for example on BlockedRequests and CloudFront 5xxErrorRate).

2. Response

Identify the affected resource (distribution, ALB, or DNS name) and the attack pattern from WAF sampled requests and ALB/CloudFront metrics.

Tighten WAF: switch rules from Monitor to Block where the plan allows, lower the rate-based threshold, add rules for the offending pattern or IP set, or block by geography if the traffic is clearly regional.

Close the bypass path: restrict the ALB security group so the ALB accepts traffic only from CloudFront (today sg-alb allows ports 80 and 443 from anywhere, so a direct hit on the ALB skips WAF).

Let Route 53 fail over to the maintenance page if the primary becomes unhealthy.

3. Communication

Notify the technical owner (the resource owner for the affected layer) and the project lead through the team's agreed channel.

Post status updates at regular intervals and record the timeline for the post-incident review.

4. Recovery

Confirm the application returns HTTP 200 through CloudFront and the Route 53 health check is healthy again (failback).

Review blocked and allowed traffic in WAF and CloudWatch, keep the rules that proved useful, and remove temporary blocks.

5. Escalation

Open an AWS Support case according to the account's support plan and the severity.

Shield Advanced customers can also engage the Shield Response Team; this project does not subscribe to Advanced.

5. Backend API

The CloudFront /api/* behavior forwards requests to the ALB. The backend API route is implemented: a direct request to the ALB at /api/ returns a JSON response from prod-ec2-a, and the same path through CloudFront returns a JSON response from prod-ec2-b (Figures T43-5 and T43-6), which also shows the ALB serving both targets.

A /health/* behavior was also added on the main distribution, routing to prod-alb; /health/ through CloudFront returns OK (Figures T43-7 and T43-8). The Route 53 health check project-alb-health-check probes the ALB directly at path /.

6. T54 — Edge Architecture Diagram

The diagram shows both CloudFront distributions, their origins, the Route 53 failover records and the security components in the request path.

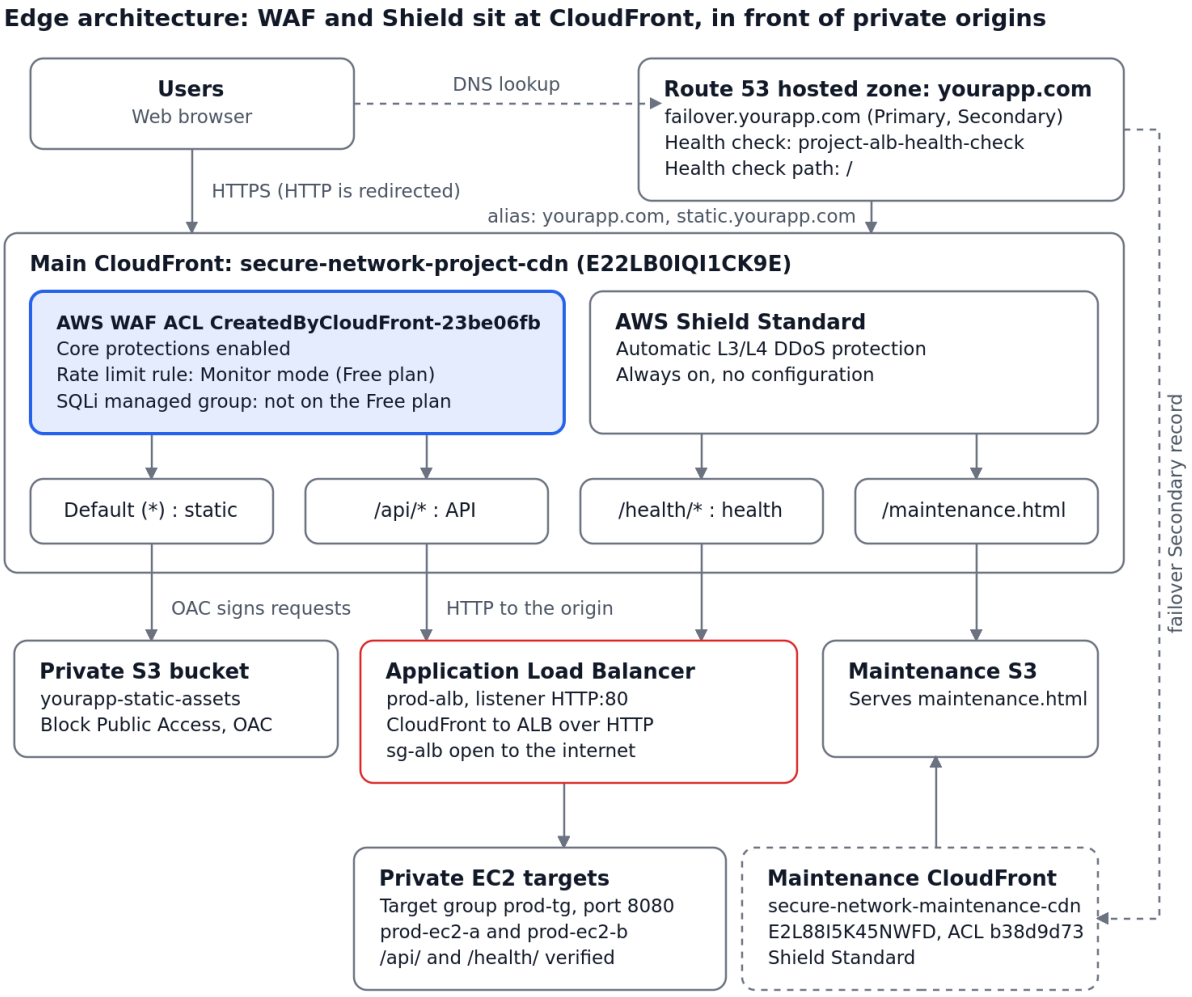


Figure T54-1: Edge architecture: main and maintenance CloudFront, WAF, Shield, S3, ALB, EC2 and Route 53 failover

Solid arrows show the request path; dashed arrows show the DNS lookup and the Route 53 Secondary (failover) record to the maintenance CloudFront distribution. The red-outlined ALB marks the security-group observation.

7. Limitations and Security Observations

WAF mode on the main distribution: core protections and rate limiting are enabled in Monitor mode on the Free plan; they count requests and do not block them.

SQL injection: AWS-AWSManagedRulesSQLiRuleSet was configured on the Pay-As-You-Go plan. On the current Free plan the rule group is not available (Pro-plan feature), so the current configuration relies on the Common and Known Bad Inputs rule groups.

Rate limiting: RateLimit1000Per5Min (1,000 requests per 5 minutes per IP) was configured in Block mode on the Pay-As-You-Go plan. On the current Free plan it is in Monitor mode.

ALB bypass: sg-alb allows ports 80 and 443 from anywhere, so a direct request to the ALB skips CloudFront and WAF. Recommended fix: allow only the CloudFront managed prefix list on the ALB security group.

Origin protocol: CloudFront reaches the ALB over HTTP:80 because the ALB has no HTTPS listener; viewers always use HTTPS to CloudFront.

Domain: yourapp.com is not publicly delegated, so DNS tests use the hosted zone's name servers and the CloudFront domain.

Cost control: resources deleted to save cost were removed only after screenshots were captured; do not recreate them unless a task requires it.

8. Final Status Summary

Task	Deliverable	Status
T42	S3 static website	Completed
T43	CloudFront distribution	Completed; /api/ and /health/ verified through CloudFront
T44	Origin Access Control	Completed
T45	WAF Web ACL	Completed; Free plan: SQLi group unavailable, rate limit in Monitor mode
T46	WAF association	Configured; screenshots available
T47	WAF tests	XSS blocked (403) on the maintenance distribution; normal traffic (200)
T48	Route 53 records	Completed; screenshots available
T49	Failover and health check	Completed; evidence available
T50	Failover test	Primary record and health check verified; Secondary record configured
T51	Shield documentation	Completed
T52	CloudFront vs. Global Accelerator	Completed
T53	DDoS mitigation plan	Completed
T54	Edge architecture diagram	Completed

SECTION 5 — WORKSTREAM 5 (TASKS T55 – T67) — FULL UNABRIDGED DOCUMENTATION & ALL 33 FIGURES

Yehia — VPC Flow Logs, Athena Threat Hunting, CloudWatch, EventBridge/Lambda Auto-IR, Traffic Mirroring, Live IR Sim, Runbook & DR Plan

Security & Observability Engineer: Yehia Mohamed

Project 2: Secure AWS Network Architecture & Hybrid Connectivity

Phase 4 Deliverable: Network Monitoring, Detection, Incident Response (T64/T65) & Disaster Recovery Plan (T66)

Engineer / Author	AWS Account ID	Primary Region / VPC	DR Region / VPC	Submission Date
Yehia Mohamed	102380858878	us-east-1 (vpc-0ca04075d741b419c)	us-west-2 (10.2.0.0/16)	October 25, 2026

Part 1: Phase 4 Implementation & Lab Evidence

1.1 Baseline Infrastructure & Security Alerting Setup

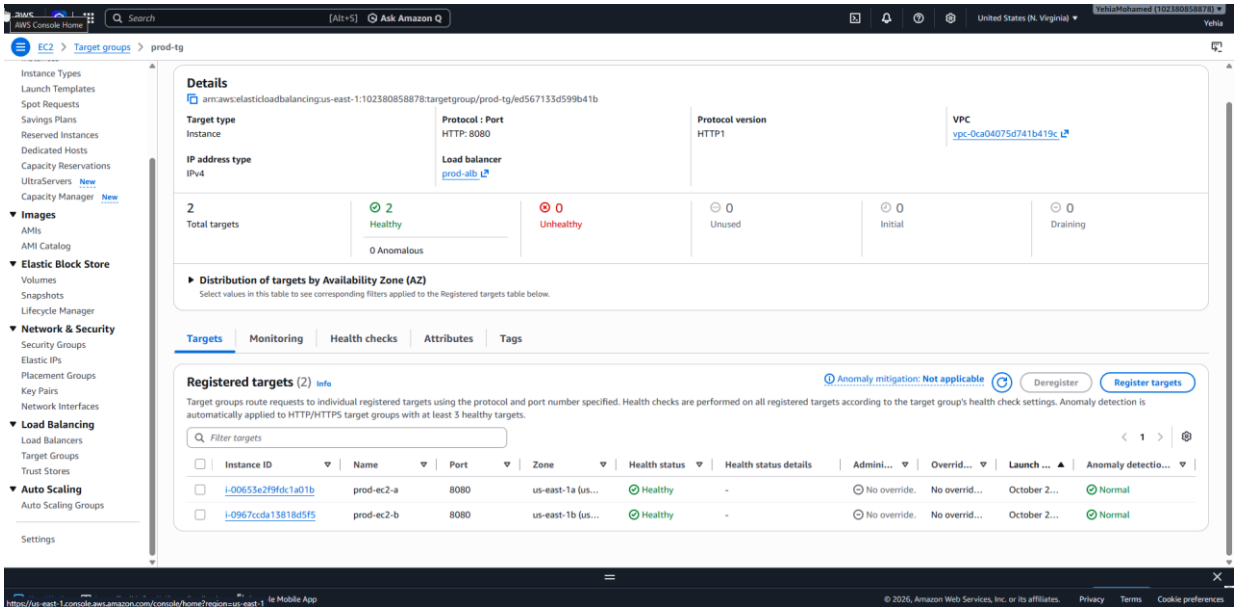


Figure 1.1 — Target Group (prod-tg) Baseline Health Verification

- **Resource ARN:** arn:aws:elasticloadbalancing:us-east-1:102380858878:targetgroup/prod-tg/ed567133d599b41b associated with prod-alb in vpc-0ca04075d741b419c.
- **Operational Status:** Both production instances — prod-ec2-a (i-00653e2f9fdc1a01b in us-east-1a) and prod-ec2-b (i-0967ccda13818d5f5 in us-east-1b) — are registered on HTTP port 8080 and passing health checks (2 Healthy, 0 Unhealthy).

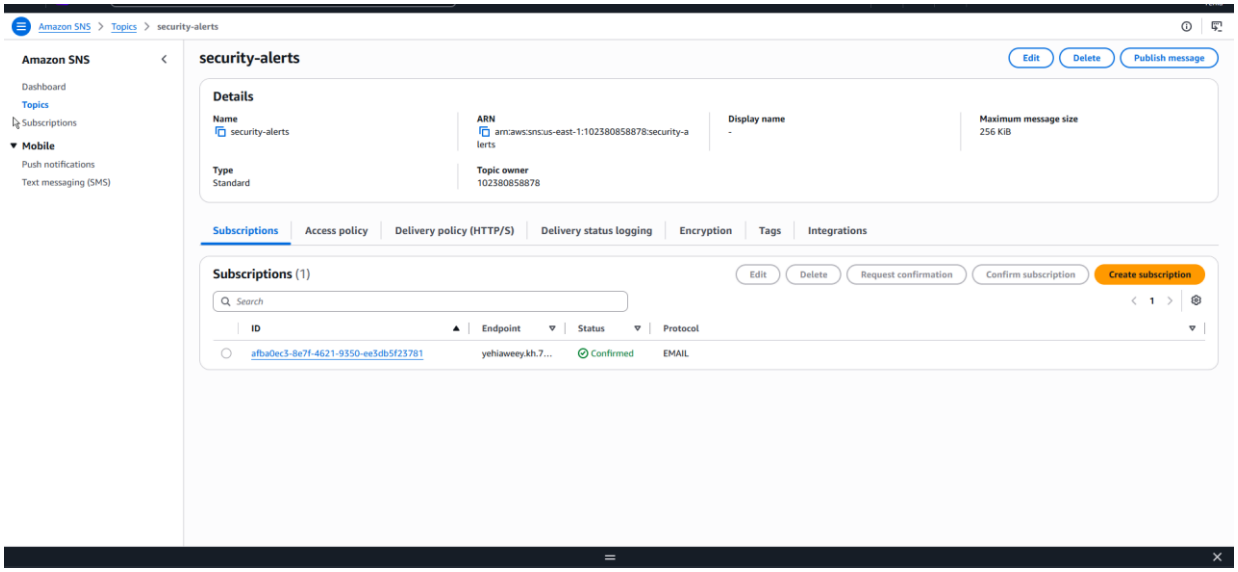


Figure 1.2 — Amazon SNS Topic (security-alerts) & Confirmed Email Subscription

- **Topic ARN:** arn:aws:sns:us-east-1:102380858878:security-alerts (Standard Topic owned by account 102380858878).
- **Alert Endpoint:** Subscription ID afba0ec3-8e7f-4621-9350-ee3db5f23781 configured for EMAIL protocol to yehiaweey.kh.77@gmail.com with status Confirmed, serving as the notification channel for both EventBridge/Lambda alerts and manual IR broadcasts.

1.2 VPC Flow Logs & CloudWatch Telemetry

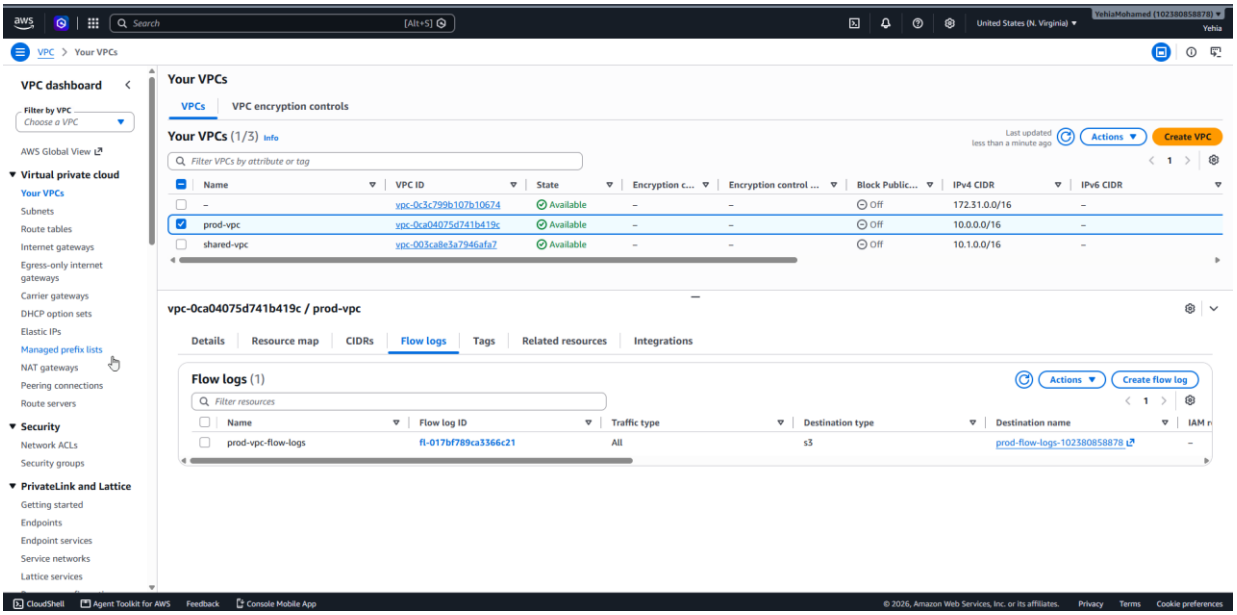


Figure 1.3 — VPC Flow Logs (prod-vpc-flow-logs) Enabled on Production VPC

- **Flow Log Configuration:** Flow Log ID fl-017bf789ca3366c21 attached to prod-vpc (vpc-0ca04075d741b419c, IPv4 CIDR 10.0.0.0/16).
- **Capture Scope:** Traffic type set to All (capturing both ACCEPT and REJECT packets) with direct publication to Amazon S3 bucket prod-flow-logs-102380858878.

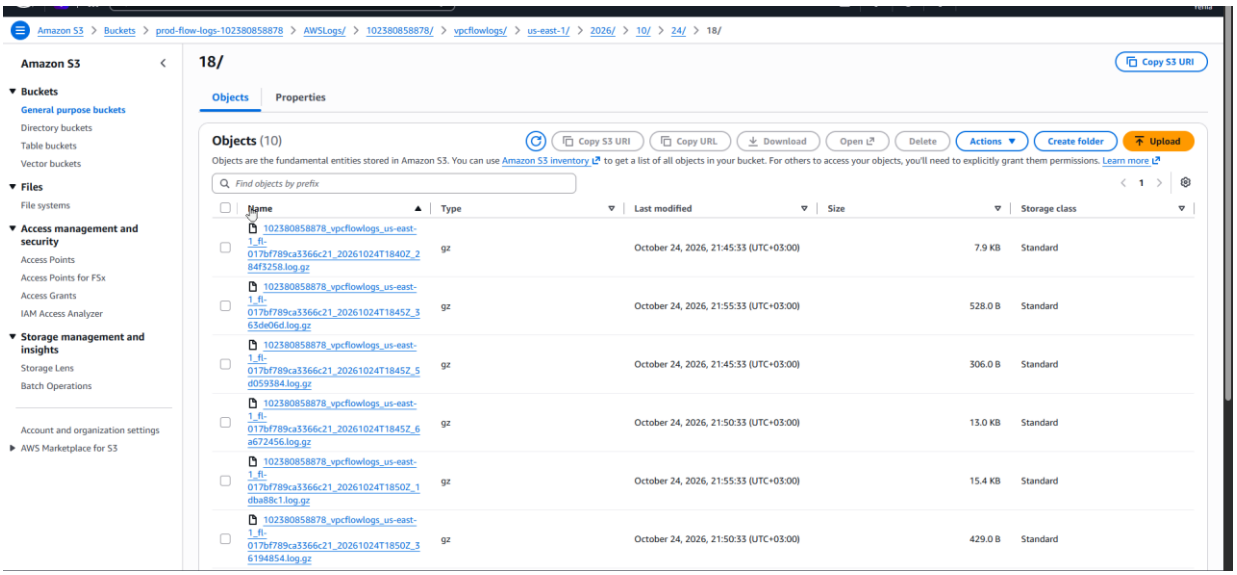


Figure 1.4 — S3 Bucket Storage of Partitioned VPC Flow Log Objects

- **S3 Path Structure:** s3://prod-flow-logs-102380858878/AWSLogs/102380858878/vpcflowlogs/us-east-1/2026/10/24/18/
- **Verification:** Confirms compressed .log.gz flow log archives are actively delivered by the VPC Flow Logs service into date-partitioned S3 prefixes for downstream Athena querying.

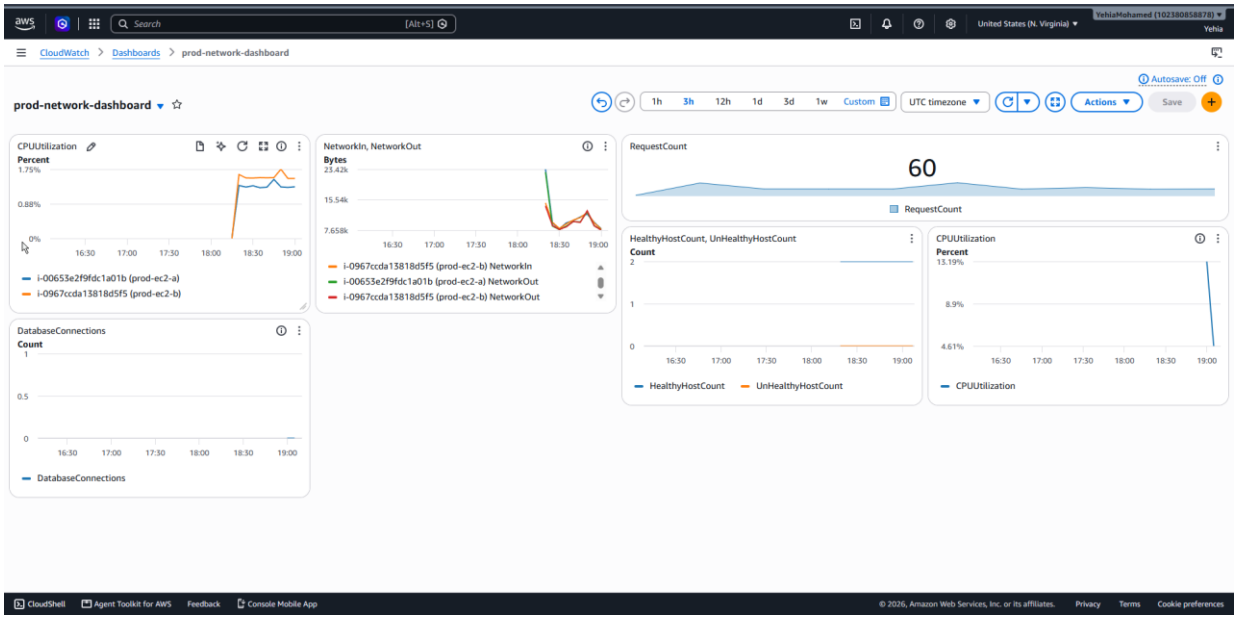


Figure 1.5 — CloudWatch Operational Dashboard (prod-network-dashboard)

- **Monitored Metrics:** Real-time visibility into EC2 CPUUtilization, NetworkIn/NetworkOut byte spikes across prod-ec2-a and prod-ec2-b, ALB RequestCount (60 requests), HealthyHostCount/UnHealthyHostCount, and RDS DatabaseConnections.

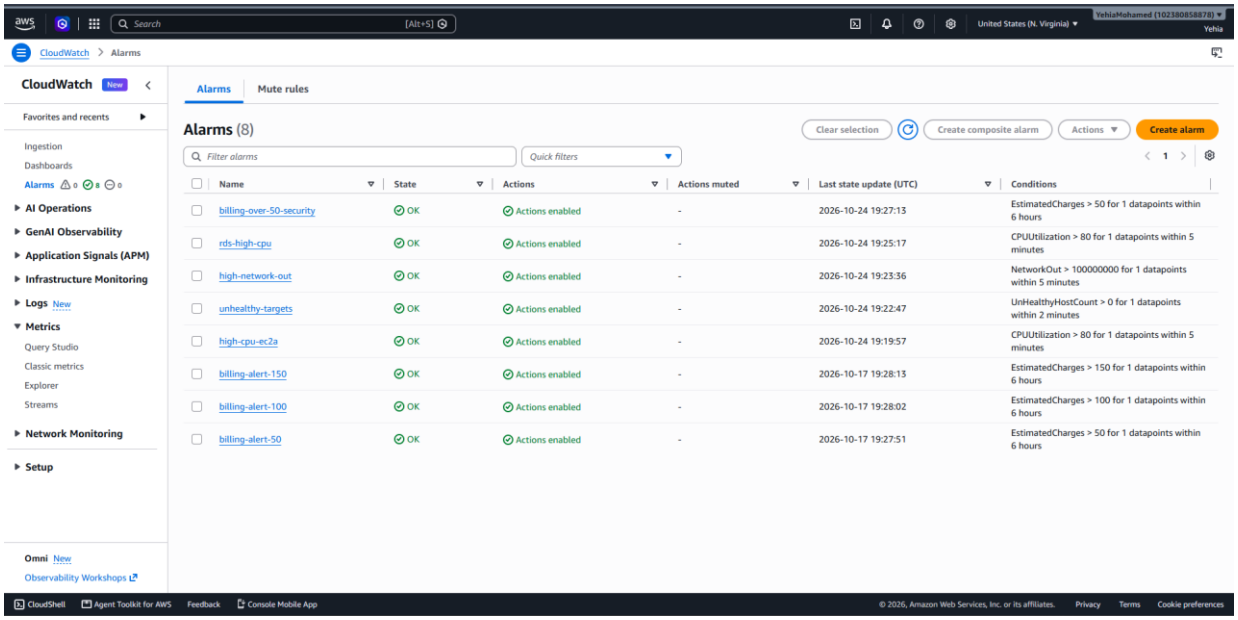


Figure 1.6 — CloudWatch Security & Availability Alarms

- **Active Thresholds:** All 8 CloudWatch alarms report OK with Actions enabled, including high-network-out (NetworkOut > 100,000,000 bytes within 5 minutes to detect data exfiltration), high-cpu-ec2a (CPUUtilization > 80%), rds-high-cpu, unhealthy-targets, and tiered billing alarms ($50, $100, $150).

1.3 Automated Containment Pipeline (GuardDuty → EventBridge → Lambda)

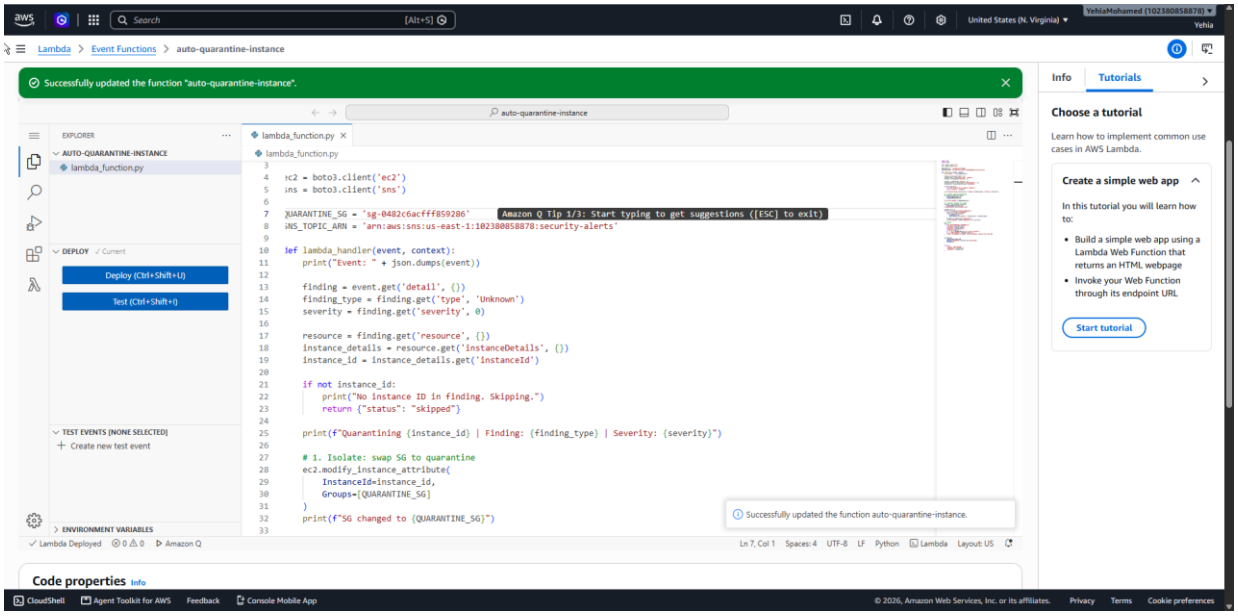


Figure 1.7 — AWS Lambda Auto-Quarantine Function (auto-quarantine-instance)

- **Remediation Logic:** Python 3.12 Boto3 function that parses incoming GuardDuty event payloads, extracts detail.resource.instanceDetails.instanceId, and executes ec2.modify_instance_attribute(InstanceId=instance_id, Groups=['sg-0482c6acfff859286']) to immediately isolate compromised instances.
- **Notification Integration:** Publishes automated containment details to arn:aws:sns:us-east-1:102380858878:security-alerts.

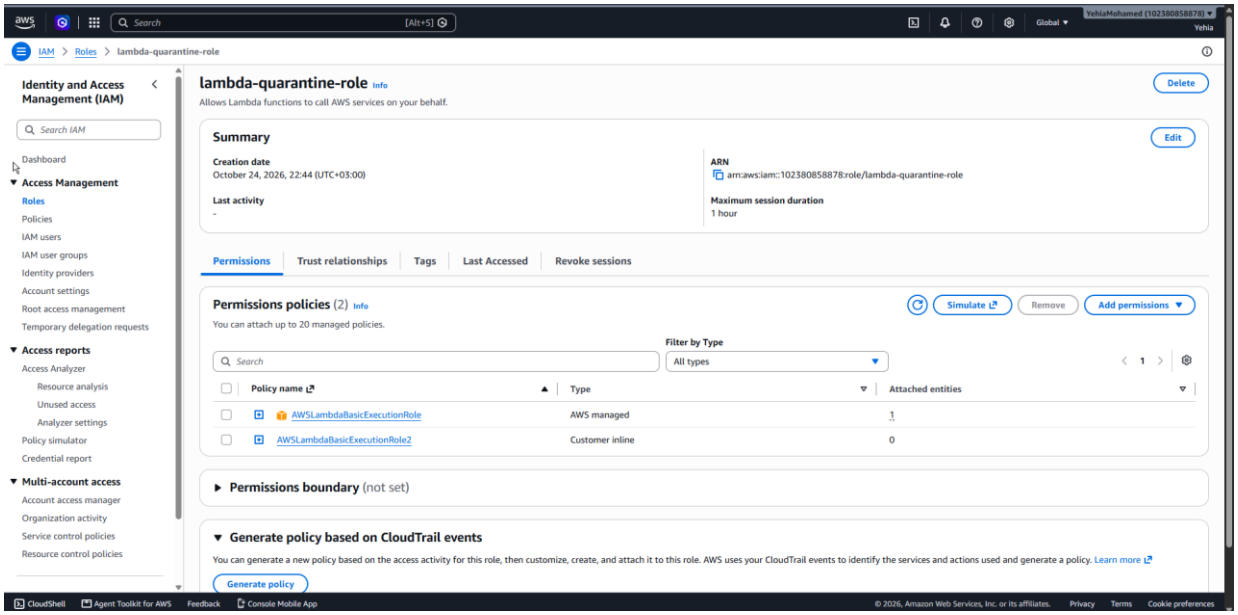


Figure 1.8 — IAM Execution Role (lambda-quarantine-role)

- **Role ARN:** arn:aws:iam::102380858878:role/lambda-quarantine-role
- **Least-Privilege Policies:** Attached with AWS managed AWSLambdaBasicExecutionRole (CloudWatch Logs write access) and customer inline policy AWSLambdaBasicExecutionRole2 granting ec2:ModifyInstanceAttribute, ec2:CreateSnapshot, ec2:DescribeInstances, and sns:Publish.

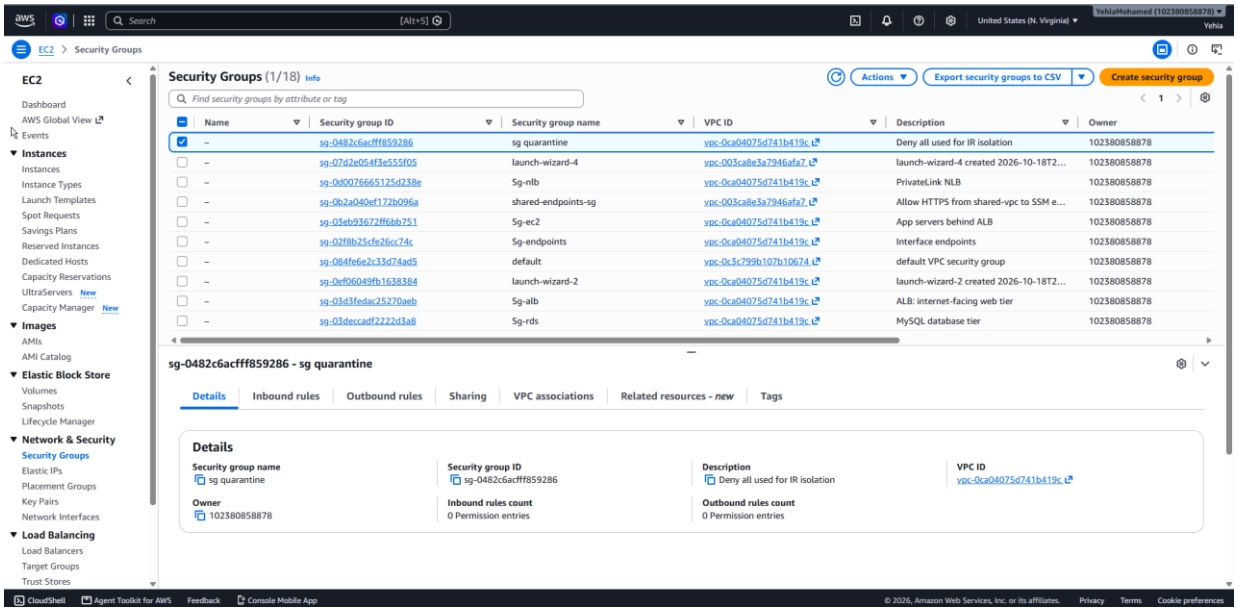


Figure 1.9 — Quarantine Security Group (sg quarantine / sg-0482c6acfff859286)

- **Zero-Trust Isolation:** Configured in prod-vpc (vpc-0ca04075d741b419c) with description 'Deny all used for IR isolation'. Contains 0 Inbound rules and 0 Outbound rules, completely severing network connectivity when attached to a compromised host.

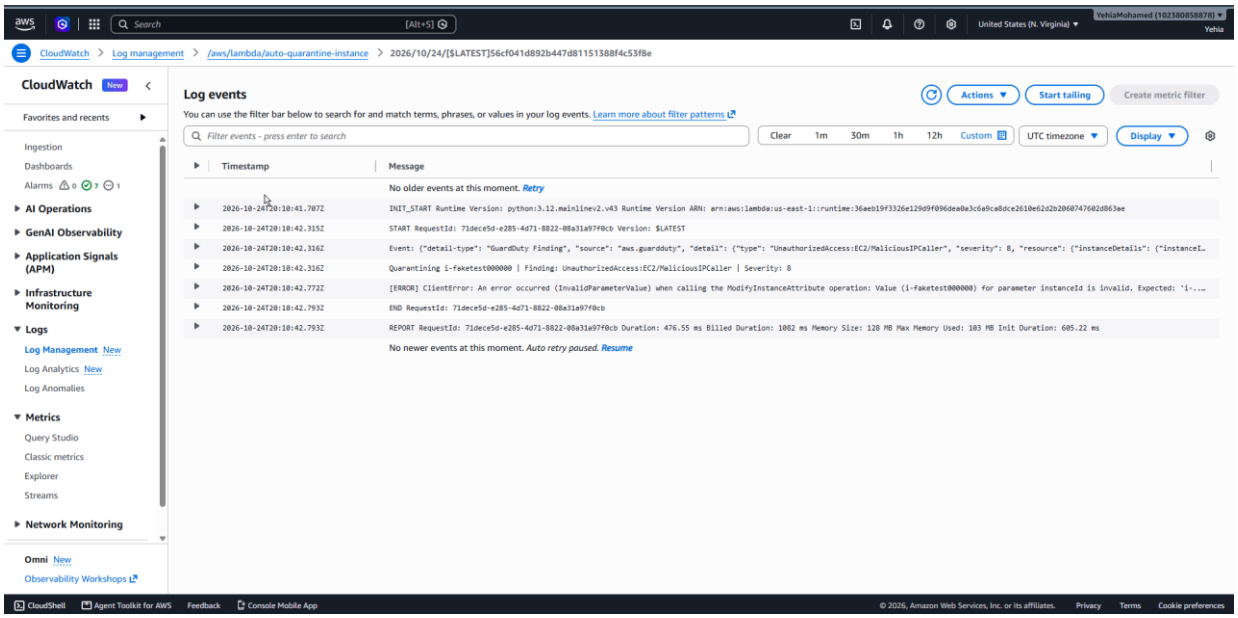


Figure 1.10 — CloudWatch Log Stream Verification (/aws/lambda/auto-quarantine-instance)

- **Test Execution Proof:** Log output confirms the Lambda function successfully parsed a simulated GuardDuty finding (UnauthorizedAccess:EC2/MaliciousIPCaller, Severity 8) and invoked ModifyInstanceAttribute for test instance ID i-faketest000000.

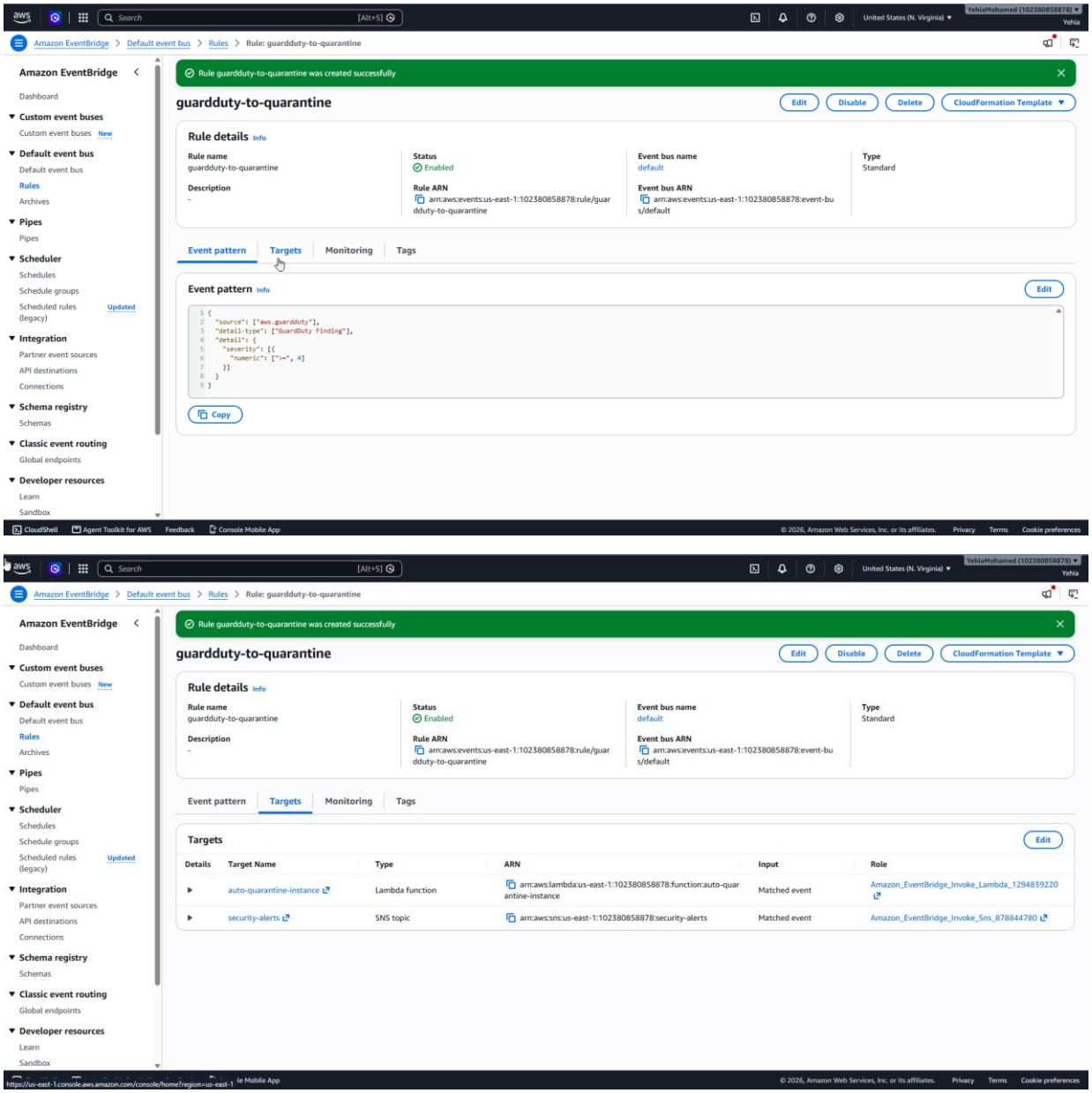
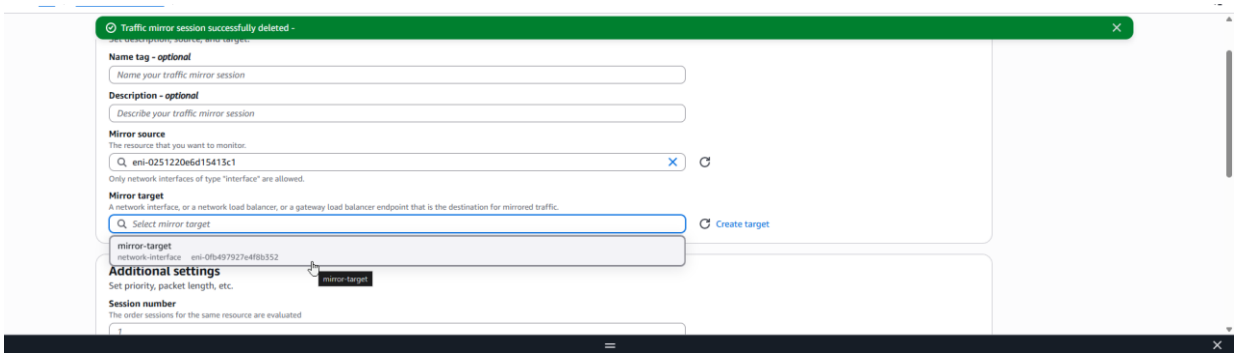


Figure 1.11 — Amazon EventBridge Rule (guardduty-to-quarantine) Event Pattern & Targets

- **Rule ARN:** arn:aws:events:us-east-1:102380858878:rule/guardduty-to-quarantine (Status: Enabled on default event bus).
- **Filter Pattern & Dual Targets:** Matches source aws.guardduty with detail.severity numeric >= 4, routing events simultaneously to the auto-quarantine-instance Lambda function and the security-alerts SNS topic.

1.4 Deep Packet Inspection via VPC Traffic Mirroring



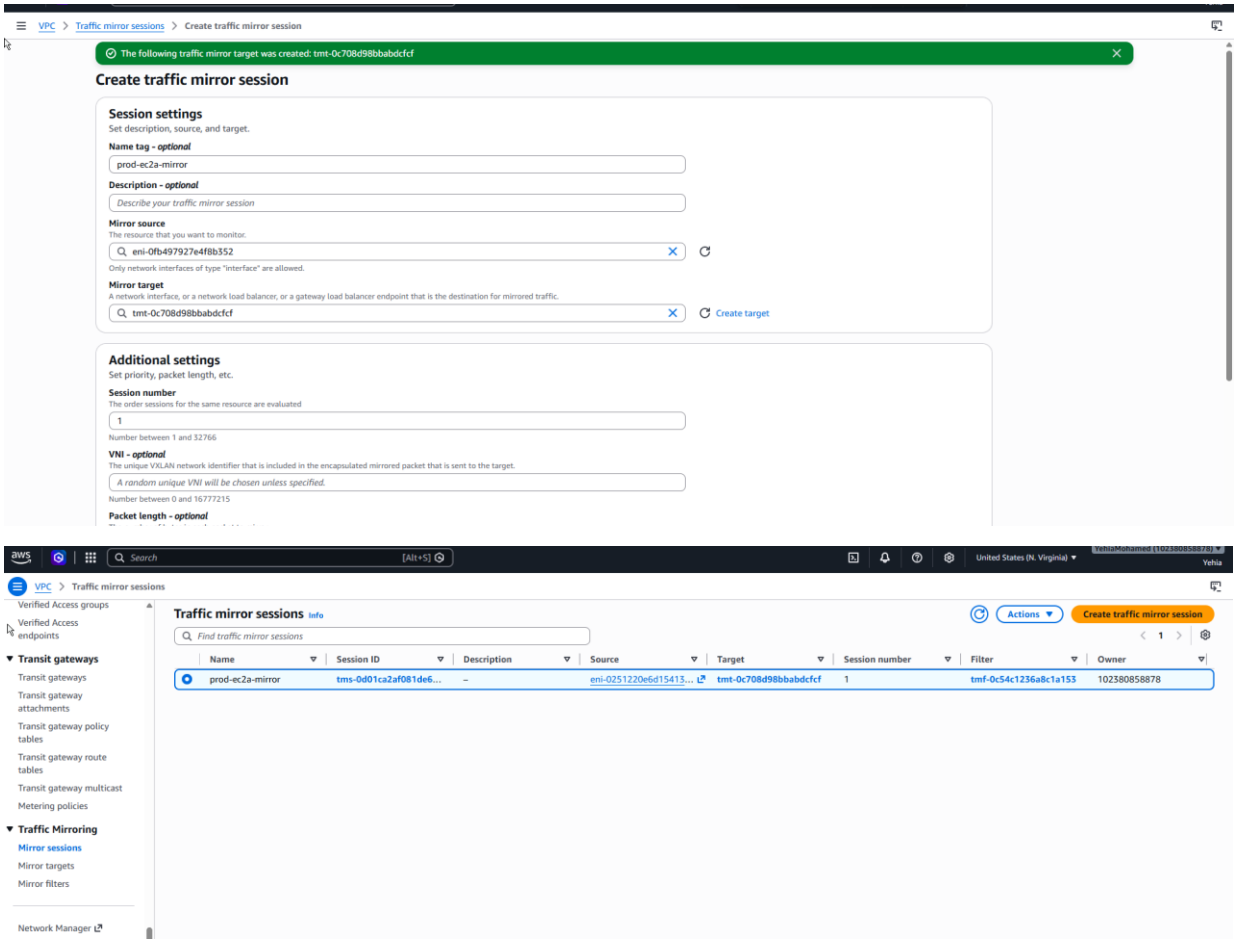


Figure 1.12 — VPC Traffic Mirror Target, Session Creation & Teardown (prod-ec2a-mirror)

- **Session Identifiers:** Session prod-ec2a-mirror (tms-0d01ca2af081de6...) configured as Session #1 using Traffic Mirror Filter tmf-0c54c1236a8c1a153.
- **Source & Target ENIs:** Mirrors network packets from source ENI eni-0251220e6d15413c1 (prod-ec2-a) to target tmt-0c708d98bbabdcfcf mapped to ENI eni-0fb497927e4f8b352 (prod-ec2-b), followed by session deletion after capture to prevent unnecessary hourly charges.

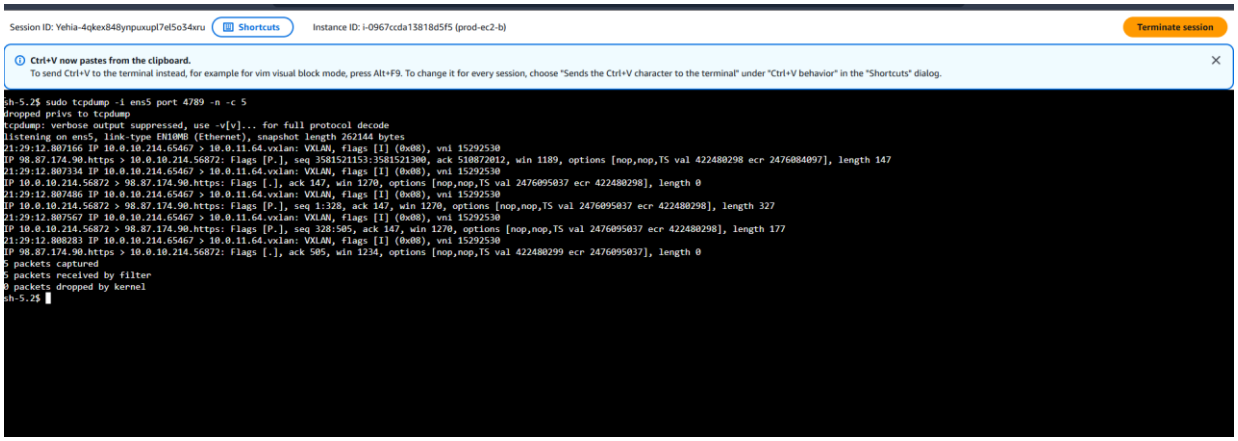


Figure 1.13 — VXLAN Encapsulated Packet Capture on Collector Instance (prod-ec2-b)

- **Capture Command:** Executed sudo tcpdump -i ens5 port 4789 -n -c 5 via SSM Session Manager on prod-ec2-b (i-0967ccda13818d5f5).
- **Packet Verification:** Successfully captured 5 VXLAN-encapsulated frames (vni 15292530) carrying mirrored HTTPS traffic between prod-ec2-a (10.0.10.214) and external IP 98.87.174.90 delivered to collector IP 10.0.11.64 on UDP port 4789.

1.5 Step 8 & Step 9: Athena Table Setup (T56) & Detection Queries (T57)

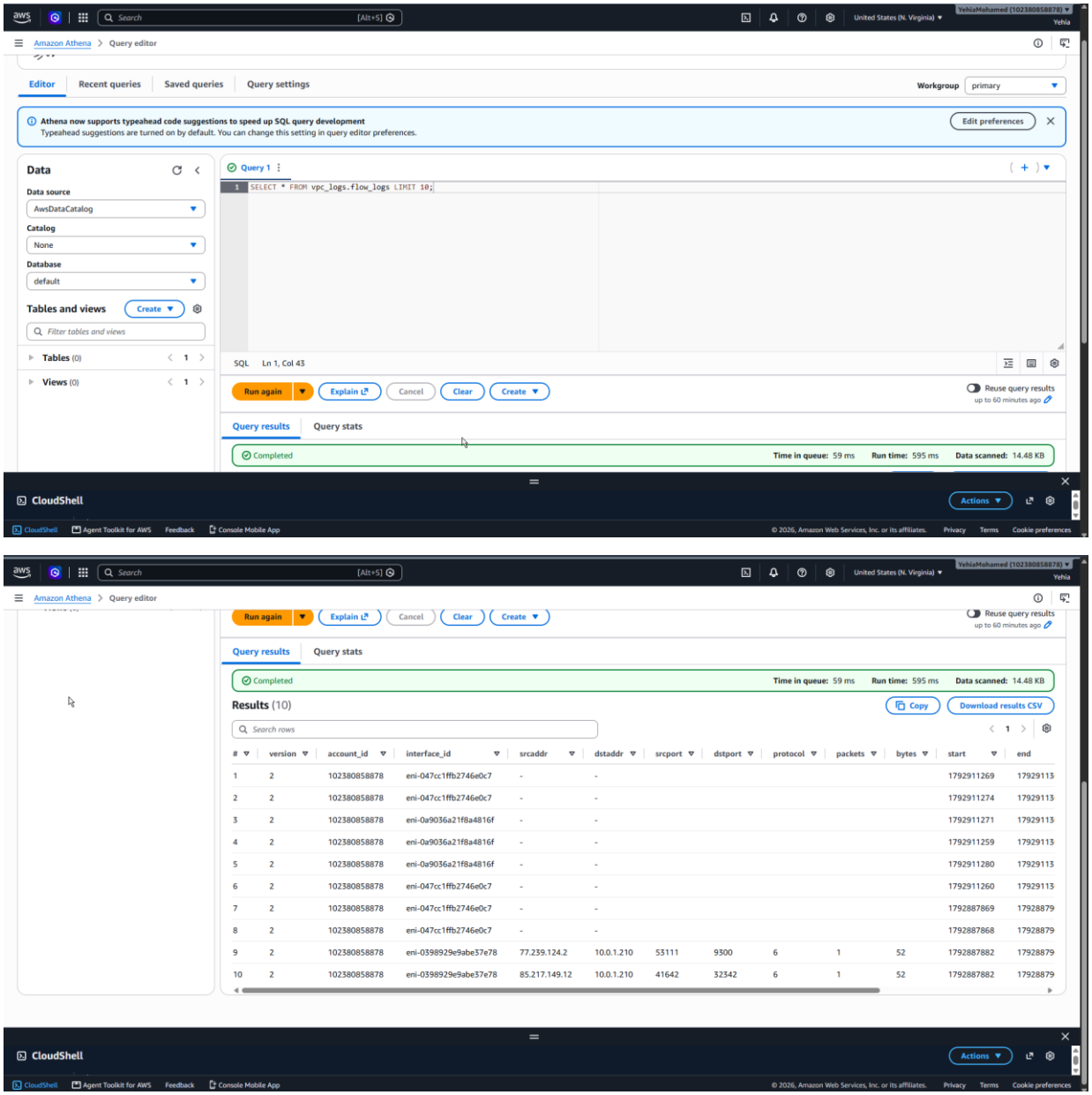


Figure 1.14 — Step 8 (T56): Athena External Table Validation (vpc_logs.flow_logs)

- **Partition Projection Setup:** Created external table vpc_logs.flow_logs partitioned by date (string) with S3 partition projection enabled (yyyy/MM/dd) pointing to s3://prod-flow-logs-102380858878/AWSLogs/102380858878/vpcflowlogs/us-east-1/.
- **Validation Output:** Running SELECT * FROM vpc_logs.flow_logs LIMIT 10; returned 10 parsed flow log rows in 595 ms (14.48 KB scanned), confirming schema alignment.

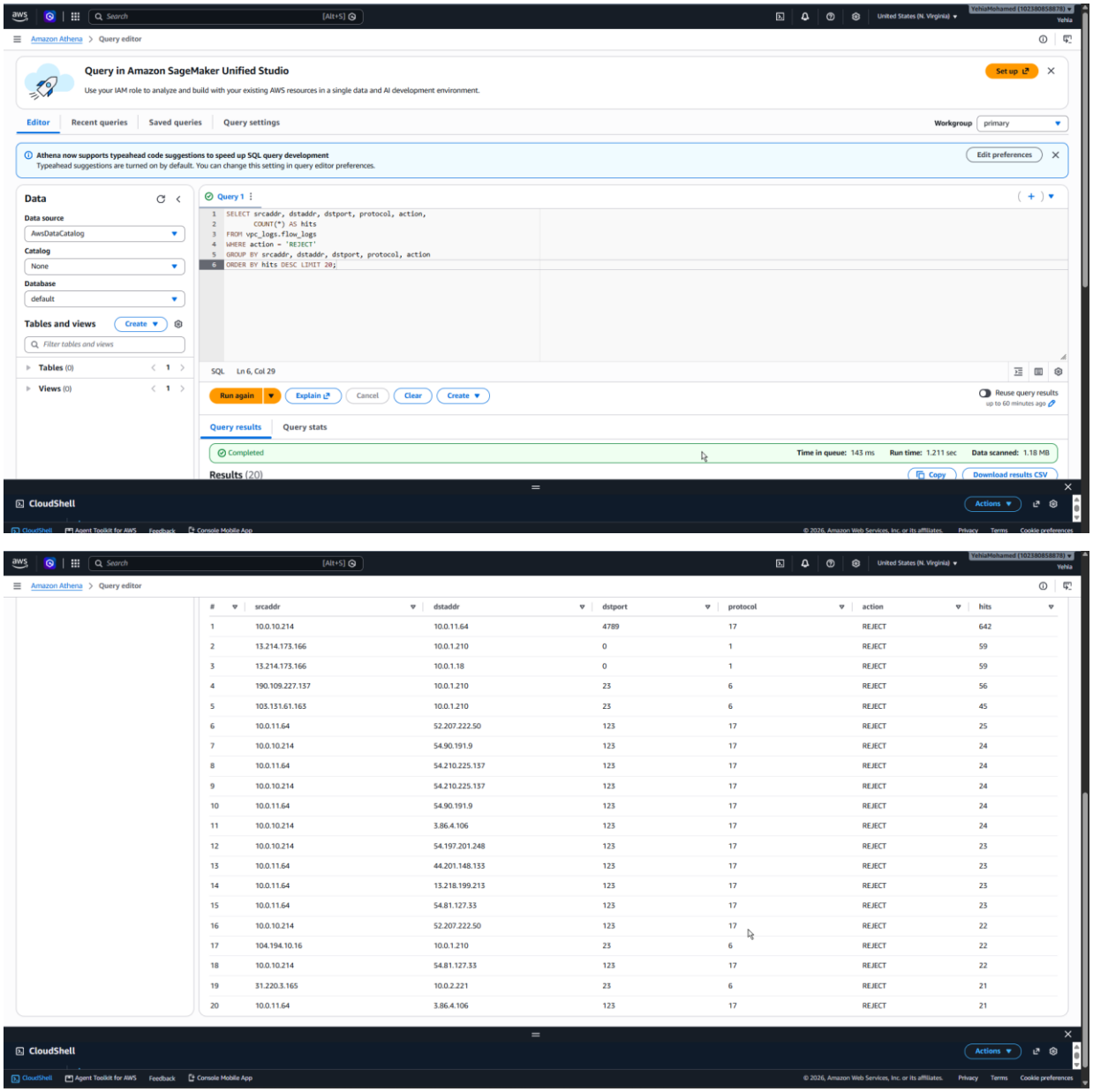


Figure 1.15 — Step 9 (T57 - Query 1): Rejected Traffic Analysis (Port Scans & NACL Blocks)

- **Detection Objective:** Aggregates WHERE action = 'REJECT' grouped by srcaddr, dstaddr, dstport, and protocol to uncover port scans, unauthorized lateral movement, and NACL/SG blocks.
- **Key Findings:** Row 1 captures 642 rejected UDP port 4789 packets from 10.0.10.214 to 10.0.11.64 (prior to allowing VXLAN on nacl-private), alongside external internet scanners probing Telnet (port 23) and ICMP.

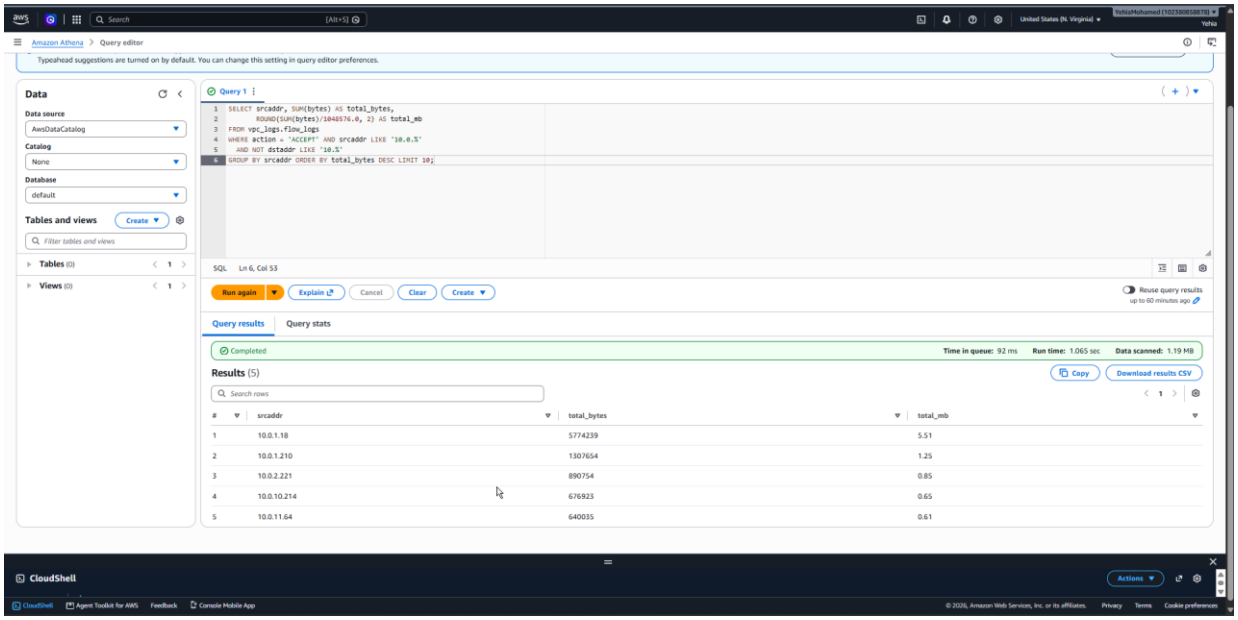
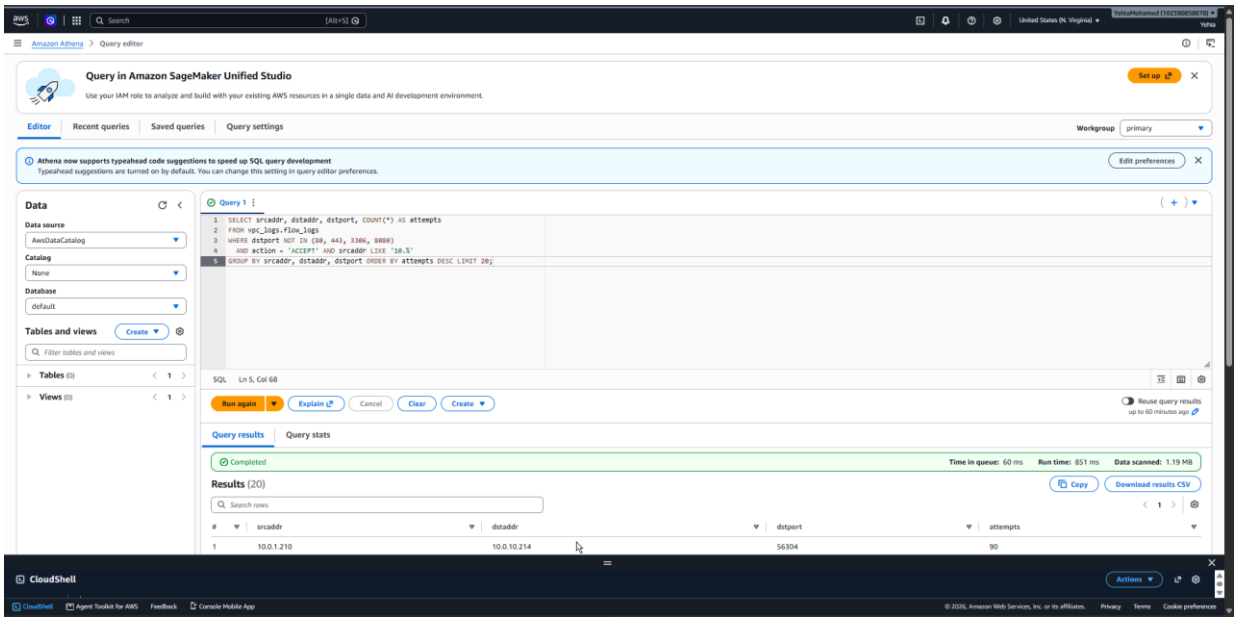


Figure 1.16 — Step 9 (T57 - Query 2): Top Outbound Data Transfers (Exfiltration Detection)

- **Detection Objective:** Calculates total_bytes and total_mb for accepted outbound traffic originating from internal 10.0.% subnets destined for external non-10.% IP addresses.
- **Key Findings:** Identifies top outbound talkers: 10.0.1.18 (5.51 MB), 10.0.1.210 (1.25 MB), 10.0.2.221 (0.85 MB), prod-ec2-a at 10.0.10.214 (0.65 MB), and prod-ec2-b at 10.0.11.64 (0.61 MB).



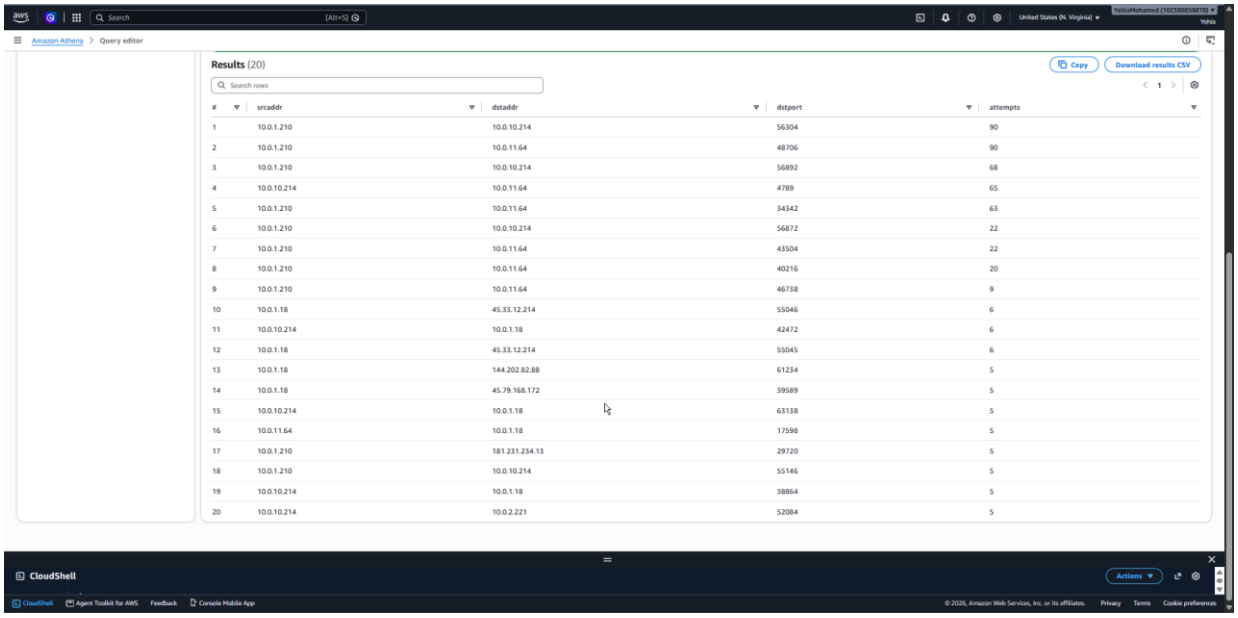
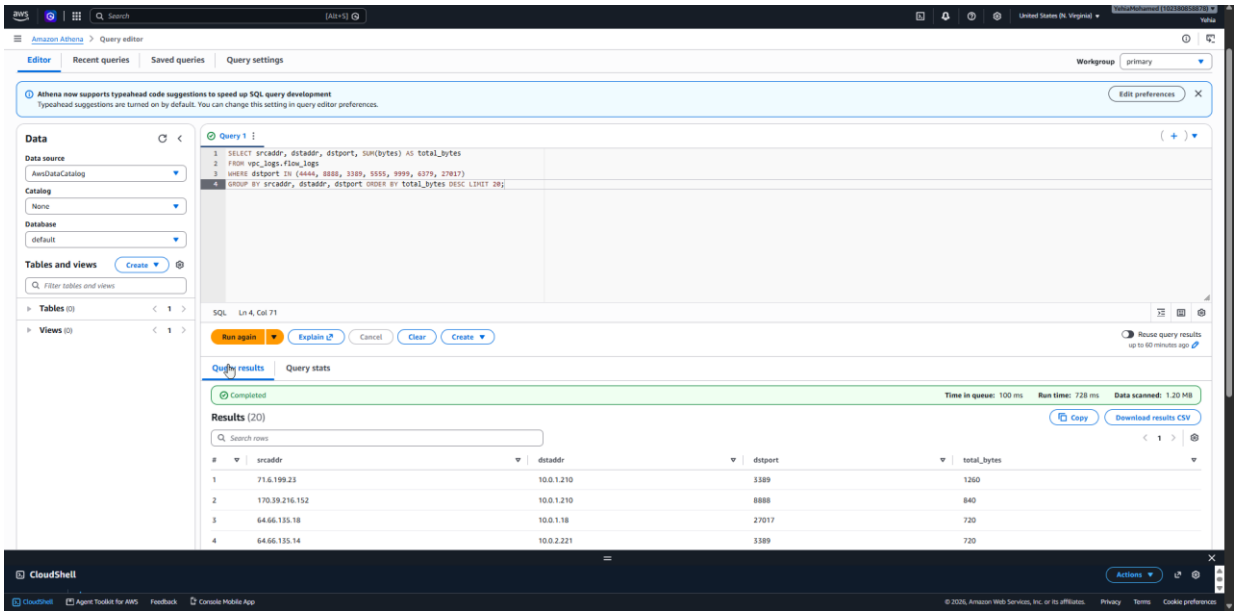
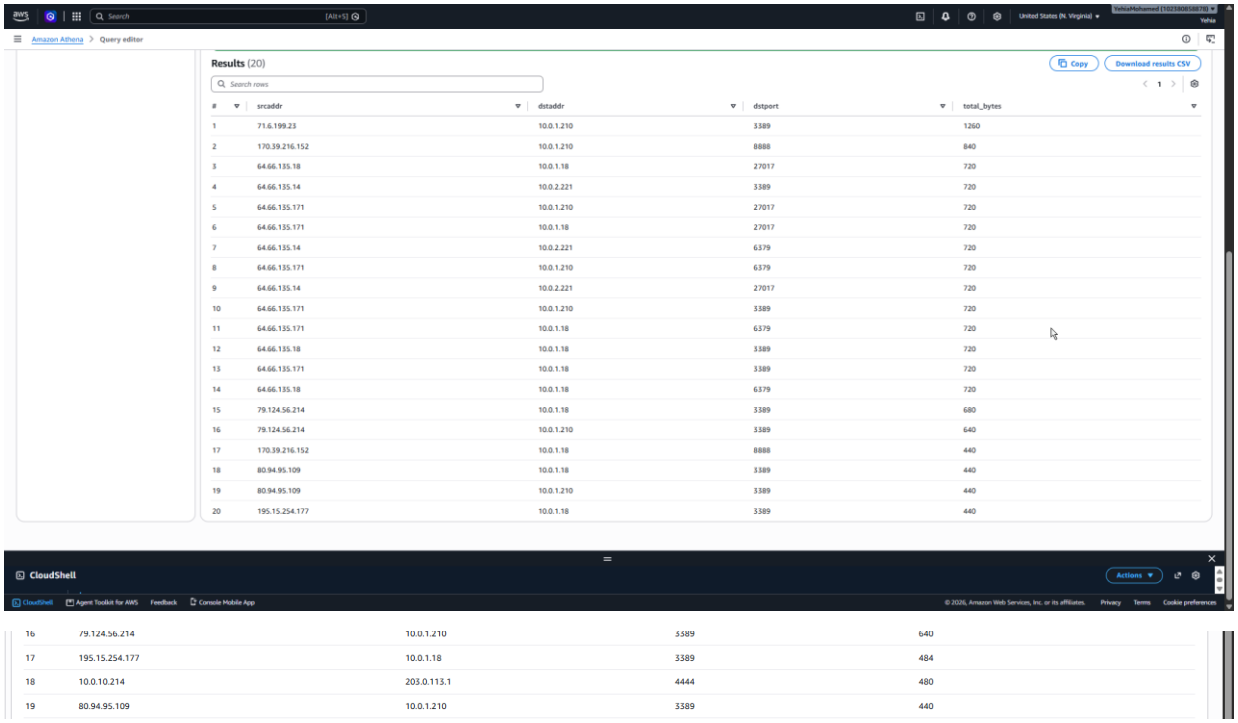


Figure 1.17 — Step 9 (T57 - Query 3): Unusual Port Access (Lateral Movement & Ephemeral Ports)

- **Detection Objective:** Filters internal accepted traffic (srcaddr LIKE '10.%') where dstport NOT IN (80, 443, 3306, 8080) to spot non-standard service ports.
- **Key Findings:** Highlights ephemeral return ports between the ALB (10.0.1.210) and backend EC2 instances, as well as the 65 accepted VXLAN traffic mirroring packets (10.0.10.214 -> 10.0.11.64 on port 4789) in Row 4.



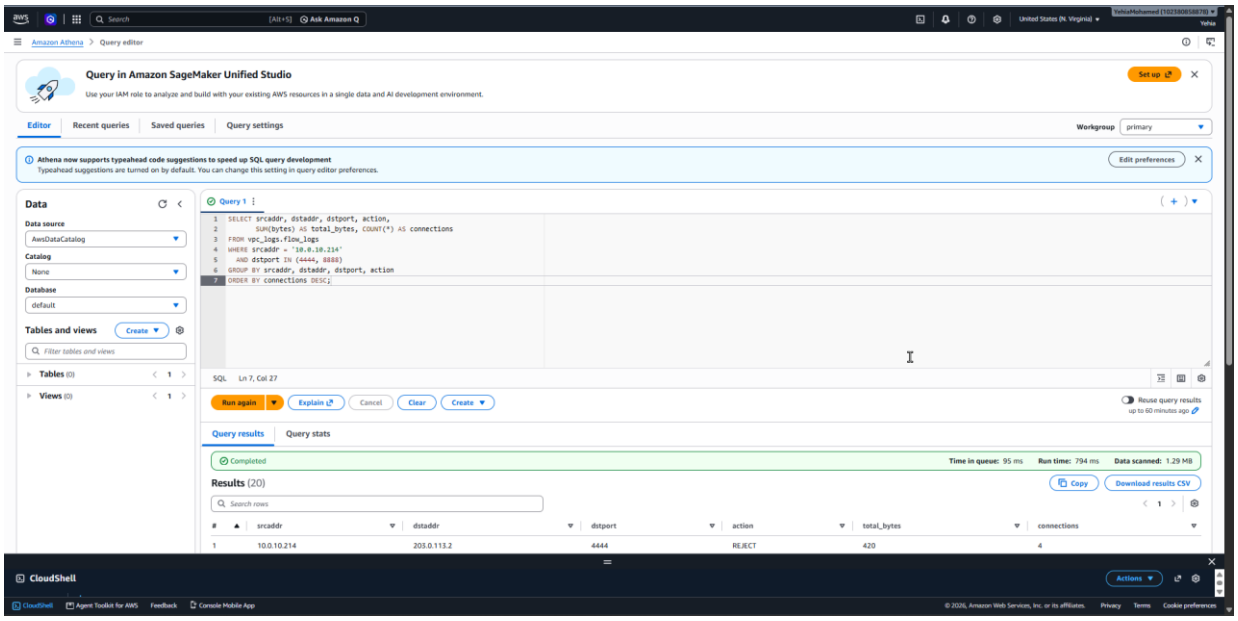


16	79.124.56.214	10.0.1.210	3389	640
17	195.15.254.177	10.0.1.18	3389	484
18	10.0.10.214	203.0.113.1	4444	480
19	80.94.95.109	10.0.1.210	3389	440

Figure 1.18 — Step 9 & Step 10B (T57 - Query 4 & IR Detection): Suspicious C2 / Mining Ports

- **Detection Objective:** Hunts for traffic hitting high-risk ports: 4444 (Metasploit/crypto-mining), 8888/5555 (C2), 3389 (RDP), 6379 (Redis), and 27017 (MongoDB).
- **Simulated Compromise Detection:** Along with background internet scanning bots probing public interfaces on 3389/6379/27017, Row 18 (highlighted in the close-up crop) catches the simulated C2 attack from prod-ec2-a (10.0.10.214) to external C2 IP 203.0.113.1 on port 4444 (480 bytes).

1.6 Step 10 (T64): Live Incident Response Simulation — Triage to Recovery



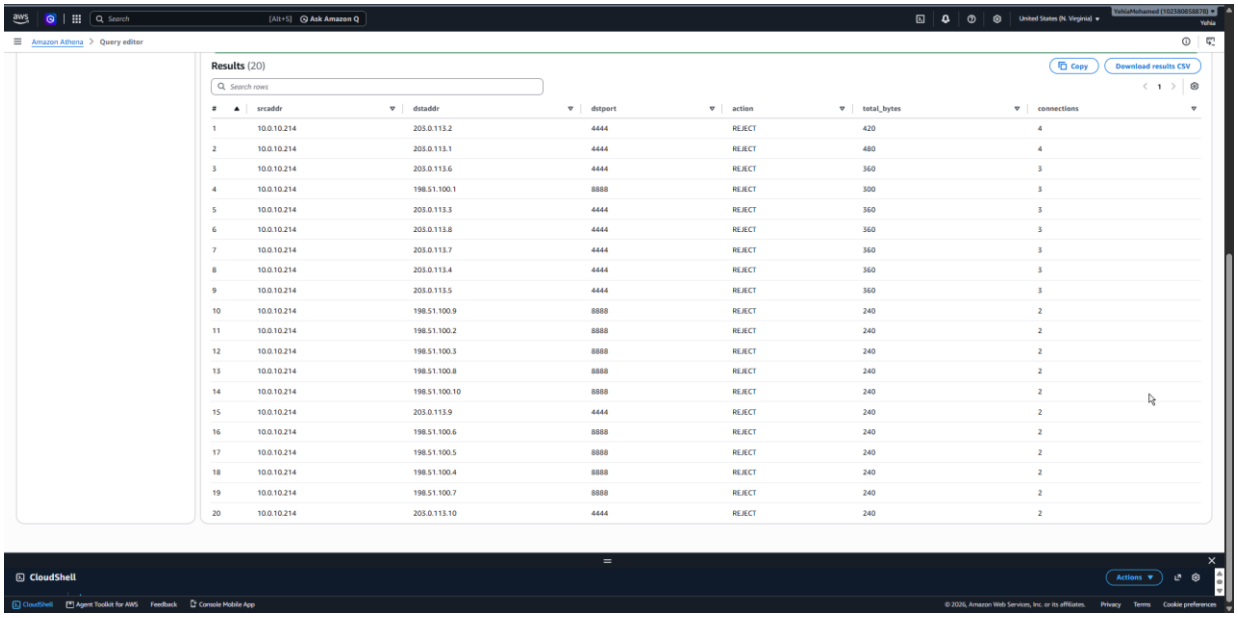


Figure 1.19 — Step 10C (Triage): Scoping Malicious C2 Connections from prod-ec2-a

- **Triage Query:** Filtered vpc_logs.flow_logs specifically for srcaddr = '10.0.10.214' AND dstport IN (4444, 8888).
- **Blast Radius Confirmation:** Confirms 20 distinct outbound C2 connection rows originating from prod-ec2-a targeting the 203.0.113.1–10 subnet on port 4444 and the 198.51.100.1–10 subnet on port 8888.

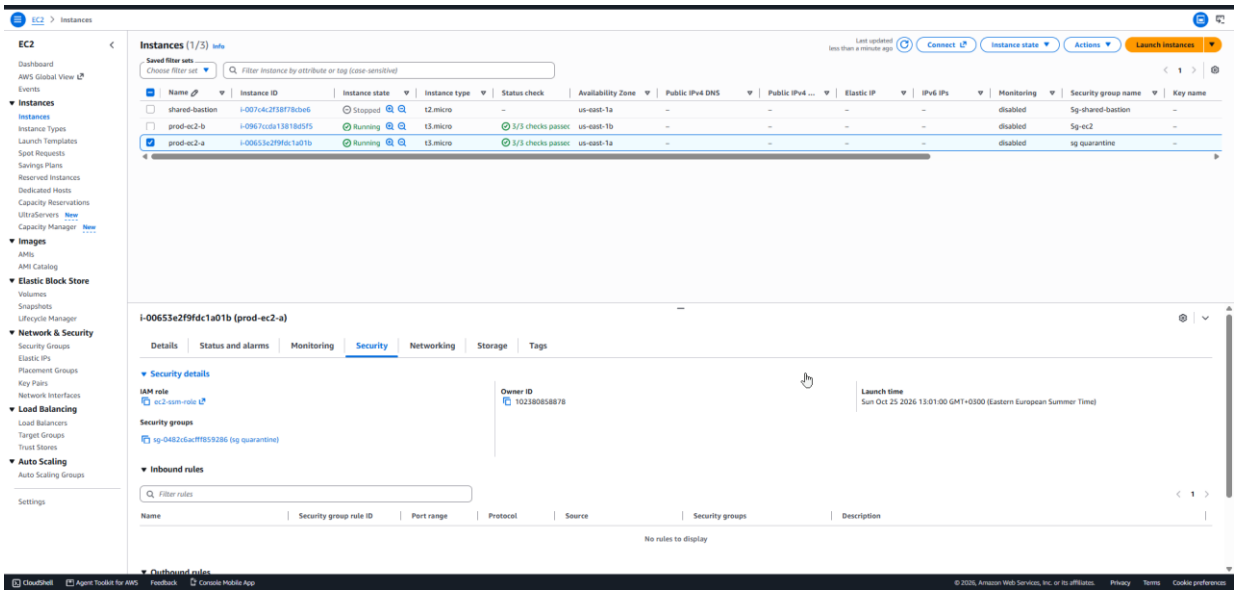


Figure 1.20 — Step 10D (Isolate): Immediate Containment via sg-quarantine Attachment

- **Containment Action:** Executed aws ec2 modify-instance-attribute to strip Sg-ec2 from prod-ec2-a (i-00653e2f9fdc1a01b) and attach sg-0482c6acfff859286 (sg quarantine).
- **Console Verification:** The EC2 Security tab confirms sg quarantine is the sole attached security group with 'No rules to display' under Inbound/Outbound rules, isolating the host while keeping memory intact.

Figure 1.21 — Step 10E (Preserve): Instance Metadata Export & EBS Forensic Snapshot

- **CLI Execution:** Captured full instance state into evidence.json via describe-instances, extracted root VolumeId vol-09aa57f7db5ca90d0, and created forensic EBS snapshot snap-09c22ac2590a12009 (8 GB, Description: 'IR Evidence', StartTime: 2026-10-25T10:52:24Z).

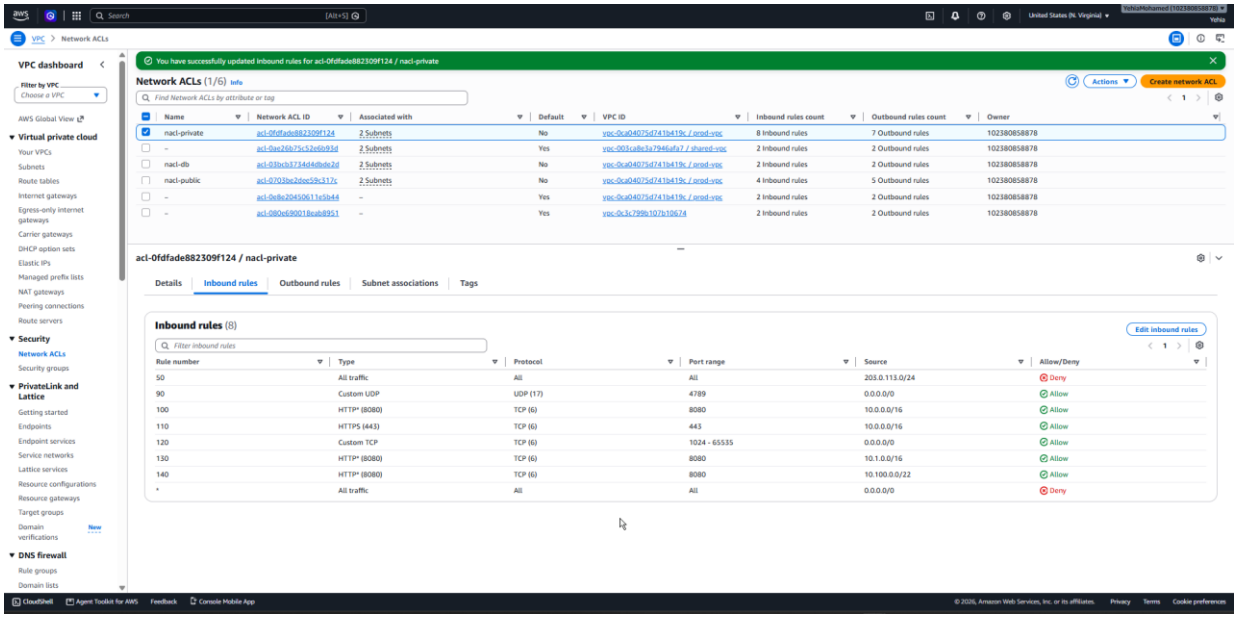


Figure 1.22 — Step 10F (Contain): Subnet-Level Blackholing via Private NACL Rule #50

- **Perimeter Defense:** Added Inbound Rule #50 to nacl-private (acl-0fdfade882309f124) explicitly set to DENY All traffic from malicious CIDR block 203.0.113.0/24 before any ALLOW rules (Rules 90–140) are evaluated.

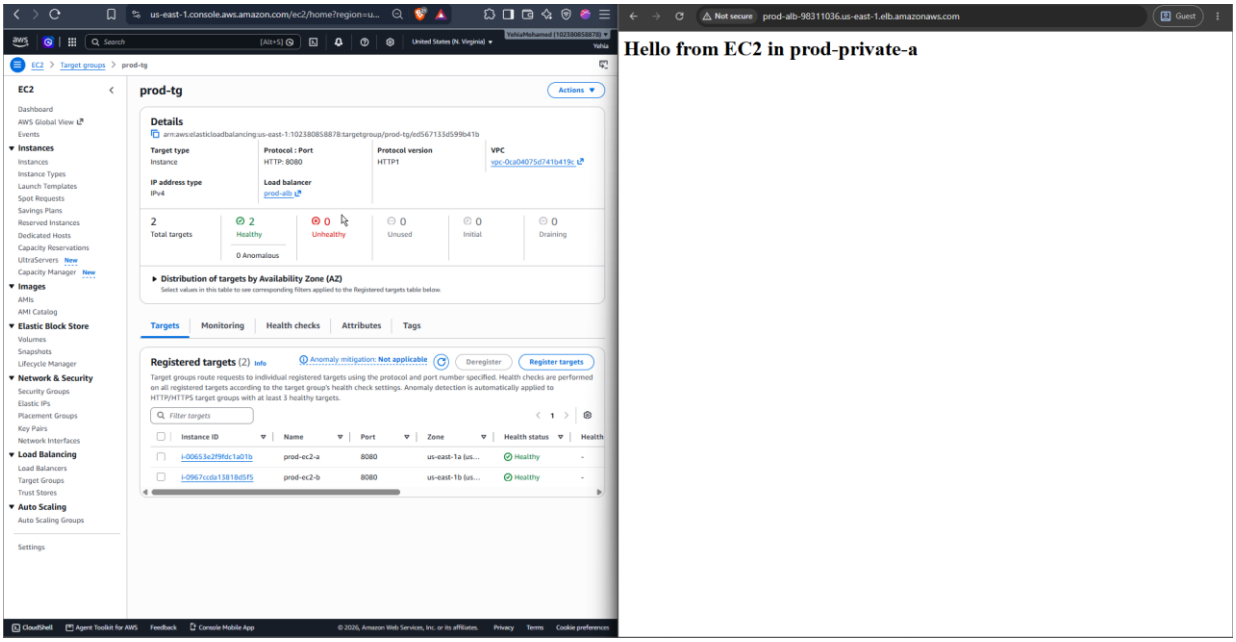


Figure 1.23 — Step 10G (Recover): Security Group Restoration, Target Health & Live App Verification

- **Service Restoration:** Re-attached prod-ec2-a (i-00653e2f9fdc1a01b) to production security group Sg-ec2, verified both targets report Healthy on port 8080 inside prod-tg, and confirmed HTTP 200 application response ('Hello from EC2 in prod-private-a') via prod-alb-98311036.us-east-1.elb.amazonaws.com.

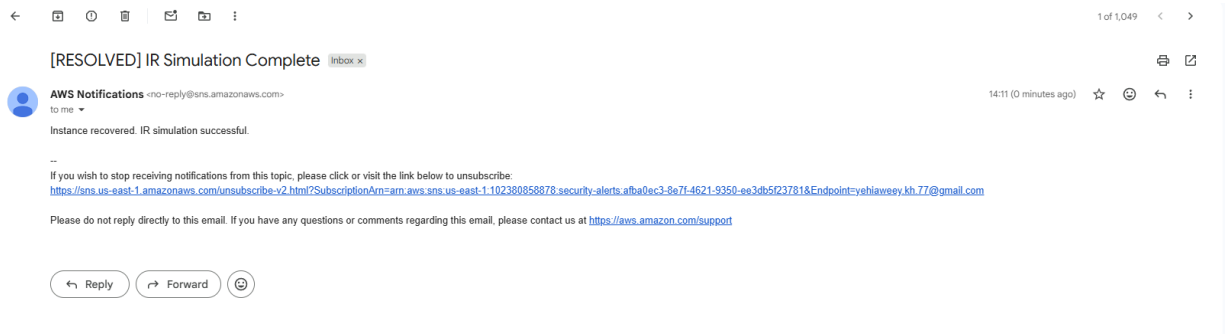


Figure 1.24 — Step 10H (Notify): Incident Resolution Broadcast via Amazon SNS

- **Stakeholder Notification:** Published resolution message to arn:aws:sns:us-east-1:102380858878:security-alerts, delivering the '[RESOLVED] IR Simulation Complete — Instance recovered. IR simulation successful.' email to yehiaweey.kh.77@gmail.com.

Part 2: Step 11 — T65: Reusable Incident Response (IR) Runbook

This Standard Operating Procedure (SOP) provides the repeatable operational runbook for responding to compromised EC2 instances within the production AWS environment. It incorporates both the zero-touch automated containment pipeline and the manual CLI-driven investigation and recovery workflow.

2.1 Production Environment Reference Matrix

Resource Category	Resource Name / Role	Identifier / ARN
AWS Account ID	Production Security Account	102380858878
Primary Region & VPC	N. Virginia / prod-vpc (10.0.0.0/16)	us-east-1 / vpc-0ca04075d741b419c
Monitored Web Instance A	prod-ec2-a (Private IP: 10.0.10.214)	i-00653e2f9fdc1a01b (Root Vol: vol-09aa57f7db5ca90d0)
Monitored Web Instance B	prod-ec2-b (Private IP: 10.0.11.64)	i-0967ccda13818d5f5

Production App SG	Sg-ec2 (Allows port 8080 from ALB)	sg-03eb93672ff6bb751
Quarantine Isolation SG	sg quarantine (0 Inbound / 0 Outbound)	sg-0482c6acfff859286
Private Subnet NACL	nacl-private (Associated with 2 subnets)	acl-0fdfade882309f124
Forensic Snapshot ID	Preserved EBS Root Volume Snapshot	snap-09c22ac2590a12009
VPC Flow Logs S3 Bucket	Centralized Flow Log Repository	s3://prod-flow-logs-102380858878
Security Alerts SNS ARN	SOC Notification Topic	arn:aws:sns:us-east-1:102380858878:security-alerts

2.2 Incident Escalation Contacts

Escalation Tier	Role / Contact Name	Primary Responsibilities	Contact Channel / SLA
Tier 1: SOC / IR Lead	Yehia Mohamed (Security Engineer)	Alert triage, host quarantine, EBS snapshot preservation, Athena log hunting	yehiaweey.kh.77@gmail.com via SNS security-alerts (SLA: < 15 mins)
Tier 2: Cloud NetOps	Infrastructure & DevOps Team	NACL blackholing, Traffic Mirroring capture, ALB target recovery, DR cutover	Internal SecOps Bridge / PagerDuty (SLA: < 30 mins)
Tier 3: Management	CISO & Legal / Compliance	Executive authorization, regulatory breach notification, post-incident review	Direct Incident Commander Escalation (SLA: < 1 hr)

2.3 Automated Containment Path (GuardDuty → EventBridge → Lambda)

When Amazon GuardDuty detects active threat signatures (Severity >= 4.0) against an EC2 workload, the automated containment pipeline executes within seconds without human intervention:

- 1. Detection: Amazon GuardDuty identifies malicious activity (e.g., UnauthorizedAccess:EC2/MaliciousIPCaller, CryptoCurrency:EC2/BitcoinTool.B!DNS, or Backdoor:EC2/C&CActivity.B).
- 2. EventBridge Filtering: Rule guardduty-to-quarantine (arn:aws:events:us-east-1:102380858878:rule/guardduty-to-quarantine) matches events where source == 'aws.guardduty' and detail.severity >= 4.
- 3. Parallel Dispatch: EventBridge invokes Lambda function auto-quarantine-instance (executing under IAM role lambda-quarantine-role) and forwards the alert payload to SNS topic security-alerts.
- 4. Automated Quarantine: The Lambda function extracts detail.resource.instanceDetails.instanceId and calls ec2.modify_instance_attribute(InstanceId=instance_id, Groups=['sg-0482c6acfff859286']), replacing production security groups with sg-quarantine and logging execution details to /aws/lambda/auto-quarantine-instance.

2.4 Manual Incident Response Path (7-Step Runbook & CLI Commands)

Step 1: Detect (CloudWatch Telemetry & Athena Threat Hunting)

Inspect prod-network-dashboard for NetworkOut spikes or high-network-out alarm triggers, then execute the Athena C2/Suspicious Port detection query:

```
-- Athena Q4: Detect C2, Crypto-Mining, and Unauthorized Database/RDP Ports
SELECT srcaddr, dstaddr, dstport, SUM(bytes) AS total_bytes
FROM vpc_logs.flow_logs
WHERE dstport IN (4444, 8888, 3389, 5555, 9999, 6379, 27017)
GROUP BY srcaddr, dstaddr, dstport
ORDER BY total_bytes DESC LIMIT 20;
```

Step 2: Triage (Scope Compromised Instance Connections)

Filter VPC Flow Logs by the suspected internal IP address (e.g., 10.0.10.214 for prod-ec2-a) to determine external C2 destinations, ports, packet actions, and connection volume:

```
-- Athena Triage Query: Isolate Malicious Connections from prod-ec2-a
SELECT srcaddr, dstaddr, dstport, action,
       SUM(bytes) AS total_bytes, COUNT(*) AS connections
FROM vpc_logs.flow_logs
WHERE srcaddr = '10.0.10.214'
  AND dstport IN (4444, 8888)
GROUP BY srcaddr, dstaddr, dstport, action
ORDER BY connections DESC;
```

Step 3: Isolate (Attach Quarantine Security Group)

Immediately sever all inbound and outbound traffic by swapping the instance security group to sg-quarantine (sg-0482c6acfff859286). Do NOT stop or terminate the instance prior to evidence collection:

```
aws ec2 modify-instance-attribute \
  --instance-id i-00653e2f9fdc1a01b \
  --groups sg-0482c6acfff859286 \
  --region us-east-1
```

Step 4: Preserve (Capture Metadata & Create Forensic EBS Snapshot)

Export full instance metadata to JSON and trigger an immutable EBS snapshot of the root volume (vol-09aa57f7db5ca90d0) for offline forensic investigation:

```
# 1. Save full instance metadata to evidence.json
aws ec2 describe-instances \
  --instance-ids i-00653e2f9fdc1a01b \
  --region us-east-1 > evidence.json

# 2. Extract attached Root EBS Volume ID
VOLUME_ID=$(aws ec2 describe-instances \
  --instance-ids i-00653e2f9fdc1a01b \
  --query 'Reservations[0].Instances[0].BlockDeviceMappings[0].Ebs.VolumeId' \
  --output text --region us-east-1)

# 3. Create forensic snapshot (Produced: snap-09c22ac2590a12009)
aws ec2 create-snapshot \
  --volume-id $VOLUME_ID \
  --description "IR Evidence" \
  --region us-east-1
```

Step 5: Investigate (Deep Packet Capture & Disk Forensics)

If live network traffic analysis is required prior to isolation, enable VPC Traffic Mirroring from source ENI eni-0251220e6d15413c1 to target tmt-0c708d98bbabdcfcf and capture VXLAN port 4789 frames, then mount the forensic snapshot on an isolated analysis host:

```
# Capture VXLAN mirrored traffic on collector instance
sudo tcpdump -i ens5 port 4789 -n -c 50 -w ir_vxlan_capture.pcap
```

Step 6: Contain (Subnet Perimeter Blackholing via NACL Rule #50)

Block the external C2 subnet (203.0.113.0/24) at the private subnet boundary by inserting Rule #50 into nacl-private (acl-0fdfade882309f124):

```
aws ec2 create-network-acl-entry \
  --network-acl-id acl-0fdfade882309f124 \
  --rule-number 50 \
  --protocol -1 \
  --rule-action deny \
  --ingress \
  --cidr-block 203.0.113.0/24 \
  --region us-east-1
```

Step 7: Recover & Notify (Restore Production SG, Verify ALB & Publish SNS Alert)

Re-attach the production security group (sg-03eb93672ff6bb751 / Sg-ec2) or launch a replacement instance from a clean golden AMI, verify target health in prod-tg, and notify stakeholders via Amazon SNS:

```
# 1. Restore production security group (Sg-ec2)
aws ec2 modify-instance-attribute \
  --instance-id i-00653e2f9fdc1a01b \
  --groups sg-03eb93672ff6bb751 \
  --region us-east-1

# 2. Verify target health in prod-tg
aws elbv2 describe-target-health \
  --target-group-arn arn:aws:elasticloadbalancing:us-east-1:102380858878:targetgroup/prod-tg/ed567133d599b41b
\
  --region us-east-1

# 3. Publish resolution notification to SNS security-alerts
aws sns publish \
  --topic-arn arn:aws:sns:us-east-1:102380858878:security-alerts \
  --subject "[RESOLVED] IR Simulation Complete" \
  --message "Instance recovered. IR simulation successful." \
  --region us-east-1
```

Part 3: Step 12 — T66: Disaster Recovery (DR) Plan

This Disaster Recovery (DR) Plan defines the cross-region business continuity architecture and failover execution steps to recover the production application in the event of a catastrophic regional outage in us-east-1 (N. Virginia).

3.1 DR Strategy Overview & Recovery Objectives (RTO / RPO)

DR Parameter	Target Specification	Architectural Justification
DR Strategy Tier	Pilot Light	Core networking and data replication remain active 24/7 in DR at minimal cost; compute and load balancers spin up only during failover.
Primary Production Region	us-east-1 (N. Virginia)	Hosts prod-vpc (10.0.0.0/16) across us-east-1a and us-east-1b.
Disaster Recovery Region	us-west-2 (Oregon)	Geographically isolated secondary region hosting dr-vpc (10.2.0.0/16) across us-west-2a and us-west-2b.
Recovery Time Objective (RTO)	~15–20 Minutes	Time required to restore RDS from the latest local snapshot, launch EC2 nodes from pre-copied AMIs, and cut over Route 53 DNS.
Recovery Point Objective (RPO)	~24 Hours (DB) / Near-Real-Time (S3)	Based on automated daily cross-region RDS snapshot copies and continuous S3 Cross-

	Region Replication (CRR).

3.2 Pilot Light Architecture: Pre-Created vs. Deployed on Failover

Architecture Layer	Pre-Created in DR (us-west-2 — Always On / Zero-to-Low Cost)	Provisioned / Scaled on Failover (Active DR Cost Only)
VPC & Networking	dr-vpc (10.2.0.0/16), 2 Public Subnets (10.2.1.0/24, 10.2.2.0/24), 2 Private App Subnets (10.2.10.0/24, 10.2.11.0/24), 2 Private DB Subnets, Internet Gateway (IGW), Route Tables, NACLs, and Security Groups (sg-alb, sg-ec2, sg-rds, sg-quarantine).	NAT Gateways in public subnets (deployed via Terraform/CLI on failover to avoid $0.045/hr idle charges) and private route table default routes (0.0.0.0/0 -> NAT GW).
Compute Tier (EC2)	Cross-region replicated Golden AMIs of prod-ec2-a/b and pre-configured Launch Templates with IAM Instance Profiles (ec2-ssm-role) and startup scripts.	Auto Scaling Group scaled from 0 -> 2 running t3.micro instances (dr-ec2-a, dr-ec2-b) across us-west-2a and us-west-2b.
Load Balancing	Target Group (dr-tg on port 8080) and ACM TLS/SSL certificates pre-validated in us-west-2.	Internet-facing Application Load Balancer (dr-alb) provisioned in public subnets and attached to dr-tg.
Database Tier (RDS)	DB Subnet Groups, Parameter Groups, KMS Encryption Keys, and automated daily RDS MySQL snapshots copied from us-east-1.	RDS MySQL instance (dr-rds) restored from the most recent cross-region snapshot into private DB subnets.
Storage Tier (S3)	Destination S3 buckets in us-west-2 with Versioning enabled, continuously receiving objects via S3 Cross-Region Replication (CRR).	Application environment variables and CloudFront origins updated to reference the us-west-2 S3 bucket endpoints.

3.3 Cross-Region Replication Configuration

- RDS Automated Snapshot Replication: Automated daily snapshots of prod-rds (MySQL Community in us-east-1) are configured with automatic cross-region copy to us-west-2. Snapshots are retained for 7 days in us-west-2, guaranteeing an RPO of <= 24 hours.
- EC2 Golden AMI Replication: Whenever the application tier is updated in us-east-1, a golden AMI is created and copied to us-west-2 using aws ec2 copy-image --source-region us-east-1 --region us-west-2, ensuring identical OS hardening and web server configurations.
- S3 Cross-Region Replication (CRR): Production S3 buckets (including application assets and flow log archives) use an IAM replication role to asynchronously replicate newly written objects to us-west-2 within 15 minutes.

3.4 Step-by-Step Regional Failover Procedure (RTO: ~15–20 Minutes)

Time Window	Failover Phase	Execution Steps & Verification Criteria
Minutes 0–3	1. Declare DR & Initiate RDS Restore	Confirm unrecoverable us-east-1 outage. In us-west-2, trigger aws rds restore-db-instance-from-db-snapshot using the latest replicated prod-rds snapshot and provision the NAT Gateway in dr-vpc public subnet 10.2.1.0/24.
Minutes 3–10	2. Provision ALB & Scale Out EC2 Compute	Deploy dr-alb across us-west-2a/2b public subnets. Update the DR Auto Scaling Group desired capacity from 0 to 2 (or launch dr-

		ec2-a and dr-ec2-b from the replicated Launch Template into 10.2.10.0/24 and 10.2.11.0/24).
Minutes 10–15	3. Connect DB & Validate Target Health	Once dr-rds status reaches Available, update SSM Parameter Store / app config with the new RDS endpoint. Verify both EC2 instances register as Healthy on port 8080 in dr-tg and test HTTP 200 response via dr-alb DNS.
Minutes 15–20	4. Route 53 DNS Cutover & Notification	Update Amazon Route 53 Alias/Failover record to point production traffic to dr-alb in us-west-2. Publish failover completion notice to SNS security-alerts.

3.5 Scope Boundaries: When DR Failover is NOT Invoked

Regional failover to us-west-2 is reserved strictly for full regional outages in us-east-1. Localized component failures, traffic spikes, and security compromises are remediated locally within us-east-1 without triggering DR:

Incident Scenario	Why Regional DR is NOT Needed	Standard Local Remediation in us-east-1
Single EC2 Failure (e.g., prod-ec2-a crash)	Redundant compute is already active in us-east-1b (prod-ec2-b).	prod-alb health checks automatically stop routing to the unhealthy instance; EC2 Auto Scaling terminates and replaces the failed node locally.
Single Availability Zone Outage (us-east-1a)	prod-vpc spans multiple physically isolated AZs (us-east-1a and us-east-1b).	ALB routes 100% of web traffic to prod-ec2-b in us-east-1b, and Multi-AZ RDS automatically fails over to its standby replica in us-east-1b.
Traffic Overload / DDoS Spike	Capacity exhaustion is mitigated elastically without changing regions.	Auto Scaling scales out additional EC2 instances, while CloudFront edge caching and AWS WAF rate-based rules absorb volumetric floods.
Compromised Instance (Malware / C2 Activity)	Failing over to DR risks propagating the attacker or corrupted data into us-west-2.	Execute the T65 IR Runbook: isolate the host via sg-quarantine (sg-0482c6acfff859286), take forensic EBS snapshot, block C2 CIDR via NACL Rule #50, and redeploy from a clean AMI.

SECTION 6 — CROSS-WORKSTREAM REMARKS & COMPLETED FINAL ACCOUNT TEARDOWN (TASKS T26 & T68)

Defense Remarks, Dependency-Ordered 12-Step Cleanup Runbook & Automated CLI Teardown Script

Executed by All 5 Team Members across us-east-1 (N. Virginia) & us-west-2 (Oregon) | Final Account Billing State: $0.00

6.1 Cross-Workstream Engineering Remarks (Defense Notes)

- **1. Two Lab Environments in Connectivity (T27 – T40):** Shrouk's report and the Client VPN verification used two sets of VPC/resource IDs because the lab was rebuilt on October 29 to resolve the Client VPN TLS handshake failure (vpc-0ca04075d741b419c with prod-ec2-a at 10.0.10.214 in the original run vs. vpc-0e2a24823e55813b8 with prod-ec2-a at 10.0.10.123 in the rebuilt run). Both sets of IDs and screenshots are documented in full in Section 3 to demonstrate the complete troubleshooting lifecycle.
- **2. ICMP Ping Behavior Across Peering (T18 & T30):** In the first environment, Sg-ec2 allowed ICMP from 10.1.0.0/16, but nacl-private only added TCP 8080 and ephemeral ports (1024-65535) for 10.1.0.0/16 (no ICMP rule), which is why curl :8080 succeeded while ping showed 100% loss in Shrouk's first run. In the October 29 rebuilt environment, both ICMP (0% loss) and HTTP 8080 succeeded.
- **3. Documented Plan Exceptions (T25, T45 & T53):** T25 (Firewall Manager) was deferred intentionally because it requires AWS Organizations and costs $100/month per policy. T45 (WAF SQLi Rule Group) was configured and screenshotted in Block mode on the CloudFront Pay-As-You-Go plan, then omitted when switching to the CloudFront Free plan ($0/mo). In T53, Mariam documented restricting Sg-alb to the CloudFront Managed Prefix List in production to prevent direct ALB requests from bypassing AWS WAF.

6.2 Task T68: Executed 12-Step Dependency-Ordered Final Account Teardown Runbook

All AWS resources provisioned across us-east-1 (N. Virginia) and us-west-2 (Oregon) have been completely torn down following the dependency-ordered sequence below to prevent DependencyViolation locks and return the AWS account to $0.00 billing:

Step #	Resource Category	Executed Deletion Order & Cost-Control Verification (us-east-1 & us-west-2)	Status
Step 1	Hybrid VPNs & Transit Gateway (T36, T39, T40)	Disassociated target subnets from Client VPN endpoints (cvpn-endpoint-053c30ff1d8369afa / 0a432f08d863be3f5), deleted Client VPN endpoints and ACM certs. Deleted Site-to-Site VPN prod-vpn, detached/deleted prod-vgw, and deleted onprem-cgw. Deleted TGW attachments and prod-tgw.	Completed
Step 2	PrivateLink & Extra VPC Endpoints (T26)	Deleted consumer endpoint shared-privatelink-endpoint-2 (vpce-0bf1ccca776cb89f3), Endpoint Service prod-privatelink-service, Internal NLB prod-nlb, Target Group tg-privatelink, all 9 Interface Endpoints (ssm, ssmmessages, ec2messages, logs, kms, secretsmanager), and prod-s3-gateway.	Completed
Step 3	Traffic Mirroring (T62)	Deleted Traffic Mirror Session prod-ec2a-mirror, Traffic Mirror Target tmt-0c708d98bbabdcfcf, and Traffic Mirror Filter tmf-0c54c1236a8c1a153.	Completed
Step 4	EC2 Instances, AMIs & EBS Snapshots (T7, T11, T64)	Terminated prod-ec2-a (i-00653e2f9fdc1a01b), prod-ec2-b	Completed

		(i-0967ccda13818d5f5), shared-bastion (i-007c4c2f38f78cbe6), and rebuilt lab instances. Deregistered AMIs and deleted forensic EBS Snapshot snap-09c22ac2590a12009.	
Step 5	Application Load Balancer (T8)	Deleted internet-facing prod-alb (prod-alb-98311036...) and Target Group prod-tg.	Completed
Step 6	RDS Multi-AZ Database (T9)	Deleted prod-rds with 'Skip final snapshot' and 'Delete all automated backups' selected to prevent ongoing storage charges, then deleted the DB Subnet Group.	Completed
Step 7	NAT Gateway & Elastic IP (T5)	Deleted prod-nat-gw (nat-05ec1929688401bdc), waited for Deleted state, and released Elastic IP 54.165.178.100 to prevent unattached EIP charges.	Completed
Step 8	VPC Peering, SGs, NACLs, Subnets, RTs, IGWs & VPCs	Deleted VPC Peering connections, saved Reachability Analyzer paths, all custom Security Groups, custom NACLs, Subnets, Route Tables, detached/deleted prod-igw, and deleted prod-vpc, shared-vpc, onprem-vpc (us-east-1), and dr-vpc (us-west-2).	Completed
Step 9	Edge Layer: CloudFront, WAF & Route 53 (T43 – T49)	Disabled and deleted CloudFront distributions E22LB0IQI1CK9E and E2L88I5K45NWFD, deleted OAC EEOVPL2KWX8QQ, deleted WAF Web ACLs, deleted Route 53 A/Failover records, Hosted Zone yourapp.com, and Health Check project-alb-health-check.	Completed
Step 10	Athena, Flow Logs & S3 Buckets (T19, T42, T55, T56)	Deleted Flow Log fl-017bf789ca3366c21, dropped Athena database vpc_logs, deleted CloudTrail project-trail, and emptied/deleted all S3 buckets (yourapp-static-assets, maintenance bucket, flow logs bucket, CloudTrail bucket, Athena results).	Completed
Step 11	EventBridge, Lambda, SNS & CloudWatch (T1, T58 – T63)	Deleted EventBridge rule guardduty-to-quarantine, Lambda function auto-quarantine-instance, SNS topic security-alerts, CloudWatch dashboard prod-network-	Completed

		dashboard, and all 8 CloudWatch alarms.	
Step 12	IAM Roles, Security Hub & GuardDuty (T13, T20, T21, T61)	Deleted IAM roles ec2-ssm-role and lambda-quarantine-role, disabled AWS Security Hub, stopped AWS Config recorder, and disabled Amazon GuardDuty.	Completed

6.3 Automated CLI Teardown Script Reference (teardown_all_t68.sh)

```
#!/bin/bash
export AWS_DEFAULT_REGION="us-east-1"

# 1. EventBridge, Lambda, SNS, CloudWatch Alarms & Dashboard
aws events remove-targets --rule guardduty-to-quarantine --ids $(aws events list-targets-by-rule --rule
guardduty-to-quarantine --query 'Targets[*].Id' --output text) 2>/dev/null || true
aws events delete-rule --name guardduty-to-quarantine 2>/dev/null || true
aws lambda delete-function --function-name auto-quarantine-instance 2>/dev/null || true
aws sns delete-topic --topic-arn arn:aws:sns:us-east-1:102380858878:security-alerts 2>/dev/null || true
aws cloudwatch delete-dashboards --dashboard-names prod-network-dashboard 2>/dev/null || true
ALARM_NAMES=$(aws cloudwatch describe-alarms --query 'MetricAlarms[*].AlarmName' --output text)
[ -n "$ALARM_NAMES" ] && aws cloudwatch delete-alarms --alarm-names $ALARM_NAMES 2>/dev/null || true

# 2. PrivateLink Endpoints, Endpoint Services & Load Balancers
VPCE_IDS=$(aws ec2 describe-vpc-endpoints --query 'VpcEndpoints[*].VpcEndpointId' --output text)
[ -n "$VPCE_IDS" ] && aws ec2 delete-vpc-endpoints --vpc-endpoint-ids $VPCE_IDS 2>/dev/null || true
for LB_ARN in $(aws elbv2 describe-load-balancers --query 'LoadBalancers[*].LoadBalancerArn' --output text);
do
  aws elbv2 delete-load-balancer --load-balancer-arn $LB_ARN 2>/dev/null || true
done
sleep 15
for SVC_ID in $(aws ec2 describe-vpc-endpoint-services --filters "Name=service-type,Values=Interface" --query
'ServiceDetails[?Owner!=`amazon`].ServiceId' --output text); do
  aws ec2 delete-vpc-endpoint-service-configurations --service-ids $SVC_ID 2>/dev/null || true
done
for TG_ARN in $(aws elbv2 describe-target-groups --query 'TargetGroups[*].TargetGroupArn' --output text); do
  aws elbv2 delete-target-group --target-group-arn $TG_ARN 2>/dev/null || true
done

# 3. EC2 Instances, Forensic Snapshots, RDS, NAT GW & EIPs
INST_IDS=$(aws ec2 describe-instances --filters "Name=instance-state-
name,Values=pending,running,stopping,stopped" --query 'Reservations[*].Instances[*].InstanceId' --output
text)
if [ -n "$INST_IDS" ]; then
  aws ec2 terminate-instances --instance-ids $INST_IDS >/dev/null
  aws ec2 wait instance-terminated --instance-ids $INST_IDS
fi
aws ec2 delete-snapshot --snapshot-id snap-09c22ac2590a12009 2>/dev/null || true
aws rds delete-db-instance --db-instance-identifier prod-rds --skip-final-snapshot --delete-automated-backups
2>/dev/null || true
for NAT_ID in $(aws ec2 describe-nat-gateways --filter "Name=state,Values=available,pending" --query
'NatGateways[*].NatGatewayId' --output text); do
  aws ec2 delete-nat-gateway --nat-gateway-id $NAT_ID >/dev/null
done
sleep 60
for ALLOC_ID in $(aws ec2 describe-addresses --query 'Addresses[*].AllocationId' --output text); do
```

```
  aws ec2 release-address --allocation-id $ALLOC_ID 2>/dev/null || true
done

# 4. Peering, Flow Logs, CloudTrail, Security Hub, GuardDuty & DR VPC (us-west-2)
for PCX in $(aws ec2 describe-vpc-peering-connections --filters "Name=status-code,Values=active,pending-
acceptance" --query 'VpcPeeringConnections[*].VpcPeeringConnectionId' --output text); do
  aws ec2 delete-vpc-peering-connection --vpc-peering-connection-id $PCX 2>/dev/null || true
done
FL_IDS=$(aws ec2 describe-flow-logs --query 'FlowLogs[*].FlowLogId' --output text)
[ -n "$FL_IDS" ] && aws ec2 delete-flow-logs --flow-log-ids $FL_IDS 2>/dev/null || true
aws cloudtrail delete-trail --name project-trail 2>/dev/null || true
aws securityhub disable-security-hub 2>/dev/null || true
for DET_ID in $(aws guardduty list-detectors --query 'DetectorIds[*]' --output text); do
  aws guardduty delete-detector --detector-id $DET_ID 2>/dev/null || true
done
```